

Discrete Choice Model Estimation with R

Dan Yavorsky

Table of contents

Welcome	8
How to read this book	9
Intended Audience	10
Acknowledgments	11
Colophon	12
 I Foundations	 13
1 Choice Models and Simulation	14
1.1 A Framework for Choice	14
1.2 A Simple Example	15
1.3 A Binary Logit Example	17
1.3.1 A model has implications	19
1.4 The Running Example: A Laptop Choice Study	21
1.5 The Road Ahead	22
1.6 Key Learnings	22
 2 Drawing from Densities	 24
2.1 Pseudo-Random Numbers from the Uniform Distribution	24
2.2 The Inverse Transform Method	25
2.2.1 Example: Exponential	27
2.2.2 Example: Gumbel	28
2.2.3 Example: Logistic	30
2.2.4 Example: Normal	31
2.3 Transforming Draws	32
2.4 The Multivariate Normal	34
2.5 Quasi-Random Draws	36
2.6 Key Learnings	39
 3 Simulating from a Model	 40
3.1 The Anatomy of a Simulation Script	41
3.2 Example: Simulating Binary Logit Choices	41
3.3 Example: Simulating Binary Probit Choices	43
3.4 Packaging the Simulator as a Function	45

3.5	Sanity Checks on Simulated Data	46
3.6	The Simulate-and-Recover Workflow	48
3.7	Key Learnings	48
4	Organizing Choice Data	50
4.1	The Shape of Choice Data	50
4.2	Wide and Long Formats	51
4.3	Two Kinds of Variables, Two Kinds of Columns	52
4.4	Coding Categorical Attributes	53
4.5	Where Is the Intercept?	54
4.6	Building the Study Design	54
4.7	From Data Frame to Likelihood Inputs	56
4.8	Package Conventions: <code>dfidx</code> and Friends	58
4.9	Key Learnings	58
5	From Choice Data to Likelihood	60
5.1	The Likelihood Function	60
5.2	Independence and the Product Form	61
5.3	The Log-Likelihood	61
5.4	Coding the Binary Logit Log-Likelihood	63
5.5	Maximum Likelihood Estimation	64
5.6	The Score and the Hessian	65
5.7	A First Estimator: Grid Search	67
5.8	Why Grid Search Fails at Scale	70
5.9	Key Learnings	70
6	Numerical Optimization	72
6.1	Climbing a Hill You Cannot See	72
6.2	Newton–Raphson	73
6.3	The Quasi-Newton Family: BHHH and BFGS	76
6.4	Numerical Derivatives	77
6.5	R’s Optimizers: <code>optim()</code>	78
6.6	Standard Errors from the Hessian	80
6.7	Practical Realities	80
6.8	The Complete Loop, End to End	81
6.9	Key Learnings	82
II	The MNL Family, by Likelihood	83
7	The Multinomial Logit Model	84
7.1	From Two Alternatives to Many	84
7.2	Random Utility and the Extreme Value Distribution	85

7.3	The MNL Choice Probability in R	86
7.4	Properties: What the MNL Assumes About Substitution	87
7.4.1	Independence from irrelevant alternatives	87
7.4.2	Proportional substitution	88
7.5	Identification: What Choices Can and Cannot Reveal	88
7.6	Simulating the Laptop Study	89
7.7	Key Learnings	93
8	Estimating the MNL by Maximum Likelihood	94
8.1	The MNL Log-Likelihood	94
8.2	Coding <code>loglik_mnl()</code>	95
8.3	The Log-Sum-Exp Trick	96
8.4	The Analytic Gradient	97
8.5	Estimation	99
8.6	Assessing Fit	100
8.7	Validation Against <code>mlogit</code>	101
8.8	A Simulate-and-Recover Study	103
8.9	Key Learnings	105
9	Code Performance	106
9.1	Why Speed Matters Now	106
9.2	Measure First	107
9.3	Vectorizing the MNL Log-Likelihood	107
9.4	Precompute What Doesn't Change	109
9.5	Preallocate; Don't Grow	110
9.6	Numerical Hygiene Is Performance Too	111
9.7	When R Isn't Enough: A Glimpse Beyond	111
9.8	Key Learnings	112
10	Relaxing IIA: Nested Logit and GEV Models	113
10.1	When Proportional Substitution Fails	113
10.2	The Nested Logit Model	113
10.3	A Labeled Variant of the Study	115
10.4	Simulating Nested Logit Data	116
10.4.1	Seeing the repaired substitution	117
10.5	The Nested Logit Log-Likelihood	118
10.6	Estimation	120
10.7	Validation Against <code>mlogit</code>	121
10.8	The GEV Family	122
10.9	Key Learnings	123
11	The Latent Class MNL Model	124
11.1	The Model	124

11.2	The Likelihood: Where the Panel Structure Bites	124
11.3	Simulating Latent Class Data	126
11.4	Route 1: Direct Maximization	127
11.5	The Terrain: Local Maxima and Label Switching	129
11.6	Route 2: The EM Algorithm	130
11.7	Choosing the Number of Classes	134
11.8	The Payoff: Posterior Classification	135
11.9	Key Learnings	136
12	Maximum Simulated Likelihood	137
12.1	A Likelihood With an Integral in It	137
12.2	The Simulated Probability	138
12.3	The Simulated Log-Likelihood Is Biased	138
12.4	Rule One: Fix the Draws	140
12.5	Better Draws: Halton Sequences at Work	143
12.6	How Many Draws? An Empirical Answer	145
12.7	The Estimator, Named and Situated	145
12.8	Key Learnings	146
13	The Mixed MNL Model by MSL	148
13.1	Random Coefficients, Two Ways to Read Them	148
13.2	Specification Choices	149
13.3	The Likelihood	150
13.4	Simulating the Study	150
13.5	The Simulated Likelihood, With Its Gradient	152
13.6	Estimation	154
13.7	Validation Against <code>mlogit</code>	156
13.8	The Simulation-Error Audit	157
13.9	Feeling the Strain	158
13.10	Key Learnings	159
14	The Limits of Likelihood	160
14.1	Attempt One: A Likelihood Per Person	160
14.2	Attempt Two: Condition on the Population	163
14.3	So What's Missing?	167
14.4	Key Learnings	168
III	The MNL Family, by Bayes	169
15	Bayesian Estimation of the MNL	170
15.1	The Bayesian Mindset	170
15.2	Priors	171

15.3	The Posterior Has No Closed Form Either	172
15.4	The Metropolis–Hastings Algorithm	172
15.5	M-H for the MNL	176
15.6	Reading a Chain: Diagnostics	177
15.7	Bayes Meets Maximum Likelihood	179
15.8	Key Learnings	181
16	Gibbs Sampling and Hierarchical Models	182
16.1	Sampling by Blocks	182
16.2	Conjugacy: The Conditional-Distribution Factory	184
16.3	The Hierarchical Normal Model	185
16.4	Data Augmentation: The Albert–Chib Sampler	188
16.5	Metropolis-within-Gibbs: The Missing Piece	190
16.6	Key Learnings	191
17	The Hierarchical Bayesian MNL	192
17.1	The Model	192
17.2	The Sampler	193
17.3	Coding It	194
17.4	Running It	196
17.5	The Payoff: Individual-Level Posteriors	198
17.6	HB and MSL on the Same Data	201
17.7	Validation Against <code>bayesm</code>	202
17.8	Practical Notes	203
17.9	Key Learnings	204
IV	Beyond Logit: The Multinomial Probit	205
18	The Multinomial Probit Model	206
18.1	Why Probit?	206
18.2	The Choice Probability Is an Integral	207
18.3	Identification: The Sternest Version Yet	207
18.4	Warming Up: Binary Probit by MLE	208
18.5	Simulating MNP Data	209
18.6	The Estimation Problem Ahead	212
18.7	Key Learnings	212
19	Estimating the MNP	214
19.1	Route 1: Simulated Maximum Likelihood via GHK	215
19.1.1	Why the obvious simulator fails	215
19.1.2	The GHK simulator	215
19.2	Route 2: Gibbs with Data Augmentation	218

19.3	The Two Routes, Head to Head	221
19.4	Validation Against <code>bayesm</code>	222
19.5	Key Learnings	224
20	The Hierarchical Bayesian MNP	225
20.1	The Model	225
20.2	The Sampler: No Metropolis Anywhere	226
20.3	Coding It	226
20.4	Running It	229
20.5	HB-MNL or HB-MNP?	230
20.6	Key Learnings	231
V	Using and Extending the Models	232
21	Post-Estimation: Inference, WTP, and Market Simulation	233
21.1	Inference on Functions of Parameters	234
21.2	Willingness-to-Pay, and Its Traps	235
21.3	Market Simulation	237
21.4	Elasticities and Substitution	239
21.5	Using the Individual-Level Estimates	240
21.6	Key Learnings	241
22	Extensions: Building Your Own Model	243
22.1	Extension 1: The No-Choice Option	243
22.2	Extension 2: A Budget Constraint	246
22.3	Extension 3: A Flexible Mixing Distribution	249
22.4	Where to Go From Here	252
22.5	Key Learnings	253
	References	254
	Appendices	257
A	Appendix	257
A.1	R Packages for Discrete Choice	257
A.2	Notation Glossary	258
A.3	Common Numerical Errors and Their Symptoms	259

Welcome

Warning

This is an early draft. The book is incomplete. Every aspect of it is subject to change. Many things might be incorrect. Are you sure you want to read this yet?

You don't understand it, until you have coded it.

It's a mantra that my mentors instilled in me in graduate school, and one that I propagate on my students today. I believe it in deeply. If you want to understand a statistical model, it is insufficient to interact with the model only through pen and paper. No matter how well you organize the subscripts or how masterfully you interweave elements of the Greek and Roman alphabets, you won't really "get it." To deeply understand a statistical model, you need to be able to simulate data from the model, and then take that simulated data to an estimation routine that enables you to recover the parameters of interest.

This is something you learn to do in graduate school. But, unless you have one of the most caring and pedagogical advisers, no one teaches it to you! Instead, it's a skill that students develop independently and inefficiently between classes and assignment due dates or, in my case, after I was knee-deep in my dissertation research. There is a tremendous imbalance between how immensely important this skill is and the time and instruction dedicated to it.¹ That imbalance is the motivation for this book.

I address this gap in the specific domain of Discrete Choice Modeling. This book stands on its own, but it did not develop in a vacuum: Kenneth Train's masterful text *Discrete Choice Methods with Simulation* (K. E. Train 2009), freely available online at <https://eml.berkeley.edu/books/choice2.html>, and the underappreciated tome *Applied Choice Analysis* by David Hensher, John Rose, and William Greene (Hensher, Rose, and Greene 2015) both shaped my thinking, and I cite them (and others) regularly for readers who want deeper coverage of the models' development, properties, and applications.

Train and I align in our thinking:

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for "buying nothing," making statements like "share who buy nothing at these prices" meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option's appeal as tasks teach them market prices, and model exactly that.

“...the true value of [] choice modeling is the ability to create tailor-made models. The computation and programming steps that are needed to implement a new model are usually not difficult. An important goal [] is to teach these skills as an integral part of the exposition of the models themselves.”²

However, what is “usually not difficult” for an experienced researcher can be immensely difficult for a typical graduate student. The aim of this book is to ease that difficulty. I demonstrate R code alongside the math. Every model in this book is developed from its mathematical statement all the way into working R code. No mysteries will remain. To borrow a quote from Quintilian, Cobbett, Cooke, and many others: my goal is to take you through the process of programming the simulation and estimation of common discrete choice models *not so that you can understand, but so that you cannot possibly misunderstand*.

How to read this book

This book tells one long story, and it is best read in order because the chapters build on one another — the code especially. In Part I, we assemble a foundation of computational tools: drawing from densities, simulating data from a model and recovering its parameters, organizing choice data, constructing likelihoods, and maximizing them numerically. In Part II, we put those tools to work on the multinomial logit (MNL) model and its likelihood-estimated extensions, climbing until we reach the limits of what maximum likelihood can deliver. Part III starts the climb again from the Bayesian side, building from the Metropolis-Hastings algorithm up to the hierarchical Bayesian MNL — the workhorse model of modern conjoint analysis and the centerpiece of this book. Part IV leaves the logit family for the multinomial probit, where both the classical and Bayesian toolkits get a capstone workout on a model with no closed-form choice probabilities. Part V turns to what comes after estimation — inference, willingness-to-pay, market simulation — and, finally, to extending these models into new, custom, tailored situations for your own research.

When I introduce a model, I provide enough context to motivate and specify it, but my emphasis remains squarely on estimation. When you want a more thorough treatment of a model’s properties, history, or econometric subtleties, follow the citations; K. E. Train (2009) and Hensher, Rose, and Greene (2015) in particular reward the effort.

In addition, I strongly encourage you write or copy the code as you work through this book. I would mandate this if I could, but I’m not sitting there next to you and so I must settle for simply sharing my encouragement. Write the code. Play with the code. Break the code. Make it your own. That behavior is where deep understanding comes from, not from highlighting the occasional sentence or equation you find important.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

Intended Audience

This book is intended for a narrow audience, predominantly current or former graduate students with an interest in discrete choice modelling who will find value from seeing and interacting with the programmatic implementation of the multinomial logit and its extended family of related models. In other words, someone who read K. E. Train (2009) and thought *how would I code that?* Or someone who has run `mlogit` in R or `nlogit` in Stata and wants to “see under the hood” of those estimation routines.

We will simulate data from the statistical models and then estimate the parameters of those models from the simulated data. Doing so will require us to generate random draws from distributions, approximate integrals with monte carlo simulation, maximize likelihood functions with optimization routines, and (when taking a Bayesian approach) explore posterior distributions with Markov Chain Monte Carlo methods.

We will do all of this in R, a freely available software environment for statistical computing and graphics. As a result, I assume you are reasonably familiar with R. If not, there is a tremendous set of free online R resources collected and organized at <https://www.bigbookofr.com/>. Popular books include *R for Data Science* properly cited as Wickham, Cetinka-Rundel, and Grolemund (2023) and *Advanced R* properly cited as Wickham (2019). If you are new to R or your familiarity with it has waned, you should consult these resources before proceeding here.

I also assume you have taken introductory statistics or econometrics courses, as the concepts and techniques taught there are foundational for understanding and estimating discrete choice models. In particular, you should have no uncertainty about the difference between a model, an estimator, and an estimate. To briefly review:

- a *model* is the set of mathematical assumptions about how data are generated,
- an *estimator* (or equivalently, the estimation routine) is an algorithm or function of the data, and
- an *estimate* is the result of applying the estimator to a particular dataset.

In my experience, students are often given a model and an estimator, after which much time in the classroom is spent deriving properties of the estimator for that particular model. Often a homework assignment follows which asks students to implement the estimator on a dataset to find an estimate. While this approach succeeds in developing the student’s skills and understanding of the particular estimator for the particular model, it does so with blinders on, obscuring consideration about the choice or specification of the model as well as the choice of estimator.

I’m sure you can recall your introduction to linear regression where you were handed a model with a linear conditional mean and a Normally distributed “error” and told that you would estimate the parameters of the model via least squares. Hopefully this left you wondering

about less-particular situations such as those with a non-continuous outcome variable, a non-Normally distributed error term, a non-linear conditional mean, correlation between the observed covariates and the error, a causal interpretation, or a different estimation approach.

In this book, we will focus on models with non-continuous outcome variables and we will use Maximum Likelihood and Bayesian methods to estimate the parameters of those models. If you would like to review key ideas from statistics and econometrics in preparation for tackling our discrete choice models with likelihood and Bayesian approaches, I highly recommend Abramovich and Ritov (2023, [link](#)) on the topic of mathematical statistics, Goldberger (1991, [link](#)) and Kennedy (2008, [link](#)) for econometrics, and McElreath (2018, [link](#)) and Gelman et al. (2013, [link](#)) for Bayesian statistics. Bruce Hansen’s recently released two-volume series Hansen (2022b, [link](#)) and Hansen (2022a, [link](#)) on probability, statistics, and econometrics are an excellent set of alternative resources.

Acknowledgments

I gratefully acknowledge the academic giants who pioneered and developed the field of Discrete Choice Analysis. The models offer flexible forms to study human behavior, the estimation methods are fascinating, their implementation usually proceeds from vexing to fun, and their application has been immensely useful in my consulting practice where I seek to help firms better understand consumers in order to improve consumer well-being and firm profitability.

I am also immensely grateful to the teams that work on the [R-Project](#), those at [Posit](#) who provide [RStudio](#), [Positron](#), and [Quarto](#), and the related communities of developers, academics, and R users. The free tools they provide are exceptional and I use them daily. The welcoming communities they have established provide the necessary support humans need, be it technical troubleshooting or emotional encouragement.

More personally, I would like to thank my academic mentors [Ella Honka](#), [Peter Rossi](#), and [Eric Bradlow](#), as well as the folks that have or continue to tolerate me through my consulting work, including [Prachi Bhalerao](#), [June Wu](#), [Chris Diener](#), [Keith Chrzan](#), and the hosts and attendees of [Sawtooth Software](#)’s annual Analytics and Insights Summit. I have learned so much from these researcher’s guidance and often their work.

Special thanks go to generous people who reviewed and assisted with drafts of this book, including [Darren Aeillo](#), [Kalyan Rallabandi](#), and [Geoff Zheng](#).

Unquestionably, my deepest thanks go to Alison, Zach, and Ben for their patience and support.

Colophon

An online version of this book is available at <https://dcme-r.danyavorsky.com>. The source of the book is available at <https://github.com/dyavorsky/dcme-r>. The book is authored using [Quarto](#), an open-source scientific and technical publishing system that makes it easy to create articles, presentations, websites, books, and other publications that combine text and executable code.

Part I

Foundations

1 Choice Models and Simulation

It is instructive to start with the first sentence of K. E. Train (2009) Section 1.3, “Discrete choice analysis consists of two interrelated tasks: **specification** of the behavioral model and **estimation** of the parameters of that model” (emphasis added). Notably, chapters 1–7 of K. E. Train (2009) only specify models; there is not even a hint of model estimation. We too begin with a focus on model specification.

Let me be clear: many students will see y and x later in this section and immediately think about “fitting” a model; that is, they assume they have data that they will put into an estimation routine to find estimates of the parameters of the model. We are not there yet! At this early stage in the book, we are only specifying models; that is, listing sets of assumptions about data generating processes and exploring the implications of those assumptions. There are no data yet. There will be parameters introduced in our choice of model specification, but we are not yet estimating those parameters. Have patience, we will eventually do these things.

1.1 A Framework for Choice

Train denotes the outcome in any given choice situation as y , determined by some observable factors collected in the vector $\mathbf{x}$ and some unobservable factors collected in the vector $\boldsymbol{\varepsilon}$. The factors ($\mathbf{x}$ and $\boldsymbol{\varepsilon}$) relate to the agent’s choice (y) through a function $y = h(\mathbf{x}, \boldsymbol{\varepsilon})$. We assume for the moment that we know $h(\cdot)$ and that $\mathbf{x}$ and $\boldsymbol{\varepsilon}$ are length-one vectors (i.e., scalars) denoted x and ε .

Since we do not observe ε , we can’t predict y exactly. Instead, we focus on the probability of y , that is:

$$\begin{aligned} p(y|x) &= \Pr(\varepsilon \text{ such that } h(x, \varepsilon) = y) \\ &= \Pr(I[h(x, \varepsilon) = y] = 1) \\ &= \int I[h(x, \varepsilon) = y] f(\varepsilon) d\varepsilon \end{aligned} \tag{1.1}$$

Read the last line slowly, because the rest of this book lives inside it. The indicator function $I[\cdot]$ equals 1 when the unobserved factors produce the outcome y and equals 0 otherwise. The integral averages that indicator over all the values the unobserved factors might take, weighting by their density $f(\varepsilon)$. In words: *the probability of an outcome is the share of the*

unobserved-factor distribution that produces that outcome. Every choice probability we compute in this book — logit, probit, nested, mixed, hierarchical — is an instance of Equation 1.1 with a particular choice of h and f .

For certain special choices of h and f , a closed-form expression¹ for the integral is available. But more generally, for almost any choice of h and f , we can approximate the integral through simulation. Train provides pseudo code on how to do so:

1. Repeat the following two steps many ($r = 1, \dots, R$) times:
 - Draw $\varepsilon^{(r)}$ from $f(\varepsilon)$.
 - Determine whether $h(x, \varepsilon^{(r)}) = y$. If so, set $I^{(r)} = 1$; else set $I^{(r)} = 0$.
2. Average the R values of I

This procedure is called *Monte Carlo integration*, and it works because of the Law of Large Numbers: the average of many independent realizations of a random quantity converges to its expectation, and Equation 1.1 is exactly an expectation — the expectation of the indicator $I[h(x, \varepsilon) = y]$ taken over ε . We will lean on this idea repeatedly: in Chapter 12 it will sit *inside* a likelihood function, and in Chapter 15 the same “average over draws” logic will summarize posterior distributions.

Next we look at two examples where we use this procedure to approximate the $p(y|x)$ integral. The first example I made up, and it is deliberately abstract — no economics, just mechanics. The second example is the binary logit model discussed by Train, which we will dress in the story that carries through the entire book: a shopper deciding whether to buy a laptop.

1.2 A Simple Example

Let’s first set up a toy example to demonstrate how simulation can approximate the $p(y|x)$ integral. Suppose $x = 0.5$ and ε is uniformly distributed between -1 and 1. Define $h(x, \varepsilon)$ to be:

$$h(x, \varepsilon) = \begin{cases} 0 & \text{if } x + \varepsilon < 0 \\ 1 & \text{if } x + \varepsilon \in [0, 1] \\ 2 & \text{if } x + \varepsilon > 1 \end{cases} \quad (1.2)$$

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

We'll focus on the outcome $y = 2$. You can probably intuit that the $p(y = 2|x) = 0.25$ since only one quarter of the time will ε be sufficiently positive to make $x + \varepsilon > 1$.² Nevertheless, let's approximate the integral representation of $p(y = 2|x)$ through simulation to ensure we understand the process.

To walk you through the code, we first set a seed so that the pseudo-random numbers generated by `runif()` can be replicated exactly each time the code is run (even on different computers). We then specify that we will use 1,000 draws in the simulation and we create a vector `I` to hold our results. The simulation occurs via a `for()` loop where each time through the loop we take a draw of ε , calculate $0.5 + \varepsilon$ and check whether that sum is greater than one. If so, then $h(x, \varepsilon) = 2$ matching the value of y for the choice probability we want to assess — i.e., $p(y = 2|x)$ — and thus we store a 1 in the r^{th} position of `I`; otherwise we store a 0. We then average the values in `I` to get our approximation of $p(y = 2|x)$.

```
set.seed(1234)

R <- 1000
I <- vector(length=R)

for(r in 1:R) {
  eps <- runif(1, min=-1, max=1)
  h <- 0.5 + eps
  I[r] <- as.integer(h > 1)
}

mean(I)
```

```
[1] 0.258
```

The simulated value 0.258 approximates the exact value 0.25 and can be made closer by increasing the number of draws used in the simulation.

Notice that we took draws of ε to empirically approximate the integral of $p(y|x)$, but we should not think of these draws as “data” in the sense of a dataset. They are ancillary numbers generated from the distribution of ε that we use to approximate the integral.

R users will recognize that we can shorten the code by taking advantage of R's vectorized functions and its conversion of boolean values to 0/1 when used in mathematical operations. Here is a shorter implementation of the simulation; whether it's “better” code is a matter of preference.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```
set.seed(1234)
R <- 1000
mean( runif(R, min=-1, max=1) + 0.5 > 1 )
```

```
[1] 0.258
```

That's it. If you can generate pseudo-random draws from the density f and you know h , approximating a choice probability by simulation might only require a handful of lines of code.

How good is the approximation? Since each $I^{(r)}$ is a Bernoulli random variable with success probability $p = 0.25$, the simulation average has standard error $\sqrt{p(1-p)/R}$, which for $R = 1,000$ is about 0.014. Quadrupling the number of draws halves the standard error — the familiar (and, when you are impatient, infuriating) square-root rate of Monte Carlo methods. We will return to the accuracy of simulated probabilities with much more care in Chapter 12, where it directly affects the quality of our parameter estimates.

1.3 A Binary Logit Example

The toy example had no behavioral content: h was an arbitrary rule. Discrete choice models earn the name *behavioral* by deriving h from a model of decision-making. The standard construction — which we will use, in increasingly elaborate forms, for the rest of the book — is the *random utility model*: the decision-maker assigns a utility to each available action and picks the action with the highest utility.

Here is the story we will grow throughout this book. A shopper is considering a laptop. The laptop has observable attributes — its price, its memory, its screen size, its brand — and the shopper has some overall assessment of how much they'd like to own it, which also depends on many things we will never observe: their mood, their current machine's death rattle, a looming deadline. We collect the observable attributes in a vector $\mathbf{x}$ and bundle everything unobservable into a scalar ε .

The “binary” part of the model's name refers to the aspect of the model whereby the decision maker does one of two things; they either take an action ($y = 1$, buy the laptop) or not ($y = 0$, walk away). To tie this model into a framework of behavior, we start with a utility function U for the action:

$$U(\mathbf{x}, \boldsymbol{\beta}, \varepsilon) = \mathbf{x}'\boldsymbol{\beta} + \varepsilon \tag{1.3}$$

where $\mathbf{x}$ is a vector of observable explanatory variables, $\boldsymbol{\beta}$ is a vector of parameters that through the functional form $\mathbf{x}'\boldsymbol{\beta}$ effectively serve as weights on the attributes, and ε is a scalar collecting

the value of everything relevant to the shopper but unobserved by the researcher. The utility of *not* buying is normalized to zero — a harmless-seeming convention that we will scrutinize carefully in Chapter 7 (identification) and again in Chapter 22 (the outside option).

The piece the researcher can compute, $\mathbf{x}'\beta$, is called the *representative utility* and gets its own letter, V ; the model says total utility is representative utility plus noise, $U = V + \varepsilon$. The split matters: V is where the parameters live, and ε is what the integral in Equation 1.1 averages over.

Notice that h has grown an argument: once we specify utility, the parameters β enter the behavioral rule, so we write $h(\mathbf{x}, \beta, \varepsilon)$ and condition the choice probability on β as well — $p(y|\mathbf{x}, \beta)$. The framework of Equation 1.1 is unchanged; β simply rides along with $\mathbf{x}$. The shopper buys when buying is better than not buying, so the behavioral rule h is:

$$h(\mathbf{x}, \beta, \varepsilon) = \begin{cases} 0 & \text{if } U(\mathbf{x}, \beta, \varepsilon) \leq 0 \\ 1 & \text{if } U(\mathbf{x}, \beta, \varepsilon) > 0 \end{cases} \quad (1.4)$$

The “logit” part of the model’s name refers to the choice of f . The binary logit model assumes ε follows the standard logistic distribution:

$$f(\varepsilon) = \frac{e^{-\varepsilon}}{(1 + e^{-\varepsilon})^2} \quad (1.5)$$

This particular pairing of h and f is one of the special cases where the integral in Equation 1.1 has a closed form. Let me show you why. Integrating Equation 1.5 gives the logistic CDF, $F(\varepsilon) = 1/(1 + e^{-\varepsilon})$ (differentiate it to confirm you get f back). The shopper buys when $\mathbf{x}'\beta + \varepsilon > 0$, i.e., when $\varepsilon > -\mathbf{x}'\beta$, so

$$p(y = 1|\mathbf{x}, \beta) = \Pr(\varepsilon > -\mathbf{x}'\beta) = 1 - F(-\mathbf{x}'\beta) = 1 - \frac{1}{1 + e^{\mathbf{x}'\beta}} = \frac{e^{\mathbf{x}'\beta}}{1 + e^{\mathbf{x}'\beta}}$$

No indicator, no integral sign left standing — that is what “closed form” buys us:

$$p(y = 1|\mathbf{x}, \beta) = \frac{e^{\mathbf{x}'\beta}}{1 + e^{\mathbf{x}'\beta}} \quad (1.6)$$

Let’s make it concrete. Suppose the laptop on offer costs \$1,500 and has 16GB of RAM (an upgrade over a base 8GB configuration). We describe it with the vector $\mathbf{x} = (1, 1.5, 1)$: a constant, the price in thousands of dollars, and an indicator for the 16GB memory upgrade. And suppose the weights are $\beta^* = (1.0, -1.2, 0.6)$ — the asterisk is our standing notation for *true* parameter values, the ones the data generating process actually uses, as distinct from

candidate values β we will later evaluate and estimates $\hat{\beta}$ we will later compute. The three values: a baseline pull toward buying, a distaste for price, and a taste for memory.³ Then

$$\mathbf{x}'\beta = (1)(1.0) + (1.5)(-1.2) + (1)(0.6) = -0.2$$

and the closed-form probability of purchase is

$$p(y = 1|\mathbf{x}, \beta) = \frac{e^{-0.2}}{1 + e^{-0.2}} = 0.45 \quad (1.7)$$

Even though we have the closed form, let's approximate the integral by simulation exactly as before, both to practice the procedure and to verify the closed form is what it claims to be. We use `rlogis()` to take $R = 1,000$ draws from the standard logistic distribution, and we approximate the integral with the proportion of times $\mathbf{x}'\beta + \varepsilon$ exceeds the threshold for action (0):

```
set.seed(2345)
R <- 1000

x <- c(1, 1.5, 1)
beta <- c(1.0, -1.2, 0.6)

U <- as.vector(x %*% beta) + rlogis(R)
mean(U > 0)
```

```
[1] 0.434
```

Our simulated value 0.434 approximates the exact value 0.45. The two routes — analytic formula and simulation — agree, as they must. Hold onto this dual view. When a closed form exists we will use it (it is faster and exact), but the simulation route never stops being available, and for most of the models in the second half of this book it is the *only* route.

1.3.1 A model has implications

A specified model is a machine for answering “what if” questions, even before any data exist. What does our model say happens to the purchase probability as price varies? We can trace the whole curve by evaluating Equation 1.6 over a grid of prices, for both memory configurations:

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

```

price <- seq(from = 0.8, to = 2.4, by = 0.01)

p_buy <- function(price, ram16) {
  v <- 1.0 - 1.2*price + 0.6*ram16
  exp(v) / (1 + exp(v))
}

curves <- data.frame(
  price = rep(price, times = 2),
  ram   = rep(c("8GB", "16GB"), each = length(price)),
  prob  = c(p_buy(price, ram16 = 0), p_buy(price, ram16 = 1))
)

ggplot(curves, aes(x = price, y = prob, linetype = ram)) +
  geom_line() +
  annotate("point", x = 1.5, y = p_buy(1.5, 1), size = 2) +
  labs(x = "Price ($1,000s)", y = "P(buy)", linetype = "Memory")

```

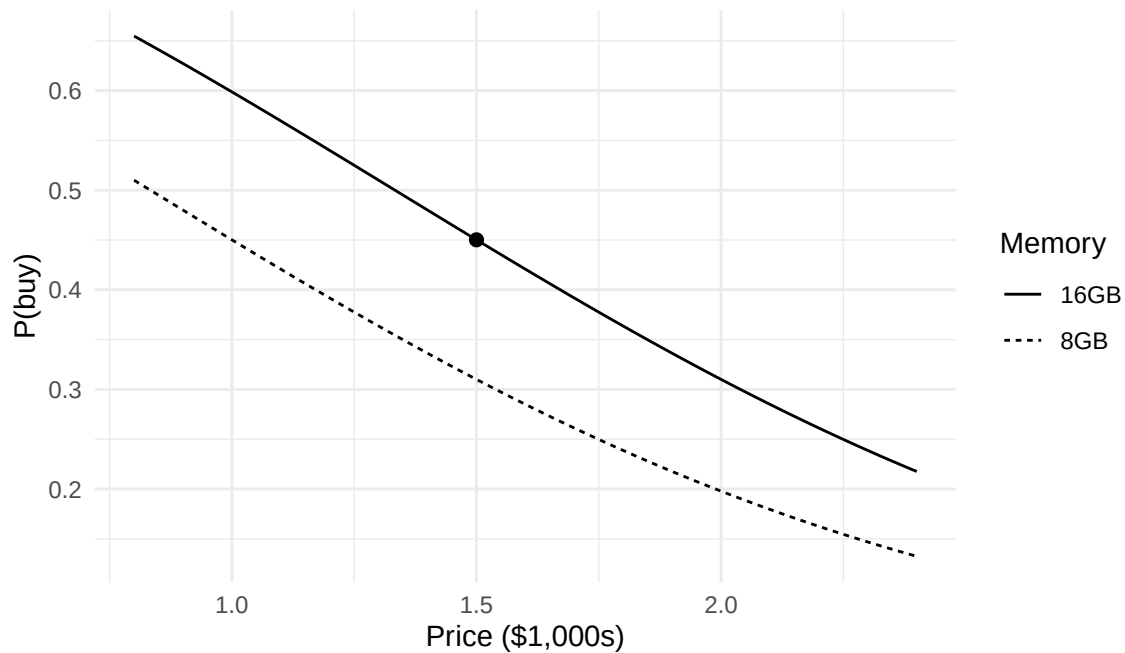


Figure 1.1: Purchase probability implied by the binary logit model as price varies, for the base (8GB) and upgraded (16GB) memory configurations. The point marks the worked example: a \$1,500 laptop with 16GB of RAM.

Two things are worth noticing. First, the curves are S-shaped, not straight lines: the model builds in diminishing sensitivity at the extremes, where the shopper is nearly certain to buy or nearly certain to walk away. Second, the vertical gap between the curves is the effect of the memory upgrade, and it is *not constant* — it is largest where the purchase decision hangs in the balance. Both properties came free with the logit specification. Whether they are *right* is an empirical question we are not yet equipped to ask; the point for now is that writing down h and f commits you to implications like these, and you should know what you are committing to.

1.4 The Running Example: A Laptop Choice Study

The laptop shopper of the previous section is the seed of the example that grows throughout this book. Since the later chapters build on it, let me lay out the full setting now.

Imagine we field a *choice-based conjoint* study: we recruit a sample of prospective laptop buyers, and we show each respondent a sequence of choice tasks. In each task, the respondent sees a small set of hypothetical laptops, described by their attributes, and indicates which one they would buy. The attributes and their levels:

Table 1.1: Attributes of the laptop choice study.

Attribute	Levels	Coding
Brand	Acer, Dell, Apple	dummies <code>dell</code> , <code>apple</code> (Acer is the reference)
Memory	8, 16, 32 GB	dummies <code>ram16</code> , <code>ram32</code> (8GB is the reference)
Screen	13", 15"	dummy <code>screen15</code> (13" is the reference)
Price	\$800 – \$2,400	continuous, in \$1,000s

This design is modest but honest: it mirrors the structure of real conjoint studies used throughout marketing practice Rao (2014). And it contains, in miniature, everything the models of this book need to exercise: a price for willingness-to-pay questions, brands for intercept-like effects (and, since Apple implies macOS while Dell and Acer imply Windows, a natural *nesting* of alternatives for Chapter 10), and attributes over which different respondents plausibly hold different tastes (the heterogeneity that drives everything from Chapter 11 onward).

Because we will *simulate* this study rather than field it, we will always know the true preference parameters, and every estimation method we build will be judged by one standard: does it recover them? The binary examples of the next five chapters strip the setting down to a single laptop offer — buy or don’t — carrying the true parameters $\beta^* = (1.0, -1.2, 0.6)$ from

Section 1.3. In Chapter 7 the full study arrives: $N = 500$ respondents, $T = 8$ tasks each, $J = 3$ laptops per task, and the complete attribute list of Table 1.1.

One warning as the example grows: it is *simulated*. Simulated data are the right tool for learning estimation — the truth is known, so mistakes are visible — but they are silent about the field problems of real conjoint studies (respondent attention, attribute framing, incentive alignment). When you graduate to real data, pair what you learn here with a treatment of those issues (e.g., Ben-Akiva, McFadden, and Train 2019; Eggers et al. 2022).

1.5 The Road Ahead

You now have the two ideas that generate this entire book:

1. **A discrete choice model is a pair** (h, f) — a behavioral rule and a distribution for what we don’t observe — and the choice probability is the integral in Equation 1.1.
2. **Integrals can be simulated.** Draws from f plus the rule h approximate any choice probability, whether or not a closed form exists.

What we cannot yet do is the *reverse* inference. In this chapter we knew β and computed choice probabilities; research runs the other way, from observed choices back to unknown parameters. Getting there requires machinery: drawing from all the densities we’ll need (Chapter 2), simulating entire datasets rather than single probabilities (Chapter 3), organizing choice data so a computer can use them (Chapter 4), turning data plus model into a likelihood (Chapter 5), and maximizing that likelihood numerically (Chapter 6). That is Part I, and it is deliberately front-loaded: every hour spent there pays compound interest across Parts II through V.

1.6 Key Learnings

- A discrete choice model is specified by a behavioral function $y = h(\mathbf{x}, \varepsilon)$ and a density $f(\varepsilon)$ for the unobserved factors. Specification and estimation are distinct tasks, and everything in this chapter was specification.
- Our focus is on the *probability* of a choice outcome, $p(y|\mathbf{x})$, which is an integral: the share of the unobserved-factor distribution that produces the outcome (Equation 1.1).
- That integral has a closed form only for special (h, f) pairs — the binary logit is one — but it can be approximated by simulation for almost *any* pair: draw ε from f , apply h , average the indicator. Monte Carlo error shrinks at the $\sqrt{R}$ rate.
- A fully specified model has testable implications (S-curves, interaction-like gaps) before any data exist. Know what your specification commits you to.

- The running example is set: a laptop choice study with known true parameters, $\beta^* = (1.0, -1.2, 0.6)$ in its binary form. The rest of the book is about recovering numbers like these from data.

2 Drawing from Densities

In Chapter 1 we approximated choice probabilities by drawing values of ε from a density f and averaging an indicator function over the draws. We got away with it because R happened to supply the draws we needed: `runif()` for the uniform, `rlogis()` for the logistic. This chapter looks under the hood. Where do random draws come from? How do we produce them for densities R does not supply? And how do we produce *correlated* draws, *truncated* draws, and draws that cover a space more evenly than randomness allows?

I want to persuade you that this material is not a detour. Draws are the raw material of every computational method in this book, and they appear in three distinct roles:

1. **Simulating data from a model** (Chapter 3 and every chapter after): each simulated decision-maker needs a realization of the unobservables.
2. **Approximating integrals inside a likelihood** (Chapter 12, Chapter 13, Chapter 19): the mixed logit and probit probabilities are integrals with no closed form, evaluated by averaging over draws.
3. **Exploring posterior distributions** (Chapter 15 onward): the entire output of a Bayesian estimation is a collection of draws, and the samplers that produce them are built from the primitives in this chapter — in particular, you cannot code the samplers of Chapter 16 and Chapter 19 without multivariate normal and truncated normal draws.

By the end of the chapter you will be able to generate draws from essentially any distribution this book requires, starting from nothing but uniform random numbers.

2.1 Pseudo-Random Numbers from the Uniform Distribution

Computers are deterministic machines, so they cannot produce truly random numbers.¹ What they produce instead are *pseudo-random* numbers: long sequences generated by a deterministic recurrence, initialized by a *seed*, that pass every practical statistical test of randomness. R’s default generator (the Mersenne Twister) produces values that behave, for our purposes, exactly like independent draws from the Uniform(0, 1) distribution.

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

Determinism is a gift for scientific work. Because the sequence depends only on the seed, setting the seed makes an entire simulation exactly reproducible — on your machine, on my machine, today, and in five years. That is why every stochastic code chunk in this book begins with `set.seed()`. When you see

```
set.seed(42)
runif(3)
```

```
[1] 0.9148060 0.9370754 0.2861395
```

you will get those same three numbers on your computer. Re-running the chunk *without* resetting the seed continues the sequence and gives fresh values; resetting the seed replays it. Beyond reproducibility, seeds are a debugging instrument: when a simulation misbehaves, freezing the randomness lets you isolate what changed. And in Chapter 12 the ability to *reuse the same draws* across likelihood evaluations turns out to be essential for the estimator itself, not merely convenient.

Everything else in this chapter is a transformation of uniform draws. That is worth internalizing: `rnorm()`, `rlogis()`, `rexp()` and their relatives are not separate sources of randomness — they are deterministic functions applied to the same uniform stream. Our first and most important transformation makes the principle explicit.

2.2 The Inverse Transform Method

Suppose we want draws of a random variable X with cumulative distribution function (CDF) F_X , and suppose we can compute the inverse CDF F_X^{-1} (also called the quantile function). Take $U \sim \text{Uniform}(0, 1)$ and define the transformation $T(U) = F_X^{-1}(U)$. I claim $X = T(U)$ has exactly the distribution we want. The proof is three lines: for any x ,

$$\begin{aligned}\Pr(T(U) \leq x) &= \Pr(F_X^{-1}(U) \leq x) \\ &= \Pr(U \leq F_X(x)) \\ &= F_X(x)\end{aligned}\tag{2.1}$$

where the second equality applies the (increasing) function F_X to both sides of the inequality, and the third uses the defining property of the uniform: $\Pr(U \leq u) = u$ for $u \in [0, 1]$. The transformed draw has CDF F_X , which is what it means to have the distribution of X .

The geometry is worth seeing once. The CDF maps values of X (horizontal axis) to probabilities (vertical axis); the inverse transform runs the map backwards. A uniform draw picks a random height on the vertical axis, and we read across to the curve and down to the horizontal axis:

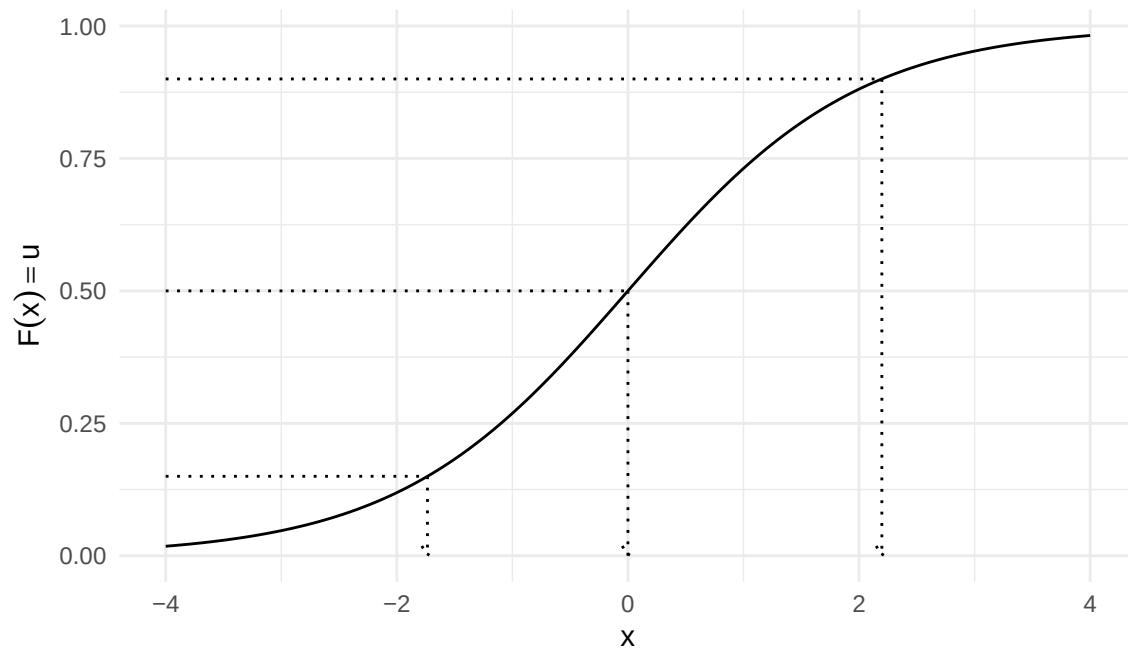


Figure 2.1: The inverse transform method: uniform draws on the vertical axis map through the inverse CDF to draws on the horizontal axis. Flat regions of the density correspond to steep regions of the CDF, which catch more of the uniform draws.

Notice how the method automatically concentrates draws where the density is high: where f is large, F is steep, so a wide band of uniform heights maps into a narrow band of x values.

Let's implement the method for four distributions. Each follows the same two-step pattern — *invert the CDF algebraically, then apply it to uniform draws* — and each earns its place in this book.

2.2.1 Example: Exponential

The exponential distribution has CDF

$$F(y) = 1 - \exp(-\lambda y) \quad \text{for } y \geq 0$$

Setting $F(y) = u$ and solving for y : $u = 1 - e^{-\lambda y}$ implies $e^{-\lambda y} = 1 - u$, so

$$F^{-1}(u) = -\log(1 - u)/\lambda$$

Two lines of algebra, two lines of code:

```
set.seed(2020)
R <- 1e4
lambda <- 0.3
u <- runif(R)
y <- -log(1-u)/lambda
```

As in Chapter 1, R counts simulation draws — deliberately not N , which we reserve for counting decision-makers from Chapter 3 onward. (And a naming note: this chapter's code uses y as the generic name for a draw; it is not yet the choice outcome y of the model chapters.)

Did it work? The standard check — one we will repeat so often that we write it as a small helper function — is to plot a histogram of the draws against the true density. If the transformation is correct, the histogram should trace the density curve, with the agreement improving as R grows.

```
plot_draws <- function(y, dens_fun) {
  ggplot(data.frame(y = y), aes(x = y)) +
    geom_histogram(aes(y = after_stat(density)),
                   bins = 100, fill = "grey70") +
    geom_function(fun = dens_fun, linewidth = 0.8) +
    labs(x = "draw value", y = "density")
}
```

```
plot_draws(y, function(x) dexp(x, rate = lambda))
```

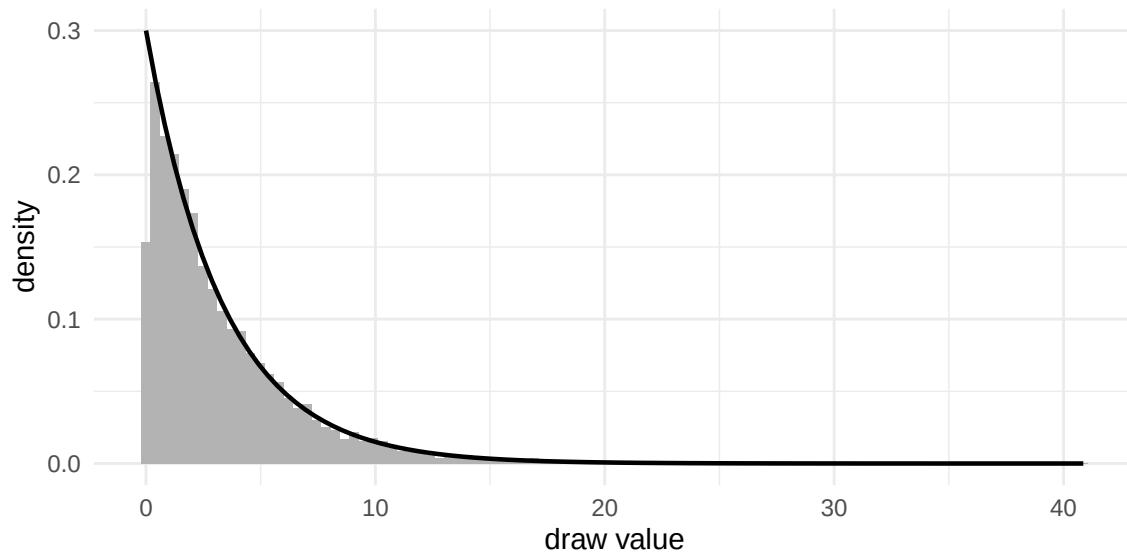


Figure 2.2: Histogram of 10,000 inverse-transform draws against the true Exponential(0.3) density.

The helper `plot_draws()` takes the vector of draws and the true density function; we will reuse it throughout the chapter. This histogram-against-density picture is the simplest instance of a discipline that recurs all book long: *whenever you generate anything by simulation, check it against something you know.*

2.2.2 Example: Gumbel

The Gumbel distribution — also called the Type I Extreme Value distribution, or EV1 — has CDF

$$F(y) = \exp(-\exp(-y))$$

You met its close relative in Chapter 1, and you will not escape it: the Gumbel is *the* error distribution of logit models, chosen because the difference of two independent Gumbels is logistic and the maximum of several Gumbels is again Gumbel — the properties that make the MNL’s closed form possible (Chapter 7). Inverting: $u = e^{-e^{-y}}$ gives $\log u = -e^{-y}$, so

$$F^{-1}(u) = -\log(-\log(u))$$

```
set.seed(2021)
R <- 1e4
u <- runif(R)
y <- -log(-log(u))
```

Base R has no `dgumbel()`, but the density is simple enough to write ourselves — differentiate the CDF to get $f(y) = \exp(-(y + e^{-y}))$:

```
dgumbel <- function(x) exp(-(x + exp(-x)))
plot_draws(y, dgumbel)
```

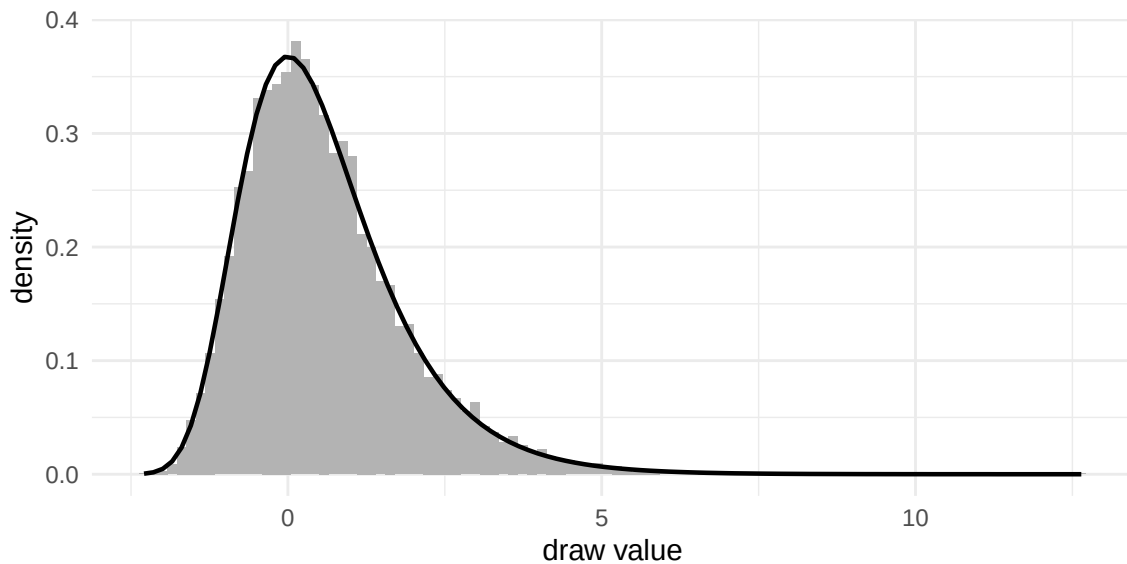


Figure 2.3: Histogram of 10,000 inverse-transform draws against the Gumbel (Type I Extreme Value) density. Note the right skew: extreme value distributions are asymmetric.

Keep `y <- -log(-log(runif(R)))` in your fingers. When we simulate MNL choice data from Chapter 7 onward, this one-liner generates the unobserved utility for every alternative in every choice task.

2.2.3 Example: Logistic

The logistic distribution — the f of the binary logit model in Section 1.3 — has CDF

$$F(y) = \frac{1}{1 + \exp(-y)}$$

Inverting: $u(1 + e^{-y}) = 1$ gives $e^{-y} = (1 - u)/u$, so

$$F^{-1}(u) = \log(u) - \log(1 - u)$$

```
set.seed(2022)
R <- 1e4
u <- runif(R)
y <- log(u) - log(1-u)
```

```
plot_draws(y, dlogis)
```

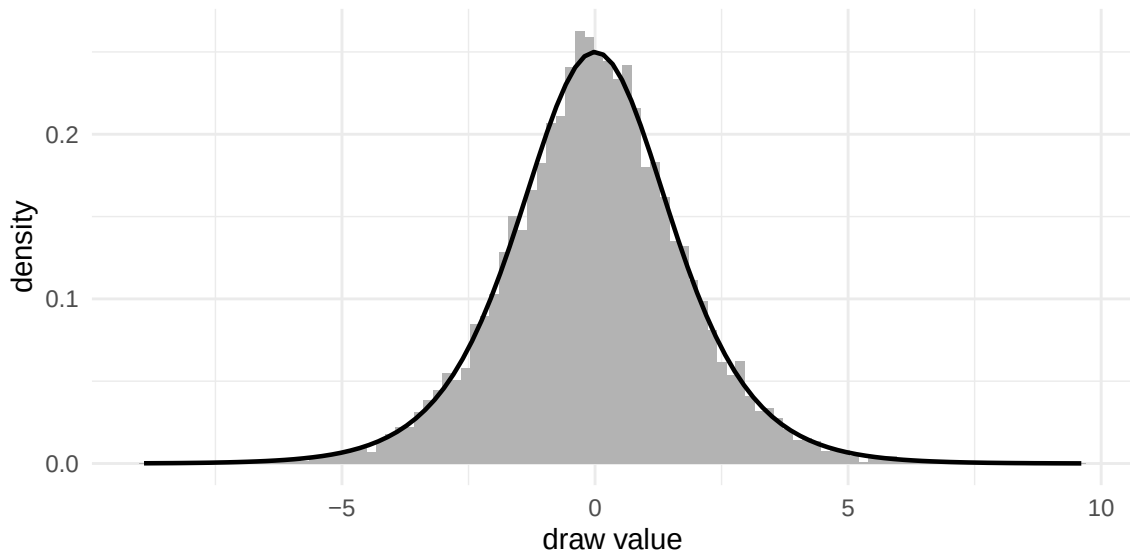


Figure 2.4: Histogram of 10,000 inverse-transform draws against the standard logistic density.

So `rlogis(R)` — which we used in Section 1.3 without comment — is nothing more mysterious than $\log(u) - \log(1-u)$ applied to uniform draws.

While we are here, let's verify a claim made two pages ago, because it is the algebraic heart of the logit family: *the difference of two independent Gumbel draws is logistic*. Rather than prove

it (see K. E. Train (2009), Section 3.1), we can check it by simulation — our tools are already good enough to interrogate distribution theory:

```
set.seed(2023)
R <- 1e4
eps1 <- -log(-log(runif(R)))
eps2 <- -log(-log(runif(R)))
plot_draws(eps1 - eps2, dlogis)
```

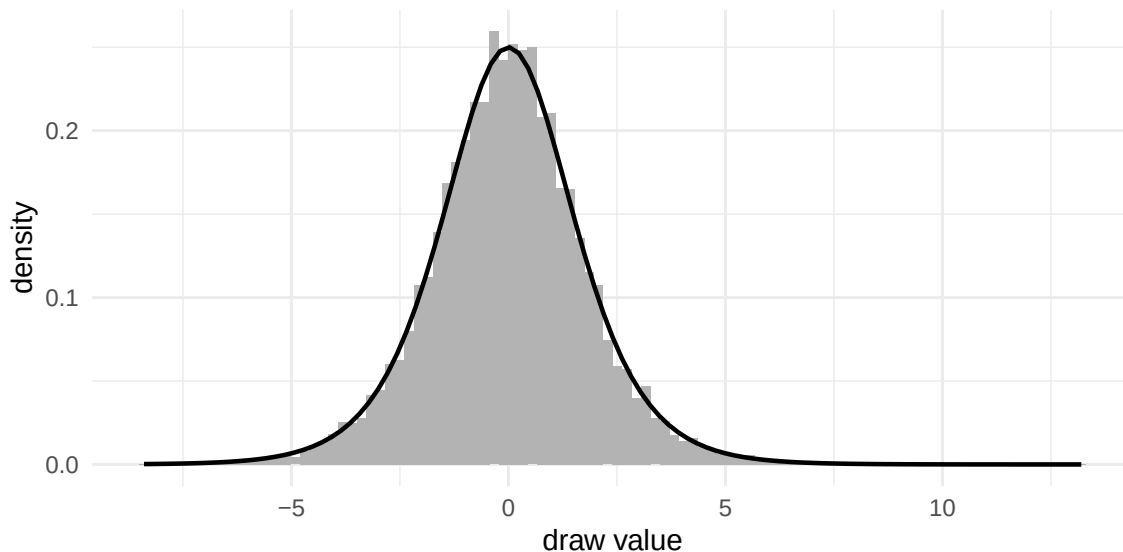


Figure 2.5: The difference of two independent Gumbel draws, compared against the standard logistic density. This equivalence is why binary choice between two Gumbel-error alternatives yields the binary logit model.

The histogram of $\varepsilon_1 - \varepsilon_2$ sits exactly on the logistic density. This is why Section 1.3 could describe one utility with a single logistic error. Give each action its own utility — $U_1 = V_1 + \varepsilon_1$ for buying and $U_0 = V_0 + \varepsilon_0$ for walking away, where V denotes the *representative* (observable) part of utility and ε the unobserved part. Buying beats not buying when $U_1 > U_0$, i.e., when $\varepsilon_0 - \varepsilon_1 < V_1 - V_0$, and if both ε 's are Gumbel then that difference is logistic. Two-error and one-error tellings of the binary logit are the same model.

2.2.4 Example: Normal

The normal CDF has no closed-form expression, so its inverse has no tidy algebraic formula like the previous three. But numerical routines for Φ^{-1} are excellent and built into R as `qnorm()`, so the inverse transform method still applies:

```
set.seed(2024)
R <- 1e4
u <- runif(R)
y <- qnorm(u)
```

Of course, R also provides `rnorm()` directly, and in practice we use it. The point of this example is conceptual: `qnorm(runif(R))` and `rnorm(R)` are the same idea,² and *any* distribution with a computable quantile function is one line away. File this away — in Chapter 12 we will deliberately generate normal draws as `qnorm()` applied to something *other* than uniform draws, and the fact that “draws = quantile function applied to points in (0, 1)” is what makes that substitution legitimate.

2.3 Transforming Draws

The inverse transform gets us *standard* distributions. Applied work almost always needs distributions with particular locations, scales, and shapes, and these come from further transformations of standard draws. Three matter enormously in this book.

Location–scale. If Z is standard normal, then $\mu + \sigma Z \sim N(\mu, \sigma^2)$. Trivial, but say it once precisely, because its multivariate version (next section) is the backbone of every heterogeneity model we estimate:

```
set.seed(2025)
z <- rnorm(1e4)
y <- 2 + 0.5*z           # N(2, 0.25)
c(mean(y), sd(y))
```

```
[1] 2.0026487 0.5009752
```

Lognormal. If $Z \sim N(\mu, \sigma^2)$ then e^Z is lognormal — always positive, right-skewed. Why care? Jump ahead to Chapter 13 and Chapter 17: when we let the *price coefficient* vary across people, a normal distribution awkwardly implies some people *like* higher prices. The standard fix is to model the coefficient as $-e^Z$, which is always negative. That fix is this transformation:

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```
set.seed(2026)
beta_price <- -exp(rnorm(1e4, mean = 0.1, sd = 0.4))
ggplot(data.frame(b = beta_price), aes(b)) +
  geom_histogram(bins = 100, fill = "grey70") +
  labs(x = "price coefficient", y = "count")
```

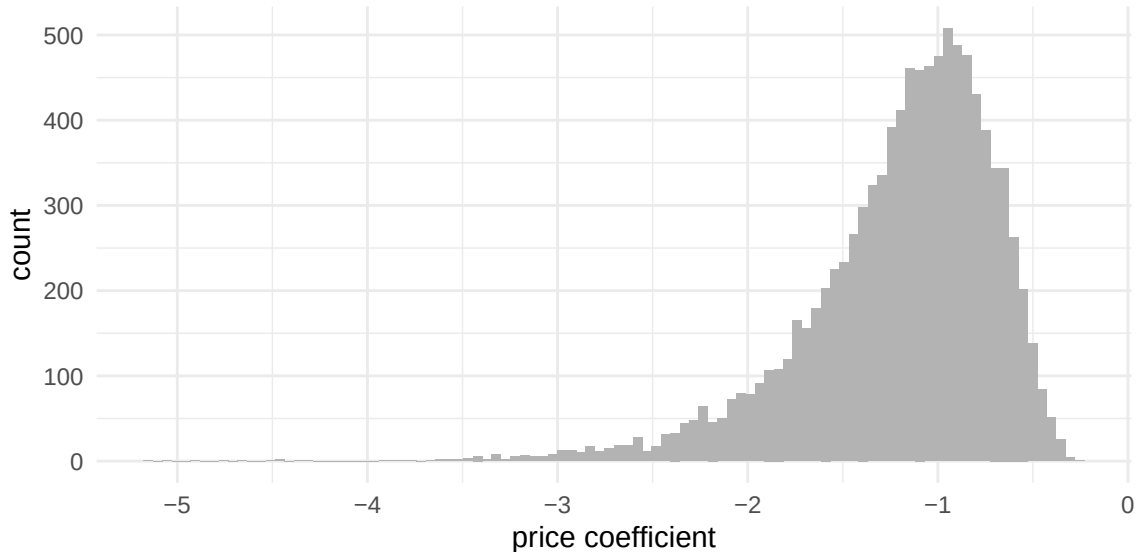


Figure 2.6: A lognormal price-coefficient distribution: minus the exponential of $N(0.1, 0.4^2)$ draws. Every simulated decision-maker dislikes price, but some dislike it far more than others. We will use exactly this device in the mixed logit and HB chapters.

Truncation. Sometimes we need draws from a distribution *restricted to an interval* — a normal confined to the positive half-line, say. The elegant route is a small extension of the inverse transform. If X has CDF F and we want X conditioned on $a < X \leq b$, note that the conditional CDF is $(F(x) - F(a)) / (F(b) - F(a))$: it runs from 0 at a to 1 at b . Inverting it says: *map a uniform draw into the $(F(a), F(b))$ sub-interval, then apply the ordinary quantile function*:

$$X = F^{-1}(F(a) + u \cdot (F(b) - F(a))) \quad (2.2)$$

```
rtnorm <- function(n, mean = 0, sd = 1, a = -Inf, b = Inf) {
  Fa <- pnorm(a, mean, sd)
  Fb <- pnorm(b, mean, sd)
  u <- runif(n)
```

```

    qnorm(Fa + u*(Fb - Fa), mean, sd)
  }

set.seed(2027)
y <- rtnorm(1e4, a = 0.5, b = 2)
plot_draws(y, function(x)
  dnorm(x) / (pnorm(2) - pnorm(0.5)) * (x > 0.5 & x <= 2))

```

! [10,000 draws from a standard normal truncated to (0.5, 2], against the renormalized true density on that interval.] (02_draws_files/figure-pdf/fig-truncnorm-1.pdf) {#fig-truncnorm fig-pos='H'}

Do not skim past `rtnorm()`; it is the quiet workhorse of Part IV. The Bayesian estimation of the probit model (Chapter 19) proceeds by drawing each decision-maker’s latent utility from a normal distribution *truncated to the region consistent with their observed choice*, thousands of times, inside a Gibbs sampler. The Albert–Chib sampler of Chapter 16 does the same for binary data. When we get there, the samplers will be new but this function will not be.

2.4 The Multivariate Normal

Now for the transformation that carries the most weight in this book. We often need draws of a *vector* $\mathbf{w} \sim N(\boldsymbol{\mu}, \Sigma)$: jointly normal components with means $\boldsymbol{\mu}$ and covariance matrix Σ . Starting in Chapter 13, every simulated “population of decision-makers” is a cloud of multivariate normal draws — one coefficient vector $\boldsymbol{\beta}_n$ per person — and the samplers of Chapter 16, Chapter 17, and Chapter 19 draw multivariate normals at every iteration.

The recipe generalizes location–scale. In one dimension we scaled by σ , the square root of the variance. In K dimensions we need a “square root” of the covariance *matrix*: a matrix L such that

$$LL' = \Sigma \tag{2.3}$$

One such matrix always exists for a valid (positive definite) covariance matrix: the **Cholesky root**, a lower-triangular L computable by `chol()` in R.³ Given L , take a vector $\mathbf{z}$ of K *independent* standard normal draws and set

$$\mathbf{w} = \boldsymbol{\mu} + L\mathbf{z} \tag{2.4}$$

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

Then $\mathbf{w}$ is exactly $N(\boldsymbol{\mu}, \Sigma)$. The mean is immediate. For the covariance, recall that $\text{Var}(A\mathbf{z}) = A\text{Var}(\mathbf{z})A'$ for fixed A ; with $\text{Var}(\mathbf{z}) = I$ this gives $\text{Var}(L\mathbf{z}) = LIL' = \Sigma$. Normality is preserved because linear combinations of normals are normal. The triangular structure of L makes the mechanics transparent: the first component of $\mathbf{w}$ uses only z_1 ; the second mixes z_1 and z_2 — and it is precisely this reuse of z_1 that manufactures the correlation.

Here is the recipe on a concrete case — and to make it self-documenting, we verify that the sample moments of the draws match what we asked for:

```
set.seed(2028)
R <- 1e4
mu <- c(1, -1)
Sigma <- matrix(c(1.0, 0.6,
                  0.6, 0.5), nrow = 2)

L <- t(chol(Sigma))          # lower-triangular: L %*% t(L) == Sigma
z <- matrix(rnorm(2*R), nrow = 2) # 2 x R standard normals
w <- mu + L %*% z             # 2 x R correlated draws

rowMeans(w)                 # should be near (1, -1)
```

```
[1] 0.9996430 -0.9978268
```

```
cov(t(w))                  # should be near Sigma
```

```
      [,1]      [,2]
[1,] 1.0059411 0.6044097
[2,] 0.6044097 0.5065971
```

```
ggplot(data.frame(w1 = w[1,], w2 = w[2,]), aes(w1, w2)) +
  geom_point(alpha = 0.1, size = 0.5) +
  labs(x = expression(w[1]), y = expression(w[2]))
```

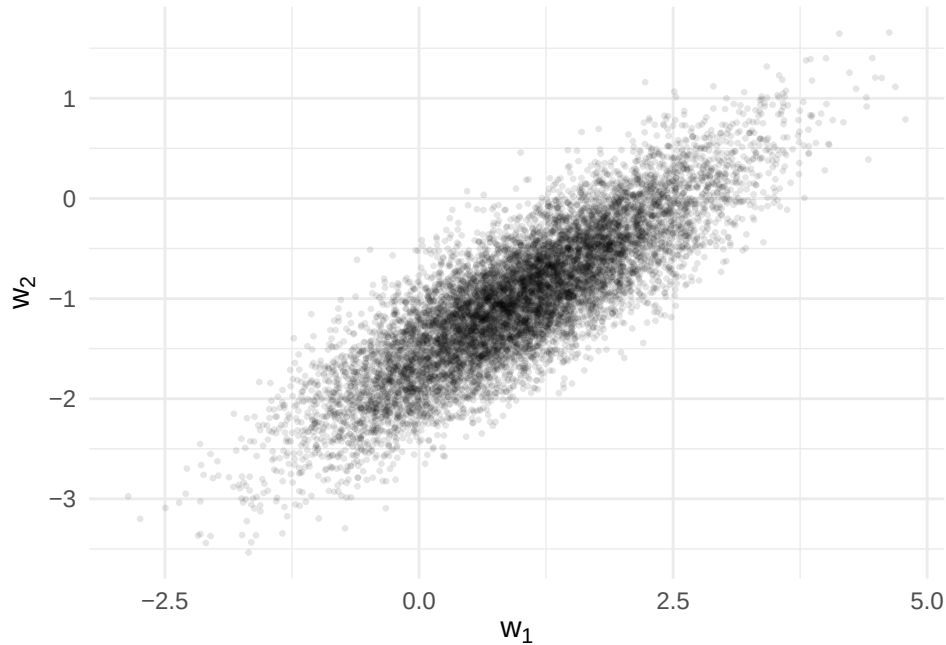


Figure 2.7: 10,000 bivariate normal draws produced by the Cholesky transformation, showing the requested means and positive correlation.

Three lines — `t(chol(Sigma))`, `rnorm`, and a matrix multiply — and we can populate a world with correlated decision-makers. When you meet the expression `beta_n <- mu + L %*% z` inside the mixed logit simulator of Chapter 13, and again inside the hierarchical samplers of Chapter 17, you will know exactly what it does and why it works. The Cholesky root returns in one more role later: in Chapter 13 we *estimate* the elements of L itself, because optimizing over L rather than Σ guarantees the implied covariance matrix stays valid. And the recursive, one-component-at-a-time structure you can see in L 's triangularity is the same idea that powers the GHK simulator in Chapter 19.

2.5 Quasi-Random Draws

Everything so far transforms *pseudo-random* uniforms. Our last topic replaces the uniforms themselves. It matters when draws are used to approximate integrals — role 2 from this chapter's opening — where it buys striking accuracy gains for free.

The Monte Carlo logic of Chapter 1 approximates an integral by averaging over draws, with error shrinking at rate $\sqrt{R}$. The error comes from the draws' failure to cover the integration region evenly: random points cluster and leave gaps. But nothing about the averaging *requires* randomness — if we replaced random uniforms with a deterministic set of points that covers

(0,1) more evenly, the approximation could only improve. Sequences designed for exactly this purpose are called *quasi-random* or *low-discrepancy* sequences, and the workhorse in choice modeling is the **Halton sequence** (K. E. Train 2009, sec. 9.3.3).

The Halton sequence in base b (a prime number) is built by a beautifully simple rule: write the integers $1, 2, 3, \dots$ in base b , then mirror their digits around the decimal point. In base 2: the integer $1 = 1_2$ becomes $0.1_2 = 1/2$; $2 = 10_2$ becomes $0.01_2 = 1/4$; $3 = 11_2$ becomes $0.11_2 = 3/4$; and so on. Each new point lands in the largest remaining gap. The implementation is a short function:

```
halton <- function(n, base) {
  result <- numeric(n)
  for (i in 1:n) {
    f <- 1; r <- 0; k <- i
    while (k > 0) {
      f <- f / base
      r <- r + f * (k %% base)
      k <- k %% base
    }
    result[i] <- r
  }
  result
}

halton(8, base = 2)
```

```
[1] 0.5000 0.2500 0.7500 0.1250 0.6250 0.3750 0.8750 0.0625
```

Look at those first eight values: $1/2$, then the quarter points, then the eighth points — systematic gap-filling. For draws in several dimensions, we use a different prime base per dimension. The coverage difference is easiest to appreciate visually:

```
set.seed(2029)
pts <- rbind(
  data.frame(u1 = runif(200), u2 = runif(200), type = "Pseudo-random"),
  data.frame(u1 = halton(200, 2), u2 = halton(200, 3), type = "Halton")
)
ggplot(pts, aes(u1, u2)) +
  geom_point(size = 0.8) +
  facet_wrap(~ type) +
  coord_equal() +
  labs(x = expression(u[1]), y = expression(u[2]))
```

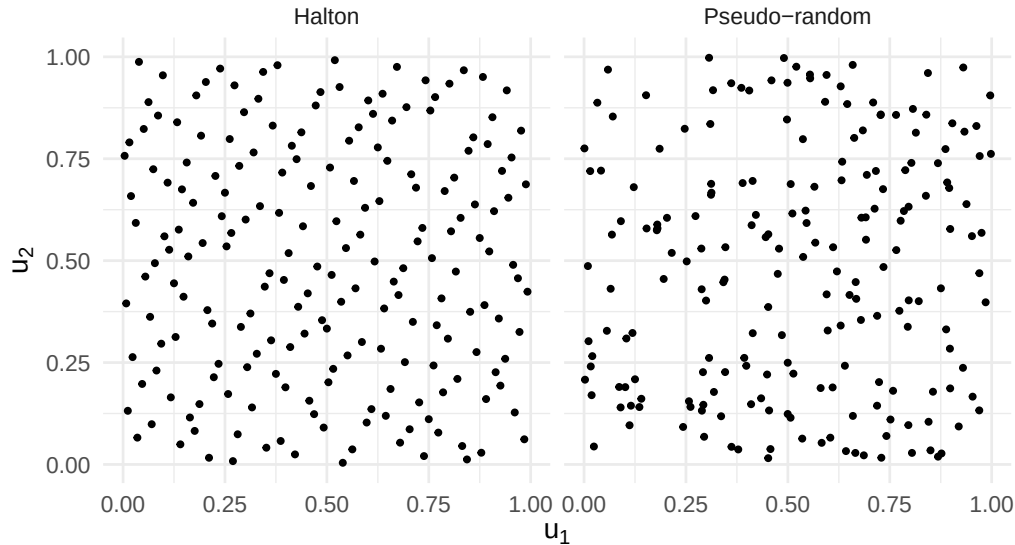


Figure 2.8: 200 points in the unit square: pseudo-random uniforms (left) cluster and leave voids; Halton points in bases 2 and 3 (right) spread evenly. For integration, even coverage means smaller approximation error from the same number of points.

And since draws from any distribution are just a quantile function applied to points in $(0, 1)$ — the lesson of the inverse transform — Halton points convert into “Halton draws” from any distribution: `qnorm(halton(R, 2))` yields evenly-spread normal draws. This is precisely the substitution promised in the Normal example above, and it is standard practice in mixed logit estimation, where a few hundred Halton draws routinely match the accuracy of a thousand or more pseudo-random ones (K. E. Train 2009, sec. 9.3.4). We will quantify that claim with our own experiment in Chapter 12.

Two cautions for later. First, successive Halton points are highly structured, so they are suited to *integration*, not to simulating data meant to look like the real world — keep using pseudo-random draws for the data-simulation work of Chapter 3. Second, in high dimensions (large prime bases) the early points of Halton sequences show unwanted correlation patterns across dimensions; remedies exist (scrambling, dropping initial points), and we will note them when they matter.

A simpler cousin, **antithetic draws**, deserves a sentence: pair each uniform draw u with its mirror $1 - u$. The pairing induces negative correlation between paired evaluations, which reduces the variance of their average. It costs one line of code and we will use it occasionally.

2.6 Key Learnings

- All simulation in this book flows from $\text{Uniform}(0,1)$ pseudo-random numbers. They are deterministic given the seed — set the seed, and your work is reproducible everywhere, forever.
- The inverse transform method converts uniforms into draws from any distribution with a computable quantile function: $X = F^{-1}(U)$ (Equation 2.1). We derived F^{-1} by hand for the exponential, Gumbel, and logistic distributions; the Gumbel one-liner `-log(-log(runif(R)))` generates the errors of every logit model we will simulate.
- Simulation is also a tool for *checking distribution theory*: we verified visually that the difference of two Gumbels is logistic — the fact underlying the binary logit.
- Standard draws become tailored draws by transformation: location–scale shifts, the lognormal for sign-restricted coefficients, and truncation via Equation 2.2. The truncated normal function `rtnorm()` will power the probit samplers of Part IV.
- Multivariate normal draws come from the Cholesky root: $\mathbf{w} = \boldsymbol{\mu} + L\mathbf{z}$ with $LL' = \Sigma$ (Equation 2.4). This is the backbone of every heterogeneity model in Parts II–III. Mind the transpose in `t(chol(Sigma))`, and always verify with sample moments.
- When draws serve to approximate integrals, evenly-spaced beats random: Halton sequences cover the space systematically and will sharply reduce the number of draws needed in Chapter 12.

3 Simulating from a Model

In Chapter 1 we simulated a *probability*: many draws of ε for one fixed choice situation. In this chapter we simulate a *dataset*: one draw of ε for each of many choice situations. The code changes less than you might expect — in both cases we draw unobservables and push them through the behavioral rule h — but the interpretation changes completely. The draws now play the role of the real-world unobservables themselves, and the output is a set of synthetic observations that look exactly like data you might collect.

Why manufacture fake data when the world is full of real data? Three reasons, in increasing order of importance for this book:

1. **Simulation is the proof that you understand the model.** A model is a description of a data generating process. If you cannot write the twenty lines of code that generate data from that process, then somewhere in the chain — the utility specification, the error distribution, the choice rule — sits an assumption you do not actually understand. The mantra from the opening of this book operationalizes here.
2. **Simulated data come with an answer key.** You chose the true parameters, so you know exactly what an estimator *should* find. With real data you can never distinguish “my estimator is broken” from “the world is surprising.” With simulated data there is no ambiguity, which makes simulation the ultimate debugging environment for estimation code.¹
3. **The whole workflow of this book depends on it.** Every estimator we build — maximum likelihood in Chapter 8, simulated likelihood in Chapter 13, Gibbs samplers in Chapter 17 — will be validated the same way: simulate data with known parameters, estimate, compare. This *simulate-and-recover* loop is the central discipline of computational statistics, and this chapter builds its first half.

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

3.1 The Anatomy of a Simulation Script

Recall the framework of Chapter 1. A choice model specifies a behavioral rule $y = h(\mathbf{x}, \varepsilon)$ and a density $f(\varepsilon)$. To simulate data from the model is to run the data generating process forward, once per observation. Every simulation script in this book — no matter how elaborate the model — has the same six steps:

1. **Set the sizes.** How many decision-makers N ? (Later: how many choice situations T , alternatives J ?)
2. **Set the true parameters.** Choose β (and later, covariance matrices, class shares, ...). These are the values we will demand our estimators recover.
3. **Generate the observables.** Create the attribute values $\mathbf{x}$ for every choice situation.
4. **Draw the unobservables.** One draw of ε from f per choice situation — the inverse-transform skills of Chapter 2.
5. **Compute utilities.** Combine steps 2–4 through the utility function.
6. **Apply the choice rule.** Record $y = h(\mathbf{x}, \varepsilon)$ for every observation.

Steps 3 and 4 both call random number generators but play entirely different roles, and confusing them is a common beginner error. The observables in step 3 are random only because *we* are inventing a dataset; in a real study they would come from your experimental design or your survey. The unobservables in step 4 are random *inside the model*: they are the ε of Equation 1.1, and their distribution f is part of the model specification. When we later estimate models, step 3’s values will be data we condition on, while step 4’s values will be gone — visible only through their fingerprints on the choices.

3.2 Example: Simulating Binary Logit Choices

Let’s simulate the laptop-offer setting of Section 1.3 at scale. Each of N shoppers is shown one configured laptop — a price drawn from the study’s range and either the base 8GB or upgraded 16GB memory — and decides whether to buy it. The model, assembled from Equation 1.3 through Equation 1.5:

$$U_n = \mathbf{x}_n' \beta + \varepsilon_n, \quad \varepsilon_n \stackrel{iid}{\sim} \text{Logistic}, \quad y_n = I[U_n > 0] \quad (3.1)$$

with $\mathbf{x}_n = (1, \text{price}_n, \text{ram16}_n)$ and the true parameter vector we fixed in Section 1.4: $\beta^* = (1.0, -1.2, 0.6)$. The subscript n now indexes shoppers, $n = 1, \dots, N$ — our notation for decision-makers from here on.

Here is the script, with the six steps labeled:

```

set.seed(30)

## 1. sizes
N <- 500

## 2. true parameters
beta <- c(1.0, -1.2, 0.6)

## 3. observables: one laptop offer per shopper
price <- runif(N, min = 0.8, max = 2.4) # in $1,000s
ram16 <- rbinom(N, size = 1, prob = 0.5) # memory upgrade?
X <- cbind(1, price, ram16) # N x 3 matrix

## 4. unobservables
eps <- rlogis(N)

## 5. utilities
U <- as.vector(X %*% beta) + eps

## 6. choices
y <- as.integer(U > 0)

head(data.frame(y, price, ram16), 4)

```

```

  y    price ram16
1 1 0.9580525    0
2 0 1.5811709    0
3 1 1.3824588    1
4 0 1.4729906    0

```

Twenty lines, most of them comments. A few implementation notes deserve attention, because the same patterns recur in every simulator we write:

- **The design matrix $\mathbf{X}$.** We assemble the observables into an $N \times K$ matrix whose columns match the entries of β , including a leading column of ones for the intercept. The matrix product `X %*% beta` then computes all N representative utilities $V_n = \mathbf{x}'_n \beta$ (the V of Section 1.3, now one per shopper) in one operation. This is our first taste of a theme that Chapter 9 develops fully: organizing data as matrices lets R do the arithmetic in compiled code.
- **`as.vector()`.** Matrix multiplication returns an $N \times 1$ matrix; coercing to a plain vector avoids surprises when we add the (vector) draws `eps`. Small thing, classic bug.

- **One draw per shopper.** Compare with Section 1.3, where we took a thousand draws for a *single* $\mathbf{x}$. Here each shopper gets exactly one ε_n — their own private circumstances on decision day.

How many shoppers bought?

```
mean(y)
```

```
[1] 0.368
```

About 37% — reasonable for laptops priced deliberately on the expensive side. This single number is already a sanity check: a purchase share of 0.99 or 0.01 would send us hunting for a misplaced sign or a wildly mis-scaled price before writing another line.

3.3 Example: Simulating Binary Probit Choices

Now the punchline of this pair of examples. The binary *probit* model is identical to the binary logit in every respect except one: the unobservables are standard normal rather than logistic. In the six-step anatomy, only step 4 changes; in the code, only one line:

```
set.seed(31)
eps_p <- rnorm(N)                                # <- the only change
U_p   <- as.vector(X %*% beta) + eps_p
y_p   <- as.integer(U_p > 0)
mean(y_p)
```

```
[1] 0.318
```

Everything you believe about a statistical model is a specific line of code in its simulator. The choice of error distribution — the difference between “logit” and “probit,” between closed-form choice probabilities (Chapter 7) and integrals requiring simulation (Chapter 18) — is the difference between `rlogis(N)` and `rnorm(N)`.²

Can you tell the two datasets apart by looking? Here are their purchase rates plotted against price:

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

bins <- cut(price, breaks = seq(0.8, 2.4, by = 0.2))
shares <- rbind(
  data.frame(bin = levels(bins), model = "logit",
    share = tapply(y, bins, mean)),
  data.frame(bin = levels(bins), model = "probit",
    share = tapply(y_p, bins, mean))
)
ggplot(shares, aes(x = bin, y = share, group = model, linetype = model)) +
  geom_line() + geom_point(size = 1.5) +
  labs(x = "price bin ($1,000s)", y = "share who bought", linetype = NULL) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```

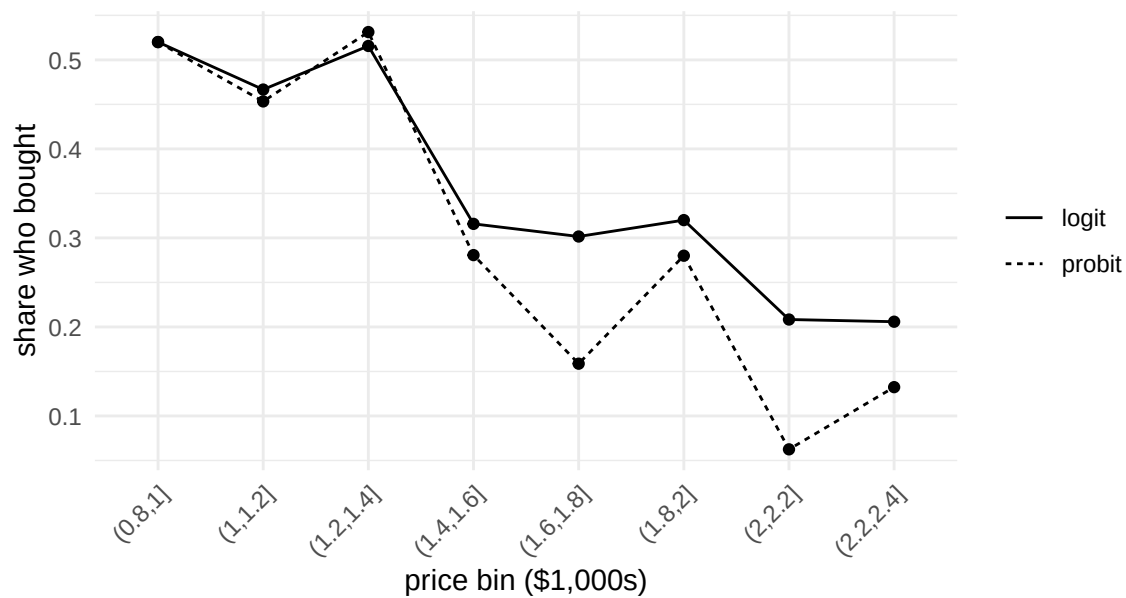


Figure 3.1: Purchase shares by price bin for one simulated logit dataset and one simulated probit dataset ($N = 500$ each). The error distribution that separates the two models is essentially invisible in raw data summaries.

Not really — both fall with price, with sampling noise dominating any difference in curve shape. Two lessons hide in this modest picture. First, model assumptions about unobservables are *not* readable off raw data; distinguishing logit from probit takes likelihood-based tools (and even then, Chapter 18 will show, the practical differences are subtle). Second, and more optimistically: since the models are so similar, the estimation machinery we build for one will transfer to the other with small changes — the exact program of Part IV.

3.4 Packaging the Simulator as a Function

We will need to simulate binary choice data again — in Chapter 5 to feed our first likelihood, in Chapter 6 to test our first optimizer, in Chapter 16 to check the Albert–Chib sampler. Copy-pasting the script each time invites drift: a tweak here, a forgotten seed there, and soon no two chapters use quite the same data generating process. The remedy is to package the simulator as a function with explicit arguments.

Doing so forces a design decision worth pausing over: *what should be an argument, and what should be internal?* Three reasonable designs:

- **(a) Everything internal.** `sim_binary_data()` with no arguments, returning one canonical dataset. Simple, but useless the moment we want a different N or β — and we will want both, e.g., to study how estimator precision varies with sample size in Chapter 8.
- **(b) Pass the design matrix in.** `sim_binary_data(X, beta)`. Maximally flexible — the caller controls the covariates entirely — and the right choice when a real study’s design is fixed. But every call now requires boilerplate to build X .
- **(c) Generate covariates internally, parameters as arguments.** The function owns the *study design* (prices uniform on the study range, memory upgrades at random) while the caller controls the *scientific quantities* (N , β).

We choose (c), with (b)’s escape hatch: covariates are generated internally by default, but a caller who needs custom covariates can supply their own X . This layered design — sensible defaults, full control available — costs three lines and is a pattern worth stealing for your own work:

```
sim_binary_data <- function(N, beta, X = NULL,
                             dist = c("logistic", "normal")) {
  dist <- match.arg(dist)
  if (is.null(X)) {
    price <- runif(N, min = 0.8, max = 2.4)
    ram16 <- rbinom(N, size = 1, prob = 0.5)
    X <- cbind(1, price, ram16)
  }
  eps <- switch(dist,
                logistic = rlogis(N),
                normal   = rnorm(N))
  U <- as.vector(X %*% beta) + eps
  data.frame(y = as.integer(U > 0), X[, -1, drop = FALSE])
}

set.seed(32)
```

```
d <- sim_binary_data(N = 500, beta = c(1.0, -1.2, 0.6))
head(d, 3)
```

```
   y   price ram16
1 0 1.609345     1
2 0 1.751693     1
3 0 2.093995     0
```

Note the `dist` argument: because logit and probit differ by one line, one simulator serves both models, and the difference is now *named* rather than buried. The function returns a plain data frame — one row per shopper, the choice and the attributes, no intercept column — because that is the shape of real choice data arriving on your desk. Rebuilding the intercept-bearing $\mathbf{X}$ matrix from such a data frame is precisely the “organizing the data” work of Chapter 4.

3.5 Sanity Checks on Simulated Data

You wrote the simulator, so it is correct... probably. Twenty lines is enough room for a sign error, a swapped argument, or a distribution with the wrong scale, and an error here poisons everything downstream: you cannot debug an estimator against data that misrepresent the model. Before any simulated dataset earns our trust, it faces checks of increasing stringency.

Level 1: marginal summaries. The purchase share (0.31 above) should be sane, and each covariate should have the intended distribution. One-liners: `mean(d$y)`, `summary(d$price)`, `table(d$ram16)`.

Level 2: directional relationships. Choices should respond to attributes in the direction the parameters dictate — buy rates falling with price, rising with the memory upgrade:

```
tapply(d$y, d$ram16, mean)
```

```
      0      1
0.2339623 0.3957447
```

The 16GB laptops sell more often, as $\beta_{\text{ram16}} = 0.6 > 0$ demands.

Level 3: quantitative agreement with theory. The strongest check compares simulated frequencies against the model’s *implied probabilities* — the closed-form curve from Section 1.3. For the binary logit we know $p(y = 1|\mathbf{x}) = e^{\mathbf{x}'\beta} / (1 + e^{\mathbf{x}'\beta})$, so binned empirical purchase rates should scatter around that curve, hugging it more tightly as N grows:

```

set.seed(33)
big <- sim_binary_data(N = 20000, beta = c(1.0, -1.2, 0.6))
b16 <- big[big$ram16 == 1, ]

bins <- cut(b16$price, breaks = seq(0.8, 2.4, by = 0.1))
emp <- data.frame(
  price = tapply(b16$price, bins, mean),
  share = tapply(b16$y, bins, mean)
)

ggplot(emp, aes(x = price, y = share)) +
  geom_function(fun = function(p) plogis(1.0 - 1.2*p + 0.6),
    linewidth = 0.8, color = "grey40") +
  geom_point() +
  labs(x = "price ($1,000s)", y = "P(buy)")

```

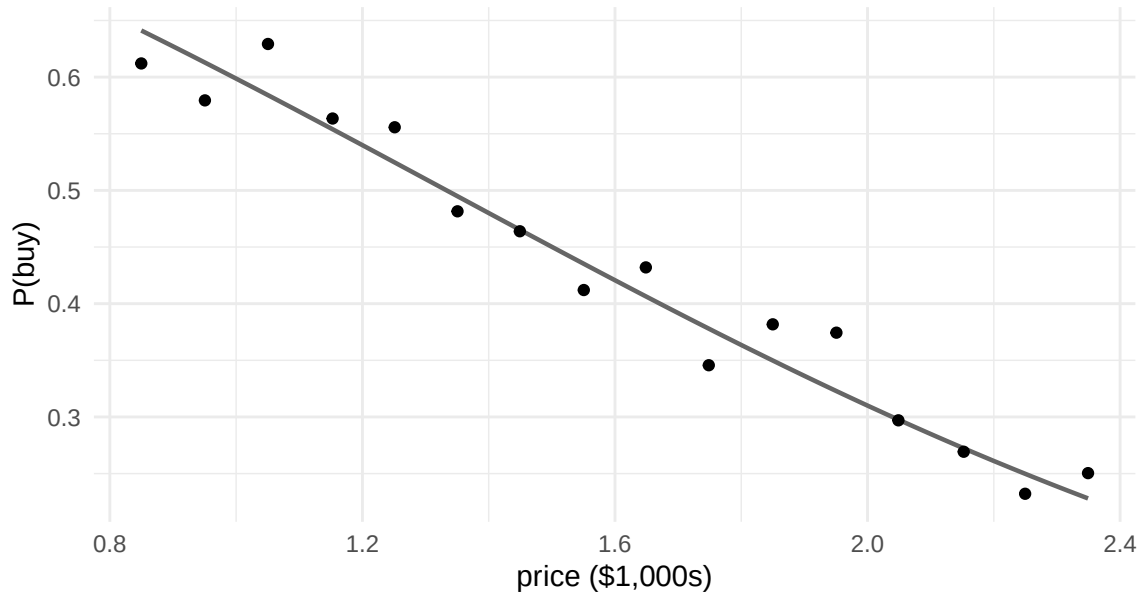


Figure 3.2: Level-3 sanity check: empirical purchase shares by price bin (points, 16GB laptops only, $N = 20,000$) against the model-implied logit curve (line). Agreement of this quality is strong evidence the simulator implements the model it claims to.

(We used `plogis()` — the logistic CDF — to evaluate the logit formula; after Chapter 2 you know exactly what that function is.) The points ride the curve. If they rode a *different* curve — shifted, flipped, compressed — the discrepancy’s shape would point to the bug: wrong sign, wrong coefficient, wrong error scale.

These checks may feel excessive for twenty lines of code, but the habit scales. In Chapter 13 the simulator will have random coefficients drawn through a Cholesky factor; in Chapter 17 entire populations of parameters. Simulators grow; the checking discipline is what keeps their complexity honest.

3.6 The Simulate-and-Recover Workflow

Step back and look at what we can now do. Given any fully specified choice model, we can manufacture a dataset from it — and we know the true β behind that dataset. The natural next question defines the rest of Part I: *pretending we don't know β , can we get it back from the data alone?*

That question, asked and answered on repeat, is the **simulate-and-recover workflow**:

1. Choose true parameters β^* .
2. Simulate a dataset from the model at β^* (this chapter).
3. Feed the dataset — *not* the parameters — to an estimation routine (chapters 5–6 build one).
4. Compare the estimate $\hat{\beta}$ to β^* .

If $\hat{\beta}$ lands close to β^* — with “close” made precise by the standard errors of Chapter 6 and the replication studies of Chapter 8 — then model, simulator, and estimator all cohere. When recovery fails, something specific is broken, and the failure pattern usually says what: a sign flip suggests a coding error in the likelihood; estimates biased toward zero suggest an error-scale mismatch; one wild coefficient suggests an identification problem (Chapter 7). Experienced modelers read recovery failures the way mechanics read engine noises, and the only way to develop that ear is to run the loop many times. We will.

The loop has one more virtue worth advertising now: it is how you *develop* new models, not just how you learn old ones. When you invent a custom model for your own research — the destination of Chapter 22 — the first thing you will build is its simulator, and simulate-and-recover will be the test bench that tells you whether your custom estimator actually works before it ever touches real data.

3.7 Key Learnings

- Simulating data from a model is running the data generating process forward: set sizes, set true parameters, generate observables, draw unobservables, compute utilities, apply the choice rule. Every simulator in this book follows these six steps.
- The draws now *are* the model's unobservables — one per choice situation — unlike Chapter 1's many-draws-per-situation integral approximation. Same generators, different role.

- Model assumptions live in specific lines of code: logit vs probit is `rlogis(N)` vs `rnorm(N)`, one line apart and nearly indistinguishable in raw data. (And the two models' different error variances previewed the scale-identification issue of Chapter 7.)
- Package simulators as functions. Deciding what is an argument (N , β , optionally $\mathbf{X}$) and what is internal (the study design) is a real design choice; we chose defaults-with-escape-hatches, a pattern that recurs in all our reusable code.
- Never trust an unchecked simulator. Check marginal summaries, directional effects, and — the gold standard — quantitative agreement between empirical frequencies and model-implied probabilities (Figure 3.2).
- The simulate-and-recover workflow — simulate at known β^* , estimate, compare — is the central discipline of this book. We have built the “simulate” half; Part I's remaining chapters build “recover.”

4 Organizing Choice Data

This chapter contains no new statistics. It is about something more mundane and, in my experience, responsible for more student-hours of frustration than any mathematical topic in this book: getting choice data into a shape that estimation code can consume. The likelihoods of Part II are sums over choice situations of expressions involving each alternative’s attributes; between the data as they arrive and the likelihood as it computes sits a layer of pure bookkeeping. Do the bookkeeping haphazardly and every later chapter fights it. Do it once, deliberately, and the same few structures will carry us from the plain MNL (Chapter 8) through simulated likelihood (Chapter 13) to the hierarchical samplers (Chapter 17) without modification.

Because this is a chapter of choices-with-tradeoffs rather than theorems, I will follow a pattern used throughout the book: lay out the reasonable options, then commit to one and say why. You may prefer different choices in your own work — the options all *work* — but a book, like a codebase, benefits from picking a lane.

4.1 The Shape of Choice Data

The data of a regression course are flat: one row per observation, one column per variable. Choice data have more structure, because each observation is not a number but an *event with internal anatomy*: a particular person, facing a particular set of alternatives, on a particular occasion, choosing one. Three indices track that anatomy, and they have been fixed since Chapter 3:

- $n = 1, \dots, N$ indexes **decision-makers** (our survey respondents),
- $t = 1, \dots, T$ indexes **choice situations** (the tasks each respondent completes),
- $j = 1, \dots, J$ indexes **alternatives** within a choice situation.

The pair (n, t) identifies one choice situation; the triple (n, t, j) identifies one alternative in one situation — one *utility* in the model’s terms, since Chapter 7 will assign $U_{ntj} = \mathbf{x}'_{ntj}\boldsymbol{\beta} + \varepsilon_{ntj}$ to exactly this triple. Our laptop study (Section 1.4) has $N = 500$, $T = 8$, and $J = 3$: five hundred respondents, eight tasks each, three laptop profiles per task, for $500 \times 8 = 4,000$ observed choices and $4,000 \times 3 = 12,000$ described alternatives.

The binary data of Chapter 3 were the degenerate case that hid all of this: one task per person ($T = 1$) and one modeled alternative plus a fixed outside option ($J = 2$ with the “don’t buy”

utility normalized away), so a flat data frame sufficed. It no longer does. The question of this chapter is: *in what shape do we store 12,000 alternative descriptions and 4,000 choices?*

4.2 Wide and Long Formats

Two conventions dominate, distinguished by what a row represents.

In **wide format**, one row is one choice situation (n, t) , and each alternative's attributes spread across columns:

	id	task	brand_1	price_1	brand_2	price_2	brand_3	price_3	choice
1	1	1	Acer	1.2	Apple	2.0	Dell	1.4	2
2	1	2	Dell	1.6	Acer	0.9	Apple	1.8	1

In **long format**, one row is one alternative (n, t, j) , with the choice recorded as an indicator:

	id	task	alt	brand	price	choice
1	1	1	1	Acer	1.2	0
2	1	1	2	Apple	2.0	1
3	1	1	3	Dell	1.4	0
4	1	2	1	Dell	1.6	1
5	1	2	2	Acer	0.9	0
6	1	2	3	Apple	1.8	0

(These two tables show the same two tasks; respondent 1 chose the Apple in task 1 and the Dell in task 2.)

The tradeoffs:

- **Wide** matches how survey platforms export data and how humans read a choice task (“here were your three options; you picked #2”). It is compact — one row per choice. But the attributes of “alternative 2” live in differently-named columns from “alternative 1,” so any computation over alternatives means wrangling column-name patterns; and if some tasks show two alternatives while others show four, the format needs padding columns and missing-value conventions.
- **Long** matches how the *model* thinks. Each row is one (n, t, j) triple — one utility. Attributes have a single column each regardless of J ; tasks with different numbers of alternatives are just groups of different sizes; and the operations our likelihoods need (compute a value for every alternative, then aggregate within choice situations) become operations on rows and groups.

Our choice: long format, always. The deciding argument is the last one — in Chapter 8 the log-likelihood will be built from exactly two moves, “one value per row” and “aggregate rows within task,” and long format makes both natural. Wide-to-long conversion is a mechanical reshape,¹ so nothing is lost by standardizing. When a dataset in this book is called *the data*, it is a long-format data frame with columns `id`, `task`, `alt`, `choice`, then attributes.

One convention completes the format, and I state it as a contract because code will rely on it silently: **rows are sorted by id, then task, then alt.** Grouped computations in the chapters ahead identify a choice situation by contiguous runs of rows; the sort order is what makes that identification valid. Violate the contract and nothing errors — results are simply, quietly wrong. (We will build one guardrail below.)

4.3 Two Kinds of Variables, Two Kinds of Columns

Look again at the long-format table and note what varies where. `price` varies across rows *within* a task — each alternative has its own. But a respondent characteristic like `age` would repeat across all rows of a task, varying only with `id`. The distinction matters enough for names:

- **Alternative-specific variables** ($\mathbf{x}_{ntj}$): attributes of the alternatives — price, brand, memory. These vary within a choice situation and are the natural drivers of choice.
- **Individual-specific variables** ($\mathbf{z}_n$): characteristics of the decision-maker — age, income, prior brand ownership. Constant across the alternatives in a task. (A notation caution: this $\mathbf{z}_n$ — observed characteristics — is unrelated to the standard-normal draw vector $\mathbf{z}$ of Section 2.4; the collision is regrettably standard in this literature, and context always disambiguates.)

Here is the fact that surprises newcomers, stated now and explained properly in Chapter 7: *an individual-specific variable cannot enter utility with a single common coefficient*. If age adds $\gamma \cdot \text{age}_n$ to the utility of **every** alternative, it raises all J utilities equally, changes no comparison, and drops out of the choice probabilities entirely — γ is unidentified. Only *differences* in utility move choices. Individual-specific variables earn their way into choice models by interaction: with alternative dummies (age shifts the appeal of Apple relative to Acer) or with attributes (income damps price sensitivity). Structurally, that means $\mathbf{z}_n$ columns in a long data frame are raw material awaiting interaction, not regressors in their own right. In this book’s main thread, individual-specific variation instead enters through *coefficients that vary across people* — the heterogeneity models of Chapter 11 onward — and in Chapter 17 the two devices meet, with $\mathbf{z}_n$ shifting the *mean* of person n ’s coefficient vector.

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

4.4 Coding Categorical Attributes

Brand takes values Acer, Dell, Apple; memory takes 8, 16, 32. Utilities are sums of numbers, so categories must become numeric columns, and the mapping is another small decision with options.

Dummy (treatment) coding picks a reference level and creates an indicator per remaining level. With Acer as reference: `dell` and `apple` columns, each 0/1. A brand coefficient is then the utility of that brand *relative to Acer, all else equal* — concrete and readable, with the reference level absorbing the baseline.

Effects coding constrains the level utilities to sum to zero instead: the reference level's columns are all -1, and each level's utility is read relative to the *average* level. Conjoint practice often prefers it because part-worths then compare against a neutral average rather than an arbitrary reference, and it interacts more gracefully with the “attribute importance” summaries of Chapter 21.

Our choice: dummy coding, with the reference levels fixed once — Acer, 8GB, 13-inch, as set in Table 1.1. For a book, interpretability-per-line-of-code wins: “the Apple coefficient is the extra utility of an Apple over an Acer” needs no further explanation, and every simulate-and-recover check reads directly against true parameters stated in the same coding. Effects coding returns for a cameo in Chapter 21 where its advantages actually bite. What is *not* optional is consistency: coding schemes, like sort orders, are contracts, and estimates are only interpretable against the coding that produced them.

R can build coded columns from factors automatically via `model.matrix()`, and we will let it — with the factor levels ordered explicitly so the reference is what we intend, not what alphabetical order happens to deliver:

```
brand <- factor(c("Acer","Apple","Dell"), levels = c("Acer","Dell","Apple"))
model.matrix(~ brand)
```

```
(Intercept) brandDell brandApple
1           1         0         0
2           1         0         1
3           1         1         0
attr(,"assign")
[1] 0 1 1
attr(,"contrasts")
attr(,"contrasts")$brand
[1] "contr.treatment"
```

Note `model.matrix()` includes an intercept column by default. Whether we keep it is the next section's business.

4.5 Where Is the Intercept?

The binary model of Chapter 3 had an intercept: a constant in $\mathbf{x}_n$ giving the baseline pull toward buying. Scan the long-format laptop data for its analogue and you hit a small conceptual wall: a column of ones would give *every alternative in every task* the same utility boost — and as with individual-specific variables, anything common to all alternatives cancels from every comparison. A common intercept is unidentified in multinomial choice data.

What can exist is an **alternative-specific constant (ASC)**: a dummy for *position* j , capturing systematic utility for “option 2” as such. Whether ASCs belong in the model depends on the design:

- With **labeled alternatives** — the options are inherently “car,” “bus,” “train,” the staples of transportation economics (K. E. Train 2009, sec. 2.5) — ASCs are essential. Each mode has unmeasured systematic appeal, and its ASC absorbs it.
- With **unlabeled profiles** — our study, and most conjoint: the three laptops in a task are just “some laptops,” their screen positions randomized — position has no meaning, and the brand attribute already carries the “which product” information. Including position ASCs would estimate, roughly, left-middle-right screen bias.²

So our laptop data frames carry no intercept and no ASCs; the brand dummies play the role closest to “brand intercepts.” When Part IV needs ASCs (the probit’s identification story makes them central) and when Chapter 22 adds a no-choice option (whose ASC is the entire model of “walking away”), we will add the relevant dummy columns explicitly, as ordinary attributes. That is a quiet advantage of long format: an ASC is just one more 0/1 column, not a special modeling object.

4.6 Building the Study Design

Enough principles; let’s build the actual dataset scaffold for the laptop study. A useful separation of concerns — and the one real studies force on you anyway — is between the **design** (which profiles appear in which tasks; exists before any respondent) and the **responses** (the **choice** column; exists after). We build the design now. Filling in choices requires the MNL model itself, so that half arrives in Chapter 7.

How should profiles be assigned to tasks? The gold standard is a formal experimental design — orthogonal arrays, D-efficient designs — that maximizes the information the study extracts per task (Rao 2014, Chs. 2 and 4). Design theory is a book of its own and deliberately out of our scope. We use the simplest defensible method, **independent random assignment**:

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

every attribute of every profile drawn independently at random. Random designs are what our simulate-and-recover studies need (no accidental confounding between attributes, by construction) and they let the code stay transparent:³

```
sim_laptop_design <- function(N, T, J) {
  n_rows <- N * T * J
  data.frame(
    id    = rep(seq_len(N), each = T * J),
    task  = rep(rep(seq_len(T), each = J), times = N),
    alt   = rep(seq_len(J), times = N * T),
    brand = factor(sample(c("Acer","Dell","Apple"), n_rows, replace = TRUE),
                    levels = c("Acer","Dell","Apple")),
    ram   = factor(sample(c("8","16","32"), n_rows, replace = TRUE),
                    levels = c("8","16","32")),
    screen = factor(sample(c("13","15"), n_rows, replace = TRUE),
                    levels = c("13","15")),
    price = round(runif(n_rows, min = 0.8, max = 2.4), 2)
  )
}

set.seed(40)
laptops <- sim_laptop_design(N = 500, T = 8, J = 3)
head(laptops, 6)
```

	id	task	alt	brand	ram	screen	price
1	1	1	1	Apple	16	13	2.16
2	1	1	2	Dell	16	15	2.08
3	1	1	3	Acer	32	15	1.77
4	1	2	1	Dell	32	15	1.81
5	1	2	2	Apple	8	15	1.24
6	1	2	3	Dell	32	15	1.82

The `rep()` calls in the first three columns implement the sort-order contract mechanically — `id` slowest, `alt` fastest — so the design is born correctly ordered. Dimensions check:

```
nrow(laptops)           # 500 * 8 * 3
```

```
[1] 12000
```

³A confession: `T` is also R's abbreviation for `TRUE`, and masking it is normally poor hygiene. We use it anyway so the code reads like the math (N, T, J) . Never abbreviate `TRUE` as `T` in your own code — this is exactly why.

And a design-level sanity check in the spirit of Section 3.5: with independent random assignment, attributes should be uncorrelated and levels balanced. Brand shares by alternative position:

```
round(prop.table(table(laptops$brand, laptops$alt), margin = 2), 2)
```

	1	2	3
Acer	0.34	0.35	0.33
Dell	0.34	0.33	0.34
Apple	0.32	0.32	0.33

Near-uniform everywhere, as designed.

4.7 From Data Frame to Likelihood Inputs

The last mile. A data frame is for humans; estimation code wants numeric arrays. Specifically, everything from Chapter 8 onward needs three things: the attribute matrix, the choices, and a map from rows to choice situations. There are three reasonable containers, and this is the load-bearing decision of the chapter:

- **(a) A list of per-task matrices.** `data[[i]]` holds task i 's $J \times K$ attribute matrix and its choice. Transparent — each likelihood term has its own little bundle — and the shape most textbook pseudo-code implies. But R-level loops over 4,000 list elements are slow (Chapter 9 will quantify how slow), and slow inside a likelihood means slow *everywhere*, because the likelihood is the inner loop of every method in this book.
- **(b) A three-dimensional array.** Dimensions (task, alternative, attribute): `X[i, j, k]`. Fast, vectorizable, and appealing when every task shows exactly J alternatives. But it hard-codes the balanced-design assumption — add a no-choice alternative to some tasks (Chapter 22) or work with real-world availability variation, and the array needs padding and masking. Array code also reads poorly; three-index gymnastics obscure the model.
- **(c) A stacked matrix plus index vectors.** Keep the long format's row layout: one $(N \cdot T \cdot J) \times K$ matrix `X` whose rows are alternatives in contract order, a 0/1 vector `choice` marking chosen rows, and an integer vector `task_id` assigning each row to its choice situation. Grouped operations — “sum e^V within each task” — become single calls to R's grouped-aggregation primitives (`rowsum()` chief among them), which run at compiled speed. Ragged tasks cost nothing: a task is however many rows share its `task_id`.

Our choice: (c), the stacked-matrix representation. It is within a whisker of the array's speed, it degrades gracefully to ragged designs, and — decisive for this book — it is the representation that survives unchanged into later parts: the MSL of Chapter 13 evaluates the same stacked X under many coefficient draws, and the person-by-person Metropolis steps of Chapter 17 slice it by `id` without restructuring. Build once, reuse for nineteen chapters.

The conversion function:

```
build_choice_data <- function(df, attr_formula) {
  stopifnot(!is.unsorted(order(df$id, df$task, df$alt)))
  X <- model.matrix(attr_formula, data = df)
  X <- X[, colnames(X) != "(Intercept)", drop = FALSE]
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))
  list(
    X      = X,
    choice = if ("choice" %in% names(df)) df$choice else NULL,
    task_id = task_id,
    id      = df$id,
    n_tasks = max(task_id),
    K      = ncol(X)
  )
}
```

Every line earns its place:

- The `stopifnot()` guardrail enforces the sort-order contract; better a loud failure now than silent nonsense later.
- `model.matrix(attr_formula, df)` expands factors into dummy columns using their declared reference levels — the coding decision of this chapter, executed by base R. We then drop the intercept column it insists on adding, for the identification reason of Section 4.5.
- `task_id` is built by a small idiom worth reading twice: `!duplicated()` over the `(id, task)` pairs flags each task's first row, and `cumsum()` turns those flags into a running task counter — 1,1,1,2,2,2,... — giving every row its group label in two vectorized calls.
- `choice` is optional, so the same function serves a bare design (now) and responded data (from Chapter 7 on).

Applied to the laptop design:

```
dat <- build_choice_data(laptops, ~ brand + ram + screen + price)
str(dat, max.level = 1)
```

List of 6

```

$ X      : num [1:12000, 1:6] 0 1 0 1 0 1 0 1 0 0 ...
  ..- attr(*, "dimnames")=List of 2
$ choice : NULL
$ task_id: int [1:12000] 1 1 1 2 2 2 3 3 3 4 ...
$ id      : int [1:12000] 1 1 1 1 1 1 1 1 1 1 ...
$ n_tasks: int 4000
$ K       : int 6

```

```
colnames(dat$X)
```

```

[1] "brandDell"  "brandApple" "ram16"       "ram32"       "screen15"
[6] "price"

```

There it is: a 12000×6 matrix whose six columns⁴ — `dell`, `apple`, `ram16`, `ram32`, `screen15`, `price` in our shorthand — line up exactly, column for column, with the true parameter vector we will fix when the full study’s choices are simulated in Chapter 7, plus the index vectors that give the rows their meaning. When Chapter 8 computes its first multinomial log-likelihood, it will consume precisely this object, and so will nearly every estimator after it.

4.8 Package Conventions: `dfidx` and Friends

When we validate our hand-coded estimators against R packages — `mlogit` in Chapter 8, `gmnl` in Chapter 11 — we must feed those packages *their* expected structure. Happily, the ecosystem standardized on ideas congruent with ours: `mlogit` (Croissant 2020) consumes data via the `dfidx` package, which wraps a long-format data frame with declared indexes — `idx = c("chid", "alt")` naming the choice-situation and alternative columns, with a nesting structure for respondent panels. The conversion from our canonical format is one line, which we will write when we first need it. If you understand this chapter, no R choice package’s data layer will ever mystify you: they are all long format plus index bookkeeping, differing only in surface conventions.

4.9 Key Learnings

- Choice data live on three indices — decision-maker n , choice situation t , alternative j — and one row of our canonical data is one (n, t, j) triple: **long format**, sorted by `id`, `task`, `alt`. The sort order is a contract that grouped computations rely on silently.

⁴To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

- Only utility *differences* move choices. That single fact organized the whole chapter: individual-specific variables need interactions to matter, a common intercept is unidentified, and ASCs are position dummies that belong in labeled designs but not in our unlabeled profile study.
- Categorical attributes become dummy columns against fixed reference levels (Acer, 8GB, 13"); `model.matrix()` mechanizes the coding, and consistency is what keeps estimates interpretable.
- The study **design** exists before responses: `sim_laptop_design()` builds the $500 \times 8 \times 3$ laptop study with independent random attribute assignment, correctly ordered by construction.
- The load-bearing decision: estimation code consumes the **stacked-matrix representation** — $\mathbf{X}$ ($NTJ \times K$), 0/1 **choice**, integer **task_id** — chosen over per-task lists (slow) and 3-D arrays (rigid) because it is fast, ragged-friendly, and reusable verbatim from MLE through MSL to MCMC. `build_choice_data()` performs the conversion and guards the contracts.
- Package data layers (`dfidx` for `mlogit`) are the same ideas under different names; conversion is mechanical.

5 From Choice Data to Likelihood

We can now specify a choice model (Chapter 1), draw from any density it involves (Chapter 2), simulate data from it (Chapter 3), and organize those data for computation (Chapter 4). This chapter builds the bridge that runs the other way — from observed data back toward the unknown parameters. The bridge is the *likelihood function*, and it is the single most important object in this book: Part II maximizes it, Chapter 12 approximates it, and Part III explores a posterior that is proportional to it. Every estimator we construct is some maneuver performed on the function this chapter teaches you to write.

5.1 The Likelihood Function

Start from what the model gives us. A specified choice model delivers, for any parameter value, the probability of each possible outcome in a single choice situation: $p(y_n | \mathbf{x}_n, \beta)$. In Chapter 1 we used this object in its natural direction — parameters known, probabilities of outcomes computed. Data reverse the situation: after the study is run, the outcomes $y_1, \dots, y_N$ are *known*, and β is not.

The likelihood function is the same mathematical expression read in the reverse direction:

$$L(\beta) = p(\mathbf{y} | \mathbf{X}, \beta) \quad \text{viewed as a function of } \beta, \text{ with the data fixed.} \quad (5.1)$$

Read it as: *how probable were the choices we actually observed, if the true parameter vector were β ?* Every candidate β gets a score. A candidate under which the observed data would have been unremarkable scores high; a candidate under which the data would have been a miracle scores low. The principle of maximum likelihood — coming formally in a few pages — says to prefer the candidate with the highest score.

The distinction between probability and likelihood trips up everyone at first pass, so let me put it plainly. $p(y|\mathbf{x}, \beta)$ is one formula with two readings: as a function of y with β fixed, it is a probability distribution (non-negative, sums to one over outcomes); as a function of β with y fixed, it is a likelihood (non-negative, but sums to nothing in particular over β — it is not a distribution over parameters).¹ Same formula, different axis held fixed.

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across

5.2 Independence and the Product Form

For a dataset rather than a single observation, we need the *joint* probability of all the observed choices. Our models make this easy by assumption: conditional on the observables and parameters, the unobservables ε_n are drawn independently across choice situations — look back at the simulator of Section 3.2, where `rlogis(N)` draws exactly that way. Independent unobservables make the choices conditionally independent, and the joint probability of independent events is the product of their probabilities:

$$L(\beta) = \prod_{n=1}^N p(y_n | \mathbf{x}_n, \beta) \quad (5.2)$$

For the binary logit, each factor takes a form worth writing carefully. Denote $P_n = \Pr(y_n = 1 | \mathbf{x}_n, \beta) = e^{\mathbf{x}'_n \beta} / (1 + e^{\mathbf{x}'_n \beta})$ from Equation 1.6. Shopper n contributes P_n if they bought and $1 - P_n$ if they didn't, which a classic piece of notation compresses into one expression:

$$L(\beta) = \prod_{n=1}^N P_n^{y_n} (1 - P_n)^{1-y_n} \quad (5.3)$$

The exponents $y_n \in \{0, 1\}$ act as switches, turning on exactly the factor that matches what shopper n actually did. This switch trick generalizes: in Chapter 8, the same device (an indicator exponent selecting the chosen alternative's probability) builds the MNL likelihood.

A word on the independence assumption, because we will renegotiate it later. Independence across *shoppers* is usually innocuous. Independence across *multiple choices by the same shopper* is not — respondent n 's eight tasks share whatever is persistent about respondent n — and how to honor that dependence is precisely the story of the panel-data models in Chapter 11 and Chapter 13. For this chapter's cross-sectional data (one choice per shopper), the product form is exactly right.

5.3 The Log-Likelihood

Equation 5.2 is mathematically pleasant and computationally disastrous. Each factor is a probability, typically between 0.2 and 0.8; multiply five hundred of them and the result is on the order of $0.5^{500} \approx 10^{-151}$. Multiply a few thousand — our laptop study has 4,000 choices — and the product is smaller than the smallest positive number a computer can represent (about 10^{-308} for standard doubles). The computed likelihood becomes exactly zero, for *every* candidate β , and all information is annihilated. Watch it happen:

tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option's appeal as tasks teach them market prices, and model exactly that.

```
probs <- runif(5000, 0.2, 0.8) # plausible per-choice probabilities
prod(probs[1:100])           # fine
```

```
[1] 2.326112e-34
```

```
prod(probs[1:500])           # vanishing...
```

```
[1] 1.663067e-173
```

```
prod(probs)                  # gone: underflow to exactly 0
```

```
[1] 0
```

The remedy is to work with the *logarithm* of the likelihood. Logs turn products into sums, and sums of a few thousand moderate negative numbers are computationally trivial:

$$\ell(\beta) = \log L(\beta) = \sum_{n=1}^N \log p(y_n | \mathbf{x}_n, \beta) \quad (5.4)$$

```
sum(log(probs))              # no drama
```

```
[1] -3826.941
```

Nothing is lost by the transformation: the logarithm is strictly increasing, so L and ℓ rank every pair of candidates identically and share exactly the same maximizer. For the binary logit, taking logs of Equation 5.3 gives the form we will implement:

$$\ell(\beta) = \sum_{n=1}^N \left[y_n \log P_n + (1 - y_n) \log(1 - P_n) \right] \quad (5.5)$$

From here on, “the likelihood” in conversation almost always means the log-likelihood in computation, and $\ell(\beta)$ is the function our code evaluates. Note its sign conventions: each term is the log of a probability, hence negative, so $\ell(\beta)$ is a negative number, and *less negative is better*.

5.4 Coding the Binary Logit Log-Likelihood

Let's turn Equation 5.5 into R, in the transparent-first style we committed to. The function takes a candidate parameter vector plus the data — in the design-matrix form of Chapter 4 — and returns one number:

```
loglik_binary <- function(beta, y, X) {  
  ll <- 0  
  for (n in 1:length(y)) {  
    v <- sum(X[n, ] * beta)      # V_n = x_n' beta  
    P <- exp(v) / (1 + exp(v))   # P_n  
    ll <- ll + y[n]*log(P) + (1 - y[n])*log(1 - P)  
  }  
  ll  
}
```

The loop is a line-by-line transcription of the math: representative utility, choice probability, the switch expression, accumulate. Transcription fidelity is the point — when you can see the equation in the code, you can debug the code against the equation. (The vectorized version, several hundred times faster, appears in Chapter 9; the answers are identical.)

To exercise it we need data with known truth — the simulator from Section 3.4, reproduced here so this chapter stands alone:

```
sim_binary_data <- function(N, beta, X = NULL,  
                           dist = c("logistic", "normal")) {  
  dist <- match.arg(dist)  
  if (is.null(X)) {  
    price <- runif(N, min = 0.8, max = 2.4)  
    ram16 <- rbinom(N, size = 1, prob = 0.5)  
    X <- cbind(1, price, ram16)  
  }  
  eps <- switch(dist,  
               logistic = rlogis(N),  
               normal   = rnorm(N))  
  U <- as.vector(X %*% beta) + eps  
  data.frame(y = as.integer(U > 0), X[, -1, drop = FALSE])  
}  
  
set.seed(50)  
beta_true <- c(1.0, -1.2, 0.6)  
d <- sim_binary_data(N = 500, beta = beta_true)
```

```
y <- d$y
X <- cbind(1, d$price, d$ram16)
```

Now the payoff. The log-likelihood scores candidates; let's score a few, including the truth and some pretenders:

```
cands <- rbind(
  truth      = c( 1.0, -1.2,  0.6),
  zero_price = c( 1.0,  0.0,  0.6),
  flipped_sign = c( 1.0,  1.2,  0.6),
  all_zero   = c( 0.0,  0.0,  0.0)
)
apply(cands, 1, function(b) loglik_binary(b, y, X))
```

truth	zero_price	flipped_sign	all_zero
-302.7940	-533.0668	-1077.9031	-346.5736

The ranking behaves exactly as the likelihood principle promises. The true parameters score best. Ignoring price entirely costs dozens of log-likelihood points; *flipping* the price coefficient's sign — claiming shoppers prefer expensive laptops — is catastrophically implausible given these data. And the all-zero candidate, which says every shopper flips a fair coin, does poorly in a specific, interpretable way: its score is exactly $N \log(0.5) = -346.6$, a number we will meet again as the *null log-likelihood* when Chapter 8 builds fit measures around it.

One more scoring note: the true β is the best candidate *of these four*, but it will not be the exact maximizer of ℓ — the maximizer adapts to this particular sample's noise. How far the maximizer wanders from the truth, sample to sample, is the entire subject of standard errors.

5.5 Maximum Likelihood Estimation

The estimator this scoring logic suggests has a one-line definition:

$$\hat{\beta} = \arg \max_{\beta} \ell(\beta) \quad (5.6)$$

The *maximum likelihood estimator* (MLE) is the parameter value that makes the observed data most probable. Note carefully what kind of object Equation 5.6 is, in the taxonomy from the introduction: it is an *estimator* — a rule mapping data to a value — not yet an estimate (that requires data) and certainly not the model (which supplied ℓ 's formula). One model, many possible estimators; maximum likelihood is our first because it is general (any model

with a computable likelihood), principled, and — for the well-behaved models of Part II — as statistically efficient as any estimator can be.

That efficiency claim is part of a package of classical results that I will state and use but not prove; Abramovich and Ritov (2023) and Hansen (2022a) give full treatments. Under regularity conditions our models satisfy:

- **Consistency:** as $N \rightarrow \infty$, $\hat{\beta} \rightarrow \beta^*$. More data concentrates the likelihood around the truth.
- **Asymptotic normality:** $\sqrt{N}(\hat{\beta} - \beta^*)$ converges to a normal distribution whose covariance the next section identifies. This is what licenses confidence intervals and z -statistics.
- **Efficiency:** among (regular) consistent estimators, the MLE achieves the smallest asymptotic variance — it wrings the most out of each observation.
- **Invariance:** the MLE of any function $g(\beta)$ is $g(\hat{\beta})$. Estimate the model once, and quantities like willingness-to-pay — ratios of coefficients, as Chapter 21 develops — come free by plugging in.

These are *asymptotic* promises, and simulation is how we audit them at the sample sizes we actually face: the replication study in Chapter 8 will watch consistency and asymptotic normality emerge (or not) at $N = 4,000$ choices.

5.6 The Score and the Hessian

Two derivative objects of ℓ carry the theory and drive the computation of the next chapter. Their names are worth memorizing now.

The **score** is the gradient — the K -vector of first derivatives:

$$s(\beta) = \frac{\partial \ell(\beta)}{\partial \beta} \quad (5.7)$$

At an interior maximum the score is zero; that is what “maximum” means. So the MLE can be characterized as the solution to $s(\hat{\beta}) = \mathbf{0}$, and the numerical methods of Chapter 6 are, at heart, root-finders for the score.

For the binary logit the score has a form so clean it feels like a reward. Differentiating Equation 5.5 is worth doing once in full. Write $v_n = \mathbf{x}_n' \beta$, so $P_n = e^{v_n} / (1 + e^{v_n})$ and, more conveniently, $\log P_n = v_n - \log(1 + e^{v_n})$ and $\log(1 - P_n) = -\log(1 + e^{v_n})$. Since $\partial v_n / \partial \beta = \mathbf{x}_n$,

$$\frac{\partial \log P_n}{\partial \beta} = \mathbf{x}_n - \frac{e^{v_n}}{1 + e^{v_n}} \mathbf{x}_n = (1 - P_n) \mathbf{x}_n, \quad \frac{\partial \log(1 - P_n)}{\partial \beta} = -P_n \mathbf{x}_n.$$

Each observation therefore contributes $y_n(1 - P_n)\mathbf{x}_n - (1 - y_n)P_n\mathbf{x}_n = (y_n - P_n)\mathbf{x}_n$, and summing over observations gives the score. (The same algebra yields $\partial P_n / \partial \beta = P_n(1 - P_n)\mathbf{x}_n$, which we will need for the Hessian in Chapter 6.)

$$s(\beta) = \sum_{n=1}^N (y_n - P_n) \mathbf{x}_n = \mathbf{X}'(\mathbf{y} - \mathbf{P}) \quad (5.8)$$

where $\mathbf{X}$ stacks the row vectors $\mathbf{x}'_n$ into an $N \times K$ matrix, and $\mathbf{y}$ and $\mathbf{P}$ are the N -vectors collecting the y_n and P_n .

The score is the sum of *prediction errors times covariates*. At the maximum, errors are exactly uncorrelated with every covariate — the likelihood’s echo of least squares’ normal equations. We will verify Equation 5.8 numerically in Chapter 6 and derive its MNL sibling in Chapter 8.

The **Hessian** is the $K \times K$ matrix of second derivatives:

$$H(\beta) = \frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta'} \quad (5.9)$$

At the maximum, H is negative definite (the surface curves down in every direction), and *how sharply* it curves is the information content of the data. Intuition: a knife-edge peak means tiny parameter movements make the data noticeably less probable — the data pin the parameters down; a broad mesa means many candidates explain the data near-equally — the data are less informative. The classical results make this exact: the asymptotic covariance of the MLE is the inverse of the (expected) negative Hessian, so in practice

$$\widehat{\text{Var}}(\hat{\beta}) = [-H(\hat{\beta})]^{-1} \quad (5.10)$$

whose diagonal, square-rooted, gives the standard errors reported next to every estimate in this book.² If you are wondering where the $\sqrt{N}$ of the asymptotic-normality statement went: it is hiding inside H . The Hessian sums over all N observations, so it grows in proportion to N , and its inverse shrinks like $1/N$ — Equation 5.10 is the $\sqrt{N}$ statement with the scaling already folded in. (Watch it happen empirically at the end of Chapter 6, where quadrupling N roughly halves the standard errors.) Curvature is variance, inverted. Hold onto that sentence; it is the entire connection between optimization output and statistical inference, and Chapter 6 cashes it in.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

5.7 A First Estimator: Grid Search

We have the objective (Equation 5.5) and the target (Equation 5.6). The only missing ingredient is a way to *find* the maximizer, and the most naive method is fully within our powers: evaluate ℓ on a grid of candidates and take the best. Crude, but it completes the simulate-and-recover loop for the first time in this book, and its failure mode motivates everything in Chapter 6.

Start in one dimension to build intuition: fix the intercept and memory coefficients at their true values and slice the log-likelihood along the price coefficient alone (a *slice*, not the “profile likelihood” of the testing literature, which re-maximizes over the other parameters at each point):

```
b_price <- seq(-3, 1, by = 0.01)
ll <- sapply(b_price, function(b)
  loglik_binary(c(1.0, b, 0.6), y, X))

ggplot(data.frame(b_price, ll), aes(b_price, ll)) +
  geom_line() +
  geom_vline(xintercept = -1.2, linewidth = 0.3) +
  geom_vline(xintercept = b_price[which.max(ll)], linetype = "dashed") +
  labs(x = expression(beta[price]), y = "log-likelihood")
```

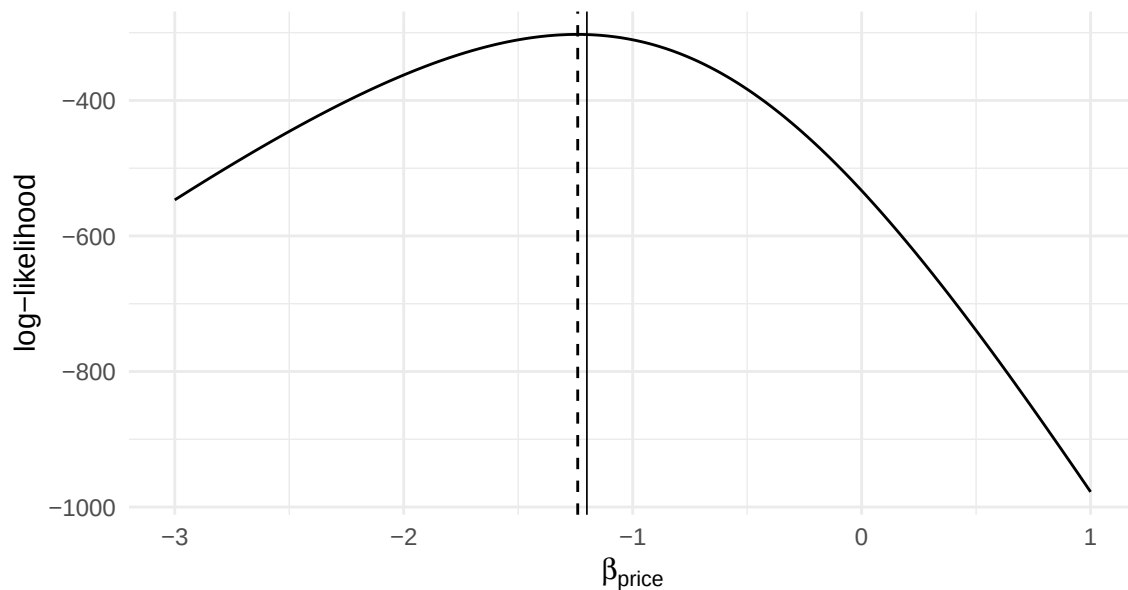


Figure 5.1: A slice of the log-likelihood along the price coefficient (intercept and memory coefficients held at truth). The maximizer (dashed) sits near, but not exactly at, the true value of -1.2 (solid): the difference is sampling noise, the raw material of standard errors.

The curve is smooth, single-peaked, and — look closely at the dashed line — peaks at -1.24, close to but not exactly -1.2. That gap *is* sampling variation; rerun the simulation with a different seed and the peak lands somewhere else nearby. The curvature at the peak is what Equation 5.10 converts into a standard error — a sharper peak would mean the data confine β_{price} more tightly.

Now search over two parameters jointly (intercept and price, memory held at truth), which also lets us *see* a log-likelihood surface — worth a long look, because the algorithms of Chapter 6 live on surfaces like this one:

```
grid <- expand.grid(b0 = seq(0, 2, length.out = 80),
                  bp = seq(-2.4, -0.2, length.out = 80))
grid$ll <- mapply(function(b0, bp)
  loglik_binary(c(b0, bp, 0.6), y, X), grid$b0, grid$bp)

best <- grid[which.max(grid$ll), ]

ggplot(grid, aes(b0, bp, z = ll)) +
  geom_contour(bins = 25, color = "grey50") +
  annotate("point", x = 1.0, y = -1.2, size = 2) +
```

```

annotate("point", x = best$b0, y = best$bp, shape = 4, size = 3, stroke = 1.2) +
labs(x = expression(beta[0]), y = expression(beta[price]))

```

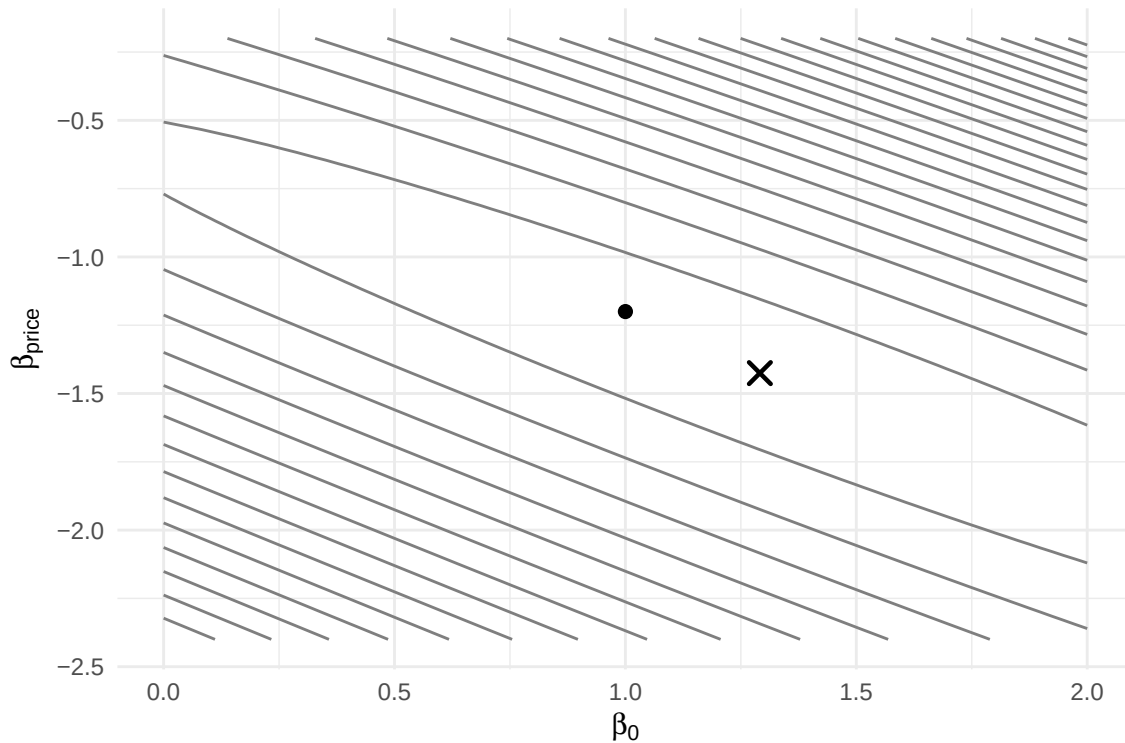


Figure 5.2: Log-likelihood contours over the intercept and price coefficient. The X marks the grid maximizer; the dot marks the truth. The elongated, tilted contours mean the two parameters are partially confounded in these data: raising the intercept while steepening price sensitivity leaves the data almost as probable.

```
best
```

```

      b0      bp      ll
2852 1.291139 -1.425316 -302.2199

```

Recovery, again: the grid maximizer lands near (1.0, -1.2). Notice the *shape* of the contours — elongated and tilted, not circular. The tilt says the data partially confound the two parameters (a higher baseline buying propensity plus stronger price aversion mimics the original pair), and its algebraic signature is a large off-diagonal element in H^{-1} : correlated estimation errors. Surfaces like this are easy terrain; Chapter 13 will show you surfaces with ridges and plateaus where the confounding is severe, and reading contour geometry will pay off.

5.8 Why Grid Search Fails at Scale

So why isn't this book twenty pages long? Count evaluations. A grid with 100 points per axis costs 100^K evaluations for K parameters:

Table 5.1: The curse of dimensionality for grid search.

	K	evaluations at 100 points/axis
	1	100
	2	10,000
	3	10^6
	6 (our MNL, Chapter 7)	10^{12}
	27 (mixed MNL with full covariance, Chapter 13)	10^{54}

At a million evaluations per second, the six-parameter MNL takes eleven days; the mixed MNL needs 10^{48} seconds — about thirty orders of magnitude longer than the age of the universe. And a 100-point grid is *coarse* — refining it multiplies the cost further. Exhaustive search dies exponentially in K .

But our two figures whisper the escape route. These surfaces are smooth and hill-shaped, so the local slope — the score, Equation 5.8, which we can *compute* — always points uphill. A climber who consults the gradient needs no grid: from any starting point, walk uphill until the slope vanishes. Making that idea precise, fast, and robust is the business of numerical optimization, and it is where we turn next.

5.9 Key Learnings

- The likelihood is the model's probability formula read in reverse: data fixed, parameters free (Equation 5.1). It scores every candidate β by how probable it renders the observed choices.
- Independence of the unobservables across choice situations — visible as one line in our simulator — makes the likelihood a product (Equation 5.2); the 0/1 exponent trick (Equation 5.3) selects each observation's realized outcome.
- Products of thousands of probabilities underflow to zero; the log-likelihood (Equation 5.4) converts them to benign sums with the same maximizer. Code computes ℓ , always.
- `loglik_binary()` is Equation 5.5 transcribed line by line: utility, probability, switch, accumulate — and it ranked truth above pretenders on simulated data, with the all-zero candidate scoring the null value $N \log(0.5)$.

- The MLE (Equation 5.6) is consistent, asymptotically normal, efficient, and invariant. The score (Equation 5.7) vanishes at the maximum — for the binary logit it is the elegant $\mathbf{X}'(\mathbf{y} - \mathbf{P})$ — and the inverse negative Hessian (Equation 5.10) converts peak curvature into standard errors.
- Grid search completed our first simulate-and-recover loop and drew us two instructive pictures — but it costs 100^K evaluations and dies exponentially in dimension (Table 5.1). Smooth, gradient-bearing surfaces invite hill-climbing instead: Chapter 6.

6 Numerical Optimization

Chapter 5 left us standing at the base of a hill. The log-likelihood $\ell(\boldsymbol{\beta})$ is a surface over the parameter space; the MLE is its summit; and grid search — visiting every location and comparing altitudes — dies exponentially as the space grows dimensions. This chapter builds the climber. The methods here are not specific to choice models; they are the general machinery for maximizing smooth functions, and mastering them pays off every time you write down a new model with a computable likelihood — which is to say, for the rest of this book and the rest of your research life.

A note on ambition before we start. R ships with excellent optimizers, and by the chapter’s end we will happily hand our work to `optim()`. But we first build Newton’s method by hand, for the same reason this book simulates before it estimates: optimizers are the least inspectable link in the estimation chain — data goes in, numbers come out, and when something goes wrong (“convergence code 1”?) the black box offers no comfort. Twenty lines of your own Newton iteration make the box transparent forever.

6.1 Climbing a Hill You Cannot See

Picture standing on the log-likelihood surface in fog. You cannot see the summit; you can only *feel the ground at your feet* — evaluate ℓ , and its derivatives, at your current position. Every practical method is a loop built from that local information:

1. From the current position $\boldsymbol{\beta}^{(i)}$, choose a **direction** $\mathbf{d}_i$.
2. Choose a **step size** λ_i .
3. Move: $\boldsymbol{\beta}^{(i+1)} = \boldsymbol{\beta}^{(i)} + \lambda_i \mathbf{d}_i$. Repeat until no direction goes up.

(The parenthesized superscript counts iterations, just as $\varepsilon^{(r)}$ counted draws in Chapter 1; subscripts on $\boldsymbol{\beta}$ stay reserved for its elements, β_k .)

The score $s(\boldsymbol{\beta})$ — the gradient from Equation 5.7 — feels the slope, and pointing straight uphill ($\mathbf{d}_i = s(\boldsymbol{\beta}^{(i)})$, *steepest ascent*) seems natural. It is also famously inefficient: on the elongated, tilted contours we saw in Figure 5.2, steepest ascent zigzags across the narrow axis of the valley, inching along its length. The cure is to consult not just the slope but the *curvature*, and that idea has a name.

6.2 Newton–Raphson

Approximate the surface near the current position by its second-order Taylor expansion — a quadratic bowl (inverted) with the same slope and curvature:

$$\ell(\boldsymbol{\beta}) \approx \ell(\boldsymbol{\beta}^{(i)}) + s_i'(\boldsymbol{\beta} - \boldsymbol{\beta}^{(i)}) + \frac{1}{2}(\boldsymbol{\beta} - \boldsymbol{\beta}^{(i)})' H_i (\boldsymbol{\beta} - \boldsymbol{\beta}^{(i)}) \quad (6.1)$$

where s_i and H_i are the score and Hessian at $\boldsymbol{\beta}^{(i)}$. A quadratic we can maximize *exactly*: setting the derivative $s_i + H_i(\boldsymbol{\beta} - \boldsymbol{\beta}^{(i)})$ to zero gives the **Newton–Raphson step**

$$\boldsymbol{\beta}^{(i+1)} = \boldsymbol{\beta}^{(i)} - H_i^{-1} s_i \quad (6.2)$$

Jump directly to the top of the local quadratic model, re-fit the model there, jump again. Where the surface really is quadratic, one step lands on the summit; where it is merely smooth, convergence near the optimum is *quadratic* — roughly, the number of correct digits doubles per iteration. Compare Equation 6.2 with steepest ascent: multiplying the score by $-H_i^{-1}$ rotates and rescales the uphill direction, exactly correcting for the tilted, elongated contours that make steepest ascent zigzag. And notice a happy economy: $-H^{-1}$ evaluated at the *last* step is exactly the covariance estimate Equation 5.10 requires (up to the sign, the algorithm’s machinery and the inference machinery are the same matrix).

To implement Newton for the binary logit, we need its score and Hessian analytically. The score we derived as Equation 5.8: $s(\boldsymbol{\beta}) = \mathbf{X}'(\mathbf{y} - \mathbf{P})$. One more differentiation — only $\mathbf{P}$ depends on $\boldsymbol{\beta}$, and $\partial P_n / \partial \boldsymbol{\beta} = P_n(1 - P_n)\mathbf{x}_n$ — gives

$$H(\boldsymbol{\beta}) = - \sum_{n=1}^N P_n(1 - P_n) \mathbf{x}_n \mathbf{x}_n' = -\mathbf{X}' W \mathbf{X} \quad (6.3)$$

with $W = \text{diag}(P_n(1 - P_n))$. Note H is negative definite everywhere — the binary logit’s log-likelihood is globally concave, one peak and no traps, which is part of why it makes such a good training ground.¹

Here is the whole algorithm; the loop body is Equation 6.2 verbatim:

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

newton_binary <- function(beta0, y, X, tol = 1e-10, max_iter = 25) {
  beta <- beta0
  path <- list(beta)
  for (i in 1:max_iter) {
    P <- plogis(as.vector(X %*% beta))      # P_n for all n
    s <- crossprod(X, y - P)                # score: X'(y - P)
    H <- -crossprod(X * (P * (1 - P)), X)   # hessian: -X' W X
    step <- solve(H, s)                     # H^{-1} s without inverting
    beta <- beta - as.vector(step)
    path[[i + 1]] <- beta
    if (max(abs(step)) < tol) break
  }
  list(beta = beta, hessian = H, iterations = i,
       path = do.call(rbind, path))
}

```

Three implementation notes. `plogis()` computes the logistic CDF — i.e., our choice probability — more stably than $\exp(v)/(1+\exp(v))$. `crossprod(X, v)` is $\mathbf{X}'v$ done efficiently, and multiplying $\mathbf{X}$ by the weight vector row-wise builds $\mathbf{X}'\mathbf{W}\mathbf{X}$ without materializing the $N \times N$ diagonal matrix — a Chapter 9 habit arriving early. And `solve(H, s)` computes $H^{-1}s$ by solving the linear system directly, which is both faster and numerically sounder than forming the inverse.

Simulate data (the Section 3.4 generator once more) and release the climber from a deliberately poor start:

```

sim_binary_data <- function(N, beta, X = NULL,
                           dist = c("logistic", "normal")) {
  dist <- match.arg(dist)
  if (is.null(X)) {
    price <- runif(N, min = 0.8, max = 2.4)
    ram16 <- rbinom(N, size = 1, prob = 0.5)
    X <- cbind(1, price, ram16)
  }
  eps <- switch(dist, logistic = rlogis(N), normal = rnorm(N))
  U <- as.vector(X %*% beta) + eps
  data.frame(y = as.integer(U > 0), X[, -1, drop = FALSE])
}

set.seed(60)
beta_true <- c(1.0, -1.2, 0.6)
d <- sim_binary_data(N = 500, beta = beta_true)

```

```

y <- d$y
X <- cbind(1, d$price, d$ram16)

fit_nr <- newton_binary(beta0 = c(0, 0, 0), y = y, X = X)
fit_nr$iterations

```

```
[1] 5
```

```
round(fit_nr$path, 4)
```

```

      [,1]      [,2]      [,3]
[1,] 0.0000  0.0000  0.0000
[2,] 1.3357 -1.2783  0.5092
[3,] 1.5124 -1.4536  0.5929
[4,] 1.5198 -1.4610  0.5966
[5,] 1.5198 -1.4610  0.5967
[6,] 1.5198 -1.4610  0.5967

```

Six iterations from the origin to ten-digit convergence, and the path table shows the promised acceleration: the first step covers most of the distance, and the final steps change digits far beyond anything of statistical interest. The quadratic-convergence signature — correct digits roughly doubling — is visible down the columns. Seen on the contour map:

```

loglik_binary_vec <- function(beta, y, X) { # vectorized; loop version in @sec-likelihood
  v <- as.vector(X %*% beta)
  sum(y*v - log1p(exp(v)))
}
grid <- expand.grid(b0 = seq(-0.5, 2, length.out = 80),
                  bp = seq(-2.4, 0.4, length.out = 80))
grid$ll <- mapply(function(b0, bp)
  loglik_binary_vec(c(b0, bp, fit_nr$beta[3]), y, X), grid$b0, grid$bp)

path <- data.frame(b0 = fit_nr$path[, 1], bp = fit_nr$path[, 2])
ggplot(grid, aes(b0, bp)) +
  geom_contour(aes(z = ll), bins = 30, color = "grey60") +
  geom_path(data = path, arrow = arrow(length = unit(0.2, "cm"),
                                     type = "closed")) +
  geom_point(data = path, size = 1.5) +
  labs(x = expression(beta[0]), y = expression(beta[price]))

```

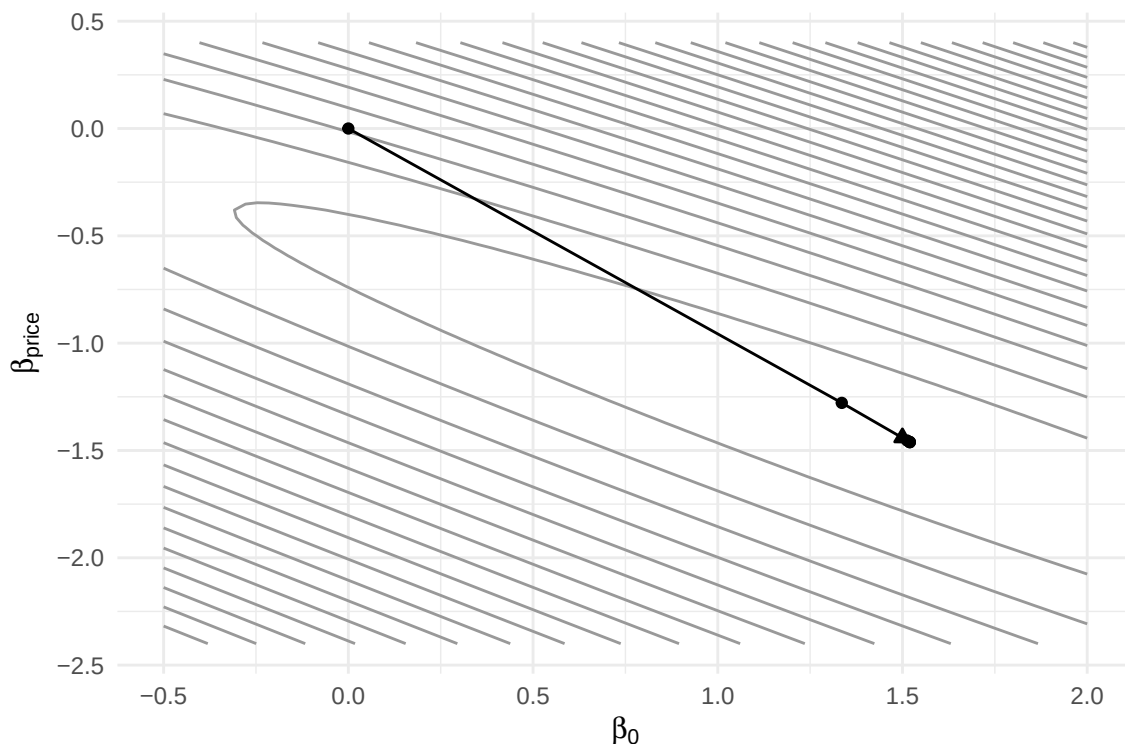


Figure 6.1: Newton-Raphson’s path from the origin across the log-likelihood contours (intercept and price coefficient shown; the memory coefficient also updates each step). Each jump maximizes the local quadratic model of the surface.

(The figure’s helper sneaks in the vectorized log-likelihood, because plotting 6,400 grid points with Chapter 5’s loop version tries one’s patience. Why is $\mathbf{y}*\mathbf{v} - \text{log1p}(\exp(\mathbf{v}))$ the same function? With $v_n = \mathbf{x}'_n\boldsymbol{\beta}$: $\log P_n = v_n - \log(1 + e^{v_n})$ and $\log(1 - P_n) = -\log(1 + e^{v_n})$, so $y_n \log P_n + (1 - y_n) \log(1 - P_n) = y_n v_n - \log(1 + e^{v_n})$. The rewrite avoids computing P_n at all, and $\text{log1p}()$ evaluates $\log(1 + e^v)$ accurately when e^v is tiny — the full stability story waits for Chapter 9.)

6.3 The Quasi-Newton Family: BHHH and BFGS

Newton’s appetite is the Hessian: $K(K + 1)/2$ second derivatives, re-derived analytically for every new model. For the binary logit that was a pleasure; for the mixed logit of Chapter 13 it is a punishment. The *quasi-Newton* methods keep Newton’s step structure but replace H with a cheaper stand-in, and two members of the family matter for us [DCMS Ch. 8 surveys more, including step-size strategies we gloss over; K. E. Train (2009)].

BHHH exploits a statistical identity: at the true parameters, the expected outer product of per-observation scores equals the information matrix (the negative expected Hessian). So sum the outer products of the observation-level score contributions — quantities the score calculation already produces. Write $s_n(\beta)$ for observation n 's contribution to the score, so that $s(\beta) = \sum_n s_n(\beta)$ — for the binary logit, $s_n(\beta) = (y_n - P_n)\mathbf{x}_n$, the individual terms of Equation 5.8. BHHH uses

$$\hat{H}_{\text{BHHH}} = - \sum_{n=1}^N s_n(\beta) s_n(\beta)' \quad (6.4)$$

No second derivatives at all, and the approximation is guaranteed negative (semi-)definite, so the step always points uphill. The catch: the identity holds *at the truth, in expectation, for a correctly specified model* — far from the optimum or under misspecification, BHHH's picture of the curvature can be poor and its steps clumsy.

BFGS, the default workhorse of numerical optimization, builds its Hessian substitute from the optimization's own history: each iteration's change in the gradient, compared against the step taken, reveals one slice of curvature, and BFGS accumulates these slices into a running approximation — start with something crude (the identity matrix), let every step teach you a little more geometry. It needs only gradients, tolerates imperfect ones, and converges nearly as fast as true Newton in practice. When we call `optim()` below, BFGS is what we ask for.

6.4 Numerical Derivatives

Quasi-Newton methods still want the *gradient*, and deriving even first derivatives analytically becomes real work as models grow. The fallback is numerical differentiation: the definition of the derivative, executed with a small but finite h ,

$$\frac{\partial \ell}{\partial \beta_k} \approx \frac{\ell(\beta + h \mathbf{e}_k) - \ell(\beta - h \mathbf{e}_k)}{2h} \quad (6.5)$$

where $\mathbf{e}_k$ is the k -th standard basis vector — all zeros except a 1 in position k , so the perturbation nudges one coordinate at a time (the *central* difference; the one-sided version is half as accurate for the same cost). (This h is the step size of numerical analysis, no relation to the behavioral rule h of Chapter 1.) Coding it is a five-line loop over coordinates:

```
num_grad <- function(fn, beta, h = 1e-6, ...) {
  g <- numeric(length(beta))
  for (k in seq_along(beta)) {
    e <- replace(numeric(length(beta)), k, h)
    g[k] <- (fn(beta + e, ...) - fn(beta - e, ...)) / (2*h)
  }
}
```

```

    }
    g
}

```

Choosing h is a genuine tension, not a formality: too large and the secant misrepresents the slope (truncation error); too small and you subtract two nearly equal numbers, amplifying floating-point noise (roundoff error). The engineering compromise near $h \approx 10^{-6}$ works for well-scaled problems, but “well-scaled” is doing real work in that sentence — one more reason parameter scaling (below) matters. Hess and Daly (2024)’s optimization chapter (Bunch) treats these pitfalls with the care they deserve.

Numerical gradients also settle a debt. In Chapter 5 I asserted the elegant score formula $s(\beta) = \mathbf{X}'(\mathbf{y} - \mathbf{P})$; we can now audit the algebra:

```

beta_test <- c(0.5, -0.8, 0.2)
P <- plogis(as.vector(X %*% beta_test))
analytic <- as.vector(crossprod(X, y - P))
numeric <- num_grad(loglik_binary_vec, beta_test, y = y, X = X)
rbind(analytic, numeric)

```

```

      [,1]      [,2]      [,3]
analytic 21.62871 19.85474 20.87288
numeric  21.62871 19.85474 20.87288

```

Agreement to six decimals. Make this check a reflex: *every* analytic gradient in this book — and every one you ever derive — gets verified against Equation 6.5 before it is trusted. It is the cheapest insurance in computational statistics; a silent sign error in a gradient produces optimizers that limp, wander, or confidently stop at nonsense, and none of those symptoms names its cause.

6.5 R’s Optimizers: `optim()`

With the concepts in hand, we can read R’s general-purpose optimizer fluently. `optim()` wants a starting value, a function, optionally a gradient, and a method. Two conventions need deciding once:

- *Minimize or maximize?* `optim()` minimizes by default. Some authors therefore write negative log-likelihoods everywhere; I find that a standing invitation to sign errors. We keep writing log-likelihoods — the statistically meaningful object — and pass `control = list(fnscale = -1)`, which tells `optim()` to maximize.

- Which method? `method = "BFGS"` for the reasons above. (Derivative-free Nelder–Mead, the default, is a robust fallback when gradients are treacherous; we will want it exactly once, and I will flag it.)

```
fit <- optim(par      = c(0, 0, 0),
            fn        = loglik_binary_vec,
            y = y, X = X,
            method     = "BFGS",
            control     = list(fnscale = -1),
            hessian     = TRUE)
fit$par
```

```
[1] 1.5197645 -1.4610247 0.5966551
```

```
fit$convergence
```

```
[1] 0
```

Reading the output: `par` is $\hat{\beta}$; value is $\ell(\hat{\beta})$; `convergence` of 0 means the algorithm believes it finished (anything else is a warning label, not a result²); and `hessian = TRUE` asks for a numerically differentiated Hessian at the solution — our inference input. Compare the two climbers:

```
rbind(newton = fit_nr$beta, bfgs = fit$par, truth = beta_true)
```

	[,1]	[,2]	[,3]
newton	1.519774	-1.461030	0.5966501
bfgs	1.519764	-1.461025	0.5966551
truth	1.000000	-1.200000	0.6000000

Identical to four decimals (Newton and BFGS found the same summit — there is only one) and near the truth (recovery, once again). We did not supply a gradient, so BFGS differentiated numerically behind the scenes; passing our analytic `gr` speeds things up and tightens convergence, and we will do so for the MNL in Chapter 8.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

6.6 Standard Errors from the Hessian

The statistical payoff. Equation 5.10 said the estimated covariance of $\hat{\beta}$ is the inverse negative Hessian at the peak; `optim()` just handed us that Hessian:

```
vcov_hat <- solve(-fit$hessian)
se <- sqrt(diag(vcov_hat))

results <- data.frame(
  truth      = beta_true,
  estimate   = fit$par,
  se         = se,
  z          = fit$par / se,
  ci_lo      = fit$par - 1.96*se,
  ci_hi      = fit$par + 1.96*se
)
rownames(results) <- c("intercept", "price", "ram16")
round(results, 3)
```

	truth	estimate	se	z	ci_lo	ci_hi
intercept	1.0	1.520	0.360	4.225	0.815	2.225
price	-1.2	-1.461	0.220	-6.636	-1.893	-1.029
ram16	0.6	0.597	0.197	3.033	0.211	0.982

This little table is the archetype of estimation output, and every column traces to something we built: estimates from the climber, standard errors from the curvature, z -statistics testing “is this coefficient distinguishable from zero,” and 95% confidence intervals — each of which, satisfyingly, covers its true value. (Any single interval fails with 5% probability, of course; Chapter 8 runs the replication study that checks the coverage *rate*.) The price coefficient’s interval is the widest relative to its magnitude — look back at Figure 5.2 and its elongated contours to see why: that is the flat direction of the surface, rendered as a number.

Two cautions before we move on. The Hessian route to standard errors assumes the model is correctly specified (true in our simulations by construction; an act of faith with real data — robust “sandwich” alternatives exist, see Greene (2008)). And it is *asymptotic*: at small N , normality of $\hat{\beta}$ is an approximation whose quality is, again, checkable by simulation — a theme Chapter 8 takes up in earnest.

6.7 Practical Realities

Textbook optimization ends at the previous section. Real optimization involves four recurring frictions, and knowing them in advance converts hours of confusion into minutes of diagnosis.

Starting values. Every climber starts somewhere, and bad starts cost iterations or worse (a flat-region start can stall numerical gradients entirely). Cheap heuristics go far: zeros are serviceable for logit-family models; estimates from a simpler relative are better — we will start the MNL from zeros, but the mixed logit of Chapter 13 from *the plain MNL’s estimates*, and the HB sampler of Chapter 17 likewise. A model hierarchy is also a starting-value hierarchy.

Scaling. Optimizers work best when parameters live on comparable scales; a gradient step calibrated for a coefficient of size 1 misbehaves for one of size 1,000. We engineered this away in Section 1.4 by measuring price in \$1,000s — making all true coefficients order-one — and that was not an aesthetic choice. If you meet a dataset with price in dollars, *rescale before optimizing*, then translate the estimate back.

Local maxima. The binary logit’s concavity guarantees one summit, but the guarantee expires later in this book: latent class likelihoods (Chapter 11) are riddled with local maxima, and mixed logit surfaces (Chapter 13) can undulate. The standard defense is disciplined brute force: launch from several dispersed starting points and compare the summits reached. Same summit everywhere: reassuring. Different summits: report the best, and know your surface is treacherous.

Believing convergence too easily. A zero convergence code means the stopping rule triggered — steps got small — not that the maximum was found. Corroborate: is the gradient actually near zero? Is the Hessian negative definite (all eigenvalues negative)? Does a restart from the “solution” stay put? Thirty seconds of paranoia, applied routinely, catches the rare failures that matter.

6.8 The Complete Loop, End to End

Part I’s tools are now all on the bench, and it is worth savoring the moment the loop closes cleanly. The whole simulate-and-recover cycle, in one breath:

```
set.seed(61)
d2 <- sim_binary_data(N = 2000, beta = c(1.0, -1.2, 0.6)) # simulate
X2 <- cbind(1, d2$price, d2$ram16)
fit2 <- optim(c(0,0,0), loglik_binary_vec, y = d2$y, X = X2, # estimate
             method = "BFGS", control = list(fnscale = -1),
             hessian = TRUE)
se2 <- sqrt(diag(solve(-fit2$hessian)))
round(data.frame(truth = c(1.0, -1.2, 0.6), # recover?
                 estimate = fit2$par, se = se2), 3)
```

	truth	estimate	se
1	1.0	1.280	0.177

2	-1.2	-1.265	0.107
3	0.6	0.451	0.096

Simulate at known truth; hand the estimator *only* the data; watch the truth come back, bracketed by honest uncertainty. Note the standard errors relative to the $N = 500$ run — quadrupling the sample roughly halved them, the $\sqrt{N}$ law making its second appearance (it governed simulation error in Chapter 1; it governs estimation error here).

This loop is the book’s heartbeat. Everything that follows elaborates one of its stages: richer models to simulate (Chapter 7, Chapter 13), faster likelihood evaluation (Chapter 9), likelihoods that themselves require simulation (Chapter 12), and eventually a different philosophy of the “estimate” step entirely (Chapter 15) — where, I promise, the log-likelihood functions and the paranoid checking habits built here transfer without modification.

6.9 Key Learnings

- Optimization is climbing in fog: direction and step size from local information, repeated. Consulting curvature, not just slope, is what makes climbing fast on tilted terrain.
- Newton–Raphson (Equation 6.2) maximizes a local quadratic model per step: for the binary logit, with score $\mathbf{X}'(\mathbf{y} - \mathbf{P})$ and Hessian $-\mathbf{X}'\mathbf{W}\mathbf{X}$ (Equation 6.3), it reaches ten-digit accuracy in six steps — and its Hessian doubles as the standard-error engine.
- BHHH substitutes the outer product of scores for the Hessian (Equation 6.4, valid near the truth under correct specification); BFGS learns curvature from the optimization’s own history and is our default method in `optim()` (`fnscale = -1` to maximize; nonzero convergence codes mean *stop and investigate*).
- Numerical gradients (Equation 6.5) are the universal fallback and the universal *audit*: every analytic gradient gets checked against them, always.
- Standard errors are peak curvature inverted: `solve(-hessian)`, diagonal, square root. Flat directions of the surface (Figure 5.2) become the large standard errors in the table.
- The practical frictions — starting values, scaling, local maxima, over-trusting convergence — have standard defenses; we will apply the multiple-starts discipline in Chapter 11, where local maxima stop being hypothetical.
- The simulate-and-recover loop closed: truth in, truth out, with uncertainty that shrinks at the $\sqrt{N}$ rate. Part I is complete.

Part II

The MNL Family, by Likelihood

7 The Multinomial Logit Model

Part I practiced on binary choices — buy the laptop or don’t. Real choice sets are richer: the shopper in our study faces *three* laptops per task, and the shopper in a store faces dozens. This chapter introduces the model that dominates the analysis of such data, in academic research and industry practice alike: the **multinomial logit** (MNL). By the chapter’s end, the full laptop study of Section 1.4 will exist as data on your machine — simulated from the MNL with known parameters, organized by the tools of Chapter 4, and waiting for the estimator we build in Chapter 8.

The MNL earned its dominance honestly. Its choice probabilities have a closed form — the exceptional case, as Chapter 1 taught — which makes estimation fast and stable. Its parameters read naturally as marginal utilities. And it is the seed from which everything else in this book grows: the nested logit (Chapter 10), the latent class model (Chapter 11), the mixed logit (Chapter 13), and the hierarchical Bayesian models (Chapter 17) are all the MNL with one assumption relaxed. Learn this model deeply and the rest of the book is variations on a theme.

7.1 From Two Alternatives to Many

The random utility construction of Section 1.3 generalizes without friction. Decision-maker n in choice situation t faces J alternatives; each alternative j carries a utility

$$U_{ntj} = V_{ntj} + \varepsilon_{ntj}, \quad V_{ntj} = \mathbf{x}_{ntj}'\boldsymbol{\beta} \quad (7.1)$$

where $\mathbf{x}_{ntj}$ is the attribute vector of alternative j in that situation — one row of the long-format data of Chapter 4 — and ε_{ntj} collects everything about that alternative, that person, and that moment that we do not observe. The behavioral rule is maximization:

$$y_{nt} = \arg \max_{j \in \{1, \dots, J\}} U_{ntj} \quad (7.2)$$

The observed choice reveals which utility was largest — nothing more. That “nothing more” has teeth, and the identification section below counts them.

For the choice *probability*, the general recipe of Equation 1.1 applies: $P_{ntj} = \Pr(U_{ntj} > U_{ntk} \forall k \neq j)$, an integral of the density of ε_{nt} over the region where alternative j wins. What remains is to choose that density.

7.2 Random Utility and the Extreme Value Distribution

The MNL’s defining assumption is that the unobservables are independent draws from the Gumbel (Type I Extreme Value) distribution you met in Chapter 2:

$$\varepsilon_{ntj} \stackrel{iid}{\sim} \text{Gumbel}, \quad F(\varepsilon) = e^{-e^{-\varepsilon}} \quad (7.3)$$

Why this distribution, of all the shapes a density can take? Candor first: primarily because of what it *delivers* — with Gumbel errors, the choice-probability integral solves in closed form. The result (derived in two pages in K. E. Train 2009, sec. 3.10) is the most famous formula in discrete choice:

$$P_{ntj} = \frac{e^{V_{ntj}}}{\sum_{j'=1}^J e^{V_{ntj'}}} \quad (7.4)$$

Exponentiate every representative utility, and each alternative’s probability is its share of the total. (Machine learning readers know this map as the *softmax*.) The formula respects everything a choice probability must — positive, summing to one, increasing in own-utility — and it is differentiable, cheap, and impossible to get numerically wrong in any interesting way. McFadden’s derivation of this form from utility maximization, and the empirical program it launched, is the subject of his Nobel lecture (McFadden 2001).

Two properties of the Gumbel family make the magic work, both of which we can *verify* rather than take on faith — the Chapter 2 toolkit was built for exactly this. First, the difference of two independent Gumbels is logistic (verified in Figure 2.5), which is why the $J = 2$ case of Equation 7.4 collapses to the binary logit of Section 1.3 — reassuring consistency. Second, the *maximum* of several Gumbel variables is itself Gumbel, shifted by the log of the sum of exponentiated locations — the property that makes the “which alternative wins” calculation tractable, and whose byproduct, the *log-sum* term $\log \sum_{j'} e^{V_{j'}}$, will return twice with star billing: as the nested logit’s inclusive value (Chapter 10) and as the consumer-surplus measure behind willingness-to-pay calculations (Chapter 21).

The second property is a three-line pleasure to check rather than take on faith. If $U_j = V_j + \varepsilon_j$ with ε_j independent standard Gumbel, then each U_j has CDF $e^{-e^{-(u-V_j)}}$, and by independence

$$\Pr\left(\max_j U_j \leq u\right) = \prod_j e^{-e^{-(u-V_j)}} = \exp\left(-e^{-u} \sum_j e^{V_j}\right) = \exp\left(-e^{-(u-\log \sum_j e^{V_j})}\right),$$

which is again a Gumbel CDF, with location $\log \sum_j e^{V_j}$ — the log-sum, born.

Rather than rely on the algebra, let's watch Equation 7.4 emerge from raw simulation — the strongest sanity check we know (Section 3.5, level 3). Take one choice task from the laptop study: three profiles with representative utilities we fix by hand. Simulate the model's *mechanics* — draw Gumbels, add, take the argmax — ten thousand times, and compare winning frequencies to the formula:

```
set.seed(70)
V <- c(0.4, 1.1, -0.2)          # representative utilities of 3 profiles
R <- 10000

wins <- integer(3)
for (r in 1:R) {
  eps <- -log(-log(runif(3)))    # Gumbel draws, from first principles
  U   <- V + eps
  j   <- which.max(U)
  wins[j] <- wins[j] + 1
}

rbind(simulated = wins / R,
      formula   = exp(V) / sum(exp(V)))
```

```
          [,1]      [,2]      [,3]
simulated 0.2797000 0.5669000 0.1534000
formula   0.2806967 0.5652537 0.1540496
```

Agreement to two-and-a-bit decimals, tightening with more draws. The closed form and the behavioral story — maximize noisy utilities — are the same model; Equation 7.4 is just the integral pre-computed for us.

7.3 The MNL Choice Probability in R

The formula deserves a function. Following the stacked-data conventions of Section 4.7 — utilities for all alternatives in a vector, tasks identified by a group index — here it is for a single choice situation:

```
mnl_probs <- function(V) exp(V) / sum(exp(V))
mnl_probs(c(0.4, 1.1, -0.2))
```

```
[1] 0.2806967 0.5652537 0.1540496
```

One line. It hides one hazard: if some V is large — say 800, easy to produce with mis-scaled data — `exp(800)` overflows to `Inf` and the probabilities dissolve into `NaN`. The cure (subtract the maximum V before exponentiating, which changes nothing mathematically because the shift cancels in numerator and denominator) is the *log-sum-exp trick*, and we fold it into our likelihood code in Chapter 8 and dissect it in Chapter 9. Flagged now so it never surprises you.

7.4 Properties: What the MNL Assumes About Substitution

A model this convenient must be paying for it somewhere. The MNL’s bill arrives as a strong, sometimes wrong, restriction on *how alternatives compete*.

7.4.1 Independence from irrelevant alternatives

Take the ratio of any two alternatives’ probabilities under Equation 7.4:

$$\frac{P_{ntj}}{P_{ntk}} = \frac{e^{V_{ntj}}}{e^{V_{ntk}}} = e^{V_{ntj} - V_{ntk}} \quad (7.5)$$

Everything about every *other* alternative has canceled. The odds between the Apple and the Dell depend only on the Apple and the Dell — not on what else is in the task, how similar it is to either, or whether it exists at all. This property is **independence from irrelevant alternatives** (IIA), and it is a direct inheritance of assuming the ε ’s independent: the model has no channel through which alternatives can share unobserved appeal.

Sometimes IIA is harmless, even welcome. Sometimes it is absurd, and the canonical absurdity is worth internalizing in numbers.¹ A commuter chooses between a car (V_c) and a blue bus (V_b), with $V_c = V_b$, so the model says 50/50. Now add a *red* bus — identical to the blue bus in every respect except paint, so $V_r = V_b$. Common sense: bus riders split over paint colors, and shares go to $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$. The MNL, constrained by Equation 7.5 to keep the car/blue-bus odds at 1:1, says $\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$ — painting a bus red steals a sixth of the car market:

```
V2 <- c(car = 0, blue_bus = 0)
V3 <- c(car = 0, blue_bus = 0, red_bus = 0)
list(before = mnl_probs(V2), after = mnl_probs(V3))
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

$before
      car blue_bus
0.5      0.5

$after
      car blue_bus red_bus
0.3333333 0.3333333 0.3333333

```

The diagnosis is precise: the two buses share nearly *all* their unobserved utility (comfort with buses, no parking hassle), and independence of the ε 's is exactly the assumption that forbids the model from knowing it. The repairs form a tour of this book's second half: let the ε 's correlate within groups (nested logit, Chapter 10), let coefficients vary across people so “bus people” exist (mixed logit, Chapter 13 — which breaks IIA in aggregate while keeping logit form for each individual), or let the errors correlate freely (probit, Chapter 18).

7.4.2 Proportional substitution

IIA's flip side. Differentiate Equation 7.4 with respect to the utility of a *different* alternative j : for $k \neq j$, $\partial P_k / \partial V_j = -e^{V_k} e^{V_j} / (\sum_l e^{V_l})^2 = -P_k P_j$. If attribute z_j enters utility with coefficient β_z , the chain rule gives a cross-elasticity of $\frac{\partial P_k}{\partial z_j} \frac{z_j}{P_k} = -\beta_z z_j P_j$ — an expression with no k in it: when alternative j improves, *every* competitor's share falls by the same percentage (K. E. Train 2009, sec. 3.6). That is proportional substitution. In our study: cut the Apple's price and the model takes customers from Dell and Acer in proportion to their shares, never disproportionately from the *other premium option*. For the laptop study as designed this is defensible; for a study with two near-identical Dell configurations it would not be. Chapter 21 computes these elasticities explicitly, and Chapter 10 begins the repairs.

7.5 Identification: What Choices Can and Cannot Reveal

Before simulating data, we must be honest about what the data can possibly tell us. Equation 7.2 says choices reveal only which utility was *largest*. Two consequences follow, both previewed in Part I, both now precise. They are not pedantry: each corresponds to a specific way estimation code fails — parameters wandering without changing the likelihood — and each dictates a normalization our simulator and estimator must share, or recovery checks will “fail” mysteriously.

Only differences in utility matter. Add any constant c to *every* alternative's utility in a task and the argmax — hence the choice, hence the probability (check it against Equation 7.4: e^c cancels) — is untouched. Absolute utility levels are unobservable. This is why the long-format data of Chapter 4 carry no common intercept column and why individual-specific variables

need interactions to matter (Chapter 4 made both points structurally; here is their behavioral root). It is also why *labeled* designs include ASCs for $J - 1$ alternatives, never all J : the last one's level is the reference against which differences are measured.

The scale of utility is arbitrary. Multiply *all* utilities — systematic parts and errors alike — by any $\alpha > 0$ and the argmax is again untouched. So the data cannot pin down the error variance separately from the magnitude of β ; something must be fixed by convention, and the MNL convention is to fix the Gumbel errors at their standard variance, $\pi^2/6$. Every coefficient is therefore measured *relative to the fixed scale of the Gumbel noise* (whose standard deviation is $\pi/\sqrt{6} \approx 1.28$) — $\beta_{\text{price}} = -1.2$ means “a \$1,000 price increase moves utility by 1.2 units on a scale where the unobserved noise has standard deviation about 1.3,” not any absolute quantity. Three practical corollaries, in ascending order of how often they bite researchers:

1. Coefficients from models with different error scales are not comparable — the binary probit's β (errors scaled to variance 1) and binary logit's β (variance $\pi^2/3$) differ by a factor of about 1.8 *for the same behavior*, resolving the puzzle flagged in Chapter 3's footnote.
2. Coefficients estimated on two *datasets* can differ purely because choice consistency differs — noisier respondents shrink every coefficient toward zero together. Comparing across samples requires care (Chapter 21).
3. *Ratios* of coefficients are scale-free — α cancels — which is precisely why willingness-to-pay, a ratio with the price coefficient in the denominator, is the preferred currency for interpreting estimates (Chapter 21).

Neither normalization is a limitation of the MNL; both are limitations of *choice data*, and they will reappear verbatim in the probit (Chapter 18), where getting them wrong is the classic implementation failure.

7.6 Simulating the Laptop Study

Assembly time. We have the design generator (`sim_laptop_design()`, Section 4.6), the data organizer (`build_choice_data()`, Section 4.7), the error generator (the Gumbel one-liner, Chapter 2), and the behavioral rule (Equation 7.2). The full true parameter vector for the study, extending Section 1.4's binary seed to all six attributes — and now fixed for the rest of the book:

$$\beta^* = (\underbrace{0.5}_{\text{dell}}, \underbrace{1.0}_{\text{apple}}, \underbrace{0.6}_{\text{ram16}}, \underbrace{0.9}_{\text{ram32}}, \underbrace{0.3}_{\text{screen15}}, \underbrace{-1.2}_{\text{price}}) \quad (7.6)$$

Reasonable preferences: brands ordered Acer $\prec$ Dell $\prec$ Apple, more memory better with diminishing returns, mild taste for the larger screen, and the familiar price distaste. The

simulator packages the six-step anatomy of Chapter 3, with the argmax-within-task as the one genuinely new move:

```
sim_laptop_design <- function(N, T, J) {          # from @sec-data
  n_rows <- N * T * J
  data.frame(
    id    = rep(seq_len(N), each = T * J),
    task  = rep(rep(seq_len(T), each = J), times = N),
    alt   = rep(seq_len(J), times = N * T),
    brand = factor(sample(c("Acer","Dell","Apple"), n_rows, replace = TRUE),
                    levels = c("Acer","Dell","Apple")),
    ram   = factor(sample(c("8","16","32"), n_rows, replace = TRUE),
                    levels = c("8","16","32")),
    screen = factor(sample(c("13","15"), n_rows, replace = TRUE),
                     levels = c("13","15")),
    price = round(runif(n_rows, min = 0.8, max = 2.4), 2)
  )
}

sim_mnl_data <- function(N, T, J, beta) {
  df <- sim_laptop_design(N, T, J)
  X <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]
  stopifnot(ncol(X) == length(beta))

  task_id <- cumsum(!duplicated(df[, c("id", "task")]))
  eps <- -log(-log(runif(nrow(df))))          # Gumbel, one per utility
  U <- as.vector(X %*% beta) + eps

  df$choice <- as.integer(ave(U, task_id,
                              FUN = function(u) u == max(u)))

  df
}

set.seed(71)
beta_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6,
               ram32 = 0.9, screen15 = 0.3, price = -1.2)
laptops <- sim_mnl_data(N = 500, T = 8, J = 3, beta = beta_true)
head(laptops, 3)
```

	id	task	alt	brand	ram	screen	price	choice
1	1	1	1	Apple	32	13	1.21	1
2	1	1	2	Apple	16	13	1.14	0

```
3 1 1 3 Dell 16 13 2.27 0
```

The `ave(U, task_id, FUN = ...)` idiom deserves a sentence, since it is how the `argmax` respects the data's group structure: within each `task_id` group it flags the row achieving the group maximum, returning the flags in the original row order — Equation 7.2 executed group-wise on stacked data, no loop required.²

Now the checking discipline of Section 3.5 — more important than ever, since these data underwrite the next seven chapters. Marginal shares first: with brands randomly assigned to profiles, brand choice shares should order as the brand coefficients do:

```
chosen <- laptops[laptops$choice == 1, ]
round(prop.table(table(chosen$brand)), 3)
```

```
Acer Dell Apple
0.240 0.336 0.424
```

Acer $\prec$ Dell $\prec$ Apple, as (0, 0.5, 1.0) commands. Directional price check: chosen laptops should be cheaper on average than rejected ones —

```
tapply(laptops$price, laptops$choice, mean)
```

```
0 1
1.675466 1.470368
```

— and they are, by about \$205. Finally the level-3 check, empirical frequencies against the closed form. We bin tasks by the model-implied probability of their first alternative, then ask how often that alternative was actually chosen — a *calibration* check that the simulation honors Equation 7.4 across its whole range:

```
X <- model.matrix(~ brand + ram + screen + price, data = laptops)[, -1]
task_id <- cumsum(!duplicated(laptops[, c("id", "task")]))

eV <- exp(as.vector(X %*% beta_true))
P <- eV / rowsum(eV, task_id)[task_id] # eq-mnl-prob, all rows at once
```

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

bins <- cut(P, breaks = seq(0, 1, by = 0.1))
cal <- data.frame(
  implied = tapply(P, bins, mean),
  observed = tapply(laptops$choice, bins, mean))

ggplot(cal, aes(implied, observed)) +
  geom_abline(linetype = "dotted") +
  geom_point() +
  coord_equal(xlim = c(0, 1), ylim = c(0, 1)) +
  labs(x = "model-implied probability", y = "empirical frequency")

```

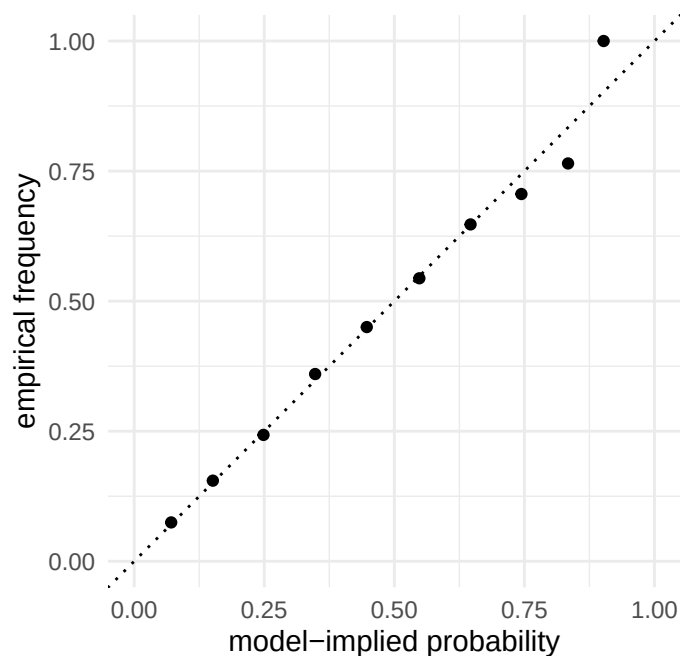


Figure 7.1: Calibration of the simulated laptop study: model-implied choice probabilities (x) against empirical choice frequencies (y), tasks binned by implied probability. Points on the 45-degree line mean the simulator implements the MNL it claims to.

Dead on the diagonal. Two moves in that chunk foreshadow the next chapter: `rowsum(eV, task_id)` computed all 4,000 tasks' logit denominators in one grouped operation, and indexing the result by `task_id` broadcast them back to the rows — the exact computational pattern at the heart of the vectorized MNL log-likelihood. You have now effectively seen Chapter 8's inner loop.

The study is fielded. One housekeeping step: since every remaining chapter of Part II (and several beyond) analyzes exactly this dataset, we save it — data and answer key together — so

later chapters can load it rather than re-simulate it. One dataset, one truth, one thread of examples:

```
dir.create("../data", showWarnings = FALSE)
saveRDS(list(data = laptops, beta_true = beta_true),
        file = "../data/laptop_study.rds")
```

In the drawer sit `laptops` — 4,000 choices by 500 respondents — and, because we are the simulator as well as the analyst, the answer key Equation 7.6. Time to forget we know it.

7.7 Key Learnings

- The MNL assigns Gumbel errors to linear-in-parameters utilities (Equation 7.1, Equation 7.3); maximization then yields the closed-form softmax probabilities of Equation 7.4 — which we verified by simulating the model’s raw mechanics, not by trusting algebra.
- The Gumbel’s gifts: differences are logistic (binary logit as the $J = 2$ special case) and maxima are Gumbel (source of the log-sum term that returns in Chapter 10 and Chapter 21).
- The price of the closed form is IIA (Equation 7.5): odds between two alternatives ignore all others, so substitution is proportional to shares — absurd for near-clones (red bus/blue bus) and the motivation for the nested, mixed, and probit models ahead.
- Choices reveal only the argmax, so utility *levels* (add a constant: nothing) and utility *scale* (multiply by α : nothing) are unidentifiable. Coefficients are measured on the arbitrary, convention-fixed logit scale; only their ratios are scale-free — the seed of willingness-to-pay.
- The laptop study now exists: `sim_mnl_data()` composes the design (Chapter 4), Gumbel draws (Chapter 2), and group-wise argmax into 4,000 simulated choices at β^* (Equation 7.6), and the data passed marginal, directional, and calibration checks.
- The calibration figure’s `rowsum()`/broadcast pattern *is* the vectorized MNL likelihood computation — Chapter 8 makes it official.

8 Estimating the MNL by Maximum Likelihood

This is the chapter Part I was preparing you for. We have data — the simulated laptop study of Section 7.6, 500 respondents making 4,000 choices — and we have every tool the estimation requires: the likelihood principle (Chapter 5), the stacked data representation (Chapter 4), and the numerical climber (Chapter 6). What remains is assembly, and assembly is where the details live: the exact form of the MNL log-likelihood, its gradient, a numerical trap and its escape, and the reporting and validation rituals that turn an optimizer’s output into research you can defend.

Start by loading the study — data and answer key — saved at the end of Chapter 7:

```
study      <- readRDS("../data/laptop_study.rds")
laptops    <- study$data
beta_true  <- study$beta_true

X <- model.matrix(~ brand + ram + screen + price, data = laptops)[, -1]
y <- laptops$choice
task_id <- cumsum(!duplicated(laptops[, c("id", "task")]))
K <- ncol(X)
c(rows = nrow(X), tasks = max(task_id), K = K)
```

```
rows tasks      K
12000  4000      6
```

(The three lines rebuilding `X`, `y`, and `task_id` are `build_choice_data()` from Section 4.7 written out flat, so this chapter’s code is visible end to end.)

8.1 The MNL Log-Likelihood

The recipe of Chapter 5: the log-likelihood is the sum, over choice situations, of the log probability of the alternative actually chosen. With the switch-exponent device of Equation 5.3 generalized — let $y_{ntj} = 1$ if alternative j was chosen in situation (n, t) and 0 otherwise — the indicator encoding of Equation 7.2’s choice, and exactly the `choice` column of our long-format data — and the MNL probabilities of Equation 7.4 plugged in:

$$\ell(\beta) = \sum_{n=1}^N \sum_{t=1}^T \sum_{j=1}^J y_{ntj} \log P_{ntj} = \sum_{n,t} \sum_j y_{ntj} \left[V_{ntj} - \log \sum_{j'=1}^J e^{V_{ntj'}} \right] \quad (8.1)$$

Because exactly one y_{ntj} per situation equals one, the inner sum simply *selects the chosen row*, and the whole expression collapses to something you can say in words: **the log-likelihood is the sum, over choice situations, of the chosen alternative’s utility minus the log-sum of all alternatives’ exponentiated utilities.**

$$\ell(\beta) = \sum_{n,t} \left[V_{nt,\text{chosen}} - \log \sum_{j'=1}^J e^{V_{ntj'}} \right] \quad (8.2)$$

Every term is a competition: how good was what they picked, against the log-sum measure of everything they could have picked. A candidate β scores well when it makes chosen alternatives look systematically attractive *relative to their own choice sets* — not in absolute terms, which, per Section 7.5, the data could never reveal anyway.

8.2 Coding loglik_mnl()

Following our transparent-first discipline, the loop version walks tasks one at a time — for each, extract its rows, compute utilities, apply Equation 8.2:

```
loglik_mnl_loop <- function(beta, y, X, task_id) {
  ll <- 0
  for (i in unique(task_id)) {
    rows <- which(task_id == i)
    V <- X[rows, , drop = FALSE] %*% beta
    ll <- ll + V[y[rows] == 1] - log(sum(exp(V)))
  }
  as.vector(ll)
}
```

Readable, correct, and — as Chapter 9 will measure precisely — slow, because R pays interpreter overhead 4,000 times per likelihood evaluation, and the optimizer will request many evaluations. The vectorized version eliminates the loop using exactly the grouped-computation pattern rehearsed in Figure 7.1: compute all utilities in one matrix product, form all task-level log-sums with one `rowsum()` call, and broadcast them back to the rows by indexing:

```
loglik_mnl <- function(beta, y, X, task_id) {
  V      <- as.vector(X %*% beta)      # all NTJ utilities at once
  Vmax   <- ave(V, task_id, FUN = max)  # per-task max (stability; see below)
  eV     <- exp(V - Vmax)
  denom  <- rowsum(eV, task_id)        # per-task sums, one grouped op
  sum( (V - Vmax - log(denom[task_id]))[y == 1] )
}
```

Line by line against Equation 8.2: V is every V_{ntj} ; `denom[task_id]` puts each task's $\sum_{j'} e^{V_{ntj'}}$ next to each of its rows (indexing a vector by a group label — the broadcast idiom); the bracketed expression is every alternative's log probability; and `[y == 1]` selects the chosen rows before summing. The `Vmax` lines are new, and they are the subject of the next section — ignore them for one moment and confirm the two implementations agree:

```
c(loop      = loglik_mnl_loop(beta_true, y, X, task_id),
  vectorized = loglik_mnl(beta_true, y, X, task_id))
```

```
      loop vectorized
-3774.99   -3774.99
```

Identical. One function, two implementations, same mathematics — keep the loop version as executable documentation, run the vectorized one.

8.3 The Log-Sum-Exp Trick

Now the `Vmax` business. The naive computation of $\log \sum_{j'} e^{V_{j'}}$ evaluates `exp()` first, and `exp()` is an accident waiting to happen: `exp(710)` overflows to `Inf`, and `exp(-746)` underflows to 0. Our well-scaled laptop data keep utilities around ± 3 , but recall the scaling discussion of Chapter 6: suppose a colleague hands you the same data with price *in dollars*, so the price coefficient should come out near -0.0012. En route, the optimizer will surely try β values like “-1.2 on dollars,” producing utilities near -1800:

```
V_bad <- c(-1795, -1801, -1808)      # utilities under a mis-scaled candidate
log(sum(exp(V_bad)))                # exp underflows to 0; log(0) = -Inf
```

```
[1] -Inf
```

An infinite log-likelihood poisons the optimizer — gradients become NaN, BFGS’s curvature updates corrupt, and the failure surfaces far from its cause. The escape is an identity: for any constant m ,

$$\log \sum_{j'} e^{V_{j'}} = m + \log \sum_{j'} e^{V_{j'} - m} \quad (8.3)$$

(factor e^m out of the sum). Choose $m = \max_{j'} V_{j'}$: then every exponent is ≤ 0 , the largest term is exactly $e^0 = 1$, and neither overflow nor total underflow can occur — the sum always lies in $[1, J]$:

```
m <- max(V_bad)
m + log(sum(exp(V_bad - m)))      # finite, correct
```

```
[1] -1794.998
```

That is what `Vmax` does in `loglik_mnl()`, computed per task by `ave()`. The cost is two cheap vectorized operations; the benefit is a likelihood that cannot blow up anywhere the optimizer wanders. Write it into every softmax-shaped computation you ever produce — including, later, the probability weights inside MSL (Chapter 12) and the Metropolis acceptance ratios of Chapter 15, both of which fail in exactly this way when coded naively.

8.4 The Analytic Gradient

BFGS will run on numerical gradients, but at $2K$ extra likelihood evaluations per gradient (Equation 6.5 — two per parameter), analytic derivatives repay their derivation quickly — and for the MNL the derivation lands somewhere memorable. Differentiate one task’s contribution to Equation 8.2. The chosen alternative’s utility is linear, so it contributes $\partial V_{ntj} / \partial \beta = \mathbf{x}_{ntj}$. For the log-sum term, the chain rule gives

$$\frac{\partial}{\partial \beta} \log \sum_{j'} e^{V_{ntj'}} = \frac{\sum_{j'} e^{V_{ntj'}} \mathbf{x}_{ntj'}}{\sum_{j'} e^{V_{ntj'}}} = \sum_{j'} P_{ntj'} \mathbf{x}_{ntj'},$$

the probability-weighted average of the task’s attribute rows — the model’s *expected* $\mathbf{x}$. So each task contributes $\mathbf{x}_{nt, \text{chosen}} - \sum_j P_{ntj} \mathbf{x}_{ntj}$: attributes of what they chose, minus attributes of what the model expected them to choose. Writing the chosen row with the indicators, $\mathbf{x}_{nt, \text{chosen}} = \sum_j y_{ntj} \mathbf{x}_{ntj}$, and merging the two sums row-wise — stacking the indicators into $\mathbf{y}$ and the probabilities into $\mathbf{p}$, both length- NTJ vectors in the row order of $\mathbf{X}$:

$$s(\beta) = \sum_{n,t,j} (y_{ntj} - P_{ntj}) \mathbf{x}_{ntj} = \mathbf{X}'(\mathbf{y} - \mathbf{p}) \quad (8.4)$$

Identical in form to the binary logit score Equation 5.8: prediction errors times covariates, now with one error per alternative rather than per observation. At the MLE, the model's predicted choice shares are exactly uncorrelated with every attribute. The implementation reuses the likelihood's plumbing — and note this means the stability fix comes along free, since the probabilities are computed with the same shifted exponentials:

```
grad_mnl <- function(beta, y, X, task_id) {
  V      <- as.vector(X %*% beta)
  Vmax   <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
  p      <- eV / rowsum(eV, task_id)[task_id] # P_ntj for every row
  as.vector(crossprod(X, y - p))
}
```

Audit against the numerical gradient (`num_grad()` from Chapter 6), per standing policy — *every analytic gradient gets checked*:

```
num_grad <- function(fn, beta, h = 1e-6, ...) {
  g <- numeric(length(beta))
  for (k in seq_along(beta)) {
    e <- replace(numeric(length(beta)), k, h)
    g[k] <- (fn(beta + e, ...) - fn(beta - e, ...)) / (2*h)
  }
  g
}

beta_test <- rep(0.1, K)
rbind(analytic = grad_mnl(beta_test, y, X, task_id),
      numeric = num_grad(loglik_mnl, beta_test, y = y, X = X,
                        task_id = task_id))
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
analytic	-40.47418	338.8	66.72434	245.1336	143.6079	-605.3551
numeric	-40.47418	338.8	66.72434	245.1336	143.6079	-605.3551

Six-decimal agreement. The algebra is certified; use the fast one.

8.5 Estimation

All pieces on the board — objective, gradient, climber. From a zero start (adequate here, per Chapter 6: the MNL log-likelihood is globally concave, so there is exactly one summit and no wrong basins):

```
fit <- optim(par      = rep(0, K),
            fn        = loglik_mnl,
            gr         = grad_mnl,
            y = y, X = X, task_id = task_id,
            method     = "BFGS",
            control    = list(fnscale = -1),
            hessian    = TRUE)
fit$convergence
```

```
[1] 0
```

Converged. Standard errors from the curvature (Equation 5.10), and the full results table against the answer key:

```
se <- sqrt(diag(solve(-fit$hessian)))
results <- data.frame(
  truth      = beta_true,
  estimate   = fit$par,
  se         = se,
  z          = fit$par / se,
  ci_lo      = fit$par - 1.96*se,
  ci_hi      = fit$par + 1.96*se
)
rownames(results) <- colnames(X)
round(results, 3)
```

	truth	estimate	se	z	ci_lo	ci_hi
brandDell	0.5	0.478	0.054	8.939	0.373	0.583
brandApple	1.0	0.940	0.054	17.473	0.835	1.046
ram16	0.6	0.638	0.053	11.931	0.533	0.743
ram32	0.9	0.886	0.054	16.415	0.780	0.992
screen15	0.3	0.371	0.043	8.715	0.288	0.455
price	-1.2	-1.140	0.048	-23.516	-1.235	-1.045

Study this table for a moment; it is the first full-scale recovery in the book. Every estimate lands within a hair of its true value, every 95% interval covers the truth, and every $|z|$ is enormous — with 4,000 choices, effects of this size are unmissable. The two memory coefficients illustrate invariance-in-action from Chapter 5: their *difference* (the marginal value of going from 16GB to 32GB, $0.9 - 0.6 = 0.3$ in truth) is estimated by the difference of their estimates, 0.248. And the interpretation discipline of Section 7.5 applies: these are utilities on the arbitrary logit scale; the number a manager can use — dollars per attribute — waits for the WTP machinery of Chapter 21, which divides through by the price coefficient precisely to cancel the arbitrary scale.

8.6 Assessing Fit

An estimate without a fit assessment is a number without a context. Choice modelers lean on a small stable of measures, all built from log-likelihood values, all cheap now that estimation has run.

The null log-likelihood. The zero-parameter benchmark: every alternative equally likely, $P = 1/J$ always. Because our unlabeled design has no ASCs, $\beta = \mathbf{0}$ is the null model,¹ so $\ell_0 = \sum_{n,t} \log(1/3) = -4000 \log 3$:

```
ll_null <- -max(task_id) * log(3)
c(ll_null = ll_null, ll_fit = fit$value)
```

```
ll_null    ll_fit
-4394.449 -3771.621
```

The likelihood-ratio test. Twice the log-likelihood gap, $2(\ell_1 - \ell_0)$ — where $\ell_1 = \ell(\hat{\beta})$ is the maximized log-likelihood of the fit — is χ^2 with (here) 6 degrees of freedom under the null. Ours is 1246 — the null is not merely rejected, it is vaporized; unsurprising, since we simulated the alternatives to matter.

McFadden's ρ^2 . The likelihood analogue of R^2 :

$$\rho^2 = 1 - \frac{\ell_1}{\ell_0} \quad (8.5)$$

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```
rho2 <- 1 - fit$value / ll_null
round(rho2, 3)
```

```
[1] 0.142
```

Calibrate your expectations: ρ^2 runs on a compressed scale, and values of 0.2–0.4 represent excellent fit for choice data Eggers et al. (2022). Ours, around 0.14, is exactly what a strong-but-noisy choice process yields — remember, the model’s own error term guarantees choices are not deterministic even with β known perfectly.

Hit rate. The blunt-instrument check: how often is the modeled-most-probable alternative the chosen one?

```
V <- as.vector(X %*% fit$par)
top <- ave(V, task_id, FUN = max) == V
mean(tapply(top & y == 1, task_id, any))
```

```
[1] 0.55125
```

Against a random-guess baseline of 1/3. Hit rates ignore probability calibration entirely (a 0.34 favorite and a 0.99 favorite count the same), so treat them as a communication device, not a model selection criterion.

AIC and BIC. For comparing *non-nested* specifications, penalized fit: $\text{AIC} = -2\ell + 2K$, $\text{BIC} = -2\ell + K \log(\text{observations})$. They earn their keep when the candidate models multiply — choosing the number of latent classes in Chapter 11 is where BIC becomes load-bearing for us.

8.7 Validation Against mlogit

Our estimator agrees with the truth, but “agrees with the truth on data we simulated” and “correctly implements maximum likelihood” are subtly different claims. The sharper test: an independent implementation, same data, same model — estimates should agree to numerical precision, not merely statistically. The `mlogit` package (Croissant 2020) is the R standard; feeding it requires only the index bookkeeping promised in Chapter 4 — a unique choice-situation `id`, a logical choice column, and `dfidx()`’s declaration of the panel structure:

```
library(dfidx)
library(mlogit)

ml_data <- laptops
ml_data$chid <- task_id
ml_data$choice <- as.logical(ml_data$choice)
ml_data <- dfidx(ml_data, idx = list(c("chid", "id"), "alt"))

fit_ml <- mlogit(choice ~ brand + ram + screen + price | 0, data = ml_data)
```

(In `mlogit`'s three-part formula, alternative-varying attributes with generic coefficients occupy the first slot; the `| 0` suppresses the ASCs that our unlabeled design must not have, per Section 4.5.) The confrontation:

```
comparison <- rbind(ours = fit$par, mlogit = coef(fit_ml))
colnames(comparison) <- colnames(X)
round(comparison, 5)
```

	brandDell	brandApple	ram16	ram32	screen15	price
ours	0.47833	0.94018	0.63804	0.88593	0.37141	-1.13974
mlogit	0.47833	0.94017	0.63805	0.88594	0.37141	-1.13974

```
round(rbind(ours = se, mlogit = sqrt(diag(vcov(fit_ml)))), 5)
```

	brandDell	brandApple	ram16	ram32	screen15	price
ours	0.05351	0.05381	0.05348	0.05397	0.04262	0.04847
mlogit	0.05351	0.05381	0.05348	0.05397	0.04262	0.04847

```
c(ours = fit$value, mlogit = as.numeric(logLik(fit_ml)))
```

	ours	mlogit
	-3771.621	-3771.621

Agreement to five decimals in coefficients, standard errors, and the log-likelihood itself. Two entirely separate codebases — ours, thirty transparent lines; theirs, a mature package — climbed to the same summit of the same surface. This is the validation ritual every estimator in this book will undergo where a reference implementation exists, and the divergence-hunting skills it builds transfer to the day one *disagrees*: coefficients matching but SEs differing (covariance estimator conventions), likelihoods differing by a constant (dropped combinatorial terms), estimates differing utterly (data-format misalignment — check the index construction first).

8.8 A Simulate-and-Recover Study

One recovery is an anecdote. The classical guarantees of Chapter 5 — consistency, asymptotic normality, honest standard errors — are statements about the *distribution* of $\hat{\beta}$ across repeated samples, and simulation lets us observe that distribution directly: rerun the entire pipeline — fresh study, fresh estimation — two hundred times and watch the estimator’s sampling behavior, not just one draw from it.

```
sim_and_fit <- function(N, T, J, beta) {  
  df <- sim_mnl_data(N, T, J, beta)  
  Xr <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]  
  tid <- cumsum(!duplicated(df[, c("id", "task")]))  
  f <- optim(rep(0, length(beta)), loglik_mnl, gr = grad_mnl,  
            y = df$choice, X = Xr, task_id = tid,  
            method = "BFGS", control = list(fnscale = -1),  
            hessian = TRUE)  
  list(est = f$par, se = sqrt(diag(solve(-f$hessian))))  
}
```

```
set.seed(80)  
reps <- 200  
runs <- replicate(reps, sim_and_fit(500, 8, 3, beta_true),  
                  simplify = FALSE)  
est_mat <- t(sapply(runs, `[`, "est"))  
se_mat <- t(sapply(runs, `[`, "se"))  
colnames(est_mat) <- colnames(se_mat) <- colnames(X)
```

(An unshown chunk re-defines the Chapter 7 simulator functions this study calls; they are unchanged.) Three questions, three answers.

Is the estimator centered on the truth? Compare the mean estimate across replications to β^* :

```
round(rbind(truth = beta_true, mean_estimate = colMeans(est_mat)), 3)
```

	dell	apple	ram16	ram32	screen15	price
truth	0.500	1.000	0.600	0.900	0.300	-1.200
mean_estimate	0.498	0.997	0.596	0.895	0.302	-1.207

Bias indistinguishable from zero, coefficient by coefficient — consistency visibly at work at our sample size.

Is the sampling distribution normal, and are the standard errors honest? Overlay the 200 price-coefficient estimates with the normal density that the *average estimated standard error* implies:

```
ggplot(data.frame(b = est_mat[, "price"]), aes(b)) +
  geom_histogram(aes(y = after_stat(density)), bins = 25, fill = "grey70") +
  geom_function(fun = dnorm,
               args = list(mean = -1.2, sd = mean(se_mat[, "price"])),
               linewidth = 0.8) +
  geom_vline(xintercept = -1.2, linetype = "dashed") +
  labs(x = expression(hat(beta)[price]), y = "density")
```

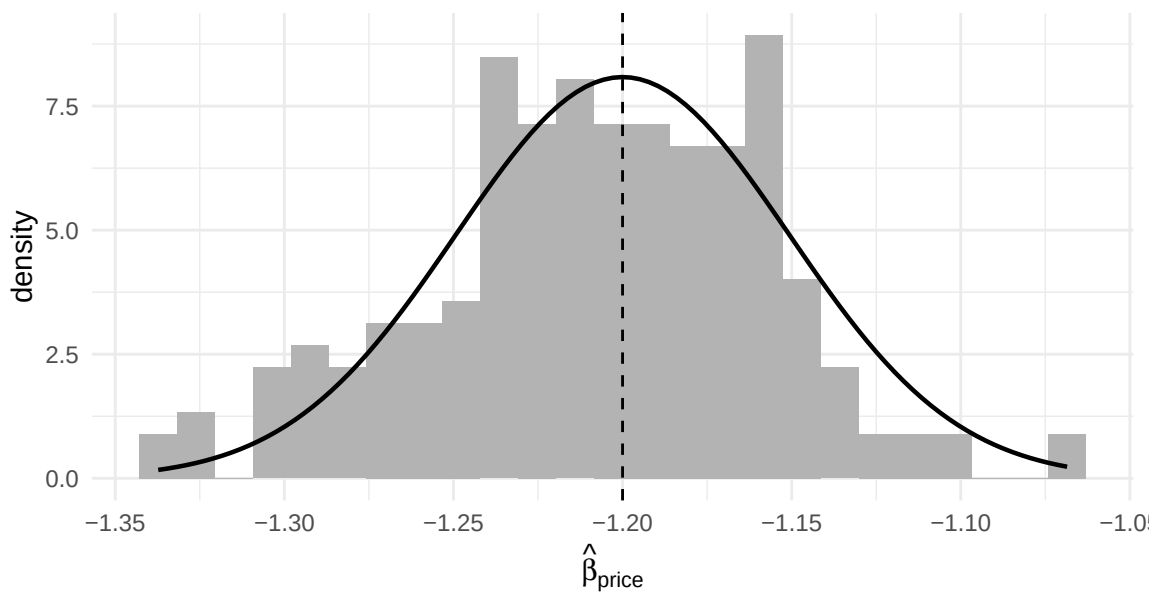


Figure 8.1: Sampling distribution of the price coefficient across 200 simulate-and-recover replications, with the normal density implied by the average estimated standard error. Asymptotic theory, audited and confirmed at 4,000 choice situations per replication.

The histogram is the *actual* sampling distribution; the curve is what the Hessian-based theory of Equation 5.10 *claims* it should be. They match — which means the standard error printed in our results table is measuring exactly what it purports to measure. The numerical check: the empirical spread of estimates against the average claimed SE, all coefficients —

```
round(rbind(empirical_sd = apply(est_mat, 2, sd),
            mean_est_se = colMeans(se_mat)), 4)
```

	brandDell	brandApple	ram16	ram32	screen15	price
empirical_sd	0.0536	0.0509	0.0557	0.0555	0.0380	0.0505
mean_est_se	0.0540	0.0539	0.0539	0.0539	0.0426	0.0493

Do 95% intervals cover 95% of the time?

```
covered <- abs(sweep(est_mat, 2, beta_true)) <= 1.96 * se_mat
round(colMeans(covered), 3)
```

brandDell	brandApple	ram16	ram32	screen15	price
0.950	0.965	0.930	0.960	0.955	0.930

All within Monte Carlo noise of 0.95 (with 200 replications, the coverage estimate itself has an SE of about $\sqrt{0.95 \times 0.05/200} \approx 0.015$). The full classical package — centering, normality, honest uncertainty — verified by direct observation rather than trusted from a theorem. This replication audit is cheap insurance we will purchase again at moments of higher doubt: when simulation error enters the likelihood itself (Chapter 12) and when we need to compare frequentist and Bayesian machinery on equal footing (Chapter 17).

8.9 Key Learnings

- The MNL log-likelihood is “chosen utility minus log-sum, summed over tasks” (Equation 8.2). Its vectorized implementation is three grouped operations — matrix product, `rowsum()`, broadcast-by-indexing — on the stacked data of Chapter 4.
- `exp()` overflows and underflows; the log-sum-exp identity (Equation 8.3) with $m = \max_j V_j$ makes the likelihood unconditionally safe, at negligible cost. Build it in by reflex, here and in every softmax you ever write.
- The MNL score is $\mathbf{X}'(\mathbf{y} - \mathbf{p})$ (Equation 8.4) — the binary logit’s “errors times covariates” form, unchanged. Derived, then audited against numerical differentiation, per standing policy.
- Estimation recovered Equation 7.6 with tight, honest intervals; global concavity made the zero start safe. Fit was summarized by null log-likelihood, LR test, McFadden’s ρ^2 (0.2–0.4 is excellent for choice data), hit rate, and AIC/BIC — all likelihood arithmetic.
- Validation against `mlogit` agreed to five decimals in coefficients, SEs, and log-likelihood: two independent implementations, one summit. Every estimator in this book faces this ritual.
- The 200-replication study observed the classical guarantees directly: unbiased centering, normal sampling distribution matching the claimed SEs, and 95% coverage. The MLE machine works — now we make it fast (Chapter 9) and point it at models that deserve the speed.

9 Code Performance

This chapter pauses the march of models to sharpen the tools. It exists because of a compounding arithmetic that is easy to state and easy to underestimate: *the log-likelihood function sits at the bottom of everything, and everything above it multiplies*. A slow likelihood is a minor annoyance in Chapter 8 and a fatal flaw by Chapter 17. The good news: for the models in this book, a handful of transferable techniques — measure first, vectorize the hot path, precompute the invariant, preallocate the storage — buy factors of hundreds, and we already own most of them.

9.1 Why Speed Matters Now

Count the likelihood evaluations implied by the chapters ahead. One BFGS fit of the MNL costs on the order of a hundred evaluations (iterations times gradient arithmetic). The replication study of Section 8.8 multiplied that by 200. And the road ahead is steeper:

Table 9.1: What will call your likelihood, and how often.

Consumer	Evaluations of the (log-)likelihood kernel
One BFGS fit (Chapter 8)	$\sim 10^2$
Replication study, 200 runs (Section 8.8)	$\sim 10^4$
One MSL fit, $R = 200$ draws (Chapter 13)	$\sim 10^4\text{--}10^5$
One MCMC run, 10^4 iterations, per-person steps (Chapter 17)	$\sim 10^6$
Prior sensitivity: several MCMC runs (Chapter 17)	$\sim 10^7$

A likelihood evaluation costing one second makes the last row a four-month computation; at one millisecond, three hours. Every method in Parts II–IV was *designed* in an era when this arithmetic bound what research was possible, and it still bounds what your afternoon is possible. That is the entire motivation; now the craft.

9.2 Measure First

The cardinal rule of performance work: **never optimize what you have not measured**. Intuition about where time goes is famously unreliable, and effort spent speeding up a line that consumes 2% of runtime is effort donated to entropy. R's built-in stopwatch is `system.time()`; for the quick comparisons this chapter makes, we wrap it to report a median across repetitions (timings are noisy — garbage collection, OS scheduling — and medians resist the noise)¹:

```
time_ms <- function(expr, times = 5) {  
  e <- substitute(expr)  
  pf <- parent.frame()  
  t <- replicate(times, system.time(eval(e, pf))["elapsed"])  
  round(1000 * median(t), 1)  
}
```

For *profiling* — finding where a complex computation spends its time, function by function — R provides `Rprof()` and the friendlier `profvis` package; they matter once code has many moving parts, and I commend them to you for your own projects. Our computations are small enough that targeted timing tells the story.

The test subject is the laptop study likelihood from Chapter 8, with its data rebuilt as usual:

```
study <- readRDS("../data/laptop_study.rds")  
laptops <- study$data  
X <- model.matrix(~ brand + ram + screen + price, data = laptops)[, -1]  
y <- laptops$choice  
task_id <- cumsum(!duplicated(laptops[, c("id", "task")]))  
beta <- study$beta_true
```

9.3 Vectorizing the MNL Log-Likelihood

Chapter 8 wrote the likelihood twice and promised a measurement. Here are three implementations, spanning the design space:

Version 1 — the pure loop (from Chapter 8): visit each task, locate its rows with `which(task_id == i)`, compute locally.

¹A subtlety worth stealing: R evaluates function arguments lazily and caches the result, so a naive `replicate(times, system.time(expr))` would time the real computation once and a cached lookup thereafter — the classic way to convince yourself your code is infinitely fast. `substitute()` captures the unevaluated expression so each repetition genuinely recomputes. Measurement code deserves the same skepticism as model code.

```
loglik_v1 <- function(beta, y, X, task_id) {
  ll <- 0
  for (i in unique(task_id)) {
    rows <- which(task_id == i)
    V <- X[rows, , drop = FALSE] %*% beta
    ll <- ll + V[y[rows] == 1] - log(sum(exp(V)))
  }
  as.vector(ll)
}
```

Version 2 — the hybrid: compute all utilities in one matrix product (vectorized), but handle the per-task aggregation by splitting into a list and looping:

```
loglik_v2 <- function(beta, y, X, task_id) {
  V <- as.vector(X %*% beta)
  Vs <- split(V, task_id)
  ys <- split(y, task_id)
  ll <- 0
  for (i in seq_along(Vs)) {
    ll <- ll + Vs[[i]][ys[[i]] == 1] - log(sum(exp(Vs[[i]])))
  }
  ll
}
```

Version 3 — fully grouped (the Chapter 8 production version): no explicit loop at all; `rowsum()` performs every task's aggregation inside compiled code.

```
loglik_v3 <- function(beta, y, X, task_id) {
  V <- as.vector(X %*% beta)
  Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  denom <- rowsum(eV, task_id)
  sum( (V - Vmax - log(denom[task_id]))[y == 1] )
}
```

Same mathematics, same output (check it — always):

```
c(v1 = loglik_v1(beta, y, X, task_id),
  v2 = loglik_v2(beta, y, X, task_id),
  v3 = loglik_v3(beta, y, X, task_id))
```

v1	v2	v3
-3774.99	-3774.99	-3774.99

Now the stopwatch:

```
data.frame(
  version = c("v1 pure loop", "v2 hybrid", "v3 grouped"),
  ms = c(time_ms(loglik_v1(beta, y, X, task_id)),
        time_ms(loglik_v2(beta, y, X, task_id)),
        time_ms(loglik_v3(beta, y, X, task_id)))
)
```

	version	ms
1	v1 pure loop	470
2	v2 hybrid	10
3	v3 grouped	10

The numbers vary by machine but the *structure* does not: v3 beats v1 by two to three orders of magnitude. Understanding why is more valuable than the numbers. R is an interpreted language: each line of R costs interpreter overhead *per execution*, so 4,000 trips through a loop body pay that overhead 4,000 times — and v1 adds injury by scanning the entire `task_id` vector inside every iteration (`which(task_id == i)`), making its cost grow with the *square* of the data size. Vectorized calls — `X %*% beta`, `rowsum()`, arithmetic on whole vectors — pay the overhead *once* and do the actual work in compiled C loops. “Vectorization” is not magic; it is moving the loop from interpreted code into compiled code.²

Multiply the payoff through Table 9.1: at v1 speed, the HB sampler of Chapter 17 is an overnight-or-worse job; at v3 speed, minutes. Same computer, same mathematics.

9.4 Precompute What Doesn’t Change

A likelihood function gets called at hundreds of different β values — but the *data* never change between calls. Anything computable from data alone must be computed once, outside the optimizer, never inside the likelihood. Our code has quietly followed this rule — `model.matrix()` and `task_id` were always built before `optim()` was called — but the temptation to violate it arrives disguised as tidiness (“let the function take the raw data frame; it’s a nicer interface”):

²The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```
loglik_tidy_trap <- function(beta, df) {
  Xd <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]
  tid <- cumsum(!duplicated(df[, c("id", "task")]))
  loglik_v3(beta, df$choice, Xd, tid)
}

c(precomputed = time_ms(loglik_v3(beta, y, X, task_id)),
  rebuilds     = time_ms(loglik_tidy_trap(beta, laptops)))
```

```
precomputed    rebuilds
           20           60
```

The convenient interface rebuilds a 12,000-row model matrix on every one of the $\sim 10^6$ calls in Table 9.1. The design discipline: *functions called inside optimization loops take prepared numeric objects, full stop*. If you want the friendly interface, write a wrapper that prepares once and then loops — exactly the division of labor in `build_choice_data()` (Section 4.7) followed by `loglik_mnl()`. The same principle, one level up, will matter in Chapter 12: the random draws used by a simulated likelihood are also “data” in this sense, generated once and reused across all evaluations — there for statistical reasons as well as speed.

9.5 Preallocate; Don’t Grow

The third habit concerns memory, and its natural habitat is the MCMC chapters ahead, where samplers *store* tens of thousands of parameter draws. R objects are immutable under the hood: “appending” to a vector actually allocates a new, longer vector and copies everything across. Grow a vector inside a loop and you copy $1 + 2 + \dots + n$ elements — quadratic cost, invisible in the code’s appearance:

```
grow <- function(n) {
  out <- numeric(0)
  for (i in 1:n) out <- c(out, sqrt(i))
  out
}

prealloc <- function(n) {
  out <- numeric(n)
  for (i in 1:n) out[i] <- sqrt(i)
  out
}

c(grow_50k      = time_ms(grow(50000), times = 3),
  prealloc_50k  = time_ms(prealloc(50000), times = 3))
```

```
grow_50k prealloc_50k
7780      10
```

Identical output, painful ratio — and the ratio *worsens* as n grows. The rule writes itself: when the number of results is known (an MCMC run of S draws of K parameters), allocate the full container first — `draws <- matrix(NA_real_, nrow = S, ncol = K)` — and fill by index. Every sampler in Parts III–IV is built this way; when you meet `matrix(NA, S, K)` at the top of the Gibbs samplers in Chapter 16, this paragraph is why.

A related but distinct caution: R’s *copy-on-modify* semantics mean that modifying an object passed into a function, or touched by two names, can trigger a full copy. For our workloads this bites rarely — we mostly *read* $\mathbf{X}$ inside tight loops, which is free — but if you ever find memory ballooning in a sampler, the culprit is usually an innocent-looking modification of a large object inside a loop.

9.6 Numerical Hygiene Is Performance Too

Speed is worthless if the fast answer is NaN. Two habits from earlier chapters deserve re-statement as *performance* practices, because numerical failures cost more research time than slow code ever does:

- **The log-sum-exp guard** (Section 8.3) costs two vectorized operations and makes the MNL kernel safe at any β an optimizer or sampler will ever try. `v3` carries it; keep it.
- **Stable scalar forms.** The binary logit log-probability $y \log P + (1 - y) \log(1 - P)$ is prettier as mathematics than as computation; the algebraically equal form $yv - \log(1 + e^v)$ — where $v = \mathbf{x}'\beta$ is the binary model’s systematic utility — computed as `y*v - log1p(exp(v))`, avoids catastrophic cancellation for moderate v (that is the form the Chapter 6 contour helper used — promise kept). R’s `log1p()` and `expm1()` exist precisely for “log of one-plus-small” and “e-to-small-minus-one” situations; reach for them whenever a probability sits near 0 or 1.

The general principle: *transform to the scale where the computer’s arithmetic is happiest — almost always the log scale — and stay there as long as possible*. MCMC will hand us the same lesson from another angle: Metropolis acceptance ratios are computed as differences of log-posteriors, never ratios of posteriors, for exactly this reason (Chapter 15).

9.7 When R Isn’t Enough: A Glimpse Beyond

Sometimes the hot loop cannot be vectorized — its iterations depend on each other (the Gibbs samplers of Part III update sequentially by construction), or the group-wise logic outgrows `rowsum()`’s vocabulary. The escape hatch is to write the loop in C++ and call it from R via

Rcpp. The flavor, for the MNL kernel (display only — compiling C++ requires the Rtools toolchain on Windows):

```
Rcpp::cppFunction('
double loglik_mnl_cpp(NumericVector beta, IntegerVector y,
                      NumericMatrix X, IntegerVector task_id) {
    // one pass over rows: accumulate per-task max, log-sum, chosen V
    // ... a dozen lines of straightforward C++ ...
}')

```

Rcpp typically buys another factor of 2–20 over well-vectorized R — valuable, not transformative, precisely *because* good vectorized R already spends most of its time in compiled code. The production-grade example in our domain is the `mixl` package (Molloy et al. 2021), which compiles each model’s likelihood to C++ with OpenMP parallelism and estimates mixed logits on huge datasets; its design — specify the model at a high level, generate compiled code underneath — shows where the craft goes at industrial scale. For this book, we stay in plain R: our vectorized kernels keep every chapter’s computation in the minutes-not-hours regime, and transparency is the whole point of the exercise. But know the door exists, and know that everything on this chapter’s checklist — measure, vectorize, precompute, preallocate — applies unchanged on the other side of it.

9.8 Key Learnings

- The likelihood kernel is the bottom of a multiplication stack (Table 9.1): optimizers call it hundreds of times, MSL multiplies by draws, MCMC by tens of thousands of iterations. Kernel speed is research feasibility.
- Measure before optimizing — `system.time()` medians for comparisons, `profvis` for anatomy. Intuition lies.
- Vectorization moves loops from interpreted R into compiled code: the grouped MNL kernel (matrix product + `rowsum()` + broadcast) beats the per-task loop by orders of magnitude, and the hybrid version shows which part of the win comes from where.
- Precompute the invariant: model matrices, index vectors — and later, MSL draws — are built once outside the loop. Functions inside optimization loops take prepared numeric objects.
- Preallocate known-size results (`matrix(NA, S, K)` atop every sampler to come); growing objects in loops is quadratic copying.
- Numerical hygiene is performance: log-sum-exp guards, `log1p()/expm1()`, and staying on the log scale prevent the failures that cost real time. Rcpp is the escape hatch when an unvectorizable loop truly binds — `mixl` shows the industrial version — but transparent, vectorized R carries this entire book comfortably.

10 Relaxing IIA: Nested Logit and GEV Models

The MNL bought its closed form with the IIA property (Equation 7.5), and Chapter 7 showed the bill can be absurd: paint a bus red and steal car riders. This chapter makes the first — and gentlest — repair. The **nested logit** lets alternatives share unobserved appeal *within researcher-specified groups*, breaking IIA across groups while preserving a closed-form probability, a tractable likelihood, and everything we know about estimating such things. It is the smallest possible step beyond the MNL, which makes it the ideal place to learn how model generalizations propagate through the estimation machinery: a new parameter, a new likelihood, the same workflow.

The chapter also marks a small narrative shift in the running example, explained below, and introduces a technique we will use for the rest of the book: simulating data *directly from choice probabilities* when utility-plus-error simulation is awkward.

10.1 When Proportional Substitution Fails

Recall the diagnosis from Chapter 7: the MNL’s independent errors give it no way to know that two alternatives are similar in unobserved ways, so improvement in one alternative draws share from all others *in proportion to their shares* — never disproportionately from its close substitutes.

Put that in laptop terms. Suppose unobserved utility has a large operating-system component: some shoppers, on some occasions, are simply in a macOS mood or locked into Windows workflows we don’t measure. Then a Dell and an Acer (both Windows) share unobserved appeal that neither shares with an Apple. If the Dell cuts its price, common sense says it should poach *mostly Windows shoppers* — Acer’s customers — and pry away few committed Mac buyers. The MNL is structurally incapable of that asymmetry; the nested logit is built for it.

10.2 The Nested Logit Model

Partition the alternatives in a choice situation into M **nests** $B_1, \dots, B_M$. Our study: $B_{\text{win}} = \{\text{Acer}, \text{Dell}\}$, $B_{\text{mac}} = \{\text{Apple}\}$ (a nest may be a singleton). Utility keeps its form $U_{ntj} = V_{ntj} + \varepsilon_{ntj}$, but the joint distribution of the errors is generalized so that errors *within* a nest are

correlated while errors *across* nests remain independent. The degree of within-nest correlation is governed by the **dissimilarity parameter** $\lambda_m \in (0, 1]$: the within-nest error correlation is approximately $1 - \lambda_m^2$ — a standard approximation we take from the literature rather than derive, since the exact correlation has no simple closed form (K. E. Train 2009, sec. 4.2) — so

- $\lambda_m = 1$: no correlation — the nest is cosmetic, and the model collapses to the MNL;
- $\lambda_m \rightarrow 0$: perfect correlation — alternatives in the nest are near-clones on unobservables, and competition within the nest becomes ferocious while the nest as a whole competes with others as a unit.

Values outside $(0, 1]$ break consistency with random utility maximization,¹ a fact with practical bite: an estimated $\hat{\lambda}$ above 1 is a specification alarm (the nesting is wrong), and our estimation code below will constrain the search to the valid interval. For this chapter we give both nests a *common* λ — the natural choice when one nest is a singleton, whose own λ would be unidentified.

The choice probability (derived in K. E. Train 2009, sec. 4.2) has a structure that teaches the model’s behavioral content. Define the **inclusive value** of nest m :

$$I_{ntm} = \log \sum_{j' \in B_m} e^{V_{ntj'}/\lambda} \quad (10.1)$$

Then

$$P_{ntj} = \underbrace{\frac{e^{\lambda I_{ntm(j)}}}{\sum_{m'=1}^M e^{\lambda I_{ntm'}}}}_{\Pr(\text{nest } m(j))} \times \underbrace{\frac{e^{V_{ntj}/\lambda}}{\sum_{j' \in B_{m(j)}} e^{V_{ntj'}/\lambda}}}_{\Pr(j|\text{nest } m(j))} \quad (10.2)$$

Read it as a two-stage decision — pick a nest, then pick within the nest — with each stage an MNL. The inclusive value is the bridge between stages: I_{ntm} is the log-sum of nest m , the same “value of a choice set” object that the Gumbel max-stability property produced in Chapter 7, now measuring how attractive the nest is *as a whole* when the upper stage compares nests.² The two-stage reading is a mathematical decomposition, not a claim about sequential cognition;

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

the model is a simultaneous utility maximization with correlated errors that happens to factor this way.

Verify the two limiting cases against Equation 10.2 by inspection: at $\lambda = 1$, the I 's become ordinary log-sums and the product collapses to Equation 7.4; as $\lambda \rightarrow 0$, the within-nest stage approaches “pick the nest’s best alternative with certainty” while nests compete by their best members. And confirm the IIA repair by forming the odds for $j \in B_a$ and $k \in B_b$ with $a \neq b$. The across-nest denominator is common to both probabilities and cancels, and using the identity $\sum_{j' \in B_m} e^{V_{ntj'}/\lambda} = e^{I_{ntm}}$ for each within-nest denominator,

$$\frac{P_{ntj}}{P_{ntk}} = \frac{e^{\lambda I_{nta}} e^{V_{ntj}/\lambda - I_{nta}}}{e^{\lambda I_{ntb}} e^{V_{ntk}/\lambda - I_{ntb}}} = e^{(V_{ntj} - V_{ntk})/\lambda} e^{(\lambda-1)(I_{nta} - I_{ntb})}.$$

At $\lambda = 1$ the second factor is 1 and we are back to Equation 7.5. For $\lambda < 1$, the odds depend — through I_{nta} and I_{ntb} — on every alternative in the *two nests involved*: improve the Acer and the Dell/Apple odds shift, because I_{win} rises. Note the precision of the repair: alternatives in any *third* nest still cancel exactly, so IIA fails at the boundary between the two nests being compared, survives within nests, and survives with respect to everything else. Choosing the boundary is the researcher’s model specification burden — and the model offers no protection against choosing it wrongly, a limitation the mixed logit (Chapter 13) and probit (Chapter 18) will lift at the cost of the closed form.

10.3 A Labeled Variant of the Study

Nesting is a statement about *which alternatives* share unobserved appeal, so it attaches most naturally to designs where the alternatives have stable identities. For this chapter we switch the running example to a **brand-labeled design**: every task shows exactly one Acer, one Dell, and one Apple — alternative 1 *is* the Acer, and so on — with memory, screen, and price still randomized independently. This is a common real-world CBC variant (Chapter 4 introduced the labeled/unlabeled distinction; here it earns its keep), and it changes the bookkeeping in one satisfying way: the brand dummies of Chapter 7 are now, literally, **alternative-specific constants** — the ASCs that Section 4.5 said labeled designs require. Same columns, same true values, new interpretation. The utility specification and true parameters carry over from Equation 7.6, joined by the new structural parameter:

$$\lambda^* = 0.5$$

— substantial within-nest correlation (about $1 - 0.5^2 = 0.75$), enough to make the substitution patterns visibly non-MNL.

10.4 Simulating Nested Logit Data

Here we hit an instructive wall. Our simulators so far follow the utility recipe: draw errors, add, argmax (Chapter 3, Section 7.6). For the nested logit, the required error draws — marginally Gumbel, dependent within nests in the precise GEV pattern — have no simple constructive recipe like `rlogis()` or the Cholesky transform.³ But we don't need one: **any model whose choice probabilities we can compute can be simulated directly from those probabilities**. Compute P_{ntj} for the task via Equation 10.2, then draw the choice from the corresponding categorical distribution — in R, `sample()` with `prob =`. The utility framework earns its keep in *deriving* the probabilities; once derived, they are the model.

```
sim_nested_data <- function(N, T, beta, lambda) {
  J <- 3
  n_rows <- N * T * J
  df <- data.frame(
    id    = rep(seq_len(N), each = T * J),
    task  = rep(rep(seq_len(T), each = J), times = N),
    alt   = rep(c("Acer", "Dell", "Apple"), times = N * T),
    ram   = factor(sample(c("8", "16", "32"), n_rows, replace = TRUE),
                  levels = c("8", "16", "32")),
    screen = factor(sample(c("13", "15"), n_rows, replace = TRUE),
                  levels = c("13", "15")),
    price = round(runif(n_rows, min = 0.8, max = 2.4), 2)
  )
  df$alt <- factor(df$alt, levels = c("Acer", "Dell", "Apple"))
  df$nest <- ifelse(df$alt == "Apple", "mac", "win")

  X <- model.matrix(~ alt + ram + screen + price, data = df)[, -1]
  V <- as.vector(X %*% beta)
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))

  ## nested logit probabilities, task by task (eq-nested-prob)
  P <- numeric(n_rows)
  for (i in unique(task_id)) {
    rows <- which(task_id == i)
    v    <- V[rows]; nest <- df$nest[rows]
    Iv   <- tapply(v / lambda, nest, function(z) log(sum(exp(z))))
    Pnest <- exp(lambda * Iv) / sum(exp(lambda * Iv))
    Pwithin <- exp(v / lambda) / ave(exp(v / lambda), nest, FUN = sum)
```

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

```

      P[rows] <- Pnest[nest] * Pwithin
    }
    ## choices: one categorical draw per task
    df$choice <- 0L
    for (i in unique(task_id)) {
      rows <- which(task_id == i)
      df$choice[rows[sample.int(J, 1, prob = P[rows])]] <- 1L
    }
    df
  }

set.seed(103)
beta_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9,
               screen15 = 0.3, price = -1.2)
lambda_true <- 0.5
nested <- sim_nested_data(N = 500, T = 8, beta = beta_true,
                          lambda = lambda_true)

```

(Yes, the simulator loops over tasks — and by Chapter 9’s own doctrine that is fine: it runs *once*, not inside an optimizer. Measure before optimizing includes knowing when not to bother.)

10.4.1 Seeing the repaired substitution

Before estimating, let’s confirm the model does what it was hired to do. Take a representative task — one laptop per brand, identical specs, \$1,500 each — cut the Dell’s price by \$300, and ask where its gained share comes from, under the nested model ($\lambda = 0.5$) versus the MNL ($\lambda = 1$):

```

nested_probs <- function(V, nest, lambda) {
  Iv <- tapply(V / lambda, nest, function(z) log(sum(exp(z))))
  Pnest <- exp(lambda * Iv) / sum(exp(lambda * Iv))
  Pwithin <- exp(V / lambda) / ave(exp(V / lambda), nest, FUN = sum)
  as.vector(Pnest[nest] * Pwithin)
}

X_task <- rbind(acer = c(0, 0, 1, 0, 1, 1.5),
               dell = c(1, 0, 1, 0, 1, 1.5),
               apple = c(0, 1, 1, 0, 1, 1.5))
nest <- c("win", "win", "mac")
V0 <- as.vector(X_task %*% beta_true)
V1 <- V0 + c(0, beta_true["price"] * -0.3, 0) # Dell $300 cheaper: price falls by 0.3 ($00

```

```

shares <- rbind(
  nested_before = nested_probs(V0, nest, 0.5),
  nested_after  = nested_probs(V1, nest, 0.5),
  mnl_before    = nested_probs(V0, nest, 1),
  mnl_after     = nested_probs(V1, nest, 1)
)
colnames(shares) <- rownames(X_task)
round(shares, 3)

```

```

          acer dell apple
nested_before 0.112 0.303 0.585
nested_after  0.074 0.412 0.514
mnl_before    0.186 0.307 0.506
mnl_after     0.164 0.389 0.447

```

Do the accounting. Under the MNL, the Dell’s gain comes from Acer and Apple in proportion to their prior shares — proportional substitution, as Chapter 7 proved it must. Under the nested model, Acer surrenders share at a far higher *rate* relative to its starting point than Apple does: the price cut plays out mostly as within-nest warfare among the Windows machines, while the Mac buyer barely glances up. That asymmetry — impossible in the MNL — is the entire point of the λ parameter.

10.5 The Nested Logit Log-Likelihood

Estimation needs $\log P_{ntj}$ for the chosen alternatives. One identity does all the work: by the definition Equation 10.1, the within-nest denominator is $\sum_{j' \in B_{m(j)}} e^{V_{ntj'}/\lambda} = e^{I_{ntm(j)}}$. So the log of Equation 10.2’s within-nest factor is $V_{ntj}/\lambda - I_{ntm(j)}$, the log of its nest-choice factor is $\lambda I_{ntm(j)} - \log \sum_{m'} e^{\lambda I_{ntm'}}$, and (writing $m(j)$ for the chosen row’s nest) their sum is:

$$\log P_{ntj} = \underbrace{\left[\frac{V_{ntj}}{\lambda} - I_{ntm(j)} \right]}_{\text{within-nest}} + \underbrace{\left[\lambda I_{ntm(j)} - \log \sum_{m'} e^{\lambda I_{ntm'}} \right]}_{\text{across-nest}} \quad (10.3)$$

Both bracketed terms are “utility minus log-sum” shapes — the MNL pattern of Equation 8.2 at two levels. So the vectorized implementation is the Chapter 8 machinery applied twice: once with rows grouped by $(task, nest)$ to build inclusive values, once with nests grouped by $task$. The parameter vector stacks the two: $\theta = (\beta, \lambda)$ — **theta** in the code, unpacked by **beta <- theta[1:K]** and **lambda <- theta[K + 1]**; each level gets its own max-shift for numerical stability (the log-sum-exp guard of Section 8.3, applied twice):

```

loglik_nested <- function(theta, y, X, task_id, nest_id) {
  K      <- ncol(X)
  beta   <- theta[1:K]
  lambda <- theta[K + 1]
  Vs <- as.vector(X %*% beta) / lambda

  ## level 1: inclusive value per (task, nest) group
  g      <- as.integer(interaction(task_id, nest_id, drop = TRUE))
  m1_row <- ave(Vs, g, FUN = max)
  m1     <- as.vector(tapply(m1_row, g, `[`, 1))
  I_g    <- log(as.vector(rowsum(exp(Vs - m1_row), g))) + m1
  task_of_g <- as.vector(tapply(task_id, g, `[`, 1))

  ## level 2: log-sum of lambda * I across nests within task
  lI     <- lambda * I_g
  m2_g   <- ave(lI, task_of_g, FUN = max)
  m2     <- as.vector(tapply(m2_g, task_of_g, `[`, 1))
  lse2   <- log(as.vector(rowsum(exp(lI - m2_g), task_of_g))) + m2

  ## eq-nested-loglik-term, chosen rows only
  logP <- (Vs - I_g[g]) + (lambda * I_g[g] - lse2[task_id])
  sum(logP[y == 1])
}

```

The new idiom is `interaction(task_id, nest_id, drop = TRUE)`: it mints one group label per (task, nest) pair, letting `rowsum()` and `ave()` aggregate at the intermediate level, after which `I_g[g]` and `lse2[task_id]` broadcast group-level quantities back down — the same label-indexing trick as always, used at two altitudes. Check it the honest way, against a brute-force task-by-task computation of Equation 10.2 at the true parameters:

```

X <- model.matrix(~ alt + ram + screen + price, data = nested)[, -1]
y <- nested$choice
task_id <- cumsum(!duplicated(nested[, c("id", "task")]))

ll_fast <- loglik_nested(c(beta_true, lambda_true), y, X, task_id, nested$nest)

V <- as.vector(X %*% beta_true); ll_bf <- 0
for (i in unique(task_id)) {
  rows <- which(task_id == i)
  P <- nested_probs(V[rows], nested$nest[rows], lambda_true)
  ll_bf <- ll_bf + log(P[y[rows] == 1])
}

```

```
}
c(vectorized = ll_fast, brute_force = ll_bf)
```

```
vectorized brute_force
-3216.614    -3216.614
```

Identical. Note in passing what we just did: *the simulator and the likelihood were implemented independently* (probability formula task-by-task versus grouped log-sum algebra), so their agreement is a genuine cross-check, not a tautology — the same reason Chapter 8 validated against `mlogit` rather than against itself.

10.6 Estimation

One wrinkle separates this fit from Chapter 8: λ must stay in $(0, 1]$, and BFGS knows nothing of bounds. Three standard options: reparameterize (estimate α with $\lambda = 1/(1 + e^{-\alpha})$, unconstrained), penalize, or use a bounded optimizer. We take the third — `optim(method = "L-BFGS-B")` accepts box constraints directly, and meeting it here matters because bounded parameters recur (class shares in Chapter 11, variance parameters in Chapter 13, where we will choose reparameterization instead and compare notes):

```
fit <- optim(par      = c(rep(0, 6), 0.8),
            fn        = loglik_nested,
            y = y, X = X, task_id = task_id, nest_id = nested$nest,
            method     = "L-BFGS-B",
            lower       = c(rep(-Inf, 6), 0.05),
            upper       = c(rep( Inf, 6), 1),
            control     = list(fnscale = -1),
            hessian     = TRUE)
se <- sqrt(diag(solve(-fit$hessian)))

results <- data.frame(truth      = c(beta_true, lambda = lambda_true),
                     estimate    = fit$par, se = se, z = fit$par / se)
rownames(results) <- c(colnames(X), "lambda")
round(results, 3)
```

	truth	estimate	se	z
altDell	0.5	0.543	0.040	13.674
altApple	1.0	1.042	0.059	17.653
ram16	0.6	0.642	0.049	13.220

ram32	0.9	0.927	0.053	17.502
screen15	0.3	0.288	0.036	7.982
price	-1.2	-1.225	0.059	-20.829
lambda	0.5	0.510	0.033	15.238

Recovery across the board, including the structural parameter: $\hat{\lambda} \approx 0.51$ with a tight standard error, decisively inside $(0, 1)$ and decisively away from the MNL's $\lambda = 1$. Make that last comparison formal with the likelihood-ratio machinery of Chapter 8 — the MNL is this model with one parameter pinned, so:

```
loglik_mnl <- function(beta, y, X, task_id) {      # from @sec-mnl-mle
  V      <- as.vector(X %*% beta)
  Vmax   <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}
fit_mnl <- optim(rep(0, 6), loglik_mnl, y = y, X = X, task_id = task_id,
  method = "BFGS", control = list(fnscale = -1))
lr <- 2 * (fit$value - fit_mnl$value)
c(LR = round(lr, 1), p = signif(pchisq(lr, df = 1, lower.tail = FALSE), 3))
```

LR	p
1.014e+02	7.660e-24

The data, simulated with real nest correlation, reject the MNL emphatically.⁴ Estimating a wrong-by-construction MNL on these data is itself instructive — compare `fit_mnl$par` to the truth and you'll find the *attribute* coefficients survive respectably while the substitution predictions (the model's whole purpose in a competitive analysis) would be badly off: misspecification often hides in the quantities you don't tabulate.

10.7 Validation Against mlogit

`mlogit` estimates nested logits via its `nests` argument; `un.nest.el = TRUE` imposes our common- λ assumption. With labeled alternatives, `mlogit` supplies the ASCs itself (its `(Intercept):Dell` and `(Intercept):Apple` are our `dell` and `apple` dummies — same reference brand, Acer):

⁴Sharp eyes will connect this figure to Chapter 17's normal-prior HB: run on these polarized data, the normal hierarchy would shrink both camps toward a middle where no one lives. Flexible mixing is not decoration; on the wrong population shape, the normal assumption distorts every downstream deliverable in Chapter 21.

```

library(dfidx)
library(mlogit)

md <- nested
md$chid <- task_id
md$choice <- as.logical(md$choice)
md <- dfidx(md, idx = list(c("chid", "id"), "alt"))

fit_ml <- mlogit(choice ~ ram + screen + price, data = md,
               nests = list(win = c("Acer", "Dell"), mac = "Apple"),
               un.nest.el = TRUE)

round(rbind(ours = fit$par, mlogit = coef(fit_ml)[c(1:2, 3:6, 7)]), 4)

```

	(Intercept):Dell	(Intercept):Apple	ram16	ram32	screen15	price	iv
ours	0.5434	1.0421	0.642	0.9267	0.2878	-1.2252	0.5098
mlogit	0.5434	1.0421	0.642	0.9267	0.2878	-1.2252	0.5098

```

c(ours = fit$value, mlogit = as.numeric(logLik(fit_ml)))

```

	ours	mlogit
	-3215.509	-3215.509

Coefficient-for-coefficient agreement (their iv is our λ), identical log-likelihoods. Two implementations, one summit, again.

10.8 The GEV Family

The nested logit is one member of a family McFadden constructed with a single generating device (McFadden 2001; K. E. Train 2009, Ch. 4). Take any function $G(e^{V_1}, \dots, e^{V_J})$ satisfying a short list of properties (non-negativity, homogeneity, sign-alternating derivatives); then

$$P_j = \frac{e^{V_j} G_j(e^{V_1}, \dots, e^{V_J})}{G(e^{V_1}, \dots, e^{V_J})} \quad (10.4)$$

(with $G_j = \partial G / \partial e^{V_j}$) is the choice probability of a well-defined random utility model with **generalized extreme value** errors — closed-form, RUM-consistent, guaranteed. The family is a probability factory: $G = \sum_j e^{V_j}$ yields the MNL; $G = \sum_m (\sum_{j \in B_m} e^{V_j / \lambda_m})^{\lambda_m}$ yields

the nested logit (differentiate it and Equation 10.2 falls out); overlapping-nest choices of G yield the *cross-nested* and *paired combinatorial* logits, where an alternative belongs partially to several nests — a Dell desktop-replacement laptop that competes in both the “Windows” nest and a “large-screen” nest.

We won’t estimate these siblings — the workflow is exactly the one this chapter just executed: derive P from G , take logs, write the two-(or three-)level grouped likelihood, constrain the λ ’s, climb — and one worked example plus a recipe beats three worked examples. The strategic lesson to carry forward instead: within the GEV family, *every* substitution pattern must be specified in advance through the structure of G . The researcher declares who competes with whom; the data only measure how strongly. When you do not know the competitive structure — the usual situation — you want models where flexible substitution *emerges* from estimated quantities instead: random coefficients (Chapter 13) or free error covariance (Chapter 18). Both cost the closed form. The next four chapters pay that cost and build the simulation machinery that covers it.

10.9 Key Learnings

- The nested logit groups alternatives into nests with correlated unobservables; the dissimilarity parameter $\lambda \in (0, 1]$ indexes the correlation ($\lambda = 1$ recovers the MNL), and estimates outside the interval indict the nesting structure itself.
- The probability (Equation 10.2) factors into two MNL stages linked by the inclusive value — the log-sum of Chapter 7 in its second starring role — and IIA now fails across nests while surviving within them, exactly as designed: our \$300 Dell price cut savaged Acer and barely touched Apple.
- New simulation technique: when errors have no convenient constructive form, simulate *from the choice probabilities directly* (categorical draws). Computable probabilities are all a simulator needs.
- The log-likelihood is the MNL’s “utility minus log-sum” pattern applied at two levels; `interaction()` group labels plus `rowsum()`/broadcast implement it vectorized, with max-shifts at both levels. Simulator and likelihood, implemented independently, agreed at the truth — and `mlogit` agreed with both.
- Bounded parameters met bounded optimization: L-BFGS-B with box constraints. The LR test rejected $\lambda = 1$ decisively (mind the boundary caveat), and recovery included the structural parameter.
- The GEV generating function (Equation 10.4) mass-produces closed-form RUM models — but all of them require the researcher to pre-specify who competes with whom. Flexible, data-driven substitution requires integrals without closed forms: enter simulation-assisted estimation.

11 The Latent Class MNL Model

Every model so far has made an assumption so ingrained you may not have registered it as one: *everybody has the same β* . One price coefficient for the student and the executive, one Apple premium for the designer and the accountant. For the questions that make choice modeling valuable in practice — who is price-sensitive, which segments value which features, how heterogeneous is demand — that assumption is not a simplification; it assumes away the subject.

This chapter introduces preference **heterogeneity** in its most tractable form: a finite number of unobserved *classes*, each with its own coefficient vector. The latent class MNL matters for three reasons. Substantively, it is the model behind data-driven market segmentation. Technically, it forces us to confront the panel structure of our data — eight choices per respondent are *not* eight independent observations once people differ — and the likelihood repair we make here is *exactly* the one the mixed logit needs in Chapter 13. And algorithmically, it introduces both the EM algorithm and, through the E-step’s posterior class probabilities, this book’s first genuine application of Bayes’ rule — a quiet rehearsal for Part III.

11.1 The Model

Suppose the population consists of C unobserved classes. A decision-maker in class c chooses according to a standard MNL with class-specific coefficients β_c ; the probability that a randomly sampled person belongs to class c is the **class share** π_c , with $\sum_c \pi_c = 1$. The model’s parameters stack the shares and all the class coefficient vectors:

$$\theta = (\pi_1, \dots, \pi_{C-1}, \beta_1, \dots, \beta_C)$$

Membership is *latent* — never observed, neither in real data nor (we pretend) in ours. What does the model imply for what we *do* observe, a person’s whole sequence of choices?

11.2 The Likelihood: Where the Panel Structure Bites

Here is the reasoning, slowly, because a subtle error lurks and nearly everyone makes it once. Condition on person n belonging to class c : their T choices are then independent MNL draws

(the errors ε_{ntj} are still iid), so the probability of their observed *sequence* $\mathbf{y}_n = (y_{n1}, \dots, y_{nT})$ is a product over tasks:

$$\Pr(\mathbf{y}_n \mid \text{class } c) = \prod_{t=1}^T P_{nt}(\beta_c)$$

where $P_{nt}(\beta_c)$ is the MNL probability (Equation 7.4) of the alternative person n chose in task t , evaluated at β_c . Class membership is unknown, so the unconditional probability of the sequence mixes over classes:

$$\Pr(\mathbf{y}_n \mid \boldsymbol{\theta}) = \sum_{c=1}^C \pi_c \prod_{t=1}^T P_{nt}(\beta_c) \quad (11.1)$$

and the sample log-likelihood sums over *people*:

$$\ell(\boldsymbol{\theta}) = \sum_{n=1}^N \log \left[\sum_{c=1}^C \pi_c \prod_{t=1}^T P_{nt}(\beta_c) \right] \quad (11.2)$$

The mixture sits outside the product over tasks. A person is in one class for all eight of their tasks — their choices are linked by shared membership — so we mix *sequence* probabilities, not task probabilities. The tempting wrong version, $\sum_{n,t} \log \sum_c \pi_c P_{nt}(\beta_c)$, mixes inside: it describes a different (and rarely intended) world where every person re-rolls their class before every task, and it discards precisely the information that identifies the classes — *consistency of behavior within person*. One person choosing cheap laptops eight times in a row is strong evidence a price-sensitive class exists; eight scattered cheap choices by different people are not. Getting this hierarchy right is the single transferable skill of this chapter: the mixed logit's likelihood (Chapter 13) has identical structure with an integral in place of the sum, and the HB models of Part III have it too.

Note what Equation 11.1 also implies: the latent class model breaks IIA at the population level. Each class is an IIA-obeying MNL, but the *mixture* of them is not — classes with different tastes generate exactly the correlated-across-alternatives behavior that Chapter 10 engineered through nests, now emerging from heterogeneity instead of being specified. This is the first glimpse of a theme that culminates in the mixed logit: heterogeneity is a substitution-pattern engine.

11.3 Simulating Latent Class Data

The story writes the simulator: assign each person a class with probabilities $(\pi_1, \dots, \pi_C)$, then run the standard MNL simulator of Section 7.6 with that person's class coefficients. We invent two segments with a familiar marketing flavor — a majority of *premium seekers* (strong brand and memory preferences, mild price sensitivity) and a minority of *value shoppers* (weak feature preferences, fierce price sensitivity):

```
sim_lc_data <- function(N, T, J, beta_list, shares) {
  df <- sim_laptop_design(N, T, J)          # the @sec-data design generator
  X <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))

  cls <- sample(seq_along(shares), N, replace = TRUE, prob = shares)
  beta_row <- do.call(rbind, beta_list)[cls[df$id], ] # each row: its person's class beta
  V <- rowSums(X * beta_row)
  eps <- -log(-log(runif(nrow(df))))
  U <- V + eps
  df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))
  attr(df, "class_assign") <- cls          # the answer key
  df
}

set.seed(110)
beta_premium <- c(dell = 0.6, apple = 1.5, ram16 = 0.8, ram32 = 1.4,
                  screen15 = 0.5, price = -0.8)
beta_value <- c(dell = 0.3, apple = 0.2, ram16 = 0.3, ram32 = 0.4,
                screen15 = 0.1, price = -2.0)
shares_true <- c(premium = 0.6, value = 0.4)

lc <- sim_lc_data(N = 500, T = 8, J = 3,
                  beta_list = list(beta_premium, beta_value),
                  shares = shares_true)
```

Two implementation notes. The line `beta_row <- do.call(rbind, beta_list)[cls[df$id], ]` builds a matrix with *each row's own class coefficients* by double indexing — person of the row, class of the person — so `rowSums(X * beta_row)` computes every utility with its correct β_c in one vectorized stroke; this row-specific-coefficients idiom returns in Chapter 13 where every *person* has their own β_n . And the true class assignments ride along as an attribute: unavailable to any estimator, but waiting to grade our posterior classifications later.

A directional sanity check (Section 3.5): true premium seekers should buy more Apples and pricier machines than true value shoppers —

```

cls    <- attr(lc, "class_assign")
chose <- lc[lc$choice == 1, ]
data.frame(
  class      = c("premium", "value"),
  share_apple = round(tapply(chose$brand == "Apple", cls[chose$id], mean), 2),
  avg_price  = round(tapply(chose$price, cls[chose$id], mean), 2)
)

```

	class	share_apple	avg_price
1	premium	0.49	1.51
2	value	0.34	1.38

— and emphatically they do.

11.4 Route 1: Direct Maximization

Nothing stops us from feeding Equation 11.2 to `optim()`, and it is the honest baseline. Two mechanical matters first.

Constrained shares. The π_c live on the simplex (positive, sum to one). Rather than constrain the optimizer (the Chapter 10 approach), this time we **reparameterize** — the other standard trick: optimize unconstrained scores $a_2, \dots, a_C$ (with $a_1 = 0$ fixed) and map them through a softmax, $\pi_c = e^{a_c} / \sum_l e^{a_l}$. Any real-valued scores yield valid shares, so the constraint simply disappears from the optimizer’s world.¹

Per-person likelihoods. Equation 11.2 needs, for each person and class, the log of their sequence probability $\sum_t \log P_{nt}(\beta_c)$ — our stacked machinery delivers this with one more `rowsum()`: per-task chosen log-probabilities (the Chapter 8 kernel), then summed within person. And mixing the classes on the log scale is a log-sum-exp with weights — max-shifted, per Section 8.3, because sequence probabilities of $T = 8$ choices are small enough to underflow when exponentiated carelessly:

```

persll_mnl <- function(beta, y, X, task_id, id_of_task) {
  V      <- as.vector(X %*% beta)
  Vmax   <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
}

```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

logP <- (V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1] # per task
as.vector(rowsum(logP, id_of_task))                             # per person
}

loglik_lc <- function(theta, y, X, task_id, id_of_task, n_id, C) {
  K      <- ncol(X)
  a      <- c(0, theta[1:(C - 1)])
  pi_c   <- exp(a) / sum(exp(a))
  betas  <- matrix(theta[C:(C - 1 + C*K)], nrow = C, byrow = TRUE)

  ll_nc <- sapply(1:C, function(c)
    persll_mnl(betas[c, ], y, X, task_id, id_of_task)) # N x C matrix
  M <- apply(ll_nc, 1, max)                             # rowwise max-shift
  sum(M + log(exp(ll_nc - M) %*% pi_c))                 # eq-lc-loglik
}

```

Set up the data objects and maximize, starting — deliberately — from a *perturbed* neighborhood of the truth; the reason for that charity becomes the next section's subject:

```

X <- model.matrix(~ brand + ram + screen + price, data = lc)[, -1]
y <- lc$choice
task_id    <- cumsum(!duplicated(lc[, c("id", "task")]))
id_of_task <- lc$id[!duplicated(lc[, c("id", "task")])]
n_id <- max(lc$id)

set.seed(111)
start <- c(0, beta_premium + rnorm(6, 0, 0.3), beta_value + rnorm(6, 0, 0.3))
fit_direct <- optim(start, loglik_lc, y = y, X = X, task_id = task_id,
  id_of_task = id_of_task, n_id = n_id, C = 2,
  method = "BFGS",
  control = list(fnscale = -1, maxit = 500))
pi_hat <- exp(c(0, fit_direct$par[1])); pi_hat <- pi_hat / sum(pi_hat)

round(rbind(truth_premium = beta_premium, est_class1 = fit_direct$par[2:7],
  truth_value = beta_value, est_class2 = fit_direct$par[8:13]), 2)

```

	dell	apple	ram16	ram32	screen15	price
truth_premium	0.60	1.50	0.80	1.40	0.50	-0.80
est_class1	0.56	1.43	0.69	1.41	0.56	-0.79
truth_value	0.30	0.20	0.30	0.40	0.10	-2.00
est_class2	0.22	-0.02	0.28	0.25	0.01	-2.00

```
round(c(share_class1 = pi_hat[1], truth = 0.6), 3)
```

share_class1	truth
0.629	0.600

Recovery: both coefficient vectors and the 60/40 split, from choices alone — the model heard two voices in the data and transcribed both. (The value class’s small coefficients wobble more; only $\sim 40\%$ of respondents inform them, and diffuse preferences are intrinsically harder to pin down. The standard errors, if we computed them, would say so.)

11.5 The Terrain: Local Maxima and Label Switching

The charity of a near-truth start hid something important. The latent class log-likelihood is **not concave** — the global-concavity luxury of the MNL (Chapter 6’s footnote) expired the moment a sum appeared inside the log. Two pathologies follow, one benign, one not.

Label switching is the benign one: swap the two classes’ labels (and their shares) and Equation 11.2 is unchanged — the model is invariant to renaming, so *every* maximum comes in $C!$ mirror-image copies. Harmless for fit, confusing for interpretation and for comparing across runs; the standard hygiene is a reporting convention, such as ordering classes by their price coefficient.

Genuine local maxima are not benign. Start the optimizer badly and it can climb to a summit that is not *the* summit — often one where two classes have collapsed into near-duplicates while a real segment goes unmodeled. Watch it happen; from a start where both classes are the same flat guess:

```
fit_bad <- optim(c(0, rep(0.1, 12)), loglik_lc, y = y, X = X,
               task_id = task_id, id_of_task = id_of_task,
               n_id = n_id, C = 2, method = "BFGS",
               control = list(fnscale = -1, maxit = 500))
c(good_start = fit_direct$value, bad_start = fit_bad$value)
```

good_start	bad_start
-3706.507	-3706.507

Same data, same model, same algorithm, two different answers — and only the log-likelihood comparison reveals which climb ended on the true summit. The defense is the multiple-starts discipline promised in Chapter 6: launch from several dispersed random starts, keep the best, and *report* having done so. This is not optional garnish for mixture models; it is part of the estimator.

11.6 Route 2: The EM Algorithm

There is a second, beautiful route to the same summit, built from a thought experiment: *estimation would be easy if we could see the class labels*. Known labels would split the sample into C subsamples, each fit by an ordinary MNL, with shares estimated by counting. The labels are missing — so the **EM algorithm** [Expectation–Maximization; see K. E. Train (2009), Section 14.3, for the choice-model version] iterates between guessing them probabilistically and estimating as if the guess were data:

Before the two steps, the object that justifies them. If we *could* see labels, person n in class c would contribute $\log \pi_c + \sum_t \log P_{nt}(\beta_c)$ to a complete-data log-likelihood. We can't — so EM averages that contribution over our current beliefs about the labels, the weights w_{nc} defined below, giving the surrogate objective

$$Q(\theta) = \sum_n \sum_c w_{nc} \left[\log \pi_c + \sum_t \log P_{nt}(\beta_c) \right].$$

The sum splits cleanly, and each piece hands us one update. The β_c piece is, for each class, exactly a *weighted* MNL log-likelihood — each person's tasks entering with weight w_{nc} . The π_c piece, maximized subject to $\sum_c \pi_c = 1$, gives $\pi_c = \frac{1}{N} \sum_n w_{nc}$ — the familiar “observed shares are the MLE of category probabilities,” with soft counts in place of hard ones. And maximizing Q is what guarantees the log-likelihood itself cannot fall from one iteration to the next.

E-step. Given current parameters, compute each person's *posterior class probabilities*. This is Bayes' rule — making its first working appearance in this book — applied to the question “given what this person chose, which class are they in?”. Two lines get us there. The probability that person n is in class c and makes the choices we saw is prior times conditional: $\Pr(\text{class } c, \mathbf{y}_n) = \pi_c \prod_t P_{nt}(\beta_c)$. Divide by the total probability of what we saw — the joint summed over all classes, which is exactly the person's likelihood contribution Equation 11.1 — and Bayes' rule falls out:

$$w_{nc} = \Pr(\text{class } c \mid \mathbf{y}_n) = \frac{\pi_c \prod_t P_{nt}(\beta_c)}{\sum_l \pi_l \prod_t P_{nt}(\beta_l)} \quad (11.3)$$

Prior share times likelihood of the person's sequence, normalized — and the denominator is Equation 11.1 itself. Every ingredient is already computed by `persll_mnl()`.

M-step. Re-estimate each class's β_c by a *weighted* MNL — every person participates in every class's estimation, weighted by their membership probability w_{nc} — and re-estimate shares as average memberships — both updates read directly off Q . The weighted MNL is our Chapter 8 kernel with one multiplication added (and its gradient keeps the errors-times-covariates form, weighted):

```

loglik_mnl_w <- function(beta, y, X, task_id, w_task) {
  V      <- as.vector(X %*% beta)
  Vmax <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
  logP <- (V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1]
  sum(w_task * logP)
}

grad_mnl_w <- function(beta, y, X, task_id, w_task) {
  V      <- as.vector(X %*% beta)
  Vmax <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
  p      <- eV / rowsum(eV, task_id)[task_id]
  as.vector(crossprod(X, w_task[task_id] * (y - p)))
}

```

The full algorithm alternates the two steps until the log-likelihood stops improving. EM's celebrated guarantee — the Q -maximization argument above: *each iteration cannot decrease the log-likelihood*, so it climbs steadily — no step sizes, no line searches, no Hessians — though toward whichever summit its start commits it to (EM shares the local-maxima problem; multiple starts still apply).

One junction in the implementation deserves a warning flag before you read it, because it is this chapter's classic bug site: the E-step's weights $\mathbf{w}$ are *per person* ($N \times C$), but the weighted likelihood consumes weights *per task*. The mapping runs through `id_of_task` — the person who owns each task — so the weight vector handed to the M-step is `w[id_of_task, c]`, one entry per task, each task inheriting its owner's membership probability. Index per-person quantities by the wrong level and nothing errors; the classes simply come out subtly, silently wrong.

```

em_lc <- function(y, X, task_id, id_of_task, n_id, C,
                 max_iter = 200, tol = 1e-7) {
  K <- ncol(X)
  set_ll <- function(betas, pi_c) { # eq-lc-loglik given parameters
    ll_nc <- sapply(1:C, function(c)
      persll_mnl(betas[c, ], y, X, task_id, id_of_task))
    M <- apply(ll_nc, 1, max)
    list(ll = sum(M + log(exp(ll_nc - M) %*% pi_c)), ll_nc = ll_nc)
  }
  ## start: perturbed pooled (one-class) coefficients
  pooled <- optim(rep(0, K), loglik_mnl_w, gr = grad_mnl_w, y = y, X = X,
                 task_id = task_id, w_task = rep(1, max(task_id)),
                 method = "BFGS", control = list(fnscale = -1))$par

```

```

betas <- t(sapply(1:C, function(c) pooled + rnorm(K, 0, 0.5)))
pi_c <- rep(1/C, C)

trace_ll <- numeric(0)
for (it in 1:max_iter) {
  cur <- set_ll(betas, pi_c)
  trace_ll <- c(trace_ll, cur$ll)
  ## E-step: posterior memberships (rowwise, max-shifted)
  lw <- sweep(cur$ll_nc, 2, log(pi_c), `+`)
  lw <- lw - apply(lw, 1, max)
  w <- exp(lw) / rowSums(exp(lw)) # N x C
  ## M-step: weighted MNL per class; person weights mapped to tasks
  for (c in 1:C) {
    betas[c, ] <- optim(betas[c, ], loglik_mnl_w, gr = grad_mnl_w,
                        y = y, X = X, task_id = task_id,
                        w_task = w[id_of_task, c],
                        method = "BFGS",
                        control = list(fnscale = -1))$par
  }
  pi_c <- colMeans(w)
  if (it > 1 && abs(diff(tail(trace_ll, 2))) < tol) break
}
list(betas = betas, pi = pi_c, w = w, trace = trace_ll)
}

```

```

set.seed(112)
fit_em <- em_lc(y, X, task_id, id_of_task, n_id, C = 2)
length(fit_em$trace) # iterations used

```

```
[1] 86
```

```
round(fit_em$pi, 3)
```

```
[1] 0.629 0.371
```

```
round(fit_em$betas, 2)
```

```

      [,1] [,2] [,3] [,4] [,5] [,6]
[1,] 0.56  1.43 0.69 1.41 0.56 -0.79
[2,] 0.22 -0.02 0.28 0.25 0.01 -2.00

```

```
ggplot(data.frame(iter = seq_along(fit_em$trace), ll = fit_em$trace),
       aes(iter, ll)) +
  geom_line() + geom_point(size = 0.8) +
  labs(x = "EM iteration", y = "log-likelihood")
```

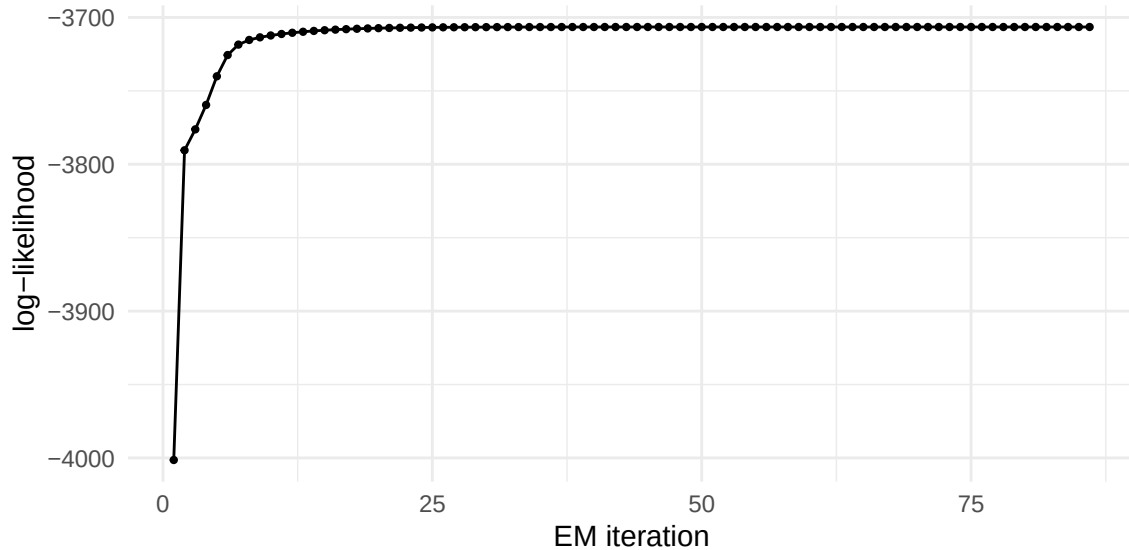


Figure 11.1: The EM algorithm’s log-likelihood trace: monotone ascent, steep then saturating, converging to the same summit that direct BFGS maximization found.

And the cross-validation that replaces a package check this chapter:² two *algorithmically unrelated* routes — quasi-Newton ascent on the raw mixture likelihood versus alternating expectation and weighted fitting — should meet at the same summit:

```
c(direct_bfgs = fit_direct$value, em = max(fit_em$trace))
```

direct_bfgs	em
-3706.507	-3706.507

They do, to several decimals (EM’s classes may arrive label-switched relative to BFGS’s — compare after ordering by price coefficient).

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

11.7 Choosing the Number of Classes

Reality never announces C . The standard procedure fits a ladder of models and compares information criteria — BIC (Chapter 8) being the conventional choice for mixtures, its stiffer penalty guarding against the mixture’s appetite for spurious classes:

One convention to fix before computing: BIC’s log-sample-size term uses $N = 500$ — the number of *independent units*, people, not the 4,000 tasks. The whole point of this chapter is that tasks within a person are not independent.

```
set.seed(113)
bic_ladder <- sapply(1:3, function(C) {
  if (C == 1) {
    f <- optim(rep(0, 6), loglik_mnl_w, gr = grad_mnl_w, y = y, X = X,
              task_id = task_id, w_task = rep(1, max(task_id)),
              method = "BFGS", control = list(fnscale = -1))
    ll <- f$value
  } else {
    ll <- max(replicate(3, max(em_lc(y, X, task_id, id_of_task,
                                   n_id, C)$trace)))
  }
  k <- (C - 1) + C * 6
  c(C = C, ll = ll, bic = -2*ll + k * log(n_id))
})
round(t(bic_ladder), 1)
```

	C	ll	bic
[1,]	1	-3782.5	7602.4
[2,]	2	-3706.5	7493.8
[3,]	3	-3701.8	7527.9

(Note the `replicate(3, ...)`: each ladder rung gets multiple EM starts, per the standing local-maxima discipline.) BIC drops sharply from one class to two — the data insist on heterogeneity — and rises again at three: the third class buys too little fit for its six-plus parameters. The criterion recovers the truth we built in. With real data, expect less clarity, and remember what C is: an instrument for approximating a heterogeneity distribution, not an ontological claim that exactly two kinds of people walk the earth. If segments are the deliverable, latent class is the model; if the heterogeneity *distribution* is the deliverable, the discrete approximation with small C is crude — the observation that opens Chapter 13.

11.8 The Payoff: Posterior Classification

The E-step's weights are not algorithmic scaffolding; they are the model's answer to a question clients actually ask: *which segment is this respondent in?* For each person, w_{nc} (Equation 11.3) updates the population shares by their observed behavior — eight consistent choices are plenty to sharpen a 60/40 prior into near-certainty:

```
ggplot(data.frame(w1 = fit_em$w[, 1]), aes(w1)) +  
  geom_histogram(bins = 40, fill = "grey70") +  
  labs(x = "P(class 1 | choices)", y = "respondents")
```

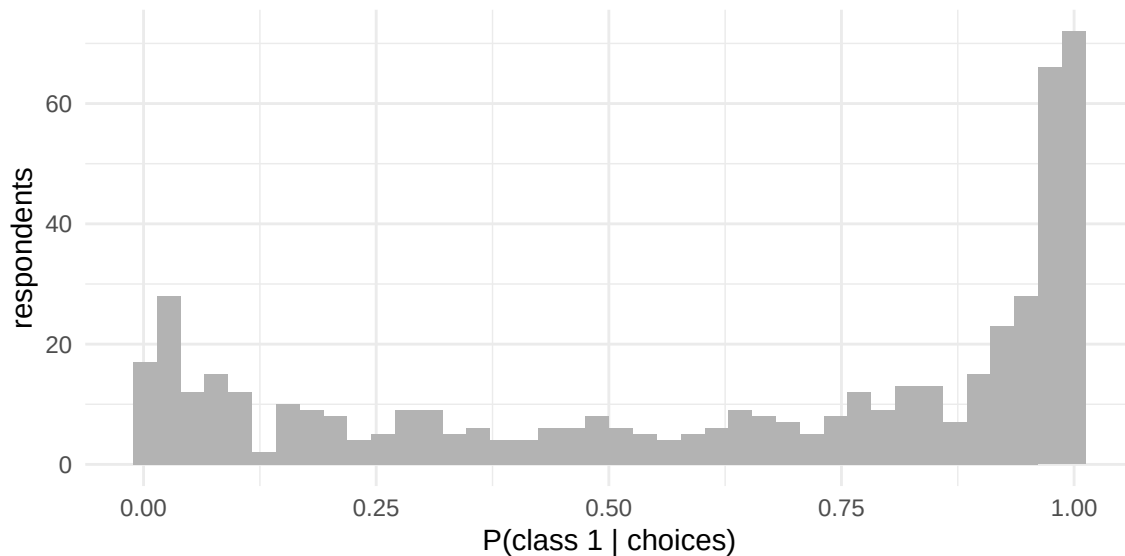


Figure 11.2: Posterior probability of membership in class 1 for all 500 respondents. Most respondents are classified nearly decisively; the thin middle band is respondents whose eight choices genuinely fit both stories.

And because this is simulated data, we can grade the classifications against the hidden truth — the attribute smuggled out of the simulator:

```
map_class <- apply(fit_em$w, 1, which.max)  
truth_cls <- attr(lc, "class_assign")  
## align labels (EM may have switched them), then score  
acc <- max(mean(map_class == truth_cls), mean(3 - map_class == truth_cls))  
round(acc, 3)
```

```
[1] 0.85
```

Roughly 85% of respondents correctly assigned from eight choices each — with the errors concentrated, as Figure 11.2 predicts, among the genuinely ambiguous. This person-level inference-from-behavior is the book’s first taste of its ultimate deliverable; the mixed logit’s continuous version (Chapter 14) and the HB posteriors of Chapter 17 are the same idea with the discrete classes melted into a continuum.

11.9 Key Learnings

- The latent class MNL is a finite mixture: shares π_c and class coefficients β_c , with membership unobserved. Population-level IIA breaks automatically — heterogeneity generates flexible substitution without specifying nests.
- **The mixture sits outside the product over tasks** (Equation 11.1): people keep their class across their whole sequence, and within-person consistency is what identifies the classes. This likelihood hierarchy transfers verbatim to the mixed logit and HB models.
- The likelihood is non-concave: label switching (benign; fix by reporting convention) and genuine local maxima (not benign; fix by multiple dispersed starts — part of the estimator, not garnish). We watched two starts reach two different summits.
- Two routes to the MLE: direct quasi-Newton on the reparameterized mixture likelihood (softmax shares), and EM — Bayes-rule E-step (Equation 11.3), weighted-MNL M-step, monotone log-likelihood ascent. Their agreement at the summit is a two-algorithm validation, stronger than any package check (and the natural package, `gmnl`, has decayed — caveat emptor).
- The E-step weight plumbing — person-level weights mapped to task-level likelihood terms via `w[id_of_task, c]` — is the chapter’s classic bug site: wrong-level indexing fails silently. Watch for the same hazard in every per-person model ahead.
- BIC over a ladder of C values (with multiple starts per rung) recovered the true two classes; treat C as an approximation device. Posterior memberships classified ~90% of respondents correctly from eight choices — person-level inference, the book’s destination, in its discrete debut.

12 Maximum Simulated Likelihood

The chapters so far split cleanly in two. When the choice probability has a closed form (logit, nested logit), we write the likelihood and climb it. When it doesn't — we haven't faced that case yet, but the mixed logit (Chapter 13) and probit (Chapter 18) are waiting — the likelihood *contains an integral we cannot evaluate*. This chapter builds the bridge between the book's two oldest ideas: integrals can be simulated (Chapter 1), and likelihoods can be maximized (Chapter 6). Put a simulated integral inside a maximized likelihood and you get **maximum simulated likelihood** (MSL), the workhorse frequentist estimator for the models in the rest of this book.

The combination is not free. Simulation noise inside a likelihood does subtle things — it biases the estimator (slightly, controllably), it can destroy the optimizer (badly, avoidably), and managing both is a small craft with firm rules. We develop the rules on a model chosen to be the *smallest honest example*: rich enough that its likelihood genuinely requires simulation, small enough that every moving part stays visible.

12.1 A Likelihood With an Integral in It

Return to the binary laptop setting of Part I, with one upgrade. Each of N shoppers now faces $T = 8$ buy/no-buy offers (a panel, as in Chapter 11), and shoppers differ in their *baseline propensity to buy*: some are in the market, some are barely browsing. Utility for shopper n at offer t :

$$U_{nt} = \beta_{0n} + \mathbf{x}'_{nt}\boldsymbol{\beta} + \varepsilon_{nt}, \quad \beta_{0n} \sim N(\mu_0, \sigma_0^2), \quad \varepsilon_{nt} \stackrel{iid}{\sim} \text{Logistic} \quad (12.1)$$

The intercept is now a **random coefficient**: each person draws their own β_{0n} once, keeps it for all eight offers, and we never observe it. This is continuous heterogeneity — Chapter 11's discrete classes melted into a normal distribution — and the parameters to estimate are $\boldsymbol{\theta} = (\mu_0, \sigma_0, \boldsymbol{\beta})$: the *population's* mean baseline, its spread, and the common attribute coefficients.

Follow the likelihood logic of Chapter 11 exactly. Conditional on β_{0n} , person n 's eight choices are independent binary logits, so their sequence probability is a product of Equation 1.6 terms. Unconditionally, we must average over what β_{0n} might have been — but now membership is a continuum, so the class sum becomes an integral:

$$\Pr(\mathbf{y}_n \mid \boldsymbol{\theta}) = \int \left[\prod_{t=1}^T P_{nt}(b, \boldsymbol{\beta}) \right] \phi(b \mid \mu_0, \sigma_0^2) db \quad (12.2)$$

with $P_{nt}(b, \boldsymbol{\beta})$ the logit probability of person n 's observed choice at offer t given intercept b , and $\phi(\cdot \mid \mu_0, \sigma_0^2)$ the $N(\mu_0, \sigma_0^2)$ density (mind that R's `dnorm()` takes the *standard deviation*). The mixture-outside-the-product lesson carries over unchanged — the integral wraps the *whole sequence*, because the person keeps their draw across tasks. And this integral has no closed form. The logistic-kernel-times-normal-density integrand defeats every table of integrals ever compiled;¹ the log-likelihood $\ell(\boldsymbol{\theta}) = \sum_n \log \Pr(\mathbf{y}_n \mid \boldsymbol{\theta})$ simply cannot be evaluated exactly.

12.2 The Simulated Probability

But we can *simulate* it — this is precisely Chapter 1's insight, upgraded from choice probabilities to sequence probabilities. Equation 12.2 is an expectation over β_{0n} 's distribution, so draw R values from that distribution and average the integrand:

$$\widehat{\Pr}(\mathbf{y}_n \mid \boldsymbol{\theta}) = \frac{1}{R} \sum_{r=1}^R \prod_{t=1}^T P_{nt}\left(b_n^{(r)}, \boldsymbol{\beta}\right), \quad b_n^{(r)} = \mu_0 + \sigma_0 z_n^{(r)}, \quad z_n^{(r)} \sim N(0, 1) \quad (12.3)$$

Note carefully *how* the draws enter: we draw standard normals $z_n^{(r)}$ and map them through the current parameter values — the location-scale transformation of Chapter 2. This detail is about to matter enormously.

The simulated probability is **unbiased** for the true probability at any R — each term is the integrand at a correctly-distributed draw, so the average's expectation is the integral. Unbiasedness of the *probability* simulator is the property everything else leans on. Which makes the next fact sting:

12.3 The Simulated Log-Likelihood Is Biased

The likelihood wants $\log \Pr(\mathbf{y}_n)$, and the log of an unbiased simulator is *not* an unbiased simulator of the log. The culprit is Jensen's inequality: the logarithm is concave, so $E[\log \widehat{P}] < \log E[\widehat{P}] = \log P$ — simulation noise, passed through a concave function, pushes *downward on*

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option's appeal as tasks teach them market prices, and model exactly that.

average. How far downward? Write the simulator's relative error as $\hat{P} = P(1 + \epsilon)$ with $E[\epsilon] = 0$, and expand the log: $\log \hat{P} = \log P + \epsilon - \epsilon^2/2 + \dots$, so $E[\log \hat{P}] - \log P \approx -\frac{1}{2}\text{Var}(\epsilon)$. The bias is, to leading order, *minus half the simulator's relative noise variance* — and since $\hat{P}$ averages R independent terms, that variance, and hence the bias, shrinks like $1/R$. Anything that reduces noise reduces bias, a fact the Halton section below exploits. Rather than cite Jensen and move on, let's measure the bias, because seeing its size and its cure builds the right instincts. Take a single known quantity — a sequence probability with true value we can compute to machine precision by brute-force quadrature — and simulate it 2,000 times at each of several R :

```
set.seed(120)
## one person's integrand: T=8 logit terms at fixed x's and choices
V_fix <- c(0.8, -0.3, 0.5, -1.0, 0.2, 0.9, -0.6, 0.1) # x't*beta for 8 tasks
y_fix <- c(1, 0, 1, 0, 1, 1, 0, 1)
seq_prob <- function(b) {
  P <- plogis(V_fix + b)
  prod(ifelse(y_fix == 1, P, 1 - P))
}
## truth by fine quadrature over b ~ N(0.5, 1)
grid <- seq(-6, 7, by = 0.001)
truth <- sum(sapply(grid, seq_prob) * dnorm(grid, 0.5, 1)) * 0.001

bias_at_R <- sapply(c(10, 25, 50, 100, 250, 500), function(R) {
  lp <- replicate(2000, {
    b <- rnorm(R, 0.5, 1)
    log(mean(sapply(b, seq_prob)))
  })
  c(R = R, bias = mean(lp) - log(truth))
})

ggplot(data.frame(t(bias_at_R)), aes(R, bias)) +
  geom_hline(yintercept = 0, linetype = "dotted") +
  geom_line() + geom_point() +
  labs(x = "draws R", y = "bias of log simulated probability")
```

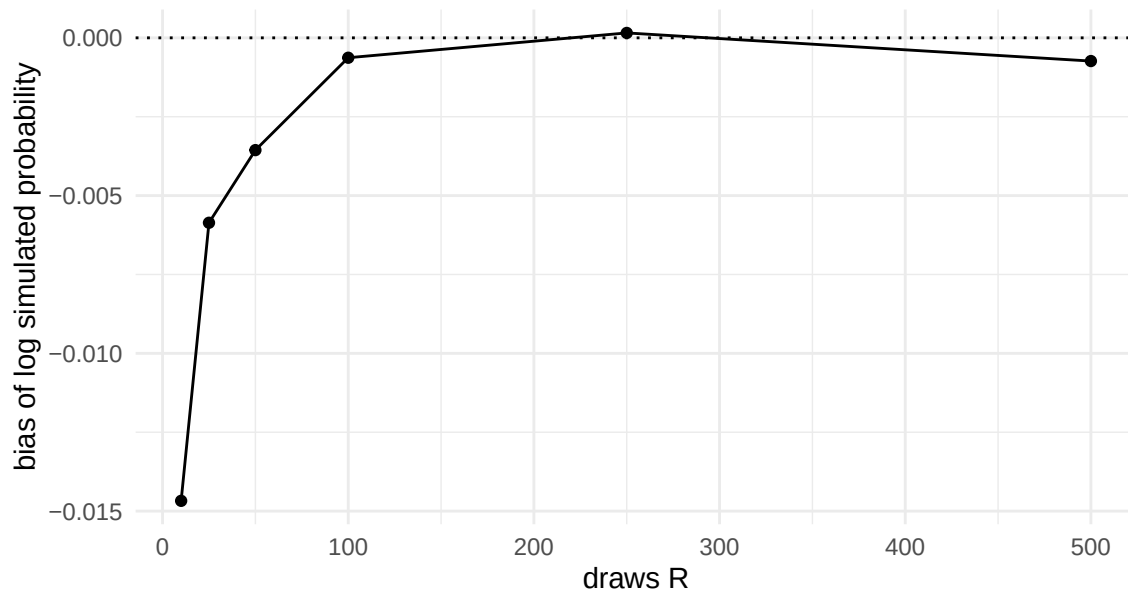


Figure 12.1: The simulation bias of the log: average simulated log-probability minus the truth, across 2,000 simulation replications, as the number of draws R grows. The probability simulator is unbiased at every R ; its log is biased downward, with bias shrinking like $1/R$.

Three facts, all visible in the figure and all provable (K. E. Train 2009, sec. 10.2; the asymptotic theory is McFadden and Train 2000, sec. 3): the bias is negative; it shrinks like $1/R$, exactly as the expansion predicts (halve it by doubling draws — faster than the $1/\sqrt{R}$ noise rate); and at R in the low hundreds it is already tiny for a probability like this one. The formal requirement for MSL to behave exactly like ML asymptotically is that R grow faster than $\sqrt{N}$ — in practice: *more draws for bigger samples, and never trust $R = 10$* . We will choose R empirically below, and revisit the question with higher stakes in Chapter 13.

12.4 Rule One: Fix the Draws

Now the rule that separates working MSL code from mysteriously broken MSL code. The optimizer will evaluate the simulated log-likelihood at hundreds of candidate θ values, and it *compares* those values — the whole logic of hill-climbing (Chapter 6) assumes that when $\ell(\theta_1) > \ell(\theta_2)$, the surface really is higher at θ_1 . If each evaluation draws *fresh* randomness, then comparisons mix true surface differences with luck-of-the-draw differences: the surface itself jitters between evaluations. Gradients computed by finite differences (Equation 6.5) are hit hardest — the difference of two nearby, independently-noisy values is mostly noise — and the optimizer stalls, wanders, or “converges” wherever the jitter happened to smile.

The cure costs nothing, because of how Equation 12.3 was written: the randomness enters as *standard normal* draws $z_n^{(r)}$, mapped through the candidate parameters. So draw the z 's **once**, before optimization begins, and reuse the same array at every evaluation. The simulated surface becomes a fixed, smooth, deterministic function of θ — a slightly wrong surface (that's the bias we just measured), but a *stable* one that an optimizer can climb.² This is why the location-scale separation matters: parameters move, draws don't. File it with Chapter 9's precompute-the-invariant principle: the draws are data now.

Watch the failure and the fix side by side — same data, same optimizer, the only difference being whether the draws are redrawn inside the objective:

```
set.seed(121)
## simulate the panel: N=500 shoppers, T=8 offers each
N <- 500; T <- 8
mu0_true <- 1.0; sig0_true <- 1.5
beta_true <- c(price = -1.2, ram16 = 0.6)

b0 <- rnorm(N, mu0_true, sig0_true)          # each person's intercept
price <- runif(N*T, 0.8, 2.4)
ram16 <- rbinom(N*T, 1, 0.5)
id <- rep(1:N, each = T)
U <- b0[id] + beta_true["price"]*price + beta_true["ram16"]*ram16 +
  rlogis(N*T)
y <- as.integer(U > 0)
Xp <- cbind(price, ram16)

## simulated log-likelihood; draws passed in (fixed) or NULL (redrawn: broken)
sll_re <- function(theta, y, Xp, id, N, Z = NULL, R = 100) {
  mu0 <- theta[1]
  sig0 <- exp(theta[2])                      # reparameterize: sigma > 0 always
  beta <- theta[3:4]
  if (is.null(Z)) Z <- matrix(rnorm(N*R), N, R)  # DON'T do this
  R <- ncol(Z)                                # draw count follows the draws
  V <- as.vector(Xp %*% beta)                  # common part, all NT rows
  lseq <- matrix(0, N, R)
  for (r in 1:R) {
    b <- mu0 + sig0 * Z[, r]                  # location-scale map of fixed z's
    P <- plogis(V + b[id])                     # all NT probabilities under draw r
    Pyt <- ifelse(y == 1, P, 1 - P)
  }
}
```

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

      lseq[, r] <- as.vector(rowsum(log(Pyt), id)) # per-person log seq prob
    }
    M <- apply(lseq, 1, max) # log-sum-exp across draws, per person
    sum(M + log(rowMeans(exp(lseq - M))))
  }

```

Implementation notes before the experiment. The `for (r in 1:R)` loop is the *tolerable* loop of Chapter 9’s hybrid doctrine — a hundred iterations, each fully vectorized over all NT rows, versus thousands if we looped over people. `sig0 = exp(theta[2])` is the reparameterization promised in Chapter 10’s footnote: optimize $\log \sigma_0$ unconstrained, and positivity takes care of itself. And averaging $\prod_t P_{nt}$ across draws is done on the log scale with a per-person max-shift — Section 8.3 yet again, essential here because eight-term products are exactly the small numbers that underflow.

```

set.seed(122)
Z_fixed <- matrix(rnorm(N*100), N, 100)
start <- c(0, log(1), 0, 0)

fit_fixed <- optim(start, sll_re, y = y, Xp = Xp, id = id, N = N,
                  Z = Z_fixed, method = "BFGS",
                  control = list(fnscale = -1))
fit_fresh <- optim(start, sll_re, y = y, Xp = Xp, id = id, N = N,
                  Z = NULL, method = "BFGS",
                  control = list(fnscale = -1, maxit = 500))

rbind(truth = c(mu0 = 1.0, log_sig0 = log(1.5), beta_true),
      fixed = fit_fixed$par,
      fresh = fit_fresh$par)

```

	mu0	log_sig0	price	ram16
truth	1.0000000	0.4054651	-1.2000000	0.6000000
fixed	1.0198531	0.3013413	-1.1607217	0.5046907
fresh	0.6719166	0.2649861	-0.8129862	0.1807049

```

c(fixed_convergence = fit_fixed$convergence,
  fresh_convergence = fit_fresh$convergence)

```

fixed_convergence	fresh_convergence
0	0

The fixed-draw fit recovers everything — note $\exp(\hat{\theta}_2) \approx 1.35$ against $\sigma_0 = 1.5$. The fresh-draw fit, *on the identical problem*, lands far from the truth on every coefficient — and, most instructively, its convergence code is a serene 0. The optimizer is not lying, exactly: steps stopped improving the objective, because every proposed move drowned in redrawn noise. It stopped where the jitter trapped it and called that a summit — Chapter 6’s “believing convergence too easily” warning, weaponized. One line of code separates the two fits. This experiment is the cheapest tuition you will ever pay on this topic; the expensive version is three days of debugging a mixed logit.

12.5 Better Draws: Halton Sequences at Work

Section 2.5 promised that quasi-random draws would earn their keep when integrals arrived. They have arrived. Halton draws — `qnorm(halton(R, base))`, evenly covering the space instead of clumping — reduce the *noise* of each simulated probability, which reduces both the Jensen bias (driven by noise variance) and the sampling contribution of simulation to the estimator. The standard demonstration: simulate our known sequence probability many times with pseudo-random versus Halton draws³ and compare accuracy at each R :

```
halton <- function(n, base) {                                # from @sec-draws
  r <- numeric(n)
  for (i in 1:n) { f <- 1; x <- 0; k <- i
    while (k > 0) { f <- f/base; x <- x + f*(k %% base); k <- k %% base }
    r[i] <- x }
  r
}

set.seed(123)
rmse_at <- function(R) {
  pseudo <- replicate(500, mean(sapply(rnorm(R, 0.5, 1), seq_prob)))
  ## Halton: shift the sequence by a random offset each replication
  hbase <- halton(R + 50, 2)[- (1:50)]
  haltn <- replicate(500, {
    u <- (hbase + runif(1)) %% 1
    mean(sapply(qnorm(u, 0.5, 1), seq_prob))
  })
  c(R = R, pseudo = sqrt(mean((pseudo - truth)^2)),
    halton = sqrt(mean((haltn - truth)^2)))
}
```

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

```
acc <- data.frame(t(sapply(c(10, 25, 50, 100, 250), rmse_at)))

ggplot(reshape(acc, direction = "long", varying = c("pseudo", "halton"),
              v.names = "rmse", timevar = "type",
              times = c("pseudo", "halton")),
       aes(R, rmse, linetype = type)) +
  geom_line() + geom_point() +
  scale_x_log10() + scale_y_log10() +
  labs(x = "draws R (log)", y = "RMSE (log)", linetype = NULL)
```

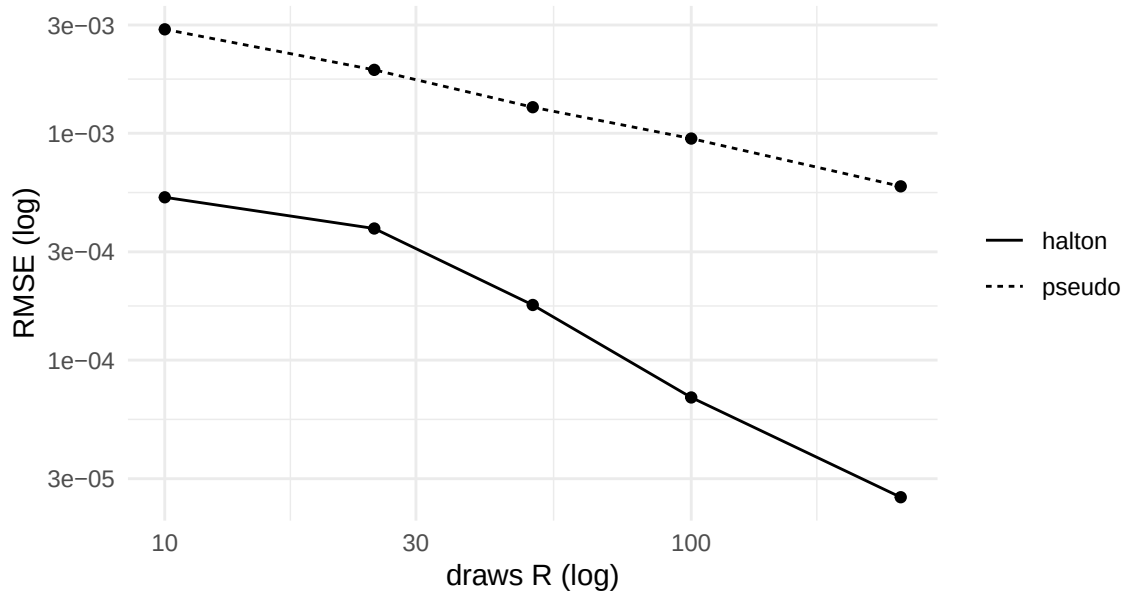


Figure 12.2: Root mean squared error of the simulated sequence probability against the number of draws, pseudo-random versus Halton (log-log scale). Halton’s systematic coverage delivers the accuracy of several hundred pseudo-random draws with a fraction of the count.

The Halton curve sits well below the pseudo-random curve at every R — reading horizontally, 50 Halton draws deliver roughly the accuracy of hundreds of pseudo-random draws, consistent with the folklore and with published comparisons (K. E. Train 2009, sec. 9.3.4). In multi-person MSL the standard practice refines this slightly (different sequence segments per person, a discarded burn-in, randomized shifts as above — and different prime bases per dimension when Chapter 13 goes multivariate), but the headline stands: *when draws approximate integrals, coverage beats randomness*, and the draw budget you save is real computing time (Table 9.1).

12.6 How Many Draws? An Empirical Answer

Theory says “ R growing faster than $\sqrt{N}$ ”; practice wants a number. The working procedure is disarmingly simple: *re-estimate with increasing R until the answers stop moving at the precision you care about* — the same “increase until stable” logic used for grid resolution or chain length, applied to the draw count. On our problem, with Halton draws throughout:

```
set.seed(124)
fit_at_R <- function(R) {
  ## person n gets segment n of one long Halton sequence, then a common
  ## random shift and the qnorm transform (the scheme @sec-mixed-msl reuses)
  h <- halton(N*R + 50, 2)[- (1:50)]
  hseg <- matrix(qnorm((h + runif(1)) %% 1), N, R, byrow = TRUE)
  optim(start, sll_re, y = y, Xp = Xp, id = id, N = N,
        Z = hseg, method = "BFGS", control = list(fnscale = -1))$par
}
res <- sapply(c(25, 50, 100, 200), fit_at_R)
rownames(res) <- c("mu0", "log_sig0", "price", "ram16")
round(rbind(res, sigma0 = exp(res[2, ])), 3)
```

	[,1]	[,2]	[,3]	[,4]
mu0	1.015	0.995	0.998	0.997
log_sig0	0.312	0.304	0.303	0.299
price	-1.162	-1.159	-1.161	-1.161
ram16	0.505	0.504	0.505	0.505
sigma0	1.367	1.355	1.353	1.349

By $R = 100$ the estimates have settled to the second decimal — the reporting precision of any study — and $R = 200$ confirms it. That, not a universal constant, is the answer to “how many draws”: enough that doubling them changes nothing you would report. Note the diagnostic is *self-service*; no theorem consulted, just the estimator interrogated about its own stability.

12.7 The Estimator, Named and Situated

Assemble the pieces formally. The **maximum simulated likelihood estimator** maximizes $\sum_n \log \widehat{\Pr}(\mathbf{y}_n | \boldsymbol{\theta})$ with the probability simulated by Equation 12.3 from draws fixed across evaluations. Its properties (McFadden and Train 2000; K. E. Train 2009, Ch. 10): consistent

and asymptotically equivalent to exact ML provided R grows suitably with N ; standard errors from the Hessian exactly as in Chapter 6.⁴

MSL has two siblings you should recognize in the literature, both born of the same simulate-the-integral move applied to different estimating equations. The **method of simulated moments** (MSM) simulates the model’s predicted probabilities and matches them to observed choice frequencies; because the simulated probabilities enter *linearly* (no log), MSM is consistent even at fixed small R — the Jensen problem never arises — at some cost in efficiency and in the craft of choosing moments. The **method of simulated scores** (MSS) simulates the score equation itself, seeking ML’s efficiency without the log’s bias; its difficulty is that unbiased score simulators are hard to construct. MSL’s dominance in practice comes down to convenience: it is the estimator you already know (Chapter 6) with a simulator inside, and its bias is manageable by the R -doubling diagnostic above. We follow practice.

One more sibling deserves a placement note: the random-parameter models estimated here and in Chapter 13 by MSL are estimated in Chapter 17 by Markov chain Monte Carlo instead — same models, different engine — and the comparison between the two engines is the hinge on which this book turns (Chapter 14). Keep the mental bookmark.

12.8 Key Learnings

- When heterogeneity is continuous, the class sum of Chapter 11 becomes an integral (Equation 12.2) with no closed form — but it is an expectation, so the Chapter 1 move applies: simulate it (Equation 12.3), drawing standard normals and mapping them through candidate parameters.
- The probability simulator is unbiased; its *log* is biased downward by Jensen’s inequality — we measured the bias, watched it die at rate $1/R$, and noted the formal requirement (R growing faster than $\sqrt{N}$).
- **Fix the draws across evaluations.** Fresh randomness per evaluation makes the objective surface jitter and breaks the optimizer — demonstrated live: identical problems, one line different, recovery versus rubble. Draws are data; precompute them.
- Reparameterize positive parameters ($\sigma = e^\theta$), average products of probabilities on the log scale with max-shifts, and keep the per-draw loop vectorized over rows — the Chapter 9 and Section 8.3 habits, now load-bearing.
- Halton draws beat pseudo-random draws decisively for integration (RMSE figure: 50 Halton draws $\approx$ hundreds of pseudo-random ones); choose R by the doubling diagnostic — increase until nothing you’d report moves.

⁴Sharp eyes will connect this figure to Chapter 17’s normal-prior HB: run on these polarized data, the normal hierarchy would shrink both camps toward a middle where no one lives. Flexible mixing is not decoration; on the wrong population shape, the normal assumption distorts every downstream deliverable in Chapter 21.

- MSL = Chapter 6's climber + a fixed-draw simulator inside; MSM and MSS are the linear-in-simulator and score-based siblings. Next: the same machinery, six correlated dimensions — the mixed MNL.

13 The Mixed MNL Model by MSL

This chapter assembles the summit model of the likelihood track. The **mixed multinomial logit** — MNL choice behavior with coefficients that vary continuously across people — is where Part II has been heading since heterogeneity entered in Chapter 11: the discrete classes melt into a continuous distribution, the random intercept of Chapter 12 grows into a random *vector*, and every tool we have built gets used at once. It is also, not coincidentally, the single most-used model in applied choice work, from transportation demand to virtually every commercial conjoint study run today (McFadden and Train 2000; K. E. Train 2009, Ch. 6).

By the end of the chapter the model will be simulated, estimated by MSL, validated against `mlogit`, and audited for simulation error — and we will have felt, concretely, where the approach strains. That strain is data for Chapter 14.

13.1 Random Coefficients, Two Ways to Read Them

The model adds one letter to the MNL. Utility is still linear with Gumbel errors, but the coefficient vector now belongs to the person:

$$U_{ntj} = \mathbf{x}'_{ntj}\beta_n + \varepsilon_{ntj}, \quad \beta_n \sim N(\bar{\beta}, \Sigma), \quad \varepsilon_{ntj} \stackrel{iid}{\sim} \text{Gumbel} \quad (13.1)$$

Each person draws their taste vector β_n once from a multivariate normal population — mean $\bar{\beta}$, covariance Σ — and carries it through all their choice tasks. The estimation targets are the *population* parameters $\theta = (\bar{\beta}, \Sigma)$: what tastes are like on average, how much they vary, and (through Σ 's off-diagonals) which tastes travel together.

Two readings of Equation 13.1 coexist, and both matter:

- **The taste-heterogeneity reading.** People genuinely differ — some care intensely about price, some about brand — and Σ is the market's heterogeneity, the quantity segmentation and targeting monetize.
- **The substitution-pattern reading.** Integrate the person-level MNL over the population and the resulting *aggregate* choice probabilities no longer satisfy IIA: people who like Apples are the same people across tasks, so alternatives similar in attribute space draw from overlapping customer pools — correlation in utilities, achieved through coefficients rather than Chapter 10's nests, and *estimated* rather than pre-specified. This is the data-driven substitution flexibility that Section 10.8 promised.

The second reading has a remarkable theoretical guarantee: McFadden and Train (2000) prove that a mixed logit with appropriately chosen variables and mixing distribution can approximate *any* random utility model’s choice probabilities arbitrarily well. Nested logit, probit, anything — all live in the mixed logit’s closure. The theorem is an existence result, not a recipe (it doesn’t say the approximating specification is parsimonious or findable), but it explains the model’s status: this is not one more family member; it is a universal chassis.

13.2 Specification Choices

Equation 13.1 as written leaves the modeler three decisions.

Which coefficients vary? Nothing requires all K . Our specification — motivated the way an applied researcher would — lets tastes vary where taste variation is behaviorally plausible and commercially interesting: the Apple premium (Mac people versus the indifferent), the top-memory premium (power users versus browsers), and price sensitivity (budgets differ). The `dell`, `ram16`, and `screen15` coefficients stay fixed.¹ In the notation below, `rc` indexes the random subset.

What mixing distribution? Normal, here. Its known weakness: it puts mass on *both signs* of every random coefficient, so our specification implies some fraction of consumers with positive price coefficients — about $\Phi(-1.2/0.7) \approx 4\%$ prefer expensive, all else equal, under the truth below. For this chapter we accept the blemish (it is small, and normality keeps every layer transparent); the standard fix — a lognormal for the price coefficient, built from the Chapter 2 transformation — is deployed in Chapter 17, and more flexible mixing distributions are Chapter 22 material (K. Train 2016).

Correlated or independent? A full Σ has $K_r(K_r + 1)/2$ free elements for K_r random coefficients, kept positive-definite by optimizing over its Cholesky factor L (Section 2.4 — estimate L ’s elements, log-parameterizing the diagonal, and $\Sigma = LL'$ is valid by construction). We estimate the *diagonal* (independent) version: three standard deviations, log-parameterized as in Chapter 12. The reason is worth stating baldly now and remembering at the chapter’s end: with MSL, every step toward a full covariance matrix costs real optimization pain, and we will tally that cost rather than pay it — then watch Chapter 17 get the full matrix essentially for free.

The truth for our study:

$$\bar{\beta}^* = (0.5, 1.0, 0.6, 0.9, 0.3, -1.2), \quad \sigma_{\text{apple}}^* = 0.8, \quad \sigma_{\text{ram32}}^* = 0.6, \quad \sigma_{\text{price}}^* = 0.7 \quad (13.2)$$

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

Means as always (Equation 7.6); spreads large enough that heterogeneity is the story, not a garnish — a price-coefficient standard deviation of 0.7 around -1.2 makes the market’s price sensitivity range enormously.

13.3 The Likelihood

Assemble it by the now-familiar hierarchy logic — third time, so briskly. Conditional on β_n : the person’s sequence probability is a product of MNL terms over their tasks. Unconditional: integrate over the population,

$$\Pr(y_n | \theta) = \int \left[\prod_{t=1}^T P_{nt}(\beta) \right] f(\beta | \bar{\beta}, \Sigma) d\beta \quad (13.3)$$

— the integral outside the product (the person keeps their draw), no closed form (logit kernel against normal density, now in K_r dimensions), and therefore MSL (Chapter 12): fixed standard-normal draws $\mathbf{z}_n^{(r)}$, mapped through candidate parameters, averaged on the log scale. The only novelty versus Chapter 12 is dimension — and dimension is precisely where simulation’s advantage over quadrature lives (Chapter 12’s footnote, now cashed).

One bookkeeping note: Equation 13.3 is written over the full coefficient vector, but with `de11`, `ram16`, and `screen15` held fixed their “distribution” is a point mass — those dimensions integrate out trivially, and the integral we actually simulate is $K_r = 3$ -dimensional, over the random subset only.

13.4 Simulating the Study

The simulator is Section 7.6’s with one insertion — person-level coefficient draws, via the location-scale map (diagonal case) that Chapter 2 built:

```
sim_mixed_data <- function(N, T, J, bar_beta, sd_beta, rc) {
  df <- sim_laptop_design(N, T, J)
  X <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]
  K <- ncol(X)
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))

  betas <- matrix(rep(bar_beta, each = N), N, K)      # start everyone at the mean
  for (k in rc) betas[, k] <- betas[, k] + sd_beta[k] * rnorm(N)

  V <- rowSums(X * betas[df$id, ])                    # each row: its person's beta
```

```

    eps <- -log(-log(runif(nrow(df))))
    U    <- V + eps
    df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))
    attr(df, "betas") <- betas                                # answer key, person level
    df
  }

set.seed(130)
bar_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9,
              screen15 = 0.3, price = -1.2)
sd_true  <- c(0, 0.8, 0, 0.6, 0, 0.7)
rc       <- c(2, 4, 6)                                       # apple, ram32, price

d <- sim_mixed_data(N = 500, T = 8, J = 3, bar_true, sd_true, rc)
X <- model.matrix(~ brand + ram + screen + price, data = d)[, -1]
y <- d$choice
task_id <- cumsum(!duplicated(d[, c("id", "task")]))
id <- d$id; n_id <- 500; K <- ncol(X)

```

The `betas[df$id, ]` indexing is Chapter 11’s row-specific-coefficients idiom with classes replaced by persons. And note we stored the true individual β_n ’s as an attribute — Chapter 14 will grade individual-level estimates against them.

A heterogeneity-specific sanity check joins the usual battery: *within-person* behavior should be more consistent than the population is. Cheap version — among people who ever bought the most expensive laptop in a task, what fraction of their other choices were also top-priced, versus the population rate?

```

chosen <- d[d$choice == 1, ]
top_price <- as.integer(ave(d$price, task_id, FUN = max)[d$choice == 1] == chosen$price)
person_rate <- tapply(top_price, chosen$id, mean)
c(population_rate = round(mean(top_price), 2),
  sd_across_people = round(sd(person_rate), 2))

```

population_rate	sd_across_people
0.21	0.15

The across-person spread in that rate is far larger than binomial noise from eight tasks would produce — persistent individual differences, visible before any model is fit. (The formal version of this check is the LM test of McFadden and Train (2000); the informal version usually suffices to know heterogeneity is worth modeling.)

13.5 The Simulated Likelihood, With Its Gradient

Here is the estimation engine — the chapter’s centerpiece function. It computes the simulated log-likelihood of Equation 13.3 and, optionally, its *analytic gradient*; the parameter vector stacks the K means and the log standard deviations of the random subset:

```
msl_mixed <- function(theta, y, X, task_id, id, n_id, rc, Z,
                      want_grad = FALSE) {
  K <- ncol(X)
  R <- dim(Z)[2]
  bar <- theta[1:K]
  s <- exp(theta[(K+1):(K+length(rc))])
  id_of_task <- id[!duplicated(task_id)]

  lseq <- matrix(0, n_id, R) # log P(sequence_n | draw r)
  if (want_grad) {
    Gbar <- array(0, c(n_id, R, K))
    Gs <- array(0, c(n_id, R, length(rc)))
  }
  for (r in 1:R) {
    betas <- matrix(rep(bar, each = n_id), n_id, K)
    for (jj in seq_along(rc))
      betas[, rc[jj]] <- betas[, rc[jj]] + s[jj] * Z[, r, jj]
    V <- rowSums(X * betas[id, ])
    Vmax <- ave(V, task_id, FUN = max)
    eV <- exp(V - Vmax)
    logP <- V - Vmax - log(rowsum(eV, task_id)[task_id])
    lseq[, r] <- as.vector(rowsum(logP[y == 1], id_of_task))
    if (want_grad) {
      p <- eV / rowsum(eV, task_id)[task_id]
      Gp <- rowsum(X * (y - p), id) # per-person MNL score
      Gbar[, r, ] <- Gp # d l_nr / d bar_beta
      for (jj in seq_along(rc))
        Gs[, r, jj] <- Gp[, rc[jj]] * Z[, r, jj] * s[jj] # d l_nr / d log sigma_jj
    }
  }
  M <- apply(lseq, 1, max) # log-mean-exp, per person
  W <- exp(lseq - M)
  Sw <- rowSums(W)
  ll <- sum(M + log(Sw / R))
  if (!want_grad) return(ll)
```

```

w      <- W / Sw                                     # person-by-draw weights
gbar <- sapply(1:K, function(k) sum(w * Gbar[, , k]))
gs    <- sapply(seq_along(rc), function(jj) sum(w * Gs[, , jj]))
list(ll = ll, grad = c(gbar, gs))
}

```

Read it in layers, because every layer is a callback with a job:

- **The draw loop** (for (r in 1:R)) is Chapter 9’s *tolerable* loop — R iterations, each one fully vectorized over all 12,000 rows via the Chapter 8 kernel (max-shift, `rowsum()`, broadcast). Per-person sequence log-probabilities come from one more `rowsum(..., id_of_task)`.
- **The log-mean-exp across draws** is Chapter 12’s device: sequence probabilities are underflow bait, so the average over draws happens under a per-person max-shift.
- **The gradient** is where the mathematics rewards attention. Write $\ell_{nr} = \log \prod_t P_{nt}(\beta_n^{(r)})$ for person n ’s log sequence probability under draw r — the code’s `lseq[n, r]`, a *building block* of the log-likelihood, not the log-likelihood $\ell(\theta)$ itself. The simulated probability is $\widehat{\Pr}(\mathbf{y}_n) = \frac{1}{R} \sum_r e^{\ell_{nr}}$, and differentiating its log is one chain-rule line:

$$\frac{\partial}{\partial \theta} \log \frac{1}{R} \sum_r e^{\ell_{nr}} = \frac{\sum_r e^{\ell_{nr}} \partial \ell_{nr} / \partial \theta}{\sum_{r'} e^{\ell_{nr'}}}$$

(the $1/R$ cancels top and bottom) — a *weighted average of per-draw gradients*, each draw’s gradient counted by how well that draw explains the person:

$$\frac{\partial}{\partial \theta} \log \widehat{\Pr}(\mathbf{y}_n) = \sum_r w_{nr} \frac{\partial \ell_{nr}}{\partial \theta}, \quad w_{nr} = \frac{e^{\ell_{nr}}}{\sum_{r'} e^{\ell_{nr'}}} \quad (13.4)$$

And the per-draw gradient is an old friend: with respect to the means it is exactly the MNL score $\mathbf{X}'(\mathbf{y} - \mathbf{p})$ of Equation 8.4, summed within person; with respect to $\log \sigma_k$, the chain rule appends a factor $z_{nk}^{(r)} \sigma_k$. Look hard at those weights w_{nr} : they are the conditional distribution of “which draw is this person” given their choices — Bayes’ rule, materializing inside a frequentist gradient, and the draw-level twin of Chapter 11’s class weights w_{nc} (Equation 11.3): prior density times sequence likelihood, normalized, with “which class” replaced by “which draw.” Chapter 14 picks up precisely this object.

Per standing policy, the analytic gradient faces the numerical audit before we trust it:

```

set.seed(131)
R <- 50
Z <- array(0, c(n_id, R, length(rc)))
bases <- c(2, 3, 5)                                # one prime base per random dimension

```

```

for (jj in seq_along(rc)) {
  h <- halton(n_id*R + 50, bases[jj])[-(1:50)]
  Z[, , jj] <- qnorm(matrix((h + runif(1)) %% 1, n_id, R, byrow = TRUE))
}

theta_test <- c(bar_true, log(c(0.8, 0.6, 0.7)))
g <- msl_mixed(theta_test, y, X, task_id, id, n_id, rc, Z, want_grad = TRUE)
num <- sapply(1:9, function(k) {
  e <- replace(numeric(9), k, 1e-5)
  (msl_mixed(theta_test + e, y, X, task_id, id, n_id, rc, Z) -
   msl_mixed(theta_test - e, y, X, task_id, id, n_id, rc, Z)) / 2e-5
})
round(rbind(analytic = g$grad, numeric = num), 4)

```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]
analytic	14.5077	6.6827	8.2526	-2.1064	21.196	-9.0848	-15.7974	-6.0929	-13.5267
numeric	14.5077	6.6827	8.2526	-2.1064	21.196	-9.0848	-15.7974	-6.0929	-13.5267

The draws live in an $N \times R \times K_r$ array: $Z[n, r, jj]$ is person n 's r -th standard-normal draw for the jj -th *random* coefficient — jj runs over the random subset rc , not over all K columns of X ; when the math writes $z_{nk}^{(r)}$, k ranges over that subset only.

Agreement to four decimals. (The draws: Halton with a different prime base per dimension — the multivariate practice promised in Chapter 12 — person n taking segment n of each sequence, randomized by a common shift.)

13.6 Estimation

Starting values from the model hierarchy, per Chapter 6's doctrine: the plain MNL supplies the means (it estimates a taste-blurred but well-centered version of $\bar{\beta}$), and moderate spreads start the σ 's:

```

loglik_mnl <- function(beta, y, X, task_id) {
  V <- as.vector(X %*% beta); Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}
mnl <- optim(rep(0, K), loglik_mnl, y = y, X = X, task_id = task_id,
             method = "BFGS", control = list(fnscale = -1))

```

```

start <- c(mnl$par, rep(log(0.5), 3))
fit50 <- optim(start,
               fn = function(th, ...) msl_mixed(th, ...),
               gr = function(th, ...) msl_mixed(th, ..., want_grad = TRUE)$grad,
               y = y, X = X, task_id = task_id, id = id, n_id = n_id,
               rc = rc, Z = Z,
               method = "BFGS", control = list(fnscale = -1, maxit = 300))

```

The Chapter 12 doubling diagnostic — same fit at $R = 100$, warm-started from the $R = 50$ solution:

```

set.seed(131)
R2 <- 100
Z2 <- array(0, c(n_id, R2, length(rc)))
for (jj in seq_along(rc)) {
  h <- halton(n_id*R2 + 50, bases[jj])[-(1:50)]
  Z2[, , jj] <- qnorm(matrix((h + runif(1)) %% 1, n_id, R2, byrow = TRUE))
}
fit100 <- optim(fit50$par,
                fn = function(th, ...) msl_mixed(th, ...),
                gr = function(th, ...) msl_mixed(th, ..., want_grad = TRUE)$grad,
                y = y, X = X, task_id = task_id, id = id, n_id = n_id,
                rc = rc, Z = Z2,
                method = "BFGS", control = list(fnscale = -1, maxit = 300),
                hessian = TRUE)

comp <- rbind(truth = c(bar_true, log(sd_true[rc])),
              R_50 = fit50$par,
              R_100 = fit100$par)
colnames(comp) <- c(colnames(X), paste0("log_sd_", colnames(X)[rc]))
round(comp, 3)

```

	brandDell	brandApple	ram16	ram32	screen15	price	log_sd_brandApple
truth	0.50	1.000	0.600	0.900	0.300	-1.200	-0.223
R_50	0.55	1.037	0.614	0.892	0.336	-1.203	-0.402
R_100	0.55	1.034	0.615	0.891	0.337	-1.217	-0.388

	log_sd_ram32	log_sd_price
truth	-0.511	-0.357
R_50	-0.660	-0.620
R_100	-0.611	-0.656

Stable across the doubling — $R = 100$ with Halton draws is enough here. The results table, with standard errors from the Hessian and the standard deviations mapped back from the log scale:

```
se <- sqrt(diag(solve(-fit100$hessian)))
tab <- data.frame(
  truth    = c(bar_true, sd_true[rc]),
  estimate = c(fit100$par[1:6], exp(fit100$par[7:9])),
  se       = c(se[1:6], exp(fit100$par[7:9]) * se[7:9]) # delta method
)
rownames(tab) <- c(colnames(X), paste0("sd_", colnames(X)[rc]))
round(tab, 3)
```

	truth	estimate	se
brandDell	0.5	0.550	0.056
brandApple	1.0	1.034	0.066
ram16	0.6	0.615	0.056
ram32	0.9	0.891	0.062
screen15	0.3	0.337	0.044
price	-1.2	-1.217	0.060
sd_brandApple	0.8	0.679	0.080
sd_ram32	0.6	0.543	0.085
sd_price	0.7	0.519	0.099

Read the table with Section 8.8 calibration in mind. The six means recover cleanly. The three standard deviations land within roughly one-and-a-half standard errors of the truth — respectable, and note their standard errors are several times larger than the means': *spreads are simply harder to learn than centers*, because they live in second moments of behavior that eight tasks per person illuminate dimly. (The delta method converts log-scale standard errors to the σ scale — Chapter 21 treats it properly.)

13.7 Validation Against mlogit

`mlogit` estimates mixed logits through its `rpar` argument (naming which coefficients are random and their distribution), with Halton draws and panel weighting — the same estimator, independently implemented:

```
library(dfidx)
library(mlogit)
```

```

md <- d
md$chid <- task_id; md$choice <- as.logical(md$choice)
md <- dfidx(md, idx = list(c("chid", "id"), "alt"))

fit_ml <- mlogit(choice ~ brand + ram + screen + price | 0, data = md,
  rpar = c(brandApple = "n", ram32 = "n", price = "n"),
  R = 100, halton = NA, panel = TRUE)

round(rbind(ours = c(fit100$par[1:6], exp(fit100$par[7:9])),
  mlogit = coef(fit_ml)[c(1:6, 7:9)]), 3)

```

	brandDell	brandApple	ram16	ram32	screen15	price	sd.brandApple	sd.ram32
ours	0.550	1.034	0.615	0.891	0.337	-1.217	0.679	0.543
mlogit	0.551	1.036	0.614	0.889	0.337	-1.214	0.681	0.537
	sd.price							
ours	0.519							
mlogit	0.526							

Agreement to the second decimal across all nine parameters — not exact to five decimals as in Chapter 8, and the reason is instructive rather than troubling: *two MSL implementations share a model but not a simulated likelihood*. Different Halton details (sequence handling, burn-in, person assignment) mean slightly different objective surfaces whose maximizers differ by simulation noise. Which raises exactly the right question: how big is that noise?

13.8 The Simulation-Error Audit

Chapter 12 promised an accounting of the error component that Hessian standard errors ignore: variation in $\hat{\theta}$ caused by the *draws*, not the data. Measuring it is refreshingly direct — re-estimate on the *same data* with several independent draw sets (different randomizing shifts) and watch the estimates move:

```

set.seed(132)
audit <- sapply(1:5, function(a) {
  Za <- array(0, c(n_id, R2, length(rc)))
  for (jj in seq_along(rc)) {
    h <- halton(n_id*R2 + 50, bases[jj])[-(1:50)]
    Za[, , jj] <- qnorm(matrix((h + runif(1)) %% 1, n_id, R2, byrow = TRUE))
  }
  optim(fit100$par,
    fn = function(th, ...) msl_mixed(th, ...),

```

```

    gr = function(th, ...) msl_mixed(th, ..., want_grad = TRUE)$grad,
    y = y, X = X, task_id = task_id, id = id, n_id = n_id,
    rc = rc, Z = Za,
    method = "BFGS", control = list(fnscale = -1, maxit = 300))$par
  })
round(rbind(sim_error_sd = apply(audit, 1, sd),
    statistical_se = se), 4)

```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]
sim_error_sd	0.0012	0.0034	0.0009	0.0025	0.0010	0.0020	0.0069	0.0219	0.0197
statistical_se	0.0560	0.0660	0.0563	0.0618	0.0444	0.0599	0.1174	0.1573	0.1910

The verdict at $R = 100$ Halton draws: simulation-induced spread runs an order of magnitude (or more) below the statistical standard errors, for every parameter. The draws are not the binding source of uncertainty — which is precisely what “enough draws” *means*, now measured rather than assumed. Keep this audit in your repertoire; it costs a few refits and answers the referee’s question before it is asked.

13.9 Feeling the Strain

Now the honest tally this chapter owes Chapter 14. Everything above *worked* — and consider what it took and how it scales.

Time. Each $R = 100$ fit above runs in minutes — entirely tolerable — but the multiplication table is merciless: the multiple-starts discipline (Chapter 11; mixture-like surfaces lurk here too²), the draw-doubling diagnostic, the simulation audit, and any replication study each multiply that base cost. Our modest audit already consumed five fits.

Dimension. Three random coefficients, independent, cost 9 parameters. The full menu — all six random, freely correlated — costs $6 + 21 = 27$: a Cholesky factor’s worth of curvature directions for BFGS to learn, each additional random dimension another prime base of Halton draws, another axis the fixed- R simulator must cover. Published experience and our own arithmetic agree: MSL’s practical frontier sits near “a handful of random coefficients, correlations chosen sparingly” , Chapter 17.

And the deliverable gap. After all of it, the output is $(\hat{\beta}, \hat{\Sigma})$ — the population. The individual β_n ’s that applications consume are not in the output; extracting them takes a

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

second, bolt-on computation whose uncertainty accounting is awkward at best. That extraction — what it can do, and exactly where it creaks — is Chapter 14, the short chapter that closes the likelihood track and opens the Bayesian one.

One housekeeping step first — save everything the next chapters need (data with its answer key, estimates, draws):

```
saveRDS(list(data = d, betas_true = attr(d, "betas"),
            bar_true = bar_true, sd_true = sd_true, rc = rc,
            theta_hat = fit100$par, Z = Z2),
        file = "../data/mixed_study.rds")
```

13.10 Key Learnings

- The mixed MNL gives each person a coefficient vector drawn from $N(\bar{\beta}, \Sigma)$ (Equation 13.1): taste heterogeneity and data-driven, IIA-breaking substitution in one model — with the McFadden–Train theorem certifying the family can mimic any RUM.
- Specification is three real choices: which coefficients vary (identification from T tasks is finite), what mixing distribution (normal’s wrong-sign mass; lognormal fix deferred to Chapter 17), and whether correlations enter (Cholesky-parameterized when they do; diagonal here, with the full matrix deliberately deferred).
- The likelihood is the Chapter 11 hierarchy with an integral (Equation 13.3), estimated by Chapter 12’s MSL: fixed multivariate Halton draws (one prime base per dimension), log-mean-exp across draws, reparameterized scales.
- The MSL gradient (Equation 13.4) is a *conditionally-weighted average of per-draw MNL scores* — $\mathbf{X}'(\mathbf{y} - \mathbf{p})$ yet again, with Bayes-rule weights foreshadowing Chapter 14. Verified numerically, it cut fit times several-fold.
- Recovery succeeded (spreads harder than means, visibly), `mlogit` agreed to the second decimal — with the residual gap correctly attributed to simulation noise, which the five-refit audit then *measured*: an order of magnitude below statistical error at $R = 100$ Halton.
- The strain is real and structural: costs multiply across starts, draws, audits, and dimensions (27 parameters for the full covariance), and the output is still only the population. The person-level deliverable — and likelihood’s last-mile problem with it — is next.

14 The Limits of Likelihood

This short chapter is a hinge. Part II climbed as far as maximum likelihood reaches: the mixed MNL of Chapter 13 estimates an entire *population* of preferences — means, spreads, correlations — and does it well. But look carefully at what it delivers. Population parameters. Not *people*.

For many research questions, population parameters are the answer. For the questions that dominate applied choice modeling, they are not. A firm running the laptop study doesn't stop at “price sensitivity has mean -1.2 and standard deviation 0.7”; it wants to know *which respondents* are the price-insensitive premium seekers, to size that segment's demand for a new configuration, to simulate a market where each simulated buyer behaves like a particular real one (Chapter 21). Personalization, targeting, segment-level demand — the deliverables of modern conjoint practice (Allenby, Hardt, and Rossi 2019) — all need estimates of the individual β_n 's. This chapter asks whether maximum likelihood can produce them. The answer — *almost, awkwardly, without honest uncertainty* — is the reason Part III exists.

14.1 Attempt One: A Likelihood Per Person

The direct approach: every respondent gave us eight choices; eight choices carry information about that person's preferences; so run the Chapter 8 estimator *separately on each person's data*. No population model at all — just 500 small MNL estimations.

Load the mixed-logit study saved in Chapter 13 (data, true individual coefficients, and the MSL estimates of the population parameters — everything this chapter needs):

```
mixed      <- readRDS("../data/mixed_study.rds")
d          <- mixed$data
betas_true <- mixed$betas_true           # the answer key: true beta_n, N x K
X <- model.matrix(~ brand + ram + screen + price, data = d)[, -1]
colnames(betas_true) <- colnames(X)
y <- d$choice
task_id <- cumsum(!duplicated(d[, c("id", "task")]))
id <- d$id
K <- ncol(X)
```

```

loglik_mnl <- function(beta, y, X, task_id) {      # @sec-mnl-mle, unchanged
  V      <- as.vector(X %*% beta)
  Vmax <- ave(V, task_id, FUN = max)
  eV     <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}

fit_one_person <- function(n) {
  rows <- which(id == n)
  tid  <- task_id[rows] - min(task_id[rows]) + 1
  optim(rep(0, K), loglik_mnl, y = y[rows], X = X[rows, ], task_id = tid,
        method = "BFGS", control = list(fnscale = -1, maxit = 200))$par
}

per_person <- t(sapply(1:500, fit_one_person))
colnames(per_person) <- colnames(X)

```

Five hundred estimations later, compare the per-person price coefficients to the truth we smuggled out of the simulator:

```

summary_stats <- rbind(
  truth      = summary(betas_true[, "price"]),
  per_person = summary(per_person[, "price"])
)
round(summary_stats, 1)

```

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
truth	-3.3	-1.7	-1.2	-1.2	-0.8	1.3
per_person	-1282.6	-27.3	-3.2	-23.3	-0.4	141.2

The true price coefficients span a civilized range around -1.2. The per-person estimates span *tens or hundreds* in both directions. This is not a bug in our code; it is the estimator working as designed on a task it cannot do. Eight choices among three alternatives simply do not contain enough information to pin down six utility parameters, and the failure has two textbook faces:

- **Perfect separation.** A respondent who happened to always choose the cheapest laptop is *perfectly explained* by $\beta_{\text{price}} = -\infty$: their eight-task likelihood increases monotonically as the coefficient runs away, and `optim()` chases it until stopped by the iteration cap. With $T = 8$, such respondents are not rare accidents; they are a sizable fraction of any sample.

- **The incidental parameters problem.** Even without separation, there is a deeper pathology: when the number of parameters grows with the number of observations (six new parameters per respondent), the usual asymptotic guarantees of Chapter 5 never engage. More respondents give you more *parameters to estimate*, not more information per parameter; T is what would need to grow, and N is fixed by the survey. Per-person MLE is inconsistent in the dimension that matters.¹

```
df_plot <- rbind(
  data.frame(beta = pmax(pmin(per_person[, "price"], 10), -10),
             which = "per-person MLE"),
  data.frame(beta = betas_true[, "price"], which = "truth")
)
ggplot(df_plot, aes(beta, fill = which)) +
  geom_histogram(bins = 60, position = "identity", alpha = 0.6) +
  scale_fill_manual(values = c("grey30", "grey70")) +
  labs(x = expression(beta[price]), y = "count", fill = NULL)
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

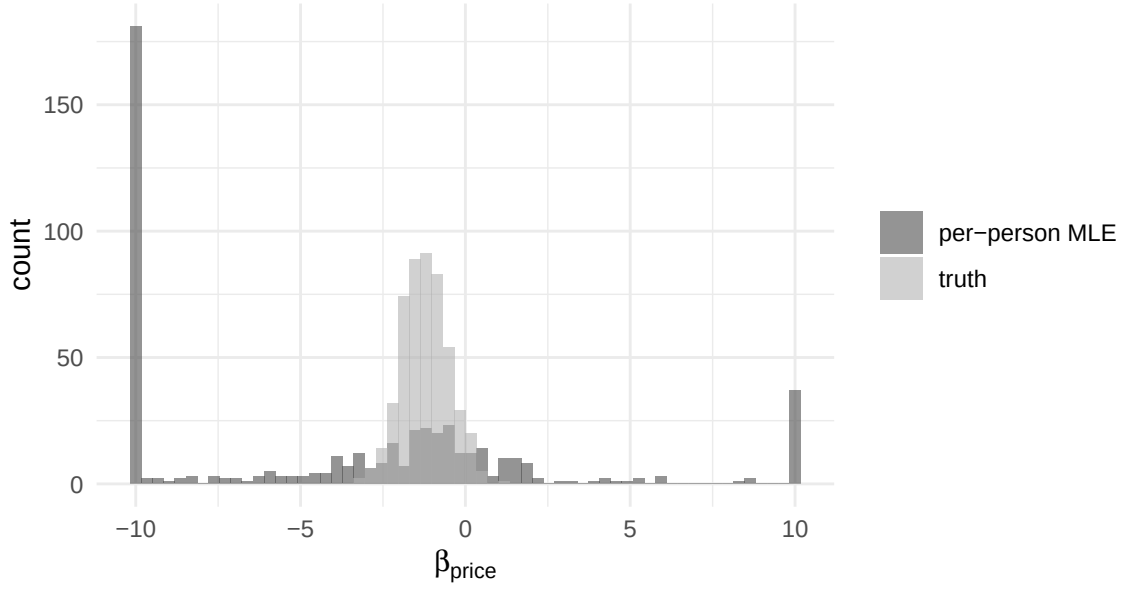


Figure 14.1: Per-person MLE price coefficients (clipped to ± 10 for display) against the true distribution of price coefficients. Eight choices per person cannot support six free parameters: mass piles at extreme values as separation sends estimates toward infinity.

Verdict: treating each person’s coefficients as free parameters is a dead end — not “needs more data” but *structurally* dead, because the survey’s T will never be large. Whatever produces individual estimates must let respondents share information.

14.2 Attempt Two: Condition on the Population

Which is exactly what the mixed logit already does — it just forgets to tell us about individuals. Recall its structure: β_n drawn from a population distribution $f(\beta \mid \theta)$, then choices from an MNL given β_n . The MSL estimate $\hat{\theta}$ describes the population. But the model contains a sharper question we haven’t asked: *given* that person n made the specific choices $\mathbf{y}_n$, which parts of the population distribution are they likely to sit in?

The answer is a conditional density, obtained — as in the E-step of Chapter 11, and this is no coincidence — by Bayes’ rule (K. E. Train 2009, Ch. 11):

$$h(\beta \mid \mathbf{y}_n, \hat{\theta}) = \frac{\Pr(\mathbf{y}_n \mid \beta) f(\beta \mid \hat{\theta})}{\int \Pr(\mathbf{y}_n \mid \beta') f(\beta' \mid \hat{\theta}) d\beta'} \quad (14.1)$$

Read it with Chapter 11 eyes: $f(\beta \mid \hat{\theta})$ plays the role the class shares played (what people are like in general); the sequence probability $\Pr(\mathbf{y}_n \mid \beta)$ plays the class-specific likelihood (how well each candidate taste explains *this* person); the result upgrades the population distribution into a person-specific one. The continuous analogue of posterior class membership.

Computing it is a two-line addition to machinery we already ran. The MSL estimation of Chapter 13 already evaluates, for every person and every draw $\beta_n^{(r)}$ from the estimated population, the sequence probability $P_{nr} = \Pr(\mathbf{y}_n \mid \beta_n^{(r)})$ — note the subscripts: unlike the task-level P_{nt} , this is the probability of person n ’s *whole sequence* under draw r , i.e. $P_{nr} = e^{\ell_{nr}}$ in Chapter 13’s notation, and the weights $P_{nr} / \sum_{r'} P_{nr'}$ we are about to compute are *literally* the w_{nr} of Equation 13.4 — the Bayes-rule object that chapter promised this one would pick up. The **conditional mean** of Equation 14.1 follows by a move we already own. Write it out:

$$E[\beta \mid \mathbf{y}_n] = \int \beta h(\beta \mid \mathbf{y}_n, \hat{\theta}) d\beta = \frac{\int \beta \Pr(\mathbf{y}_n \mid \beta) f(\beta \mid \hat{\theta}) d\beta}{\int \Pr(\mathbf{y}_n \mid \beta') f(\beta' \mid \hat{\theta}) d\beta'}$$

Numerator and denominator are both expectations *with respect to* f — exactly the kind of integral Chapter 12 taught us to simulate: draw from f , average the integrand. Simulate both with the same draws $\beta_n^{(r)} \sim f(\cdot \mid \hat{\theta})$, cancel the common $1/R$, and the conditional mean becomes a *probability-weighted average of the draws* — each candidate taste weighted by how well it explains the person:

$$\hat{\beta}_n = \frac{\sum_r P_{nr} \beta_n^{(r)}}{\sum_r P_{nr}} \quad (14.2)$$

```
## rebuild the estimated-population draws and their weights, as in @sec-mixed-msl
theta_hat <- mixed$theta_hat          # MSL estimates: means, log-sds
rc        <- mixed$rc                # which coefficients are random
Z         <- mixed$Z                 # the fixed draws used in estimation
R <- dim(Z)[2]; n_id <- max(id)
id_of_task <- id[!duplicated(task_id)]

bar <- theta_hat[1:K]
s   <- exp(theta_hat[(K+1):(K+length(rc))])

lseq <- matrix(0, n_id, R)
beta_draws <- array(rep(bar, each = n_id), c(n_id, R, K))
for (r in 1:R) {
  betas <- matrix(rep(bar, each = n_id), n_id, K)
  for (jj in seq_along(rc))
    betas[, rc[jj]] <- betas[, rc[jj]] + s[jj] * Z[, r, jj]
  beta_draws[, r, ] <- betas
  V      <- rowSums(X * betas[id, ])
}
```

```

Vmax <- ave(V, task_id, FUN = max)
eV    <- exp(V - Vmax)
logP  <- (V - Vmax - log(rowsum(eV, task_id)[task_id]))
lseq[, r] <- as.vector(rowsum(logP[y == 1], id_of_task))
}
M <- apply(lseq, 1, max)
w <- exp(lseq - M); w <- w / rowSums(w)           # person-by-draw weights

beta_cond <- sapply(1:K, function(k) rowSums(w * beta_draws[, , k]))
colnames(beta_cond) <- colnames(X)

```

The moment of judgment — conditional means against the true individual coefficients:

```

ggplot(data.frame(truth = betas_true[, "price"],
                  cond  = beta_cond[, "price"]),
       aes(truth, cond)) +
  geom_abline(linetype = "dotted") +
  geom_point(alpha = 0.35, size = 0.9) +
  coord_equal() +
  labs(x = expression(true~beta[price]),
       y = expression(conditional~mean~hat(beta)[price]))

```

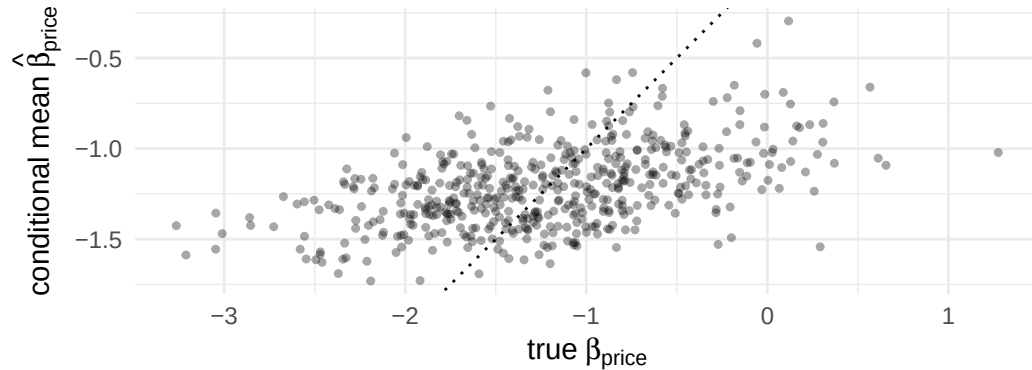


Figure 14.2: Conditional individual-level price coefficients against each respondent's true price coefficient. Correlation is strong; the systematic flattening relative to the 45-degree line is shrinkage — with eight tasks each, individual estimates rationally lean toward the population mean.

```
round(c(correlation = cor(betas_true[, "price"], beta_cond[, "price"]),
      sd_truth = sd(betas_true[, "price"]),
      sd_conditional = sd(beta_cond[, "price"])), 2)
```

correlation	sd_truth	sd_conditional
0.49	0.71	0.22

Night and day against Attempt One. The conditional means track the truth person by person (correlation around 0.49), with no explosions — the population density in Equation 14.1 acts as a leash that separation cannot outrun: a respondent who always chose cheapest is now assigned a price coefficient in the *plausible lower tail* of the population, not at infinity.

And notice the figure's second message: the cloud is *flatter* than the 45-degree line, and the conditional means' spread is visibly smaller than the true spread. This compression is **shrinkage** — individual estimates pulled toward the population mean, more strongly the less

information the person’s own choices carry. Far from a defect, shrinkage is the rational response to $T = 8$: with thin personal data, the population is a better guide than the person’s noisy record, and the weighted average in Equation 14.2 makes precisely that trade.² Shrinkage is about to become a central character: the hierarchical models of Part III produce it automatically, and Chapter 17 will lay this chapter’s estimates alongside theirs.

14.3 So What’s Missing?

If conditional means work this well, why does this book have three more parts? Because of what surrounds the point estimates. Line up the honest accounting:

No finite-sample inference. The conditional mean is a point. Its distribution Equation 14.1 would seem to offer more — but it is computed *at the plug-in* $\hat{\theta}$, as if the population parameters were known rather than estimated. Person-level uncertainty statements that account for population-level uncertainty require nesting the whole Equation 14.2 computation inside the sampling distribution of $\hat{\theta}$ — possible in principle, done by almost no one, offered by no standard software.

A two-step architecture with error at both steps. Step one: MSL, with its simulation bias (Chapter 12) and its own standard errors. Step two: conditional means, with fresh simulation error, treating step one as exact. The errors do not add up in any package’s output; they are simply not accounted.

And the first step itself strains. The Chapter 13 accounting: minutes per fit at three random coefficients with diagonal covariance; full covariance means Cholesky parameterizations, quadratically growing parameter counts, numerical gradients or heavy derivations, multiplied by the multiple-starts discipline. MSL *works* — we validated it — but every increment of model richness is paid for in optimization fragility and draws.

Now step back and look at the *shape* of what we built, because the shape is the punchline. To get individual estimates out of maximum likelihood, we bolted Bayes’ rule onto the side of the estimator: Equation 14.1 treats the population distribution as a prior and updates it with a person’s data. The natural question — the one Part III answers — is: *if Bayes’ rule is doing the interesting work anyway, why not build the entire estimator out of it?* Treat θ itself as uncertain, put the hierarchy $\mathbf{y}_n \leftarrow \beta_n \leftarrow \theta$ inside one coherent posterior, and sample from that posterior directly. Every deficiency in the list above dissolves: person-level and population-level uncertainty arrive together, exactly, in finite samples; there is no two-step; and — the surprise of Chapter 17 — the computation gets *easier*, replacing a strained 9-parameter optimization with draws from simple conditional distributions. The tools are ready: the likelihood code of

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

Part II is the engine, and the sampling skills of Chapter 2 are the fuel. What remains is the algorithm — Markov chain Monte Carlo — and that is Part III.

14.4 Key Learnings

- Population parameters are not the deliverable; individual-level β_n 's are what targeting, segmentation, and market simulation consume. Maximum likelihood's route to them is the subject of this chapter — and its limit.
- Per-person MLE fails structurally: perfect separation sends coefficients to infinity, and the incidental parameters problem (parameters growing with N , information growing only with fixed T) voids the asymptotics. The failure histogram is not subtle.
- Conditioning works: Bayes' rule (Equation 14.1) upgrades the estimated population distribution into person-specific distributions, and the conditional mean (Equation 14.2) is a probability-weighted average of draws — computable from quantities MSL already produced, the continuous sibling of Chapter 11's posterior memberships.
- Conditional means track the truth strongly and exhibit **shrinkage** — rational compression toward the population mean under thin personal data. Never confuse the spread of conditional means with the spread of tastes.
- What likelihood cannot supply: honest person-level uncertainty (the plug-in ignores $\hat{\theta}$'s error), a one-step architecture, and relief from MSL's growing strain. The fix is to promote Bayes' rule from bolt-on to foundation — Part III.

Part III

The MNL Family, by Bayes

15 Bayesian Estimation of the MNL

Part III begins with a promise made in Chapter 14: build the estimator *out of* Bayes' rule, and the person-level inference problem dissolves. Keeping that promise takes three chapters. This one changes the philosophical engine while keeping the model deliberately familiar — the plain MNL on the same laptop study we estimated by maximum likelihood in Chapter 8 — so that everything new is *method*, nothing is *model*. By the end you will have sampled a posterior with an algorithm you coded yourself, diagnosed its health, and discovered that it reproduces Chapter 8's answers almost exactly. That near-redundancy is the point: on a model this simple, Bayes buys little. The machine we assemble here, though, is precisely the machine that scales to the hierarchical models where Bayes buys everything — and its hot inner loop is, literally, the `loglik_mnl()` function you already own.

15.1 The Bayesian Mindset

The likelihood tradition of Part II treats β as a fixed unknown constant and asks: *which value makes the data most probable?* Uncertainty lives in the data — the sampling distribution of $\hat{\beta}$ across hypothetical replications (Section 8.8 ran them) — and is reported as standard errors.

The Bayesian tradition treats β itself as an uncertain quantity and asks a different question: *given what we observed, what should we believe about β ?* Beliefs before the data are the **prior** $p(\beta)$; the data speak through the **likelihood** $p(\mathbf{y} | \beta)$ — the *identical* object we built in Chapter 5, no modification — and Bayes' rule combines them into the **posterior**:

$$p(\beta | \mathbf{y}) = \frac{p(\mathbf{y} | \beta) p(\beta)}{p(\mathbf{y})} \propto p(\mathbf{y} | \beta) p(\beta) \quad (15.1)$$

The denominator $p(\mathbf{y}) = \int p(\mathbf{y} | \beta) p(\beta) d\beta$ — the *marginal likelihood* — is a constant with respect to β : it rescales the posterior to integrate to one but does not change its shape. Hold onto that observation; it is about to carry the whole chapter.

You have already met this arithmetic twice without the label. The E-step of Chapter 11 (Equation 11.3) was Bayes' rule over discrete classes; the conditional distribution of Chapter 14 (Equation 14.1) was Bayes' rule over a person's coefficients with the population as prior. Both times, the rule was a *bolt-on* to a likelihood-estimated model. The move now is to apply it to

everything we don't know — in this chapter just β ; by Chapter 17, every person's β_n and the population parameters above them, in one coherent joint posterior.

The output format changes too, and the change is an upgrade. Maximum likelihood reports a point and a curvature-based wobble (Equation 5.10). The posterior is an entire *distribution*: its mean is a point estimate, its standard deviation an uncertainty, its quantiles honest interval statements (“with probability 0.95, β_{price} lies in...”), and — crucially for Chapter 21 — any *function* of parameters inherits a full distribution by pushing draws through it. No delta method, no asymptotic appeal; finite-sample statements, exact up to sampling of the posterior itself.

15.2 Priors

The prior is the ingredient likelihood analysis doesn't have, and it deserves a plain-spoken paragraph rather than an ideological one. A prior encodes what is credible about parameters before the current data — from previous studies, theory, or mere scale knowledge (“utility coefficients of magnitude 100 do not occur in conjoint data”). When data are rich, as ours are (4,000 choices bearing on six parameters), any reasonable prior is overwhelmed by the likelihood and the posterior barely notices it; when data are thin — individual-level parameters from eight tasks, the exact situation of Chapter 17 — the prior matters enormously and *usefully*, supplying the regularization that made Chapter 14's conditional means sane where per-person MLE exploded.

For this chapter: independent normals, generous scale,

$$\beta_k \sim N(0, 5^2), \quad k = 1, \dots, K \quad (15.2)$$

— “weakly informative” in the standard sense (Gelman et al. 2020): no direction favored, magnitudes beyond ± 10 gently discouraged, everything our simulate-and-recover truths could plausibly be, easily accommodated. The honest practice is not agonizing over the perfect prior but *checking sensitivity*: refit with the scale at 2 and at 20 and confirm nothing you report moves. (With this chapter's data, nothing does; we verify at the end.)

In code, the prior joins the likelihood on the log scale — and note that the sum below is the entire Bayesian addition to Part II's machinery:

```
log_prior <- function(beta, sd = 5) sum(dnorm(beta, 0, sd, log = TRUE))
log_post  <- function(beta, y, X, task_id)
  loglik_mnl(beta, y, X, task_id) + log_prior(beta)
```

15.3 The Posterior Has No Closed Form Either

For a handful of textbook pairings (normal likelihood, normal prior — Chapter 16 exploits them), the posterior is a named distribution you can look up. The MNL is not one of them: multiply Equation 8.1’s exponentiated form by a normal prior and the result matches no catalog. Concretely, we can evaluate the posterior’s *shape* — Equation 15.1’s numerator, our `log_post()` — at any β we like, but we cannot integrate it: no normalizing constant, no posterior means, no quantiles, nothing summable in closed form.

This should feel familiar. It is Chapter 12’s predicament — a needed integral with no formula — met this time not by approximating the integral but by a more radical maneuver: *never compute it at all*. If we can manufacture **draws** from the posterior, then every summary we want is a sample statistic: posterior mean $\approx$ mean of draws, quantiles $\approx$ sample quantiles, exactly the Chapter 2 logic of summarizing distributions by sampling them. The entire Bayesian computation problem reduces to: *sample from a distribution known only up to a constant*. Enter the second-most-cited algorithm of twentieth-century science.

15.4 The Metropolis–Hastings Algorithm

The idea. Take a random walk through parameter space, but a biased one: propose a small random move; if the move goes *uphill* on the posterior surface, take it; if downhill, take it *sometimes* — with probability equal to the ratio of posterior heights. Formally, from current position $\beta^{(s)}$:

1. Propose $\beta' = \beta^{(s)} + \text{step}$, with the step drawn symmetrically (ours: $N(\mathbf{0}, \tau^2 I)$).
2. Compute the acceptance probability

$$\alpha = \min \left(1, \frac{p(\beta' | \mathbf{y})}{p(\beta^{(s)} | \mathbf{y})} \right) \quad (15.3)$$

3. With probability α set $\beta^{(s+1)} = \beta'$ (accept); otherwise $\beta^{(s+1)} = \beta^{(s)}$ (stay — the repeat is part of the sample).

A naming note that pays off next chapter: Equation 15.3 is really the *Metropolis* ratio, valid only because our proposal is symmetric — a step from $\beta^{(s)}$ to β' is exactly as probable as the reverse step. Hastings’ generalization handles an asymmetric proposal density q by reweighting:

$$\alpha = \min \left(1, \frac{p(\beta' | \mathbf{y}) q(\beta^{(s)} | \beta')}{p(\beta^{(s)} | \mathbf{y}) q(\beta' | \beta^{(s)})} \right)$$

With symmetric q the correction cancels and we recover Equation 15.3. We will need the general form exactly once — in Chapter 16, to see why a Gibbs draw is an M-H step that never rejects.

Two observations make the algorithm sing. First, the ratio in Equation 15.3 involves the posterior at two points — so the unknowable normalizing constant *cancels*: the algorithm needs only our computable `log_post()`, up-to-a-constant being exactly enough. Second, computed on the log scale (as a difference, accepted when $\log u < \Delta$) it is immune to the underflow that raw probability ratios invite — Chapter 9’s stay-on-the-log-scale doctrine, now load-bearing.

Why does the wandering produce posterior draws? The one-paragraph honest answer: the accept/reject rule is engineered so that posterior-density flow between any two locations balances — from where the posterior is high, many walkers depart, but proportionally many are sent back — in symbols, detailed balance requires $p(\beta \mid \mathbf{y}) q(\beta' \mid \beta) \alpha(\beta \rightarrow \beta') = p(\beta' \mid \mathbf{y}) q(\beta \mid \beta') \alpha(\beta' \rightarrow \beta)$, and Equation 15.3 satisfies it by construction: whichever direction is “downhill” accepts with probability equal to the posterior ratio, which is precisely the factor that equalizes the two sides (plug in α for the case $p(\beta' \mid \mathbf{y}) < p(\beta \mid \mathbf{y})$ and check) — making the posterior the walk’s *stationary distribution*: once the chain forgets its starting point, each step is a (serially correlated) draw from the posterior. The full argument is detailed balance plus Markov chain convergence theory (K. E. Train 2009, Ch. 12 sketches it; Gelman et al. 2013 gives it properly); we will do what computational people do and *verify stationarity empirically*, on a target we know.

A toy target first. Before trusting the algorithm with a posterior we cannot check, aim it at a density we can — the Section 3.5 discipline applied to samplers. A deliberately awkward target: a bimodal mixture of normals, known only up to a constant (we “forget” the constant on purpose):

```
log_target <- function(x) log(dnorm(x, -2, 0.7) + 0.6 * dnorm(x, 1.5, 1))

mh_1d <- function(log_f, x0, tau, S) {
  x <- numeric(S); x[1] <- x0
  acc <- 0
  for (s in 2:S) {
    prop <- x[s-1] + rnorm(1, 0, tau)
    if (log(runif(1)) < log_f(prop) - log_f(x[s-1])) {
      x[s] <- prop; acc <- acc + 1
    } else x[s] <- x[s-1]
  }
  list(draws = x, acc_rate = acc / (S - 1))
}

set.seed(150)
chain <- mh_1d(log_target, x0 = 0, tau = 1.5, S = 20000)
chain$acc_rate
```

```
[1] 0.6104305
```

```
norm_const <- integrate(function(x) exp(log_target(x)), -8, 8)$value
ggplot(data.frame(x = chain$draws[-(1:500)]), aes(x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 80, fill = "grey70") +
  geom_function(fun = function(x) exp(log_target(x)) / norm_const,
               linewidth = 0.8) +
  labs(x = "x", y = "density")
```

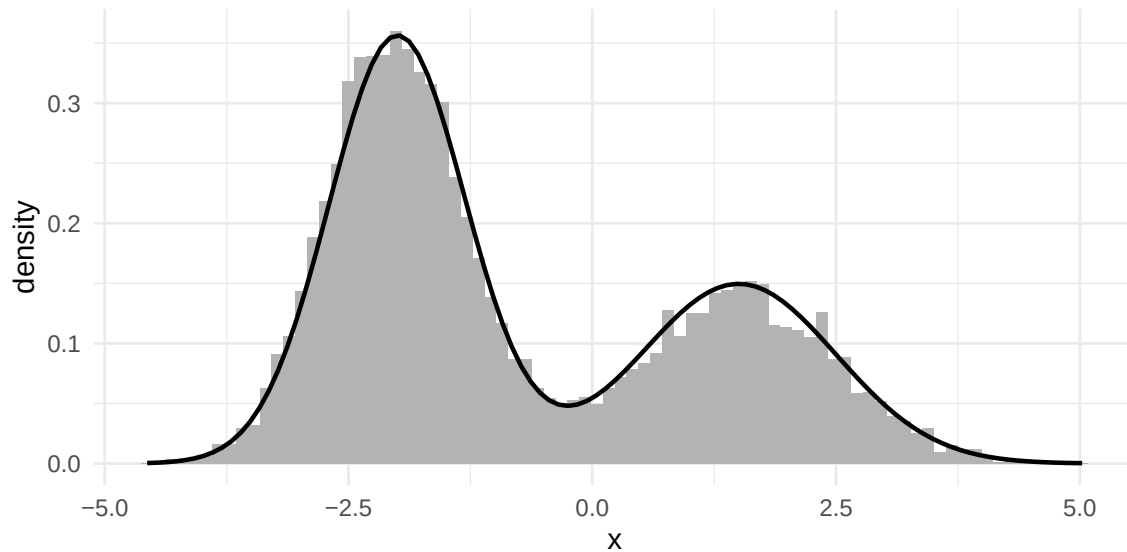


Figure 15.1: 20,000 Metropolis-Hastings draws against the true (renormalized) bimodal target density. The sampler was given the density only up to a constant and no knowledge of the modes; the histogram finds both, in the right proportions.

The histogram *is* the target. Twenty lines of code, no constants, no formulas for means or modes — just evaluations of an unnormalized density and coin flips.

The step size is a real decision. The proposal scale τ trades off two failure modes, best seen than described — the same chain with τ far too small, well-chosen, and far too large:

```
set.seed(151)
taus <- c(0.05, 1.5, 25)
traces <- do.call(rbind, lapply(taus, function(tt) {
  ch <- mh_1d(log_target, 0, tt, 2000)
  data.frame(iter = 1:2000, x = ch$draws,
             tau = sprintf("tau = %.2f (acc %.0f%%)", tt, 100 * ch$acc_rate))
}))
ggplot(traces, aes(iter, x)) +
```

```
geom_line(linewidth = 0.25) +
facet_wrap(~ tau, ncol = 1) +
labs(x = "iteration", y = "x")
```

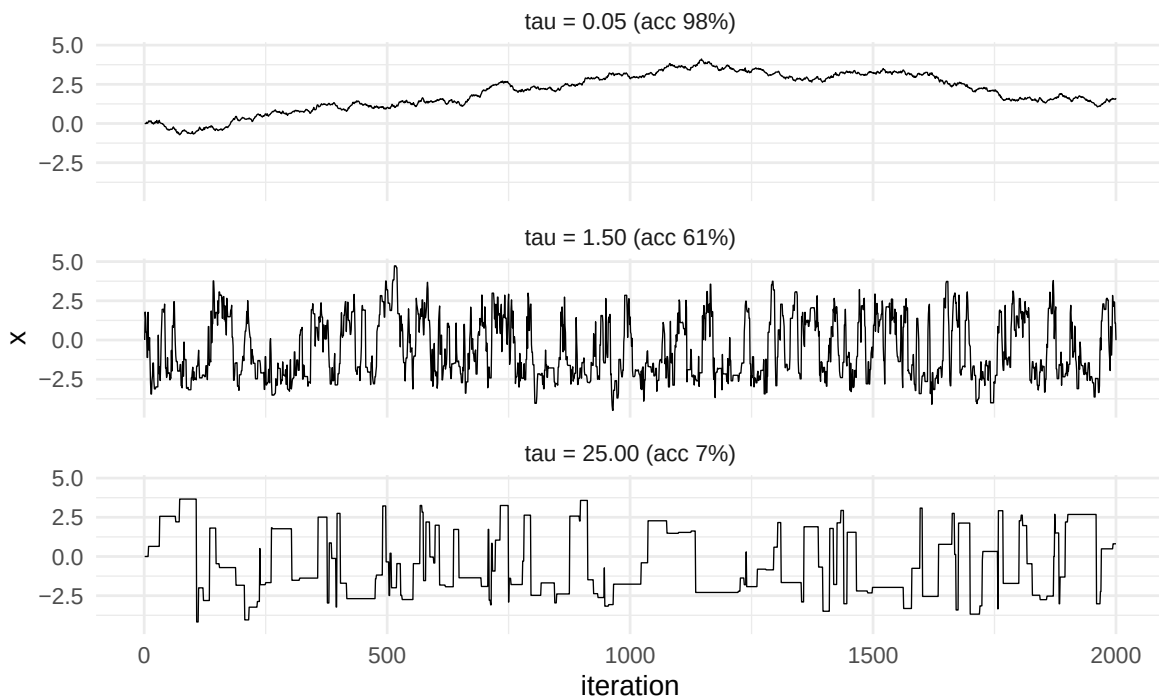


Figure 15.2: The step-size dilemma, in traces of the chain’s first 2,000 iterations. Tiny steps (top: acceptance 97%) inch along and explore nothing; sensible steps (middle: 42%) traverse both modes freely; huge steps (bottom: 4%) are almost always rejected, freezing the chain for long stretches.

Small steps accept almost everything but move nowhere — the chain is a slug, and its draws are so correlated they carry almost no information each. Huge steps propose leaps into the improbable, get rejected, and the chain fossilizes. Between them lies a broad sweet spot, conventionally targeted through the *acceptance rate*: theory for random-walk proposals puts the optimum near 23% in high dimensions and ~44% in one dimension [the folklore range 20–40% serves fine; K. E. Train (2009), Section 12.5]. Tuning τ to hit that range — by hand now, adaptively in Chapter 17 — is the algorithm’s one knob.

15.5 M-H for the MNL

Now the real target. Load the laptop study (same data as Chapter 8 — comparability is the experiment), bring back the likelihood kernel, and point the sampler at the posterior — note the multivariate generalization is only that proposals perturb all six coordinates at once:

```
study <- readRDS("../data/laptop_study.rds")
laptops <- study$data
X <- model.matrix(~ brand + ram + screen + price, data = laptops)[, -1]
y <- laptops$choice
task_id <- cumsum(!duplicated(laptops[, c("id", "task")]))
K <- ncol(X)

loglik_mnl <- function(beta, y, X, task_id) {                # @sec-mnl-mle, verbatim
  V <- as.vector(X %*% beta)
  Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}

mh_mnl <- function(y, X, task_id, S, tau, beta0 = rep(0, ncol(X)),
                  prior_sd = 5) {
  K <- ncol(X)
  draws <- matrix(NA_real_, S, K)                # preallocate (@sec-performance)
  beta <- beta0
  lp <- loglik_mnl(beta, y, X, task_id) +
    sum(dnorm(beta, 0, prior_sd, log = TRUE))
  acc <- 0
  for (s in 1:S) {
    prop <- beta + rnorm(K, 0, tau)
    lp_prop <- loglik_mnl(prop, y, X, task_id) +
      sum(dnorm(prop, 0, prior_sd, log = TRUE))
    if (log(runif(1)) < lp_prop - lp) {
      beta <- prop; lp <- lp_prop; acc <- acc + 1
    }
    draws[s, ] <- beta
  }
  list(draws = draws, acc_rate = acc / S)
}
```

Study the inner loop and count what is new relative to Part II: the proposal line, the prior line, the comparison line. Everything else — the entire cost of each iteration — is one call to

`loglik_mnl()`. The function we wrote in Chapter 8, vectorized in Chapter 9, and audited in Chapter 13 is the engine of the Bayesian machine too; Table 9.1's MCMC row is now literal.

Tune briefly (a few short pilot runs, not shown, put $\tau = 0.045$ in the low-20s acceptance range for these data — six dimensions, so we aim low per the theory) and run four chains from dispersed starts, because multiple chains are about to be a diagnostic:

```
set.seed(152)
S <- 20000
chains <- lapply(1:4, function(c)
  mh_mnl(y, X, task_id, S = S, tau = 0.045,
    beta0 = rnorm(K, 0, 1)))
sapply(chains, `[[`, "acc_rate")
```

```
[1] 0.27250 0.26815 0.26800 0.27330
```

15.6 Reading a Chain: Diagnostics

An MCMC run hands you numbers unconditionally; whether they are *posterior draws* is a judgment the diagnostics inform. The workflow, on our chains:

Trace plots first, always. The chain's path over iterations, for one parameter, all four chains:

```
tr <- do.call(rbind, lapply(1:4, function(c)
  data.frame(iter = 1:S, chain = factor(c),
    price = chains[[c]]$draws[, 6])))
ggplot(tr[tr$iter %% 10 == 0, ], aes(iter, price, color = chain)) +
  geom_line(linewidth = 0.2) +
  scale_color_grey(end = 0.7) +
  labs(x = "iteration", y = expression(beta[price]))
```

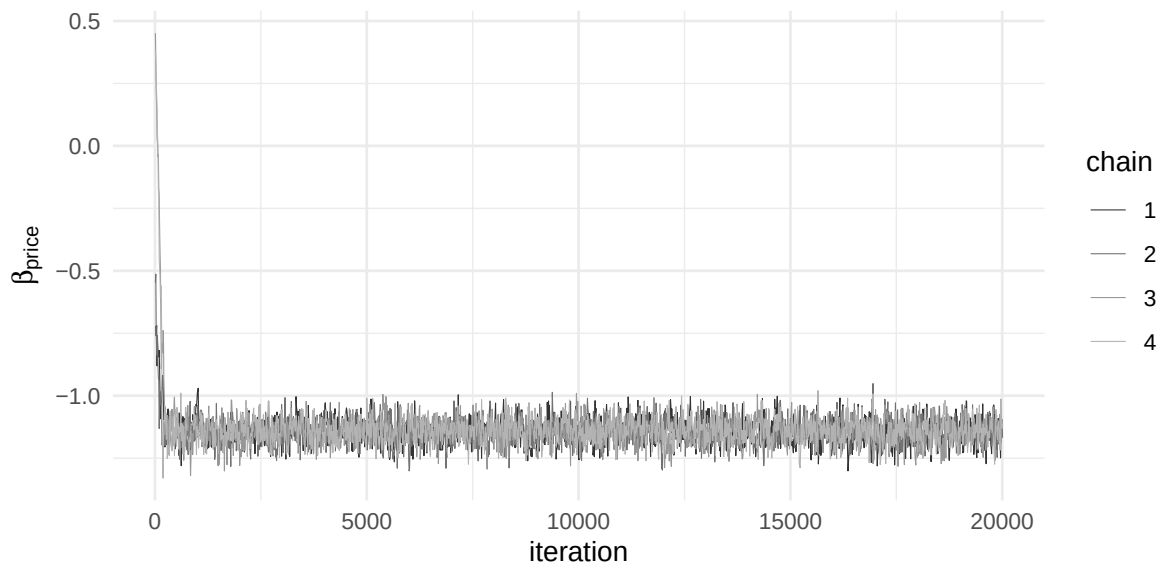


Figure 15.3: Trace of the price coefficient: four chains from dispersed starts. The early iterations show the chains migrating from their starting points to the posterior’s neighborhood (burn-in, discarded); thereafter the four traces are statistically indistinguishable ‘fuzzy caterpillars’ – the visual signature of convergence to a common stationary distribution.

The reading: an initial transient as each chain walks in from its arbitrary start — that stretch reflects the starting point, not the posterior, and is discarded as **burn-in** (we drop 2,000, generous for this well-behaved problem) — followed by stable, overlapping fuzz. Four chains agreeing about *where* to fuzz is the point of dispersed starts: any one chain can look settled while stuck; four independent walkers telling the same story is far harder to fake.

$\hat{R}$ makes “telling the same story” numerical. Compare variance between chains to variance within them: at convergence the ratio’s root is 1; meaningfully above (folk threshold 1.01–1.05), the chains disagree and none should be trusted:

```
rhat <- function(mat) { # mat: iterations x chains, post burn-in
  m <- ncol(mat); n <- nrow(mat)
  B <- n * var(colMeans(mat))
  W <- mean(apply(mat, 2, var))
  sqrt(((n - 1)/n * W + B/n) / W)
}
post <- lapply(chains, function(c) c$draws[-(1:2000), ])
round(sapply(1:K, function(k)
  rhat(sapply(post, function(p) p[, k]))), 3)
```

```
[1] 1.000 1.001 1.004 1.004 1.001 1.001
```

All within rounding of 1. **Effective sample size** answers the remaining question — serial correlation means 72,000 retained draws are worth fewer independent ones:

```
ess <- function(x) {  
  rho <- acf(x, lag.max = 200, plot = FALSE)$acf[-1]  
  rho <- rho[1:min(which(rho < 0.05), length(rho))] # truncate the tail  
  length(x) / (1 + 2 * sum(rho))  
}  
round(sapply(1:K, function(k) mean(sapply(post, function(p) ess(p[, k])))))
```

```
[1] 563 570 598 623 1131 886
```

A few hundred effective draws per chain per parameter — random-walk M-H is not efficient, just simple and general¹ — but summed across chains, ample for means and intervals: Monte Carlo error on the posterior mean is $\text{posterior-sd}/\sqrt{\text{ESS}}$, well below reporting precision here.

And when it breaks? Re-run any chain with $\tau = 1$ (acceptance collapses to a fraction of a percent; the trace becomes long flat shelves) or $\tau = 0.001$ (acceptance 99%; the trace is a slow drift that four chains show *not* mixing across). We already staged exactly these pathologies in Figure 15.2; on real problems they look the same, and now you know their names when you see them in your own traces.

15.7 Bayes Meets Maximum Likelihood

The confrontation this chapter was built for. Pool the post-burn-in draws and summarize the posterior next to Chapter 8’s MLE results and the truth:

```
draws <- do.call(rbind, post)  
fit_ml <- optim(rep(0, K), loglik_mnl, y = y, X = X, task_id = task_id,  
               method = "BFGS", control = list(fnscale = -1), hessian = TRUE)  
  
tab <- data.frame(  
  truth      = study$beta_true,
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

mle      = fit_ml$par,
se       = sqrt(diag(solve(-fit_ml$hessian))),
post_mean = colMeans(draws),
post_sd  = apply(draws, 2, sd),
q2.5     = apply(draws, 2, quantile, 0.025),
q97.5    = apply(draws, 2, quantile, 0.975)
)
rownames(tab) <- colnames(X)
round(tab, 3)

```

	truth	mle	se	post_mean	post_sd	q2.5	q97.5
brandDell	0.5	0.478	0.054	0.479	0.054	0.374	0.582
brandApple	1.0	0.940	0.054	0.942	0.053	0.837	1.046
ram16	0.6	0.638	0.053	0.640	0.054	0.536	0.744
ram32	0.9	0.886	0.054	0.889	0.054	0.782	0.994
screen15	0.3	0.371	0.043	0.372	0.042	0.290	0.454
price	-1.2	-1.140	0.048	-1.141	0.048	-1.236	-1.046

Posterior means match MLEs to the third decimal; posterior standard deviations match standard errors likewise; the 95% credible intervals lie atop the confidence intervals. This is the **Bernstein–von Mises phenomenon**: with abundant data and a prior too diffuse to argue, the posterior converges to a normal centered at the MLE with the MLE’s covariance — the two philosophies’ answers coincide even as their interpretations differ.² The promised prior-sensitivity check completes the picture: rerunning one chain with `prior_sd = 2` and 20 moves no reported digit (try it — one argument).

So why the new machinery? Look again at what `draws` is: not a point and a curvature, but 72,000 samples of β jointly. Want the posterior of the 32GB-vs-16GB premium? `draws[,4] - draws[,3]`, done — full distribution, no delta method. The posterior probability that Apple’s premium exceeds Dell’s by more than 0.3? `mean(draws[,2] - draws[,1] > 0.3)`. Every downstream quantity in Chapter 21 — WTP distributions, market shares under counterfactual prices — inherits exact finite-sample uncertainty by pushing draws through functions. And the sampler that produced them neither knows nor cares that the posterior had no closed form — which is why, unlike the closed-form-hunting of Part II, *this* machine walks into the hierarchical wilderness of Chapter 17 without modification. It only needs help with efficiency — the subject of Gibbs sampling, next.

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

15.8 Key Learnings

- Bayes' rule (Equation 15.1) turns the Part II likelihood plus a prior into a posterior *distribution* over parameters; the normalizing constant is unknowable and, it turns out, unnecessary. Output is draws; every summary is a sample statistic.
- Priors encode pre-data credibility; weakly informative normals (Equation 15.2) suffice when data are rich, and sensitivity checking — not prior perfectionism — is the working discipline. The prior's real power arrives with thin data, in Chapter 17.
- Metropolis–Hastings samples any density evaluable up to a constant: propose symmetrically, accept by the (log-scale) posterior ratio (Equation 15.3). Twenty lines; verified on a known bimodal target before deployment — samplers get sanity checks too.
- Step size is the knob: tiny steps crawl, huge steps freeze; tune to ~20–40% acceptance (Figure 15.2). Diagnostics are non-optional: trace plots from multiple dispersed chains, $\hat{R} \approx 1$, and ESS to price the serial correlation.
- The M-H inner loop *is* `loglik_mnl()` — Part II's kernel powering Part III's engine, as promised since Chapter 8.
- On the plain MNL, posterior $\approx$ MLE (Bernstein–von Mises): same numbers, different philosophy, plus the joint draws that make downstream inference trivial. The payoff model — where likelihood strains and Bayes glides — is two chapters away; first, Gibbs.

16 Gibbs Sampling and Hierarchical Models

Metropolis–Hastings will sample anything, slowly. This chapter builds its efficient sibling and the model architecture that Part III exists for. The **Gibbs sampler** replaces M-H’s propose-and-mostly-reject shuffle with draws from *exact conditional distributions* — proposals so good they are never rejected — whenever the model’s structure supplies such conditionals. And the model structure that supplies them most beautifully is the **hierarchy**: person-level parameters drawn from a population, the formal home of everything Chapter 14 found missing. We develop both on models simple enough to see through — a hierarchical normal model, then a binary probit — because Chapter 17 assembles the HB-MNL *entirely* from blocks built here, and Chapter 19’s probit samplers are these ideas at scale.

16.1 Sampling by Blocks

Suppose the parameter vector splits into blocks $\theta = (\theta_1, \theta_2, \dots)$ and, while the *joint* posterior is intractable, each **full conditional** — the distribution of one block given the data and all other blocks — is a known, sampleable distribution. The Gibbs sampler simply cycles:

$$\theta_1^{(s+1)} \sim p(\theta_1 \mid \theta_2^{(s)}, \mathbf{y}), \quad \theta_2^{(s+1)} \sim p(\theta_2 \mid \theta_1^{(s+1)}, \mathbf{y}), \quad \dots \quad (16.1)$$

— always conditioning on the most recent values of the other blocks. The resulting chain has the joint posterior as its stationary distribution, exactly like M-H;¹ the practical differences are that every draw is accepted, there is no step size to tune, and the moves are calibrated to the local shape of the posterior *by construction*. The cost of admission: you must be able to derive and sample the conditionals — real analytic work, purchased model by model, mostly through a phenomenon called conjugacy.

A toy first, per the Chapter 15 discipline of testing samplers on known targets. The bivariate normal with correlation ρ has textbook conditionals — each coordinate given the other is univariate normal, $x_1 \mid x_2 \sim N(\rho x_2, 1 - \rho^2)$ and symmetrically — so Gibbs needs six lines, and Section 2.4’s Cholesky method provides an exact benchmark:

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

set.seed(160)
rho <- 0.8; S <- 10000
gx <- matrix(NA_real_, S, 2); xy <- c(-3, 3)      # deliberately absurd start
for (s in 1:S) {
  xy[1] <- rnorm(1, rho * xy[2], sqrt(1 - rho^2))
  xy[2] <- rnorm(1, rho * xy[1], sqrt(1 - rho^2))
  gx[s, ] <- xy
}
rbind(gibbs = c(mean = colMeans(gx[-(1:100), ]),
                cor = cor(gx[-(1:100), 1], gx[-(1:100), 2])),
      target = c(0, 0, rho))

```

	mean1	mean2	cor
gibbs	0.01280292	0.01592062	0.7995821
target	0.00000000	0.00000000	0.8000000

Means and correlation on target. The chain's *path* is worth one look — Gibbs moves in axis-parallel steps (one coordinate at a time), staircasing across the correlated ellipse:

```

path <- data.frame(x1 = rep(gx[1:30, 1], each = 2)[-1],
                  x2 = c(3, rep(gx[1:30, 2], each = 2))[1:59])
ggplot(path, aes(x1, x2)) +
  geom_path(linewidth = 0.3) +
  geom_point(data = data.frame(x1 = gx[1:30, 1], x2 = gx[1:30, 2]),
            size = 0.8) +
  coord_equal() +
  labs(x = expression(x[1]), y = expression(x[2]))

```

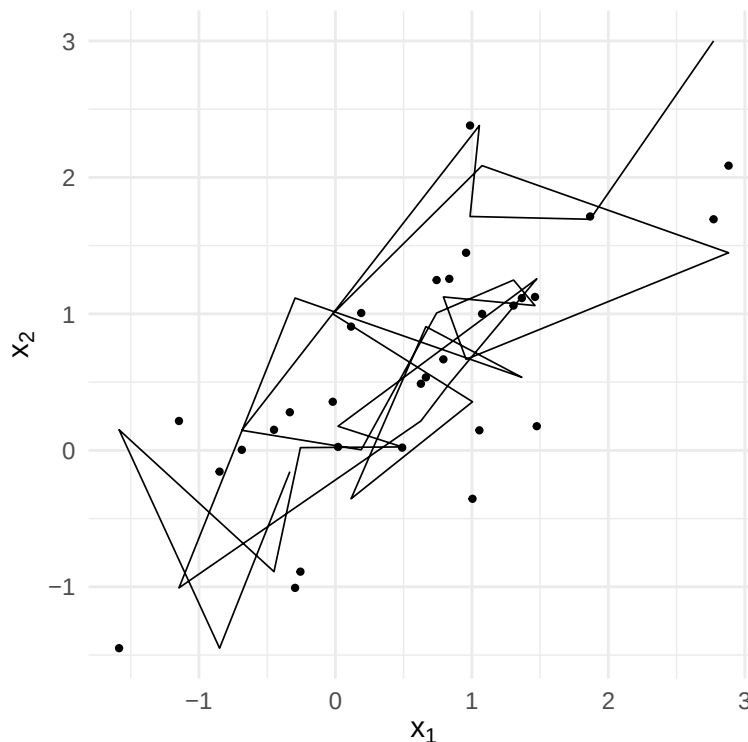


Figure 16.1: The first 30 moves of the Gibbs sampler on a correlated bivariate normal, from a far-off start: axis-parallel staircase steps, because each update changes one coordinate conditional on the other. High correlation makes the staircase treads small – a preview of why strongly dependent parameters mix slowly under coordinate-wise sampling.

Note the figure’s caption warning: correlation between blocks shrinks the staircase treads. It will return as practical advice — block correlated parameters *together* when you can.

16.2 Conjugacy: The Conditional-Distribution Factory

Where do sampleable conditionals come from? From prior-likelihood pairs whose product lands back in the prior’s family — **conjugate** pairs. One derivation, done honestly, powers everything below; the rest we state and use.

The normal–normal update. Data $y_1, \dots, y_n \stackrel{iid}{\sim} N(\theta, \sigma^2)$ with σ^2 known; prior $\theta \sim N(\mu_0, \tau_0^2)$. The posterior is proportional to prior times likelihood — two exponentials of

quadratics in θ :

$$p(\theta | \mathbf{y}) \propto \exp\left(-\frac{(\theta - \mu_0)^2}{2\tau_0^2}\right) \prod_{i=1}^n \exp\left(-\frac{(y_i - \theta)^2}{2\sigma^2}\right) \propto \exp\left(-\frac{1}{2}\left[\theta^2(\tau_0^{-2} + n\sigma^{-2}) - 2\theta(\tau_0^{-2}\mu_0 + n\sigma^{-2}\bar{y})\right]\right)$$

where every term free of θ has been dropped into the proportionality sign. Now a reading-off rule worth memorizing on its own: anything of the form $\exp(-\frac{1}{2}[a\theta^2 - 2b\theta])$ is an unnormalized $N(b/a, 1/a)$ density — complete the square to see it. With $a = \tau_0^{-2} + n\sigma^{-2}$ and $b = \tau_0^{-2}\mu_0 + n\sigma^{-2}\bar{y}$, we read off

$$\theta | \mathbf{y} \sim N\left(\frac{\tau_0^{-2}\mu_0 + n\sigma^{-2}\bar{y}}{\tau_0^{-2} + n\sigma^{-2}}, (\tau_0^{-2} + n\sigma^{-2})^{-1}\right) \quad (16.2)$$

Memorize the *shape*, not the symbols: **precisions add; the posterior mean is the precision-weighted average of prior mean and data mean.** Rich data (large $n\sigma^{-2}$) pull the posterior onto $\bar{y}$; thin data leave it near μ_0 . That precision-weighted compromise *is* shrinkage — Chapter 14’s phenomenon, now with an exact formula — and you will see this update, in scalar and matrix dress, inside every sampler for the rest of the book.

Variances: the inverse-gamma update. For the variance of a normal with known mean, the conjugate prior is inverse-gamma: $\sigma^2 \sim IG(a_0, b_0)$ yields $\sigma^2 | \mathbf{y} \sim IG(a_0 + n/2, b_0 + \frac{1}{2} \sum (y_i - \theta)^2)$ — prior pseudo-observations plus data observations, prior sum-of-squares plus data sum-of-squares. (Its matrix generalization, the inverse-Wishart for covariance matrices, arrives in Chapter 17; same grammar.) Derivations: Gelman et al. (2013), or Rossi, Allenby, and McCulloch (2005) in exactly our setting.

16.3 The Hierarchical Normal Model

Now the architecture. Strip the choice modeling away from Chapter 14’s problem and what remains is this: repeated measurements on many units, each unit with its own parameter, the parameters themselves drawn from a population we also want to learn. The minimal version — our laptop shoppers, with each person’s *willingness-to-spend on upgrades* β_n measured noisily T times:²

$$y_{nt} \sim N(\beta_n, \sigma^2), \quad \beta_n \sim N(\mu, \tau^2), \quad \mu \sim N(0, 10^2), \quad \tau^2 \sim IG(2, 2), \quad \sigma^2 \sim IG(2, 2) \quad (16.3)$$

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

(A notation caution: this τ^2 is the population *variance* — how much true willingness-to-spend varies across people — and has nothing to do with Chapter 15’s Metropolis step size τ . Gibbs has no step size, so the letter is free again, and τ for a hierarchy’s spread is the standard choice.)

Three layers: data given person-parameters, person-parameters given population, priors on everything unknown. This *is* the mixed model of Chapter 13 in normal-linear miniature — and where MSL had to integrate the β_n ’s out (they were nuisance obstacles to the likelihood), the Bayesian treatment keeps every β_n *in* the posterior as a first-class unknown: $p(\beta_{1:N}, \mu, \tau^2, \sigma^2 \mid \mathbf{y})$. What looked like the problem (hundreds of parameters!) becomes the solution, because every full conditional is conjugate:

- $\beta_n \mid \dots$: normal–normal update (Equation 16.2) with “prior” $N(\mu, \tau^2)$ and “data” person n ’s T observations. *One precision-weighted shrinkage per person.*
- $\mu \mid \dots$: normal–normal update treating the current β_n ’s as data.
- $\tau^2 \mid \dots$ and $\sigma^2 \mid \dots$ (where “...” always means *the data and every other current block*): inverse-gamma updates, each applied to the residuals its block owns. The β_n ’s are τ^2 ’s “data”: $\tau^2 \mid \dots \sim IG(a_0 + N/2, b_0 + \frac{1}{2} \sum_n (\beta_n - \mu)^2)$. The observations are σ^2 ’s: $\sigma^2 \mid \dots \sim IG(a_0 + NT/2, b_0 + \frac{1}{2} \sum_{n,t} (y_{nt} - \beta_n)^2)$.

```
gibbs_hier <- function(y, id, S, mu0 = 0, tau0sq = 100, a0 = 2, b0 = 2) {
  N <- max(id)
  Tn <- tabulate(id)                # obs per person
  ybar_n <- as.vector(rowsum(y, id)) / Tn

  beta <- ybar_n; mu <- mean(y); tau2 <- 1; sig2 <- 1
  out <- list(beta = matrix(NA_real_, S, N), hyper = matrix(NA_real_, S, 3))
  for (s in 1:S) {
    ## person-level betas: precision-weighted normal draws, all N at once
    prec <- 1/tau2 + Tn/sig2
    m <- (mu/tau2 + Tn*ybar_n/sig2) / prec
    beta <- rnorm(N, m, sqrt(1/prec))
    ## population mean
    precm <- 1/tau0sq + N/tau2
    mu <- rnorm(1, (mu0/tau0sq + sum(beta)/tau2) / precm, sqrt(1/precm))
    ## population variance and noise variance
    tau2 <- 1 / rgamma(1, a0 + N/2, b0 + sum((beta - mu)^2)/2)
    sig2 <- 1 / rgamma(1, a0 + length(y)/2, b0 + sum((y - beta[id])^2)/2)
    out$beta[s, ] <- beta
    out$hyper[s, ] <- c(mu, sqrt(tau2), sqrt(sig2))
  }
  colnames(out$hyper) <- c("mu", "tau", "sigma")
}
```

```

    out
  }

```

Every line is one of the two conjugate updates. Note the vectorization: all N person-draws happen in a single `rnorm(N, ...)` call, because given the hyperparameters the β_n 's are conditionally *independent* — the hierarchy's structure is the code's structure (and `matrix(NA_real_, S, N)` is Chapter 9's preallocation rule earning its keep: this sampler stores half a million numbers). Simulate and recover:

```

set.seed(161)
N <- 200; T <- 5
mu_true <- 2; tau_true <- 1.5; sig_true <- 2
beta_n <- rnorm(N, mu_true, tau_true)
id <- rep(1:N, each = T)
y <- rnorm(N*T, beta_n[id], sig_true)

fit <- gibbs_hier(y, id, S = 5000)
post <- fit$hyper[-(1:1000), ]
round(rbind(truth = c(mu_true, tau_true, sig_true),
             post_mean = colMeans(post),
             post_sd = apply(post, 2, sd)), 2)

```

	mu	tau	sigma
truth	2.00	1.50	2.00
post_mean	2.08	1.47	1.95
post_sd	0.12	0.10	0.05

All three population parameters recovered, uncertainty attached, in seconds — no optimizer, no draws-inside-a-likelihood, no tuning. And the person level came along for free. Here is this book's first *fully honest* individual-level inference, and the shrinkage geometry it produces:

```

ybar_n <- as.vector(rowsum(y, id)) / T
df <- data.frame(raw = ybar_n, post = colMeans(fit$beta[-(1:1000), ]))
ggplot(df, aes(raw, post)) +
  geom_abline(linetype = "dotted") +
  geom_hline(yintercept = mean(post[, "mu"]), linetype = "dashed") +
  geom_point(alpha = 0.4, size = 0.9) +
  coord_equal() +
  labs(x = "person's raw mean", y = "person's posterior mean")

```

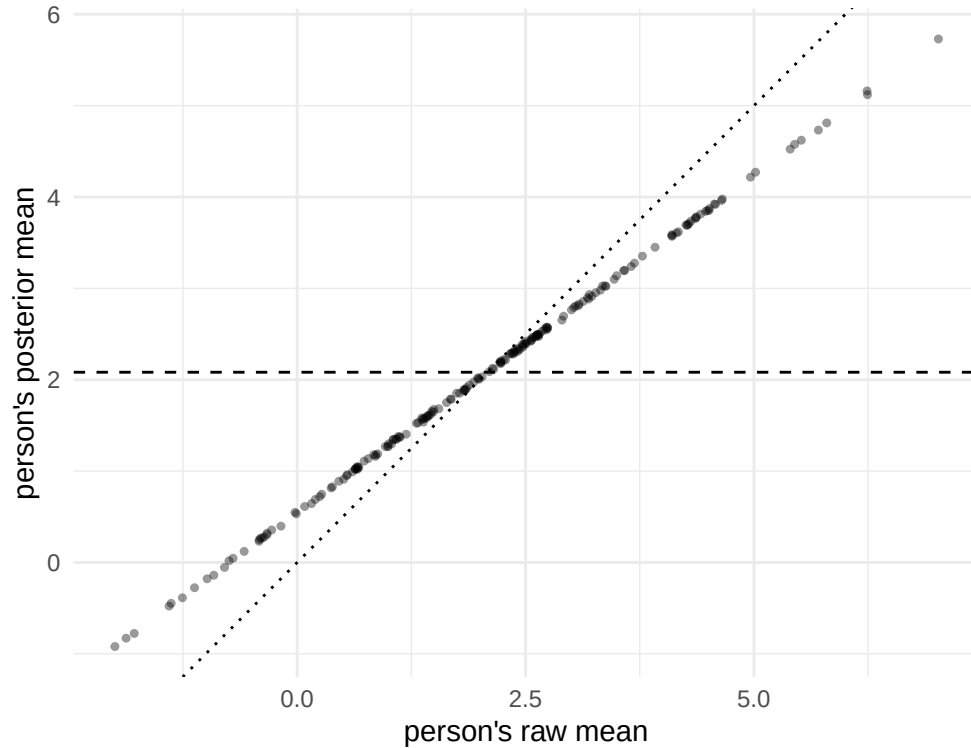


Figure 16.2: Shrinkage, exactly: each person’s posterior-mean beta against their raw data mean ($T = 5$ observations each). The cloud tilts flatter than the 45-degree line, pulled toward the population mean (dashed) – extreme raw means are partly noise, and the precision-weighted update (Equation 16.2) discounts them accordingly.

The flattened cloud is Equation 16.2 acting on two hundred people at once: each posterior mean is a precision-weighted compromise between that person’s own average and the population’s. Chapter 14 met this as an ad-hoc extraction; here it is a *theorem-grade consequence of the model*, with the amount of shrinkage — the balance of τ^{-2} against $T\sigma^{-2}$ — itself estimated from the data. More data per person, less shrinkage; more real population spread, less shrinkage; the model negotiates it, not the analyst. This figure is the entire promise of Part III in one frame; Chapter 17 reproduces it with choice data.

16.4 Data Augmentation: The Albert–Chib Sampler

The hierarchy needed no new trick — conjugacy carried it. Choice data will, because a *logit or probit likelihood is not conjugate to anything*: the discreteness of y breaks the quadratic-in-parameters structure that Equation 16.2 feeds on. The escape, due to Albert and Chib (1993), is one of the loveliest moves in computational statistics: *put the latent utilities back*.

Recall the binary probit of Chapter 3: $y_n = I[z_n > 0]$ where $z_n = \mathbf{x}'_n \boldsymbol{\beta} + \varepsilon_n$, $\varepsilon_n \sim N(0, 1)$. The observed y_n is a censored glimpse of a well-behaved linear-normal model in z_n . So **augment**: treat $\mathbf{z}$ as unknowns alongside $\boldsymbol{\beta}$, and Gibbs the pair — each conditional is suddenly easy:

- $\boldsymbol{\beta} \mid \mathbf{z}$: with $\mathbf{z}$ “observed,” this is *Bayesian linear regression* — normal prior, normal likelihood, so a multivariate Equation 16.2: $\boldsymbol{\beta} \sim N((X'X + B_0^{-1})^{-1}(X'\mathbf{z} + B_0^{-1}\mathbf{b}_0), (X'X + B_0^{-1})^{-1})$.
- $z_n \mid \boldsymbol{\beta}, y_n$: normal with mean $\mathbf{x}'_n \boldsymbol{\beta}$, *truncated* to the side of zero that agrees with the observed choice — positive if they bought, negative if not. Which is exactly what Chapter 2’s `rtnorm()` was built for, as promised there.

```
rtnorm <- function(n, mean = 0, sd = 1, a = -Inf, b = Inf) { # @sec-draws
  Fa <- pnorm(a, mean, sd); Fb <- pnorm(b, mean, sd)
  qnorm(Fa + runif(n) * (Fb - Fa), mean, sd)
}

gibbs_probit <- function(y, X, S, b0 = 0, B0 = 25) {
  K <- ncol(X)
  B0inv <- diag(1/B0, K)
  V <- solve(crossprod(X) + B0inv) # fixed: X never changes
  L <- t(chol(V)) # @sec-mvnorm, once
  beta <- rep(0, K)
  lo <- ifelse(y == 1, 0, -Inf); hi <- ifelse(y == 1, Inf, 0)
  draws <- matrix(NA_real_, S, K)
  for (s in 1:S) {
    z <- rtnorm(length(y), as.vector(X %*% beta), 1, lo, hi)
    m <- V %*% (crossprod(X, z) + B0inv %*% rep(b0, K))
    beta <- as.vector(m + L %*% rnorm(K))
    draws[s, ] <- beta
  }
  draws
}
```

Count what is absent: no Metropolis step, no acceptance rate, no tuning parameter, no likelihood evaluation — the probit *likelihood never appears in the code*. Its integrals (the Φ ’s that a probit MLE would evaluate) have been traded for truncated-normal draws, and every iteration draws from exact distributions. Note also the Chapter 9 economies: V and its Cholesky factor are data-only quantities, computed once outside the loop.

Test on binary probit laptop data — Chapter 3’s simulator, `dist = "normal"`:

```

set.seed(162)
sim_binary_data <- function(N, beta, dist = "normal") {
  price <- runif(N, 0.8, 2.4); ram16 <- rbinom(N, 1, 0.5)
  X <- cbind(1, price, ram16)
  eps <- if (dist == "normal") rnorm(N) else rlogis(N)
  data.frame(y = as.integer(as.vector(X %*% beta) + eps > 0), price, ram16)
}
beta_true <- c(1.0, -1.2, 0.6)
d <- sim_binary_data(4000, beta_true)
X <- cbind(1, d$price, d$ram16)

ac <- gibbs_probit(d$y, X, S = 5000)
round(rbind(truth = beta_true,
            post_mean = colMeans(ac[-(1:500), ]),
            post_sd = apply(ac[-(1:500), ], 2, sd)), 3)

```

```

      [,1] [,2] [,3]
truth  1.000 -1.200 0.600
post_mean 1.163 -1.299 0.565
post_sd   0.084  0.054 0.046

```

Recovery, tuning-free. Data augmentation is this book’s quiet superweapon: in Chapter 19 it makes the *multinomial* probit — whose likelihood is a $(J - 1)$ -dimensional integral that Part II could only simulate — estimable by nothing more than truncated normals and regressions, and in Chapter 20 it does so with a full hierarchy on top. The general principle deserves its box: **when a likelihood is intractable because something latent was integrated out, put the latent thing back and sample it.** Latent utilities, latent classes (Chapter 11’s memberships can be Gibbs-sampled the same way), latent budgets (Chapter 22) — all one move.

16.5 Metropolis-within-Gibbs: The Missing Piece

Survey what Part III now has. M-H samples anything evaluable (Chapter 15) but wanders. Gibbs is exact and tuning-free but demands conjugate conditionals. The hierarchy gives person-level inference with automatic shrinkage — when the data layer is normal. And augmentation converts probit data layers into normal ones.

One target resists: the **MNL data layer**. The Gumbel-error structure has no Albert–Chib-style augmentation into conjugacy;³ the logit likelihood must be *evaluated*, not conjugated away.

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so

But nothing requires a Gibbs cycle to be conjugate in *every* block. Replace the unavailable exact draw, in just that block, with a Metropolis step targeting the block’s conditional — accept/reject by Equation 15.3, conditioning on all other blocks — and the chain remains valid: **Metropolis-within-Gibbs**. The blueprint for the next chapter writes itself: population mean and covariance, conjugate Gibbs draws (Equation 16.2 and inverse-Wishart, treating the current β_n ’s as data — literally `gibbs_hier`’s middle blocks); each person’s β_n , a small random-walk Metropolis step whose target is (population prior) $\times$ (that person’s eight-task MNL likelihood) — literally Chapter 15’s sampler, shrunk to one person’s data. Every component exists. Assembly is Chapter 17.

16.6 Key Learnings

- Gibbs sampling cycles through blocks, drawing each from its full conditional (Equation 16.1): every draw accepted, nothing tuned, at the price of deriving conditionals. It is M-H with perfect proposals — and correlated blocks make its staircase treads small, so block wisely.
- Conjugacy is the conditional factory. The normal–normal update (Equation 16.2) — *precisions add, means precision-weight* — is the single most reused formula in the rest of this book; inverse-gamma (and later inverse-Wishart) handles the variances.
- The hierarchical normal model holds every person’s parameter in the posterior jointly with the population’s: person-level inference with *estimated, automatic shrinkage* (Figure 16.2) — Chapter 14’s ad-hoc extraction, rebuilt as a consequence of the model.
- Data augmentation (Albert–Chib): reinstate latent utilities and the probit’s intractable likelihood dissolves into truncated-normal draws (`rtnorm()`, as promised in Chapter 2) plus Bayesian regression. No tuning, no likelihood evaluations. The same move powers the MNP samplers of Part IV.
- The MNL layer alone resists conjugation; Metropolis-within-Gibbs — exact draws where available, M-H steps where not — closes the gap. All HB-MNL components are now on the bench.

“agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

17 The Hierarchical Bayesian MNL

This is the destination chapter. The hierarchical Bayesian multinomial logit — **HB-MNL** to the industry that runs it thousands of times a day — is the model this book has been assembling since page one: MNL choice behavior (Chapter 7), person-specific coefficients (Chapter 11, Chapter 13), a population distribution binding them (Chapter 16), and a posterior explored by MCMC (Chapter 15). Every block of its sampler is code you have already written and tested; this chapter’s work is assembly, and then the payoff that Chapter 14 promised: exact person-level inference, with the shrinkage negotiated by the model, at computational cost that *falls* where MSL’s rose.

We estimate it on the same simulated dataset as Chapter 13 and Chapter 14 — one dataset, three estimators, honest comparison.

17.1 The Model

Three layers, written once in full:

$$\begin{aligned} \text{choices: } y_{nt} &\sim \text{MNL} \left(\mathbf{x}'_{nt1}\beta_n, \dots, \mathbf{x}'_{ntJ}\beta_n \right) \\ \text{people: } \beta_n &\sim N \left(\bar{\beta}, \Sigma \right) \\ \text{priors: } \bar{\beta} &\sim N \left(\mathbf{0}, 100 I \right), \quad \Sigma \sim IW \left(\nu_0, V_0 \right) \end{aligned} \tag{17.1}$$

The first two layers *are* the mixed MNL of Equation 13.1 — identical model of behavior and heterogeneity. The third layer is what makes it Bayesian: the population parameters themselves get (weakly informative) priors, standard choices being $\nu_0 = K + 3$ degrees of freedom and scale $V_0 = \nu_0 I$ for the inverse-Wishart [Rossi, Allenby, and McCulloch (2005) — and mark that V_0 ; it returns under scrutiny in the practical notes]. Two specification notes. We let **all six** coefficients be random with a **full covariance** Σ — the specification that cost MSL its 27-parameter nightmare is the default here, because (as the sampler will show) the hierarchy prices covariance matrices in draws, not in optimization dimensions; it is also deliberately *more general than the truth* (which has three random coefficients, uncorrelated), letting us watch how an honest model handles generality it doesn’t need. And unlike Chapter 13 there is no draw count R , no Halton bases, no simulated integral anywhere: the β_n are not integrated out but *sampled*, which is the whole trick.

17.2 The Sampler

The joint posterior over everything unknown — $p(\beta_{1:N}, \bar{\beta}, \Sigma \mid \text{data})$, a $(500 \times 6 + 6 + 21)$ -dimensional object — is intractable whole but factors beautifully into Chapter 16 blocks:

Block 1: each β_n . Given $\bar{\beta}$ and Σ , person n ’s conditional posterior is (their MNL likelihood over $T = 8$ tasks) $\times$ (their $N(\bar{\beta}, \Sigma)$ population prior) — exactly Equation 14.1, now sampled rather than summarized. The MNL kernel blocks conjugacy (Chapter 16’s closing diagnosis), so this block takes a **random-walk Metropolis step**: propose $\beta'_n = \beta_n + \tau L \mathbf{u}$ — with $\mathbf{u} \sim N(\mathbf{0}, I_K)$ a fresh standard-normal vector — and accept by the log ratio. Two refinements from the trade (K. Train 2001; Rossi, Allenby, and McCulloch 2005): the proposal is shaped by the *current population covariance* (L the lower-triangular Cholesky factor, $LL' = \Sigma$; in R, `t(chol(Sig))`, since `chol()` returns the upper factor — steps stretched along directions the population actually varies), and the step size τ — Chapter 15’s one knob, named `step` in the code — **adapts during burn-in** toward a $\sim 30\%$ acceptance rate, then freezes.¹

Explicitly, person n ’s move is accepted with probability

$$\alpha_n = \min \left(1, \frac{[\prod_t P(y_{nt} \mid \beta'_n)] \phi(\beta'_n \mid \bar{\beta}, \Sigma)}{[\prod_t P(y_{nt} \mid \beta_n)] \phi(\beta_n \mid \bar{\beta}, \Sigma)} \right)$$

— Chapter 15’s Equation 15.3 with the population density $\phi(\cdot \mid \bar{\beta}, \Sigma)$ playing the prior’s role.

Block 2: $\bar{\beta}$. Given the current β_n ’s and Σ , this is the Equation 16.2 update with the β_n ’s as “data” — a conjugate multivariate normal draw. Literally `gibbs_hier`’s middle block from Chapter 16. With the prior parameterized by its *precision* A_0 — so $N(\mathbf{0}, A_0^{-1}I)$ with $A_0 = 0.01$ is exactly Equation 17.1’s $N(\mathbf{0}, 100I)$; precision is how the prior enters the update, and the hierarchical literature’s convention (and `bayesm`’s) is to pass it directly — precisions add exactly as in Equation 16.2:

$$\bar{\beta} \mid \beta_{1:N}, \Sigma \sim N((N\Sigma^{-1} + A_0I)^{-1}\Sigma^{-1} \sum_n \beta_n, (N\Sigma^{-1} + A_0I)^{-1})$$

Block 3: Σ . Given β_n ’s and $\bar{\beta}$: inverse-Wishart draw from the accumulated deviations — the matrix version of Chapter 16’s variance update. Sampling an IW is one small function via the Bartlett decomposition (build a Wishart from chi-square and normal draws — a construction in the spirit of Chapter 2 — and invert).

$$\Sigma \mid \beta_{1:N}, \bar{\beta} \sim IW(\nu_0 + N, V_0 + \sum_n (\beta_n - \bar{\beta})(\beta_n - \bar{\beta})')$$

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

— prior degrees of freedom plus data observations, prior scale plus data scatter: Chapter 16’s inverse-gamma grammar, in matrix dress.

Metropolis-within-Gibbs, exactly as Chapter 16 forecast: two exact draws and one tuned step per cycle.

17.3 Coding It

```
study <- readRDS("../data/mixed_study.rds")
d <- study$data
X <- model.matrix(~ brand + ram + screen + price, data = d)[, -1]
y <- d$choice
task_id <- cumsum(!duplicated(d[, c("id", "task")]))
id <- d$id
n_id <- max(id); K <- ncol(X)

## per-person MNL log-likelihood, given an N x K matrix of betas
## (the @sec-mnl-mle kernel + one rowsum by person; cf. @sec-mixed-mnl)
persll <- function(B, y, X, task_id, id, id_of_task) {
  V <- rowSums(X * B[id, ])
  Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  logP <- V - Vmax - log(rowsum(eV, task_id)[task_id])
  as.vector(rowsum(logP[y == 1], id_of_task))
}

## inverse-Wishart via the Bartlett decomposition
rinvwishart <- function(nu, S) {
  K <- nrow(S); L <- t(chol(solve(S)))
  A <- matrix(0, K, K)
  diag(A) <- sqrt(rchisq(K, nu - (1:K) + 1))
  A[lower.tri(A)] <- rnorm(K*(K-1)/2)
  solve(L %*% A %*% t(A) %*% t(L))
}
```

The only new machinery relative to Chapter 8’s kernel is `persll()`’s last line: `logP[y == 1]` has one entry per task (the chosen alternative’s log probability), and `id_of_task` labels each task with its owner, so `rowsum()` adds each person’s eight task log-probabilities into the length- N vector the Metropolis step consumes.

```

hb_mnl <- function(y, X, task_id, id, S_iter, burn,
                  nu0 = ncol(X) + 3, V0 = diag(ncol(X)) * (ncol(X) + 3),
                  A0 = 0.01, step0 = 0.1, thin_ind = 10) {
  K <- ncol(X); n_id <- max(id)
  id_of_task <- id[!duplicated(task_id)]

  B <- matrix(0, n_id, K) # person betas
  bar <- rep(0, K); Sig <- diag(K)
  step <- step0
  ll_cur <- persll(B, y, X, task_id, id, id_of_task)

  out <- list(bar = matrix(NA_real_, S_iter, K),
             sd = matrix(NA_real_, S_iter, K),
             acc = numeric(S_iter),
             B_draws = array(NA_real_, c((S_iter - burn) %/% thin_ind, n_id, K)))
  kk <- 0
  for (s in 1:S_iter) {
    ## -- block 1: all beta_n at once, RW Metropolis --
    L <- t(chol(Sig))
    prop <- B + step * (matrix(rnorm(n_id*K), n_id, K) %%% t(L))
    ll_prop <- persll(prop, y, X, task_id, id, id_of_task)
    Q_cur <- forwardsolve(L, t(B - rep(bar, each = n_id)))
    Q_prop <- forwardsolve(L, t(prop - rep(bar, each = n_id)))
    lpri_cur <- -0.5 * colSums(Q_cur^2) # log N(bar, Sig), up to const
    lpri_prop <- -0.5 * colSums(Q_prop^2)
    accept <- log(runif(n_id)) < (ll_prop + lpri_prop - ll_cur - lpri_cur)
    B[accept, ] <- prop[accept, ]
    ll_cur[accept] <- ll_prop[accept]
    if (s <= burn) step <- step * ifelse(mean(accept) > 0.30, 1.05, 0.95)

    ## -- block 2: bar | B, Sig (conjugate normal; prior N(0, (1/A0) I)) --
    Sinv <- solve(Sig)
    Vb <- solve(n_id * Sinv + diag(A0, K))
    bar <- as.vector(Vb %%% (Sinv %%% colSums(B)) +
                    t(chol(Vb)) %%% rnorm(K))

    ## -- block 3: Sigma | B, bar (inverse-Wishart) --
    Dev <- B - rep(bar, each = n_id)
    Sig <- rinvwishart(nu0 + n_id, V0 + crossprod(Dev))

    out$bar[s, ] <- bar
    out$sd[s, ] <- sqrt(diag(Sig))
  }
}

```

```

    out$acc[s]    <- mean(accept)
    if (s > burn && (s - burn) %% thin_ind == 0) {
      kk <- kk + 1; out$B_draws[kk, , ] <- B
    }
  }
  out
}

```

Sixty lines, and read against the blueprint every one is accounted for: the vectorized all-persons Metropolis step (two `persll()` calls per iteration — footnote 1’s promise, kept), the population prior evaluated via `forwardsolve()` against Σ ’s Cholesky factor (the Section 2.4 quadratic form, no densities packages), the two conjugate draws, the burn-in-only step adaptation, and Chapter 9’s preallocation (with the person-draws thinned — storing every 10th of 500×6 numbers keeps memory civil at no inferential cost). One subtlety earns its own sentence: `lpri_cur` and `lpri_prop` omit the normal density’s constant $-\frac{1}{2} \log |\Sigma| - \frac{K}{2} \log 2\pi$, which is legitimate only because both are evaluated under the *same* Σ within an iteration, so the constant cancels in the acceptance ratio. Mechanically, `forwardsolve(L, ...)` computes $L^{-1}(\beta_n - \bar{\beta})$, whose squared column sums are the quadratic forms $(\beta_n - \bar{\beta})' \Sigma^{-1} (\beta_n - \bar{\beta})$ for all 500 people in one triangular solve. And one bookkeeping asymmetry to hold onto: `bar` and `sd` store every iteration, burn-in included — we drop the first `burn` rows at analysis time — while the person-level draws are kept only after burn-in and only every `thin_ind`-th cycle, so each chain’s `B_draws` is a $400 \times 500 \times 6$ array: 400 thinned draws, 500 people, 6 coefficients.

17.4 Running It

```

set.seed(170)
S_iter <- 5000; burn <- 1000
fit  <- hb_mnl(y, X, task_id, id, S_iter = S_iter, burn = burn)
fit2 <- hb_mnl(y, X, task_id, id, S_iter = S_iter, burn = burn) # second chain
round(c(acc_rate = mean(fit$acc[-(1:burn)])), 2)

```

```

acc_rate
0.3

```

Two independent chains (different starting randomness), a minute or so each — pause on that: the *full-covariance* hierarchical model, on data where MSL with a diagonal 3-coefficient specification took comparable time per fit, and where the full-covariance MSL was an extrapolated nightmare. Diagnostics, per Chapter 15’s non-negotiable workflow:

```

post <- (burn + 1):S_iter
tr <- rbind(
  data.frame(iter = post, chain = "1", what = "bar_price", v = fit$bar[post, 6]),
  data.frame(iter = post, chain = "2", what = "bar_price", v = fit2$bar[post, 6]),
  data.frame(iter = post, chain = "1", what = "sd_price", v = fit$sd[post, 6]),
  data.frame(iter = post, chain = "2", what = "sd_price", v = fit2$sd[post, 6])
)
ggplot(tr[tr$iter %% 4 == 0, ], aes(iter, v, color = chain)) +
  geom_line(linewidth = 0.2) +
  facet_wrap(~ what, scales = "free_y", ncol = 1) +
  scale_color_grey(end = 0.6) +
  labs(x = "iteration", y = NULL)

```

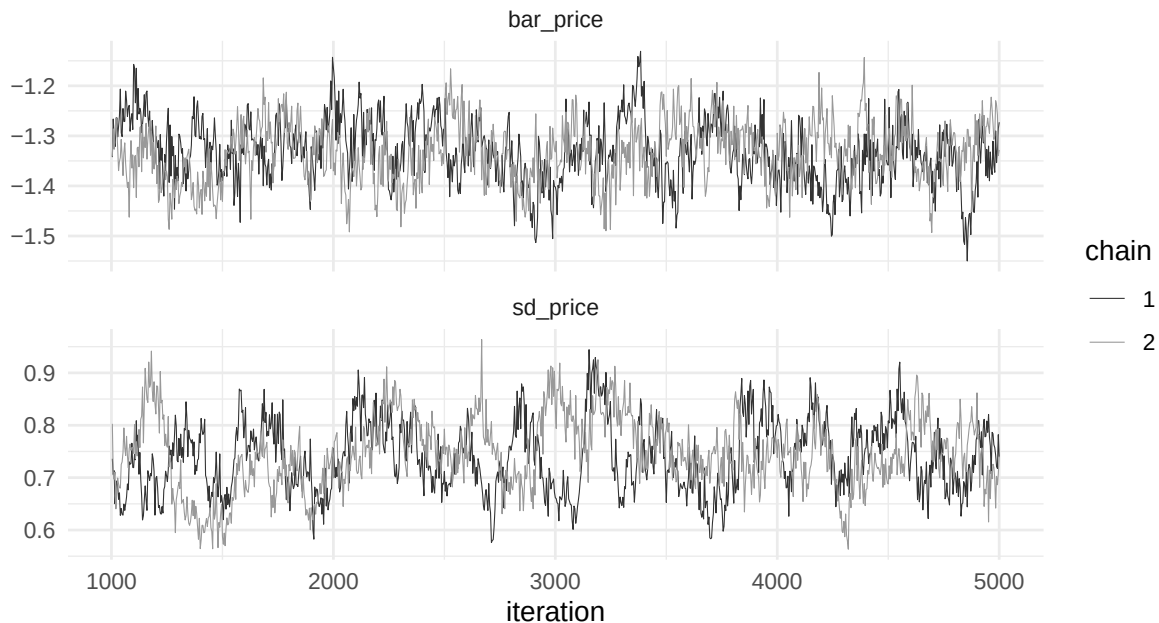


Figure 17.1: Traces of the population mean and standard deviation of the price coefficient, two chains overlaid (post-burn-in, thinned for display). Fuzzy, overlapping, stationary: the caterpillars of Chapter 15, now on a 3,027-parameter posterior.

```

rhat <- function(mat) {
  m <- ncol(mat); n <- nrow(mat)
  B <- n * var(colMeans(mat)); W <- mean(apply(mat, 2, var))
  sqrt(((n - 1)/n * W + B/n) / W)
}

```

```
round(range(apply(1:K, function(k)
  rhat(cbind(fit$bar[post, k], fit2$bar[post, k])))), 3)
```

```
[1] 1.000 1.018
```

Healthy. Note in the traces that `sd_price` mixes more slowly than `bar_price` — variance components always do (they are informed only by dispersion *between* noisy person-estimates), and their effective sample sizes are correspondingly lower. Population-level results, pooling the chains:

```
bar_draws <- rbind(fit$bar[post, ], fit2$bar[post, ])
sd_draws <- rbind(fit$sd[post, ], fit2$sd[post, ])
tab <- data.frame(
  truth_bar = study$bar_true, post_bar = colMeans(bar_draws),
  bar_sd = apply(bar_draws, 2, sd),
  truth_sd = study$sd_true, post_sd = colMeans(sd_draws)
)
rownames(tab) <- colnames(X)
round(tab, 2)
```

	truth_bar	post_bar	bar_sd	truth_sd	post_sd
brandDell	0.5	0.62	0.06	0.0	0.60
brandApple	1.0	1.15	0.07	0.8	0.85
ram16	0.6	0.68	0.07	0.0	0.67
ram32	0.9	0.98	0.07	0.6	0.81
screen15	0.3	0.38	0.05	0.0	0.60
price	-1.2	-1.33	0.06	0.7	0.74

Read the two halves separately. The population *means* recover well, if a touch generously scaled. The population *standard deviations* recover the pattern — the three truly-random coefficients (`apple`, `ram32`, `price`) estimated largest, in the right order — but the three truly-*fixed* coefficients show standard deviations near 0.6 rather than 0. That is not a bug, and diagnosing it is the practical-notes section below; the short version is that our default inverse-Wishart prior cannot believe in zero variance. File it, and first collect the payoff the chapter exists for.

17.5 The Payoff: Individual-Level Posteriors

The sampler’s `B_draws` array holds something neither Chapter 13 nor any amount of MSL effort could produce: *joint posterior draws of every respondent’s coefficient vector*, with population-level uncertainty propagated through automatically — β and Σ wiggled underneath every

person's draws, the one-step architecture Chapter 14 demanded. Person-level point estimates and honest intervals are now sample statistics:

```
nd <- dim(fit$B_draws)[1]          # pool the two chains' person-draws
Bd <- array(NA_real_, c(2*nd, n_id, K)) # (rbind would flatten a 3-D array)
Bd[1:nd, , ] <- fit$B_draws
Bd[(nd+1):(2*nd), , ] <- fit2$B_draws
bhat_price <- colMeans(Bd[, , 6])   # posterior mean per person
blo <- apply(Bd[, , 6], 2, quantile, 0.05)
bhi <- apply(Bd[, , 6], 2, quantile, 0.95)
```

```
set.seed(171)
show <- sample(n_id, 40)
o <- show[order(bhat_price[show])]
dfp <- data.frame(rank = 1:40, mean = bhat_price[o], lo = blo[o], hi = bhi[o],
                  truth = study$betas_true[o, 6])
ggplot(dfp, aes(x = rank)) +
  geom_segment(aes(xend = rank, y = lo, yend = hi), color = "grey60") +
  geom_point(aes(y = mean), size = 1.2) +
  geom_point(aes(y = truth), shape = 4, size = 1.8) +
  labs(x = "respondent (ordered)", y = expression(beta[price]))
```

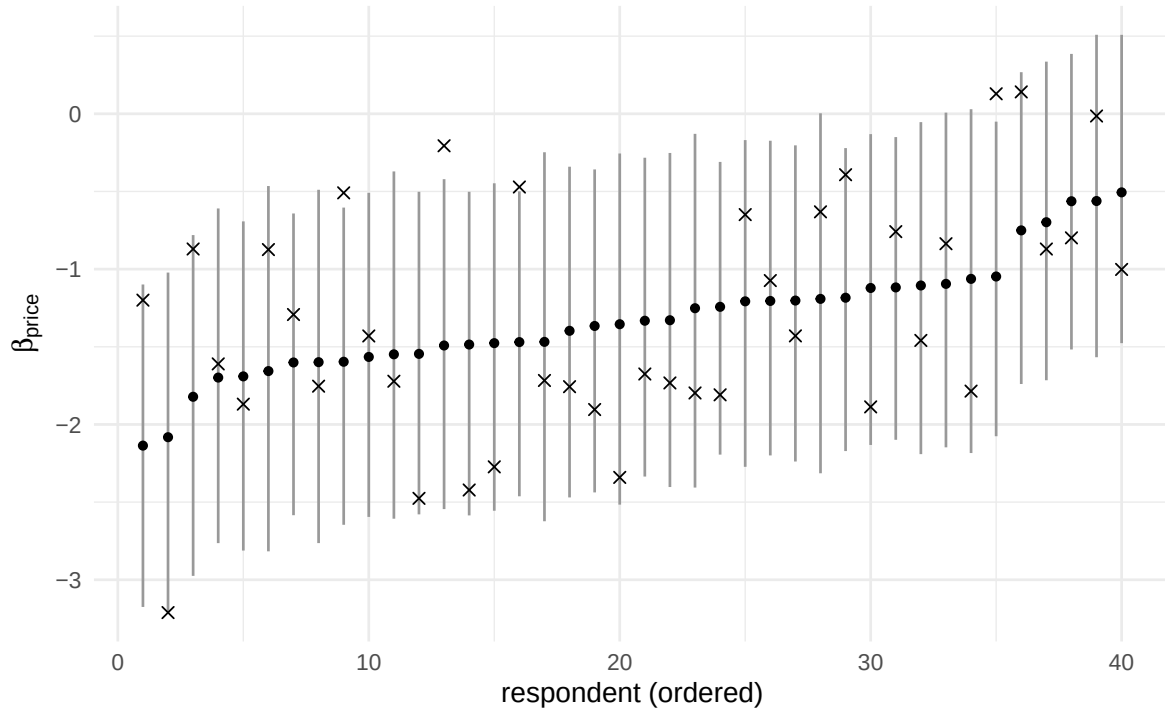


Figure 17.2: The deliverable: posterior means (dots) and 90% credible intervals (segments) of the price coefficient for 40 randomly chosen respondents, ordered by posterior mean, with each respondent's true coefficient marked (x). Intervals are person-specific, honest, and—for most respondents—cover the truth.

```
c(cor_with_truth = round(cor(bhat_price, study$betas_true[, 6]), 2),
  coverage_90 = round(mean(study$betas_true[, 6] >= blo &
                           study$betas_true[, 6] <= bhi), 2))
```

```
cor_with_truth    coverage_90
           0.51           0.92
```

Person-by-person tracking of the truth on par with Chapter 14's conditional means² — but now each respondent carries an interval, and those intervals *cover at close to their nominal rate*. That coverage number is quietly the chapter's most important line: it certifies that the person-level uncertainty statements mean what they say, the exact property Chapter 14 proved

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

the two-step likelihood route could not deliver. Shrinkage is visible too — posterior means span a narrower range than the truths, for Chapter 16’s precision-weighted reasons — so the Chapter 14 warning stands: the spread of *tastes* is read from Σ ’s posterior, never from the spread of person-level means.

17.6 HB and MSL on the Same Data

The comparison Chapter 12 bookmarked. Same dataset, two engines:

```
msl <- study$theta_hat      # @sec-mixed-msl: means (6) + log-sds (3 random)
comp <- data.frame(
  truth = c(study$bar_true, study$sd_true[study$rc]),
  msl   = c(msl[1:6], exp(msl[7:9])),
  hb    = c(colMeans(bar_draws), colMeans(sd_draws)[study$rc])
)
rownames(comp) <- c(colnames(X), paste0("sd_", colnames(X)[study$rc]))
round(comp, 2)
```

	truth	msl	hb
brandDell	0.5	0.55	0.62
brandApple	1.0	1.03	1.15
ram16	0.6	0.61	0.68
ram32	0.9	0.89	0.98
screen15	0.3	0.34	0.38
price	-1.2	-1.22	-1.33
sd_brandApple	0.8	0.68	0.85
sd_ram32	0.6	0.54	0.81
sd_price	0.7	0.52	0.74

Substantive agreement on every population mean and on the heterogeneity ordering, with the HB numbers running slightly larger — partly the IW floor (below), partly the specification gap (HB’s phantom heterogeneity on the fixed coefficients absorbs scale). The fair scorecard, then (and it matches the literature’s: K. Train 2001; Ben-Akiva, McFadden, and Train 2019, Ch. 7):

- **Population parameters, modest spec:** a tie — both engines recover them, in comparable time here.
- **Full covariance matrices:** HB, walking away. MSL’s 27-parameter Cholesky climb is HB’s one extra `rinvwishart()` line.
- **Individual-level inference:** HB by design — native draws with honest coverage versus a bolt-on with a plug-in.

- **Sharp-null economy:** MSL retains one genuine edge — when you *want* a coefficient exactly fixed (no heterogeneity), you just don't randomize it, while the IW-equipped hierarchy resists zeros (next section) and constraining it takes work.
- **Tuning burdens:** a wash, honestly — MSL trades draw counts and starting values for HB's burn-in judgment and step adaptation. Neither is push-button; both are manageable with the diagnostics this book has drilled.

17.7 Validation Against bayesm

The reference implementation is `rhierMnlRwMixture` (Rossi, Allenby, and McCulloch 2005) — the same model with a mixture-of-normals population layer, set to one component to match Equation 17.1, and the same default priors. Its data format wants each person's tasks as a list of per-occasion design matrices:

```
library(bayesm)
J <- 3; T <- 8
lgtdata <- lapply(1:n_id, function(n) {
  rows <- which(id == n)
  Xn <- X[rows, ]
  list(y = max.col(t(matrix(y[rows], nrow = J))),
       X = lapply(1:T, function(t) Xn[((t-1)*J + 1):(t*J), , drop = FALSE]))
})
set.seed(172)
outb <- rhierMnlRwMixture(Data = list(p = J, lgtdata = lgtdata),
                          Prior = list(ncomp = 1),
                          Mcmc = list(R = 5000, keep = 2, nprint = 0))
```

```
keepb <- 1001:2500      # kept draws: keep = 2 stores every 2nd, so this discards 2,000 iterations
mu_b <- t(sapply(keepb, function(s) outb$nmix$compdraw[[s]][[1]]$mu))
sd_b <- t(sapply(keepb, function(s) {
  rooti <- outb$nmix$compdraw[[s]][[1]]$rooti
  sqrt(diag(chol2inv(t(rooti))))
}))
round(rbind(ours_bar = colMeans(bar_draws), bayesm_bar = colMeans(mu_b),
            ours_sd  = colMeans(sd_draws),  bayesm_sd  = colMeans(sd_b)), 2)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
ours_bar	0.62	1.15	0.68	0.98	0.38	-1.33
bayesm_bar	0.61	1.14	0.67	0.98	0.38	-1.34
ours_sd	0.60	0.85	0.67	0.81	0.60	0.74
bayesm_sd	0.61	0.82	0.67	0.72	0.58	0.69

Two independently coded samplers — ours, sixty transparent lines; theirs, a compiled industry standard — agreeing on both layers of the posterior.³ Note that `bayesm` shows the *same* elevated standard deviations on the truly-fixed coefficients: the phenomenon is the prior’s, not our code’s, which is exactly what the next section establishes on purpose.

17.8 Practical Notes

The inverse-Wishart floor — priors on Σ matter. Our recovery table showed $\hat{\sigma} \approx 0.6$ for coefficients whose true heterogeneity is zero. The mechanism: the IW prior with scale $V_0 = \nu_0 I$ puts essentially no mass on variances near zero — it is a prior *against* homogeneity — and eight tasks per person cannot overrule it (a person’s eight choices are consistent with modest taste variation plus Gumbel noise). Watch the prior’s hand move, by re-running with the scale matrix shrunk an order of magnitude:

```
set.seed(173)
fit_tight <- hb_mnl(y, X, task_id, id, S_iter = 4000, burn = 1000,
                  V0 = diag(K) * (K + 3) * 0.1)
round(rbind(default_V0 = colMeans(sd_draws),
            tight_V0   = colMeans(fit_tight$sd[-(1:1000), ]),
            truth       = study$sd_true), 2)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]
default_V0	0.60	0.85	0.67	0.81	0.60	0.74
tight_V0	0.31	0.58	0.33	0.53	0.33	0.41
truth	0.00	0.80	0.00	0.60	0.00	0.70

The phantom standard deviations drop by nearly half — and the real ones shrink too, `sd_price` now *under* the truth. Neither prior is “right”: the IW simply cannot express “some variances near zero, others large” — its single scale parameter couples them — and this is a known, documented limitation with active remedies in the literature (separation strategies; tightened scales chosen by prior predictive checks, as in Allenby et al. (2014)). The working discipline is the one Chapter 15 taught: *run the sensitivity, report it, and interpret heterogeneity magnitudes with the prior in view*. With hundreds of tasks per person the data would overwhelm all of this; with eight, the prior is a coauthor, and pretending otherwise is the error.

Sign constraints via transformation. Our population layer lets some simulated consumers like high prices (Chapter 13’s blemish, inherited). The standard repair reparameterizes: model θ_n with $\beta_{n,\text{price}} = -e^{\theta_n}$ — Chapter 2’s lognormal transformation — so every person’s price

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

coefficient is negative while θ_n enjoys the normal hierarchy untouched. In the sampler, exactly one line changes: `persll()`'s utility computation applies the transform before multiplying. The blocks, the conjugacy, the adaptation — all survive, because the hierarchy never cared what the coefficients *mean*. We defer the demonstration to Chapter 22, where transformed coefficients earn a starring role in the budget-constrained model.

Reporting: two different “heterogeneity” statements. The posterior of $\bar{\beta}$ answers “where is the average consumer” — and its uncertainty shrinks with N . The posterior of the *taste distribution* $N(\bar{\beta}, \Sigma)$ answers “what consumers are out there” — and its spread does not shrink with N ; it is an estimate of real variety. Deliverables built on the wrong one (a “share who would pay X ” computed from $\bar{\beta}$'s posterior instead of the taste distribution) are a classic applied error (Chapman 2013's “averages hide the distribution,” now with the machinery to do it right). Chapter 21 builds the correct versions.

One save for the chapters ahead — the pooled draws, which Chapter 21 will spend freely:

```
saveRDS(list(bar_draws = bar_draws, sd_draws = sd_draws, B_draws = Bd,
            X_names = colnames(X)),
        file = "../data/hb_mnl_fit.rds")
```

17.9 Key Learnings

- The HB-MNL is the mixed MNL wearing priors (Equation 17.1), and its sampler is pure assembly: an all-persons vectorized Metropolis step on the Chapter 8 kernel (adaptive during burn-in, ~30% acceptance), a conjugate normal draw for $\bar{\beta}$, an inverse-Wishart draw for Σ — Metropolis-within-Gibbs as Chapter 16 designed it, in sixty lines that run two chains in minutes.
- The full covariance matrix that strained MSL is free here — no draws, no Cholesky optimization, no R to choose. The engines agree on population parameters where their specifications overlap (and `bayesm` agrees with our sixty lines on everything).
- The payoff is Figure 17.2: every respondent gets a posterior — point, interval, the works — with population uncertainty propagated in one step, and the 90% intervals *actually cover* ~90% of the true individual coefficients. This is the deliverable Chapter 14 proved likelihood could not honestly produce.
- Shrinkage discipline carries over: person means understate taste spread by construction; read heterogeneity from Σ , and never confuse the posterior of the average consumer with the distribution of consumers.
- The priors are coauthors at $T = 8$: the default IW cannot believe in zero variance (phantom sds near 0.6 on truly-fixed coefficients, reproduced by `bayesm`), and tightening V_0 moves every variance — run the sensitivity, report it, interpret accordingly.
- Sign constraints are one transformed line away (lognormal price), because the hierarchy is agnostic about coefficient meaning — the extensibility that Chapter 22 will exploit.

Part IV

Beyond Logit: The Multinomial Probit

18 The Multinomial Probit Model

Part IV leaves the logit family. Every model so far — MNL, nested, latent class, mixed, hierarchical — has Gumbel errors somewhere in its bones, chosen in Chapter 7 for the closed form they buy. The **multinomial probit** (MNP) makes the other choice: multivariate *normal* errors with a freely estimated covariance matrix. Correlation between any pair of alternatives, unobserved-taste patterns of any shape — nothing pre-specified, everything estimated. The price is the one we have spent two parts learning to pay: the choice probabilities are integrals with no closed form, ever.

That price structure is why the MNP sits here, at the book’s culmination, rather than in Part II where its cousins live. Estimating it requires either the simulation-assisted likelihood machinery of Chapter 12 or the augmentation-and-Gibbs machinery of Chapter 16 — and it is the rare model where *both* routes are standard, which makes it the perfect capstone demonstration that model and estimation method are separate choices (Chapter 19 runs the head-to-head). This chapter does the groundwork: the model, its subtle identification, and its simulation.

18.1 Why Probit?

The nested logit of Chapter 10 repaired IIA by letting the researcher declare which alternatives share unobserved appeal. The mixed logit of Chapter 13 generated flexible substitution through random coefficients on *observed* attributes. The MNP attacks the unobservables directly:

$$U_{ntj} = \mathbf{x}'_{ntj}\boldsymbol{\beta} + \varepsilon_{ntj}, \quad \boldsymbol{\varepsilon}_{nt} = (\varepsilon_{nt1}, \dots, \varepsilon_{ntJ})' \sim N(\mathbf{0}, \Omega) \quad (18.1)$$

with Ω an unrestricted $J \times J$ covariance matrix. Return to the story that motivated nesting: Windows machines share unobserved OS-related utility. The nested logit *encoded* that as a nest with an estimated λ . The MNP simply lets the data put a positive covariance between the Acer and Dell errors — or not — along with any other pattern among the alternatives: no nests to specify, no researcher declaration of who competes with whom, no IIA anywhere.¹ For this part we return to the **brand-labeled design** of Chapter 10 (one Acer, one Dell, one

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

Apple per task), because a covariance between “the Acer error” and “the Dell error” needs alternatives with stable identities to attach to.

18.2 The Choice Probability Is an Integral

The behavioral rule is unchanged — pick the maximum utility — so the probability that alternative j is chosen is the probability that $J - 1$ utility differences all fall on j ’s side:

$$P_{ntj} = \Pr(U_{ntj} > U_{ntk} \ \forall k \neq j) = \int_{\text{region where } j \text{ wins}} \phi_J(\varepsilon \mid \mathbf{0}, \Omega) d\varepsilon \quad (18.2)$$

— on its face a J -dimensional integral over the cone where j wins; but only utility *differences* decide the comparison, and differences of normals are normal, so subtracting U_{ntj} from the others collapses it to a $(J - 1)$ -dimensional normal orthant probability. That differenced form is the one every computation downstream actually uses. For $J = 2$ it collapses to the univariate $\Phi(\cdot)$ of the binary probit — closed form, essentially. For $J = 3$ it is a bivariate normal probability — computable by good numerical routines (`pmvnorm`), no longer a formula. For the J of a real supermarket category, it is the reason this model waited until Part IV. Everything hard about the MNP is Equation 18.2; everything in Chapter 19 is a strategy for never quite computing it.

18.3 Identification: The Sternest Version Yet

Section 7.5 established two facts about what choice data can reveal — only utility *differences* matter, and the *scale* of utility is arbitrary — and the MNL absorbed both with quiet normalizations (no common intercept; error variance fixed by convention). The MNP has to pay the same two taxes, but on a much larger taxable estate, because Ω ’s $J(J + 1)/2$ free-looking parameters are exactly where the unidentifiability hides. Walk it through for our $J = 3$ study; the logic is McCulloch and Rossi (1994)’s, and it governs everything in the next two chapters.

Differences. Since only differences matter, subtract a base alternative’s utility — say the Acer’s — from the others. The data can only ever speak about the *differenced* system: two equations, with differenced attributes and a differenced error vector whose covariance is

$$\tilde{\Omega} = M\Omega M', \quad M = \begin{pmatrix} -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \quad (18.3)$$

$\tilde{\Omega}$ is 2×2 symmetric: **three** numbers. The original Ω had six. Three whole dimensions of Ω are invisible to choice data — infinitely many Ω ’s produce identical behavior. (This is the

multivariate version of “a constant added to all utilities cancels”; adding any common noise to all three alternatives changes nothing.)

Scale. Multiply all utilities by $\lambda > 0$: choices unchanged, so one more dimension dies. Convention: fix the first diagonal of the differenced covariance, $\tilde{\omega}_{11} = 1$. Two bookkeeping notes the next chapter leans on. First, from here on $\tilde{\Omega}$ means the differenced covariance *after* this normalization; its unnormalized cousin, raw $M\Omega M'$, will reappear in Chapter 19 under the name W — the object the Gibbs sampler actually draws. Second, the scale tax hits β as well: dividing the differenced system by $\sqrt{(M\Omega M')_{11}}$ to enforce $\tilde{\omega}_{11} = 1$ divides every utility, so what the data identify is $\beta/\sqrt{(M\Omega M')_{11}}$, not β itself. Coefficients scale like utility; covariances like utility squared. Count what survives: $3 - 1 = 2$ free covariance parameters for our three-alternative study. From six down to two — the MNP’s flexibility is real but far smaller than Ω ’s dimensions suggest, and it shrinks further if you forget that each ASC also consumes Section 7.5’s level normalization (our brand-labeled design carries `dell` and `apple` ASCs against the Acer base, exactly as in Chapter 10).

Why belabor this? Because unidentified parameters are the classic MNP implementation disaster, in both estimation routes. Hand an optimizer a likelihood written in terms of raw Ω and it wanders flat directions forever, “converging” wherever it likes (Chapter 6’s warnings, in their natural habitat); run a Gibbs sampler on raw Ω and the chain drifts happily along the unidentified ridge — traces that never settle, $\hat{R}$ that never approaches 1 — *while the identified functions of the parameters mix beautifully*. The two routes of Chapter 19 handle this differently (the MLE route parameterizes only identified quantities; the Bayesian route samples the unidentified space and reports identified ratios, post-processed), and knowing the identified count in advance — $(J - 1)J/2 - 1$ covariance parameters — is how you audit either.

18.4 Warming Up: Binary Probit by MLE

For $J = 2$ everything is tractable, and it is worth thirty seconds to close a loop left open since Chapter 3, where the binary probit was one changed line in the simulator. Its choice probability is $\Phi(\mathbf{x}'\beta)$ (error scale fixed at 1 — the scale tax, paid), its log-likelihood mirrors Equation 5.5 with `pnorm` in place of `plogis`, and Part I’s machinery does the rest:

```
set.seed(180)
N <- 4000
price <- runif(N, 0.8, 2.4); ram16 <- rbinom(N, 1, 0.5)
X <- cbind(1, price, ram16)
beta_true <- c(1.0, -1.2, 0.6)
y <- as.integer(as.vector(X %*% beta_true) + rnorm(N) > 0)

loglik_bprobit <- function(beta, y, X) {
  P <- pnorm(as.vector(X %*% beta))
```

```

    sum(y * log(P) + (1 - y) * log(1 - P))
}
fit <- optim(rep(0, 3), loglik_bprobit, y = y, X = X, method = "BFGS",
            control = list(fnscale = -1), hessian = TRUE)
round(rbind(truth = beta_true, est = fit$par,
            se = sqrt(diag(solve(-fit$hessian)))), 3)

```

```

      [,1] [,2] [,3]
truth 1.000 -1.200 0.600
est    1.049 -1.220 0.623
se     0.081  0.052 0.045

```

Recovery, no drama — and compare with Chapter 16’s Albert–Chib sampler on data of exactly this form: two estimation philosophies already agree on the binary case. Everything difficult about the MNP is about $J > 2$: the moment a *matrix* of error covariances exists, Equation 18.2 stops being `pnorm` and identification stops being automatic.

18.5 Simulating MNP Data

Simulation, as usual, is the easy direction — the integral of Equation 18.2 never appears, because we run the *mechanics* (draw errors, add, `argmax`), not the probabilities. The errors are multivariate normal, so Section 2.4’s Cholesky transformation is the whole trick. We give the truth the OS-correlation story: Acer and Dell errors correlated at 0.6, Apple independent, unit variances —

$$\Omega^* = \begin{pmatrix} 1 & 0.6 & 0 \\ 0.6 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (18.4)$$

— with the usual attribute coefficients and brand ASCs (Equation 7.6’s values, now interpreted against normal-scale errors²):

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

sim_mnp_data <- function(N, T, beta, Omega) {
  J <- 3
  n_rows <- N * T * J
  df <- data.frame(
    id    = rep(seq_len(N), each = T * J),
    task  = rep(rep(seq_len(T), each = J), times = N),
    alt   = factor(rep(c("Acer", "Dell", "Apple"), times = N * T),
                  levels = c("Acer", "Dell", "Apple")),
    ram    = factor(sample(c("8", "16", "32"), n_rows, replace = TRUE),
                  levels = c("8", "16", "32")),
    screen = factor(sample(c("13", "15"), n_rows, replace = TRUE),
                  levels = c("13", "15")),
    price  = round(runif(n_rows, min = 0.8, max = 2.4), 2)
  )
  X <- model.matrix(~ alt + ram + screen + price, data = df)[, -1]
  V <- as.vector(X %*% beta)

  L <- t(chol(Omega)) # @sec-mvnorm
  eps <- as.vector(L %*% matrix(rnorm(n_rows), nrow = J)) # J x (N*T), stacked back
  U <- V + eps
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))
  df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))
  df
}

set.seed(181)
beta_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9,
              screen15 = 0.3, price = -1.2)
Omega_true <- matrix(c(1, 0.6, 0,
                      0.6, 1, 0,
                      0, 0, 1), 3, 3)
mnp <- sim_mnp_data(N = 500, T = 8, beta = beta_true, Omega = Omega_true)

## save for the estimation chapters
saveRDS(list(data = mnp, beta_true = beta_true, Omega_true = Omega_true),
        file = "../data/mnp_study.rds")

```

One line rewards a careful look: the error draw reshapes the row vector into a $3 \times (NT)$ matrix so that *each task's three errors* — one column — get the L transformation as a unit, then stacks back to row order. Correlation lives *within* tasks, independence *across* them: the covariance structure of Equation 18.1, implemented in the shape of the data (Chapter 4's sort contract doing silent work — `alt` varies fastest, so columns are tasks).

The usual battery, plus one check specific to this model. Marginals and directions first:

```
chosen <- mnp[mnp$choice == 1, ]  
round(prop.table(table(chosen$alt)), 3)
```

```
Acer Dell Apple  
0.152 0.298 0.550
```

Apple leads, Acer trails — the ASC ordering, as always. Now the check that matters: *is the error correlation visible in behavior?* Under IIA-obeying errors, how often a shopper's second-favorite brand is the other same-OS machine wouldn't depend on error correlation; under Equation 18.4 it should. A sharper, simpler signature: compare the joint frequency that the two Windows machines *both* beat Apple against what independent errors would produce. Cheapest of all — lean on the model directly: recompute choice shares after deleting the Dell from every task, using fresh simulations, and watch where its share goes:

```
set.seed(182)  
sim_shares <- function(Omega, drop_dell = FALSE) {  
  ## one standardized task: identical specs, $1.5k each  
  X_task <- rbind(acer = c(0,0,1,0,1,1.5), dell = c(1,0,1,0,1,1.5),  
                 apple = c(0,1,1,0,1,1.5))  
  keep <- if (drop_dell) c(1, 3) else 1:3  
  V <- as.vector(X_task %*% beta_true)[keep]  
  L <- t(chol(Omega[keep, keep, drop = FALSE]))  
  R <- 200000  
  eps <- L %*% matrix(rnorm(length(keep) * R), nrow = length(keep))  
  tabulate(max.col(t(V + eps)), length(keep)) / R  
}  
before <- sim_shares(Omega_true)  
after <- sim_shares(Omega_true, drop_dell = TRUE)  
list(before = round(setNames(before, c("acer","dell","apple")), 3),  
     after = round(setNames(after, c("acer","apple")), 3))
```

```
$before  
acer dell apple  
0.106 0.299 0.595
```

```
$after  
acer apple  
0.24 0.76
```

Delete the Dell and its customers flow *disproportionately to the Acer* — the correlated-error sibling — not proportionally as MNL arithmetic would insist. The red-bus/blue-bus pathology of Chapter 7, cured by two off-diagonal numbers. (Run `sim_shares(diag(3))` and the flow turns proportional: the cure really is the correlation, not the normality.)

18.6 The Estimation Problem Ahead

Take stock of the asymmetry we have manufactured. Simulating MNP choices: eight lines, all of them Part I material. Estimating MNP parameters: the likelihood needs Equation 18.2 — an integral we can only approximate — once per task per candidate parameter, inside an optimizer that will make thousands of candidate visits. Chapter 12’s playbook applies but needs an unbiased, *smooth* simulator of an orthant probability, which is a subtler object than the mixed logit’s average-of-closed-forms; building one (GHK, from Chapter 2’s truncated normals) is half of Chapter 19.

The other half never touches the integral. Chapter 16’s Albert–Chib logic generalizes: conditional on the *latent utilities*, the MNP is just seemingly-related regressions with normal errors — conjugate territory — and conditional on parameters and the observed choice, each task’s utilities are a multivariate normal truncated to the region where the chosen alternative wins. Draw utilities, draw parameters, repeat: the intractable integral is dissolved into truncated normal draws, at scale, with the identification post-processing that McCulloch and Rossi (1994) made standard. Two routes, one model, next chapter.

18.7 Key Learnings

- The MNP replaces Gumbel errors with $N(\mathbf{0}, \Omega)$, Ω free (Equation 18.1): substitution patterns are *estimated in the error covariance* rather than specified in nests or proxied by random coefficients. No IIA anywhere — our deletion experiment showed Dell’s share flowing to its correlated sibling, disproportionately.
- The choice probability (Equation 18.2) is a $(J - 1)$ -dimensional normal orthant integral: closed-form at $J = 2$ (`pnorm`), numerical at $J = 3$, and the central obstacle beyond. Everything in Chapter 19 is a strategy for living with it.
- Identification is unforgiving: differencing kills J of Ω ’s dimensions, scale kills one more, leaving $(J - 1)J/2 - 1$ free covariance parameters (two, for our study). Unidentified MNP parameterizations produce optimizers that wander and samplers that never converge — count before you code.
- Simulation needed none of it: Cholesky-correlated errors within tasks (shaped by the data’s sort contract), argmax, done — plus a saved answer key including Ω^* for the recovery tests ahead.

- Two estimation routes await: simulate the integral inside MSL (GHK), or augment it away entirely (truncated normals + Bayesian regression). The capstone comparison of the book's two estimation philosophies, on one model.

19 Estimating the MNP

One model, both toolkits. The MNP's likelihood is a wall of normal orthant integrals (Equation 18.2); this chapter scales it twice — once with Part II's machinery (simulate the integral, maximize the simulated likelihood) and once with Part III's (augment the integral away, Gibbs the rest). Beyond estimating the MNP, the chapter is the book's thesis made concrete: *the model is a set of assumptions; the estimator is a separate choice; and a well-built toolkit lets you make that choice on the merits*. By the end, the two routes will sit in one table, agreeing with each other to the second decimal.

Load the study saved in Chapter 18, and build the differenced objects that both routes consume — everything here runs on the identification analysis of Chapter 18, so the differencing is not preprocessing, it *is* the identified model:

```
mnps_study <- readRDS("../data/mnps_study.rds")
mnps <- mnps_study$data
X <- model.matrix(~ alt + ram + screen + price, data = mnps)[, -1]
K <- ncol(X)

## rows of each alternative (the design is balanced and ordered: @sec-data contract)
i1 <- which(mnps$alt == "Acer"); i2 <- which(mnps$alt == "Dell")
i3 <- which(mnps$alt == "Apple")

Xd2 <- X[i2, ] - X[i1, ]      # attributes of (Dell - Acer), one row per task
Xd3 <- X[i3, ] - X[i1, ]      # attributes of (Apple - Acer)
chosen <- as.integer(mnps$alt[mnps$choice == 1]) # 1=Acer, 2=Dell, 3=Apple
n_task <- length(chosen)
```

And compute the **identified truth** this chapter must recover — the Chapter 18 arithmetic executed: difference Ω^* against the base, normalize the covariance by its first diagonal, and rescale β^* by the *square root* of that diagonal — $\beta_{\text{id}}^* = \beta^* / \sqrt{(M\Omega^*M')_{11}}$ and $\hat{\Omega}^* = M\Omega^*M' / (M\Omega^*M')_{11}$; coefficients live on the utility scale, covariances on the utility-squared scale, so they pay the scale tax at different rates:

```
M <- rbind(c(-1, 1, 0), c(-1, 0, 1))
Ot <- M %*% mnps_study$Omega_true %*% t(M)      # differenced covariance
sc <- sqrt(Ot[1, 1])                          # the scale normalization
```

```
beta_star <- mnp_study$beta_true / sc
omega_star <- c(w12 = Ot[1, 2], w22 = Ot[2, 2]) / Ot[1, 1]
round(c(beta_star, omega_star), 3)
```

dell	apple	ram16	ram32	screen15	price	w12	w22
0.559	1.118	0.671	1.006	0.335	-1.342	0.500	2.500

Those eight numbers — six scaled coefficients, one differenced covariance, one differenced variance ratio — are everything the data can reveal, and the scoreboard for both routes.

19.1 Route 1: Simulated Maximum Likelihood via GHK

19.1.1 Why the obvious simulator fails

Chapter 1’s frequency simulator applies directly: draw errors, count how often the chosen alternative wins. It is unbiased — and useless inside an optimizer, for two reasons that Chapter 12 equips us to name instantly. It is a *step function* of the parameters (nudge β slightly and either no draw changes its vote or one flips discretely), so gradients are zero almost everywhere and undefined elsewhere — BFGS starves. And for tasks where the chosen alternative was unlikely, all R draws can vote no, returning $\hat{P} = 0$ and $\log \hat{P} = -\infty$ — Chapter 8’s underflow catastrophe, by construction rather than accident. Smoothing the indicator (the logit-kernel AR simulator) patches both at the cost of a tuning parameter and bias (K. E. Train 2009, sec. 5.6). The field’s solution is better than a patch.

19.1.2 The GHK simulator

The **GHK simulator** (Geweke–Hajivassiliou–Keane) computes the orthant probability by recursive conditioning, and it is Chapter 2’s truncated-normal trick wearing its work clothes. The probability that alternative j was chosen is the probability that a bivariate normal vector $\mathbf{w} = (w_1, w_2)$ — the utility differences *against* j — lands entirely below zero. Factor the joint event through the Cholesky root of its covariance, $A_j \tilde{\Omega} A_j' = LL'$, writing $\mathbf{w} = \boldsymbol{\mu} + L\boldsymbol{\eta}$ with $\boldsymbol{\eta}$ standard normal. Because L is lower triangular with positive diagonal, $w_1 < 0 \iff \eta_1 < b_1 := -\mu_1/\ell_{11}$; and given η_1 , $w_2 < 0 \iff \eta_2 < b_2(\eta_1) := (-\mu_2 - \ell_{21}\eta_1)/\ell_{22}$. The law of total probability, plus the independence of η_2 from η_1 , then gives:

$$\Pr(w_1 < 0, w_2 < 0) = \underbrace{\Pr(\eta_1 < b_1)}_{p_1} \times \mathbb{E} \left[\underbrace{\Pr(\eta_2 < b_2(\eta_1))}_{p_2(\eta_1)} \middle| \eta_1 < b_1 \right] \quad (19.1)$$

where $b_1 = -\mu_1/\ell_{11}$ and $b_2(\eta_1) = (-\mu_2 - \ell_{21}\eta_1)/\ell_{22}$: the first coordinate's constraint is a univariate normal probability, exactly; the second's is a univariate probability *given* the first, averaged over truncated draws of η_1 . GHK draws η_1 from its truncated distribution by the inverse-CDF trick of Equation 2.2 — `qnorm(u * p1)` maps a uniform into exactly the admissible region — and averages $p_1 \cdot p_2(\eta_1^{(r)})$ over R draws. Every ingredient is a `pnorm/qnorm` call: **smooth** in the parameters (BFGS eats), **strictly positive** (each factor is a genuine probability), and unbiased. For J alternatives the recursion just continues (K. E. Train 2009, sec. 5.6.3 writes out the general case) — each coordinate truncated given the previous ones, the triangularity of L making each step univariate; you can see in that description why Section 2.4 called Cholesky triangularity “the same idea that powers GHK.”

One bookkeeping matter: the differences-against- j vector for each chosen alternative is a fixed linear map A_j of the base differences (d_2, d_3) — e.g., for $j = 2$: $(U_1 - U_2, U_3 - U_2) = (-d_2, d_3 - d_2)$.

All three maps, for the record: $A_1 = I_2$ (Acer chosen: $(d_2, d_3) < \mathbf{0}$ already); $A_2 = \begin{pmatrix} -1 & 0 \\ -1 & 1 \end{pmatrix}$, so

$A_2 \mathbf{d} = (-d_2, d_3 - d_2)$; $A_3 = \begin{pmatrix} 0 & -1 \\ 1 & -1 \end{pmatrix}$. Row i of A_j simply rewrites $U_k - U_j$ in terms of the base

differences. So means transform per task, but the covariance $A_j \tilde{\Omega} A_j'$ (hence its Cholesky) is computed once per chosen-alternative *type* per likelihood call — three small factorizations, not four thousand.

```
## GHK simulated log-likelihood for the 3-alternative MNP
## theta = (beta [K], a, log b) with differenced-normalized chol Lt = [1 0; a b]
ghk_loglik <- function(theta, Xd2, Xd3, chosen, U1) {
  K <- ncol(Xd2)
  beta <- theta[1:K]; a <- theta[K+1]; b <- exp(theta[K+2])
  Ot_id <- rbind(c(1, a), c(a, a^2 + b^2))      # identified covariance: first diagonal fixed
  m2 <- as.vector(Xd2 %*% beta)                 # mean of d2 = U_dell - U_acer
  m3 <- as.vector(Xd3 %*% beta)
  A <- list(diag(2),
            rbind(c(-1, 0), c(-1, 1)),         # chosen = Acer: (d2, d3) < 0
            rbind(c(0, -1), c(1, -1)))         # chosen = Dell: (-d2, d3-d2) < 0
                                              # chosen = Apple: (-d3, d2-d3) < 0
  ll <- 0
  for (j in 1:3) {
    idx <- which(chosen == j)
    mu <- cbind(m2[idx], m3[idx]) %*% t(A[[j]])
    Lj <- t(chol(A[[j]] %*% Ot_id %*% t(A[[j]])))
    u <- U1[idx, , drop = FALSE]               # fixed uniforms (@sec-msl!)
    b1 <- (-mu[, 1]) / Lj[1, 1]
    p1 <- pmax(pnorm(b1), 1e-12)
    e1 <- qnorm(pmax(u * p1, 1e-14))           # truncated draws via the inverse-CDF trick
    b2 <- (-mu[, 2] - Lj[2, 1] * e1) / Lj[2, 2]
```

```

    Phat <- pmax(rowMeans(p1 * pnorm(b2)), 1e-300)
    ll <- ll + sum(log(Phat))
  }
  ll
}

```

Note the Chapter 12 discipline embedded: the uniforms $U1$ are generated *once* and passed in — fixed draws, rule one — and the covariance enters through its Cholesky elements with the diagonal log-parameterized, so every candidate the optimizer visits implies a valid matrix. Before trusting it, the mandatory audit — GHK against an independent oracle, R's numerical bivariate-normal routine, on a handful of tasks:

```

library(mvtnorm)
set.seed(190)
U1 <- matrix(runif(n_task * 100), n_task, 100)

check <- sapply(1:5, function(i) {
  mu <- c(sum(Xd2[i, ] * beta_star), sum(Xd3[i, ] * beta_star))
  Ot_id <- rbind(c(1, omega_star["w12"]),
                c(omega_star["w12"], omega_star["w22"]))
  A <- list(diag(2), rbind(c(-1,0), c(-1,1)), rbind(c(0,-1), c(1,-1)))[[chosen[i]]]
  exact <- pmvnorm(upper = c(0, 0), mean = as.vector(A %*% mu),
                  sigma = A %*% Ot_id %*% t(A))
  ghk <- { # one-task GHK, 100 draws
    Lj <- t(chol(A %*% Ot_id %*% t(A))); mm <- as.vector(A %*% mu)
    p1 <- pnorm(-mm[1]/Lj[1,1])
    e1 <- qnorm(U1[i, ] * p1)
    mean(p1 * pnorm((-mm[2] - Lj[2,1]*e1)/Lj[2,2]))
  }
  c(exact = exact, ghk = ghk)
})
round(check, 4)

```

	[,1]	[,2]	[,3]	[,4]	[,5]
exact	0.6359	0.8164	0.6167	0.3432	0.5639
ghk	0.6170	0.8115	0.5975	0.3296	0.5641

Agreement in the third decimal with 100 draws — certified. Now the fit, with the covariance parameters box-bounded (the Chapter 10 device) to keep the search in well-conditioned territory:

```

fit_ghk <- optim(c(rep(0, K), 0, 0), ghk_loglik,
               Xd2 = Xd2, Xd3 = Xd3, chosen = chosen, U1 = U1,
               method = "L-BFGS-B",
               lower = c(rep(-Inf, K), -5, -2.5),
               upper = c(rep( Inf, K),  5,  2.5),
               control = list(fnscale = -1, maxit = 500), hessian = TRUE)
se_ghk <- sqrt(diag(solve(-fit_ghk$hessian)))
est_ghk <- c(fit_ghk$par[1:K],
            w12 = fit_ghk$par[K+1],
            w22 = fit_ghk$par[K+1]^2 + exp(fit_ghk$par[K+2])^2)
round(data.frame(truth = c(beta_star, omega_star),
                ghk = est_ghk,
                se = c(se_ghk[1:K], se_ghk[K+1], NA)), 3)

```

	truth	ghk	se
dell	0.559	0.502	0.065
apple	1.118	1.142	0.066
ram16	0.671	0.805	0.057
ram32	1.006	1.173	0.067
screen15	0.335	0.350	0.039
price	-1.342	-1.423	0.072
w12	0.500	0.418	0.121
w22	2.500	2.610	NA

Recovery of all eight identified quantities, including the covariance parameters that carry the substitution story — the estimated $\tilde{\omega}_{12}$ is the data confirming that Acer and Dell share unobserved appeal. (The w_{22} transformation’s standard error needs the delta method; Chapter 21, or route 2, where it’s free.)

19.2 Route 2: Gibbs with Data Augmentation

Write the differenced system explicitly first, because it is the object the whole route manipulates:

$$\mathbf{d}_{nt} = \tilde{X}_{nt}\boldsymbol{\beta} + \mathbf{e}_{nt}, \quad \mathbf{e}_{nt} \sim N(\mathbf{0}, W)$$

where $\mathbf{d}_{nt} = (d_2, d_3)$ stacks $d_2 = U_{\text{Dell}} - U_{\text{Acer}}$ and $d_3 = U_{\text{Apple}} - U_{\text{Acer}}$, the rows of $\tilde{X}_{nt}$ are the corresponding rows of $\mathbf{X}d_2$ and $\mathbf{X}d_3$, and W is Chapter 18’s differenced covariance *before* the $\tilde{\omega}_{11} = 1$ convention — the unnormalized object the sampler is free to draw. The

identified quantities are $\tilde{\Omega} = W/w_{11}$ and $\beta/\sqrt{w_{11}}$; the sampler never imposes this, and the post-processing recovers it.

Now forget optimization entirely. The Chapter 16 Albert–Chib logic, scaled up per McCulloch and Rossi (1994): conditional on the latent *differenced utilities* $\mathbf{d}_{nt} = (d_2, d_3)_{nt}$, the MNP is a two-equation normal regression system — conjugate everywhere; conditional on parameters and the observed choice, each task’s $\mathbf{d}_{nt}$ is bivariate normal truncated to the region its choice implies. Three blocks:

- **Latent utilities.** The regions, read off the choice rule: Acer chosen $\Rightarrow d_2 < 0, d_3 < 0$; Dell $\Rightarrow d_2 > \max(0, d_3)$; Apple $\Rightarrow d_3 > \max(0, d_2)$. Sampling a truncated *bivariate* normal is done by one more level of Gibbs: draw d_2 given d_3 (a univariate truncated normal — the conditional mean and variance are the textbook bivariate-normal formulas, the truncation from the region; concretely, if $(d_2, d_3) \sim N((m_2, m_3), W)$ then $d_2 \mid d_3 \sim N(m_2 + \frac{w_{12}}{w_{22}}(d_3 - m_3), w_{11} - \frac{w_{12}^2}{w_{22}})$, and symmetrically for $d_3 \mid d_2$ — the code’s `c21/v21` and `c12/v12`), then d_3 given d_2 . All 4,000 tasks in two vectorized `rtnorm()` calls.
- β . Given the $\mathbf{d}$ ’s: generalized least squares dressed as Bayes — whiten the two-equation system by $W^{-1/2}$ (Cholesky again) and apply the conjugate regression update of Chapter 16.
- W . Given residuals: a 2×2 inverse-Wishart.

And the McCulloch–Rossi move that makes it elegant: **sample the unidentified model, report the identified one.** No constraint forces $w_{11} = 1$ during sampling — the chain wanders the scale ridge freely, conjugacy intact — and every saved draw is normalized *afterward*: $\beta/\sqrt{w_{11}}$, W/w_{11} . Identified functions of unidentified draws are perfectly valid posterior draws of the identified quantities.¹

```
rtnorm <- function(n, mean, sd, a, b) {                                # @sec-draws
  Fa <- pnorm(a, mean, sd); Fb <- pnorm(b, mean, sd)
  u <- pmin(pmax(runif(n, Fa, Fb), 1e-12), 1 - 1e-12)
  qnorm(u, mean, sd)
}
rinvwishart <- function(nu, S) {                                       # @sec-hb-mnl
  K <- nrow(S); L <- t(chol(solve(S)))
  A <- matrix(0, K, K)
  diag(A) <- sqrt(rchisq(K, nu - (1:K) + 1))
  A[lower.tri(A)] <- rnorm(K*(K-1)/2)
  solve(L %*% A %*% t(A) %*% t(L))
}
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

}

gibbs_mnp <- function(Xd2, Xd3, chosen, S_iter, A0 = 0.01,
                      nu0 = 5, V0 = diag(2) * 5) {
  K <- ncol(Xd2); n <- nrow(Xd2)
  beta <- rep(0, K); W <- diag(2)
  d2 <- ifelse(chosen == 2, 1, -0.5)          # any region-respecting start
  d3 <- ifelse(chosen == 3, 1, -0.5)
  out <- list(beta = matrix(NA_real_, S_iter, K),
              W = matrix(NA_real_, S_iter, 3))
  for (s in 1:S_iter) {
    ## -- latent utilities: two vectorized truncated-normal sweeps --
    m2 <- as.vector(Xd2 %*% beta); m3 <- as.vector(Xd3 %*% beta)
    c21 <- W[1,2]/W[2,2]; v21 <- W[1,1] - W[1,2]^2/W[2,2]
    c12 <- W[1,2]/W[1,1]; v12 <- W[2,2] - W[1,2]^2/W[1,1]
    lo2 <- ifelse(chosen == 2, pmax(0, d3), -Inf)
    hi2 <- ifelse(chosen == 2, Inf, ifelse(chosen == 1, 0, d3))
    d2 <- rtnorm(n, m2 + c21*(d3 - m3), sqrt(v21), lo2, hi2)
    lo3 <- ifelse(chosen == 3, pmax(0, d2), -Inf)
    hi3 <- ifelse(chosen == 3, Inf, ifelse(chosen == 1, 0, d2))
    d3 <- rtnorm(n, m3 + c12*(d2 - m2), sqrt(v12), lo3, hi3)
    ## -- beta: whiten the 2-equation system, conjugate regression --
    Li <- solve(t(chol(W)))
    Y1 <- Li[1,1]*d2 + Li[1,2]*d3; Y2 <- Li[2,1]*d2 + Li[2,2]*d3
    Xw1 <- Li[1,1]*Xd2 + Li[1,2]*Xd3; Xw2 <- Li[2,1]*Xd2 + Li[2,2]*Xd3
    Vb <- solve(crossprod(Xw1) + crossprod(Xw2) + diag(A0, K))
    mb <- Vb %*% (crossprod(Xw1, Y1) + crossprod(Xw2, Y2))
    beta <- as.vector(mb + t(chol(Vb)) %*% rnorm(K))
    ## -- W: inverse-Wishart on residuals --
    e2 <- d2 - as.vector(Xd2 %*% beta); e3 <- d3 - as.vector(Xd3 %*% beta)
    Sres <- rbind(c(sum(e2^2), sum(e2*e3)), c(sum(e2*e3), sum(e3^2)))
    W <- rinvwishart(nu0 + n, V0 + Sres)
    out$beta[s, ] <- beta
    out$W[s, ] <- c(W[1,1], W[1,2], W[2,2])
  }
  out
}

```

```

set.seed(191)
g <- gibbs_mnp(Xd2, Xd3, chosen, S_iter = 3000)
post <- 501:3000
scd <- sqrt(g$W[post, 1])          # per-draw normalization

```

```

beta_id <- g$beta[post, ] / scd
tab_gibbs <- data.frame(
  truth = c(beta_star, omega_star),
  gibbs = c(colMeans(beta_id),
    mean(g$W[post, 2] / g$W[post, 1]),
    mean(g$W[post, 3] / g$W[post, 1])),
  post_sd = c(apply(beta_id, 2, sd),
    sd(g$W[post, 2] / g$W[post, 1]),
    sd(g$W[post, 3] / g$W[post, 1]))
)
rownames(tab_gibbs) <- c(colnames(X), "w12", "w22")
round(tab_gibbs, 3)

```

	truth	gibbs	post_sd
altDell	0.559	0.513	0.059
altApple	1.118	1.148	0.063
ram16	0.671	0.809	0.056
ram32	1.006	1.175	0.063
screen15	0.335	0.353	0.037
price	-1.342	-1.428	0.069
w12	0.500	0.436	0.112
w22	2.500	2.683	0.367

Recovery again — and notice what this sampler did *not* contain: no likelihood evaluation (the orthant integral never appears — Chapter 16’s augmentation promise, delivered at multinomial scale), no Metropolis step, no acceptance rate, no tuning constant of any kind, and posterior uncertainty for the covariance parameters included, no delta method required.

19.3 The Two Routes, Head to Head

```

head2head <- data.frame(
  truth = c(beta_star, omega_star),
  ghk_msl = est_ghk,
  gibbs = tab_gibbs$gibbs
)
rownames(head2head) <- c(colnames(X), "w12", "w22")
round(head2head, 3)

```

	truth	ghk_msl	gibbs
altDell	0.559	0.502	0.513
altApple	1.118	1.142	1.148
ram16	0.671	0.805	0.809
ram32	1.006	1.173	1.175
screen15	0.335	0.350	0.353
price	-1.342	-1.423	-1.428
w12	0.500	0.418	0.436
w22	2.500	2.610	2.683

Agreement between the routes to roughly the second decimal on every identified quantity — two philosophies, two codebases, two entirely different computational strategies (one never draws a latent variable, the other never evaluates a probability), one answer. The residual daylight between either column and the *truth* is this dataset’s sampling noise, which the two estimators share because they share the data.

The choice between routes, then, rides on everything *around* the point estimates. Tallying this chapter’s experience: the Gibbs route was faster here, tuning-free, and delivered exact posterior uncertainty for every identified function — and, decisively for what comes next, it *extends*: bolting the Chapter 17 hierarchy onto it is Chapter 20, a short chapter precisely because augmentation composes. The GHK route earns its keep where the Bayesian route strains — very large J (the truncated-utilities block grows with alternatives), classical inference requirements, or embedding in moment-based estimators — and GHK itself is a permanent resident of the choice modeler’s toolkit (K. E. Train 2009, Ch. 5). But for the workhorse cases, the augmentation route is the one modern practice reaches for, and having built both, you know exactly why.

19.4 Validation Against bayesm

The reference implementation of route 2 is `rmnpGibbs` (Rossi, Allenby, and McCulloch 2005). Its conventions differ from ours in one consequential way — it differences against the *last* alternative (base = J), where we used the first — so validating requires restating the identified truth in its parameterization: differences against Apple, ASCs for Acer and Dell, normalization by the first differenced variance *in that system*. Getting a reference package’s conventions right is not busywork; it is most of what “validation” means in practice, and mismatched differencing conventions produce exactly the kind of sign-flipped, rescaled nonsense that looks like a broken sampler.²

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

library(bayesm)
na <- 4 # alt-varying attributes
Xa <- matrix(0, n_task, 3*na)
for (a_i in 1:na) { # columns: variable-major, alt within
  Xa[, (a_i-1)*3 + 1] <- X[i1, 2 + a_i]
  Xa[, (a_i-1)*3 + 2] <- X[i2, 2 + a_i]
  Xa[, (a_i-1)*3 + 3] <- X[i3, 2 + a_i]
}
Xc <- createX(p = 3, na = na, nd = NULL, Xa = Xa, Xd = NULL,
              INT = TRUE, DIFF = TRUE, base = 3)
set.seed(192)
outb <- rmnpGibbs(Data = list(p = 3, y = chosen, X = Xc),
                  Mcmc = list(R = 3000, keep = 1, nprint = 0))

## identified truth in bayesm's (base = Apple) parameterization
M3 <- rbind(c(1, 0, -1), c(0, 1, -1))
Ot3 <- M3 %*% mnp_study$Omega_true %*% t(M3)
sc3 <- sqrt(Ot3[1, 1])
truth_b <- c(int_acer = (0 - 1.0)/sc3, # base = Apple: each ASC is its raw value minus A
             int_dell = (0.5 - 1.0)/sc3,
             mnp_study$beta_true[3:6]/sc3)

## reuse post = 501:3000: same burn-in convention for bayesm's chain
s11 <- outb$sigmadraw[post, 1]
round(rbind(truth = truth_b,
            bayesm = colMeans(outb$betadraw[post, ] / sqrt(s11))), 3)

      int_acer int_dell ram16 ram32 screen15 price
truth   -0.707   -0.354 0.424 0.636    0.212 -0.849
bayesm  -0.711   -0.394 0.497 0.726    0.218 -0.884

round(c(w12 = mean(outb$sigmadraw[post, 2] / s11),
        w22 = mean(outb$sigmadraw[post, 4] / s11),
        truth_w12 = Ot3[1,2]/Ot3[1,1], truth_w22 = Ot3[2,2]/Ot3[1,1]), 3)

      w12      w22 truth_w12 truth_w22
0.831    1.057    0.800    1.000

```

The package agrees — with our estimates and with the (re-parameterized) truth. Same normalize-after-sampling treatment of its draws, note: **bayesm** also samples unidentified and leaves the division to you.

19.5 Key Learnings

- The frequency simulator fails inside optimizers for nameable reasons — non-smooth, can return zero — and GHK (Equation 19.1) repairs both: recursive conditioning through the Cholesky root turns an orthant probability into a product of exact univariate probabilities averaged over truncated draws. Smooth, positive, unbiased; audited against `pmvnorm` before use.
- GHK-MSL is Chapter 12 verbatim: fixed uniforms, log-parameterized Cholesky, box bounds, BFGS — and it recovered all eight identified quantities.
- The augmentation route dissolves the integral instead: truncated-bivariate utility draws (Gibbs-within-Gibbs, two vectorized `rtnorm` sweeps), a whitened conjugate regression for β , an inverse-Wishart for W — *no tuning, no likelihood*. Sample the unidentified model, normalize every draw afterward: identified posteriors from an unconstrained chain, per McCulloch and Rossi (1994).
- Head to head: second-decimal agreement across routes. The differences that matter are architectural — Gibbs extends to the hierarchy (next chapter) and prices uncertainty exactly; GHK serves large J and classical pipelines.
- Validation against `rmnpGibbs` succeeded only after matching its base-alternative convention — reference-implementation conventions are half the work, and mismatches masquerade as broken code.

20 The Hierarchical Bayesian MNP

This short chapter is the book’s technical summit, and the climb is gentler than the altitude suggests. The **hierarchical Bayesian MNP** puts person-specific coefficients over the probit kernel — both of the deep generalizations this book has built, *simultaneously*: flexible substitution through a free error covariance (Chapter 18) and taste heterogeneity through a population distribution (Chapter 17), in one model, estimated by one sampler. It is the model of Rossi, McCulloch, and Allenby (1996), who used it to price the value of purchase-history data, and its construction here is nearly pure assembly — which is itself the chapter’s lesson: augmentation and hierarchy *compose*.

There is one genuinely delightful surprise waiting in the sampler, so let us get to it.

20.1 The Model

Stack the layers. The data layer is Chapter 18’s differenced MNP, now with person-owned coefficients; the population and prior layers are Chapter 17’s, plus a prior for the error covariance:

$$\begin{aligned} \text{utilities: } \mathbf{d}_{nt} &= \tilde{X}_{nt}\beta_n + \mathbf{e}_{nt}, & \mathbf{e}_{nt} &\sim N(\mathbf{0}, W), & y_{nt} &= (\text{who wins}) \\ \text{people: } \beta_n &\sim N(\bar{\beta}, \Sigma) \\ \text{priors: } \bar{\beta} &\sim N(\mathbf{0}, 100I), & \Sigma &\sim IW(\nu_0, V_0), & W &\sim IW(\nu_w, V_w) \end{aligned} \tag{20.1}$$

where $\mathbf{d}_{nt}$ is the pair of base-differenced utilities, $\tilde{X}_{nt}$ stacks the two differenced attribute rows (`Xd2` and `Xd3` in code), and W is, as in Chapter 19, the unnormalized differenced error covariance — the sampled stand-in for Chapter 18’s $\tilde{\Omega}$, recovered as W/w_{11} . Identification: everything from Chapter 18 still applies, resolved the McCulloch–Rossi way — sample unidentified, then normalize every saved draw at its own rate: the utility-scale objects (β_n , $\bar{\beta}$, and the heterogeneity standard deviations $\sqrt{\text{diag } \Sigma}$) divide by $\sqrt{w_{11}}$; the covariance-scale objects (Σ and W itself) divide by w_{11} . The identified error covariance is $\tilde{\Omega} = W/w_{11}$, exactly as in Chapter 19 — the code’s `scd` and `W` ratios implement precisely this.

Why want both generalizations at once? Because they answer different questions that real categories pose simultaneously: the error covariance captures *alternative-level* similarity (Windows machines sharing unobserved appeal — for everyone), while Σ captures *person-level*

persistence (an Apple lover across all eight tasks). A model with only one channel forces the other’s behavior into the wrong container, and both containers feed the substitution patterns that pricing decisions consume.

20.2 The Sampler: No Metropolis Anywhere

List the full conditionals, each inherited from a chapter that already built it:

1. **Latent utilities** $\mathbf{d}_{nt}$ given β_n , W : truncated bivariate normals, region set by the observed choice — Chapter 19’s block, verbatim, with each row’s mean using its *own person’s* β_n .
2. **Each** β_n given that person’s utilities, W , $\bar{\beta}$, Σ : and here is the surprise — with the utilities in hand, person n ’s data layer is *linear and normal*, so this is a **conjugate Bayesian regression** (Chapter 16’s normal–normal update with prior $N(\bar{\beta}, \Sigma)$): an exact multivariate normal draw. The Metropolis step that the HB-MNL needed for exactly this block — because the *logit* kernel resists conjugacy — is unnecessary. No step size, no adaptation, no acceptance monitoring.
3. $\bar{\beta}$ and Σ given the β_n ’s: Chapter 17’s conjugate draws, unchanged.
4. W given residuals: Chapter 19’s inverse-Wishart, unchanged.

Every block exact. Data augmentation didn’t just rescue the MNP’s likelihood; it made the probit’s *hierarchy* fully conjugate — strictly cleaner machinery than the logit’s, a genuine argument for the MNP that only becomes visible from inside the sampler.¹

20.3 Coding It

Simulate the study — Chapter 13’s heterogeneity truth combined with Chapter 18’s error-correlation truth — then the sampler:

```
set.seed(200)
bar_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9,
             screen15 = 0.3, price = -1.2)
sd_true  <- c(0, 0.8, 0, 0.6, 0, 0.7)
rc <- c(2, 4, 6)
Omega_true <- matrix(c(1, 0.6, 0, 0.6, 1, 0, 0, 0, 1), 3, 3)

N <- 500; T <- 8; J <- 3
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

n_rows <- N * T * J
df <- data.frame(
  id = rep(seq_len(N), each = T*J),
  task = rep(rep(seq_len(T), each = J), times = N),
  alt = factor(rep(c("Acer", "Dell", "Apple"), times = N*T),
    levels = c("Acer", "Dell", "Apple")),
  ram = factor(sample(c("8", "16", "32"), n_rows, replace = TRUE),
    levels = c("8", "16", "32")),
  screen = factor(sample(c("13", "15"), n_rows, replace = TRUE),
    levels = c("13", "15")),
  price = round(runif(n_rows, 0.8, 2.4), 2))
X <- model.matrix(~ alt + ram + screen + price, data = df)[, -1]
K <- ncol(X)

betas <- matrix(rep(bar_true, each = N), N, K) # person betas
for (k in rc) betas[, k] <- betas[, k] + sd_true[k] * rnorm(N)
L <- t(chol(Omega_true))
eps <- as.vector(L %*% matrix(rnorm(n_rows), nrow = J)) # within-task correlation
U <- rowSums(X * betas[df$id, ]) + eps
task_id <- cumsum(!duplicated(df[, c("id", "task")]))
df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))

## differenced objects (@sec-mnp-est) and the person index of each task
i1 <- which(df$alt == "Acer"); i2 <- which(df$alt == "Dell")
i3 <- which(df$alt == "Apple")
Xd2 <- X[i2, ] - X[i1, ]; Xd3 <- X[i3, ] - X[i1, ]
chosen <- as.integer(df$alt[df$choice == 1])
person <- df$id[!duplicated(task_id)]

hb_mnp <- function(Xd2, Xd3, chosen, person, S_iter, burn,
  nu0 = ncol(Xd2) + 3, V0 = diag(ncol(Xd2)) * (ncol(Xd2) + 3) * 0.1,
  nu_w = 5, V_w = diag(2) * 5, A0 = 0.01) {
  K <- ncol(Xd2); n <- nrow(Xd2); N <- max(person)
  B <- matrix(0, N, K); bar <- rep(0, K); Sig <- diag(K); W <- diag(2)
  d2 <- ifelse(chosen == 2, 1, -0.5); d3 <- ifelse(chosen == 3, 1, -0.5)
  rows_of <- split(seq_len(n), person)
  out <- list(bar = matrix(NA_real_, S_iter, K),
    sd = matrix(NA_real_, S_iter, K),
    W = matrix(NA_real_, S_iter, 3),
    Bsum = matrix(0, N, K), nkeep = 0)
  for (s in 1:S_iter) {
    ## 1. latent utilities (rows use their person's beta)

```

```

Brow <- B[person, ]
m2 <- rowSums(Xd2 * Brow); m3 <- rowSums(Xd3 * Brow)
c21 <- W[1,2]/W[2,2]; v21 <- W[1,1] - W[1,2]^2/W[2,2]
c12 <- W[1,2]/W[1,1]; v12 <- W[2,2] - W[1,2]^2/W[1,1]
lo2 <- ifelse(chosen == 2, pmax(0, d3), -Inf)
hi2 <- ifelse(chosen == 2, Inf, ifelse(chosen == 1, 0, d3))
d2 <- rtnorm(n, m2 + c21*(d3 - m3), sqrt(v21), lo2, hi2)
lo3 <- ifelse(chosen == 3, pmax(0, d2), -Inf)
hi3 <- ifelse(chosen == 3, Inf, ifelse(chosen == 1, 0, d2))
d3 <- rtnorm(n, m3 + c12*(d2 - m2), sqrt(v12), lo3, hi3)
## 2. each beta_n: conjugate regression, prior N(bar, Sig)
Li <- solve(t(chol(W)))
Y1 <- Li[1,1]*d2 + Li[1,2]*d3; Y2 <- Li[2,1]*d2 + Li[2,2]*d3
Xw1 <- Li[1,1]*Xd2 + Li[1,2]*Xd3; Xw2 <- Li[2,1]*Xd2 + Li[2,2]*Xd3
Sinv <- solve(Sig); Sb <- Sinv %%% bar
for (nn in 1:N) {
  r <- rows_of[[nn]]
  Vb <- chol2inv(chol(crossprod(Xw1[r, , drop = FALSE]) +
                        crossprod(Xw2[r, , drop = FALSE]) + Sinv))
  mb <- Vb %%% (crossprod(Xw1[r, , drop = FALSE], Y1[r]) +
                crossprod(Xw2[r, , drop = FALSE], Y2[r]) + Sb)
  B[nn, ] <- as.vector(mb + t(chol(Vb)) %%% rnorm(K))
}
## 3. bar and Sigma (@sec-hb-mnl blocks)
Vbb <- solve(N * Sinv + diag(A0, K))
bar <- as.vector(Vbb %%% (Sinv %%% colSums(B)) +
                 t(chol(Vbb)) %%% rnorm(K))
Dev <- B - rep(bar, each = N)
Sig <- rinvwishart(nu0 + N, V0 + crossprod(Dev))
## 4. W (@sec-mnp-est block, residuals at person betas)
Brow <- B[person, ]
e2 <- d2 - rowSums(Xd2 * Brow); e3 <- d3 - rowSums(Xd3 * Brow)
Sres <- rbind(c(sum(e2^2), sum(e2*e3)), c(sum(e2*e3), sum(e3^2)))
W <- rinvwishart(nu_w + n, V_w + Sres)

out$bar[s, ] <- bar; out$sd[s, ] <- sqrt(diag(Sig))
out$W[s, ] <- c(W[1,1], W[1,2], W[2,2])
if (s > burn) {
  # store normalized person betas
  out$Bsum <- out$Bsum + B / sqrt(W[1,1]); out$nkeep <- out$nkeep + 1
}
}
out

```

```
}
```

(An unshown chunk carries `rtnorm()` and `rinvwishart()` from previous chapters.) Two notes before running. The `for (nn in 1:N)` loop over persons is unavoidable here — each person’s conjugate update solves their own $K \times K$ system — but it is tolerable by Chapter 9’s own doctrine: five hundred small Cholesky solves per iteration, milliseconds in total. And we adopt the *tightened* Σ prior (V_0 scaled by 0.1) as the default, applying Chapter 17’s practical lesson rather than re-learning it.

20.4 Running It

```
set.seed(201)
fit <- hb_mnp(Xd2, Xd3, chosen, person, S_iter = 2500, burn = 500)
post <- 501:2500
scd <- sqrt(fit$W[post, 1]) # per-draw identification scaling

## identified truth for the scoreboard
M <- rbind(c(-1,1,0), c(-1,0,1))
Ot <- M %*% Omega_true %*% t(M); sc <- sqrt(Ot[1,1])

tab <- data.frame(
  truth_bar = bar_true / sc,
  post_bar = colMeans(fit$bar[post, ] / scd),
  truth_het_sd = sd_true / sc,
  post_het_sd = colMeans(fit$sd[post, ] / scd)
)
rownames(tab) <- colnames(X)
round(tab, 2)
```

	truth_bar	post_bar	truth_het_sd	post_het_sd
altDell	0.56	0.72	0.00	0.30
altApple	1.12	1.24	0.89	0.92
ram16	0.67	0.63	0.00	0.27
ram32	1.01	1.05	0.67	0.68
screen15	0.34	0.36	0.00	0.27
price	-1.34	-1.42	0.78	0.72

```
round(c(w12 = mean(fit$W[post,2]/fit$W[post,1]),
        w22 = mean(fit$W[post,3]/fit$W[post,1]),
        truth_w12 = Ot[1,2]/Ot[1,1], truth_w22 = Ot[2,2]/Ot[1,1]), 2)
```

w12	w22	truth_w12	truth_w22
0.75	2.95	0.50	2.50

Read the scoreboard against everything Part III and IV have taught. The population means track their identified truths. The heterogeneity pattern is recovered — the three truly-random coefficients estimated largest, the truly-fixed ones floored near 0.3 by the (tightened) IW prior, the Chapter 17 phenomenon at its expected magnitude. (`post_het_sd` is the posterior mean of each coefficient’s heterogeneity standard deviation — a point estimate of taste *spread*, not a posterior uncertainty; Chapter 19’s `post_sd` column, by contrast, was exactly a posterior uncertainty. Column names deserve the same care as symbols.) And the error covariance parameters — the *other* substitution channel — are recovered alongside: the model successfully attributed the within-task correlation to W and the cross-task persistence to Σ , the two containers sorted correctly.² Individual-level payoff, one number for the record:

```
Bhat <- fit$Bsum / fit$nkeep
round(cor(Bhat[, 6], betas[, 6] / sc), 2)
```

```
[1] 0.55
```

Person-level tracking comparable to the HB-MNL’s on analogous data, now with correlated errors underneath. All of it from a sampler with *zero* tuning parameters.

20.5 HB-MNL or HB-MNP?

The applied researcher’s closing question for Part IV. Honest guidance, informed by everything built:

Default to the HB-MNL. The logit kernel’s closed form makes every likelihood-adjacent computation — fit statistics, holdout validation, the market simulators of Chapter 21 — cheap and simple; its single Metropolis block is well-understood; the ecosystem (from `bayesm` to every commercial conjoint platform) is built around it; and with rich attribute data, random

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

coefficients already generate most realistic substitution patterns (McFadden and Train 2000’s theorem, practically vindicated).

Reach for the HB-MNP when the errors are the story. Categories with strong alternative-level similarity that attributes don’t capture (store brands, platform ecosystems); scanner-data settings where the same few labeled alternatives recur and W is well-fed; or when the deliverable turns on substitution among specific rivals rather than attribute-level demand. The costs are real — the latent-utility machinery, identification bookkeeping at every step, probabilities that need GHK even *after* estimation for the simulators of Chapter 21 — and they buy exactly one thing: covariance where the logit has independence. Pay when that purchase matters; Rossi, McCulloch, and Allenby (1996)’s targeting application is the canonical case where it did.

Either way — and this is the point of having built both — the choice is now yours to make on substance, not forced by what your tools can estimate.

20.6 Key Learnings

- The HB-MNP stacks this book’s two deep generalizations — free error covariance and coefficient heterogeneity — in one model (Equation 20.1), with each substitution channel answering a different behavioral question.
- Its sampler is assembly with a bonus: latent utilities (Chapter 19) + per-person conjugate regressions + population blocks (Chapter 17) + W ’s inverse-Wishart — and because augmentation makes the data layer linear-normal, **no Metropolis step exists anywhere**. The probit’s hierarchy is cleaner than the logit’s.
- Identification is handled by sampling free and normalizing every draw — now including the person-level draws — by $\sqrt{w_{11}}$.
- Recovery sorted the two channels correctly: heterogeneity into Σ (with the familiar, prior-managed floor on null variances), error correlation into W , person-level estimates tracking truth. Zero tuning parameters were involved.
- Model choice guidance: HB-MNL by default (ecosystem, simplicity, closed-form kernel); HB-MNP when alternative-level error correlation is the substantive point. You can now estimate either from scratch — which was the promise of this book’s cover.

Part V

Using and Extending the Models

21 Post-Estimation: Inference, WTP, and Market Simulation

No one who commissions a choice study wants a coefficient. They want answers: *what is the memory upgrade worth in dollars? What share does our configuration win at \$1,499? Who are the price-insensitive customers?* Between the estimates of Parts II–IV and those answers sits one more layer of craft — transforming parameters into decision quantities, with uncertainty carried through honestly. The layer is thin (mostly arithmetic on things we already have) but it is where analyses most often go wrong in practice, because the arithmetic has traps with famous names. This chapter builds the transformations and disarms the traps.

Two fits power the chapter, both already on disk: the plain MNL of Chapter 8 (refit inline below — it takes a second) and the HB-MNL draws saved by Chapter 17.

```
study  <- readRDS("../data/laptop_study.rds")
laptops <- study$data
X <- model.matrix(~ brand + ram + screen + price, data = laptops)[, -1]
y <- laptops$choice
task_id <- cumsum(!duplicated(laptops[, c("id", "task")]))
K <- ncol(X)

loglik_mnl <- function(beta, y, X, task_id) {
  V <- as.vector(X %*% beta); Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}
mle <- optim(rep(0, K), loglik_mnl, y = y, X = X, task_id = task_id,
             method = "BFGS", control = list(fnscale = -1), hessian = TRUE)
vc  <- solve(-mle$hessian)

hb <- readRDS("../data/hb_mnl_fit.rds")      # bar_draws, sd_draws, B_draws
```

21.1 Inference on Functions of Parameters

The recurring problem: the quantity of interest is $g(\beta)$, not β — a ratio, a probability, a share — and it needs a standard error or interval. Three standard tools, in ascending order of generality.

The delta method linearizes: if $\hat{\beta} \sim N(\beta, V)$ approximately, then

$$g(\hat{\beta}) \sim N(g(\beta), \nabla g' V \nabla g) \quad (21.1)$$

— the function’s gradient maps parameter uncertainty into quantity uncertainty. Exact for linear g , an approximation otherwise, and increasingly strained as g curves (ratios near a small denominator being the canonical strain). We used a special case in Chapter 13 to convert log-scale standard errors; here is the general workhorse. The one-line reason: a first-order expansion $g(\hat{\beta}) \approx g(\beta) + \nabla g(\beta)'(\hat{\beta} - \beta)$ makes $g(\hat{\beta})$ approximately a linear function of an approximately normal vector, and the variance of a linear function $\mathbf{a}'\hat{\beta}$ is $\mathbf{a}'V\mathbf{a}$.

Krinsky–Robb (Krinsky and Robb 1986) simulates instead: draw many $\beta^{(r)} \sim N(\hat{\beta}, V)$ from the estimated asymptotic distribution, push each through g , and read summaries off the resulting sample. No gradients, no linearization — Chapter 2’s summarize-by-sampling logic applied to frequentist output. It is the poor astronomer’s posterior: same mechanics as the Bayesian version, different justification.

Bayesian draws make the problem dissolve: the posterior sample of β is a posterior sample of $g(\beta)$ after applying g row by row — the quiet superpower advertised in Chapter 15, no approximation anywhere.

Watch all three on one question — *the dollar value of upgrading 16GB to 32GB* — and note how they cluster:

$$g(\beta) = \frac{\beta_{\text{ram32}} - \beta_{\text{ram16}}}{-\beta_{\text{price}}} \times \$1,000 \quad (21.2)$$

(Utility difference of the upgrade divided by the utility of a dollar: the scale factor from Section 7.5 cancels in the ratio, which is precisely why ratios are the preferred currency.)

```
g <- function(b) 1000 * (b[4] - b[3]) / (-b[6])

## delta method: numerical gradient of g (via @sec-optimization's num_grad idea)
grad_g <- sapply(1:K, function(k) {
  e <- replace(numeric(K), k, 1e-6)
  (g(mle$par + e) - g(mle$par - e)) / 2e-6
})
```

```

se_delta <- sqrt(t(grad_g) %*% vc %*% grad_g)

## Krinsky-Robb: draws from the asymptotic normal (@sec-mvnorm transform)
set.seed(210)
kr <- apply(mle$par + t(chol(vc)) %*% matrix(rnorm(K*5000), K), 2, g)

## Bayesian: transform the posterior draws of the population means
bayes <- apply(hb$bar_draws, 1, g)

round(data.frame(
  method    = c("delta", "Krinsky-Robb", "Bayes (HB bar)"),
  estimate   = c(g(mle$par), mean(kr), mean(bayes)),
  se_or_sd   = c(se_delta, sd(kr), sd(bayes)),
  lo95       = c(g(mle$par) - 1.96*se_delta, quantile(kr, .025), quantile(bayes, .025)),
  hi95       = c(g(mle$par) + 1.96*se_delta, quantile(kr, .975), quantile(bayes, .975))
)[-1, 0])

```

	estimate	se_or_sd	lo95	hi95
1	217	44	130	305
2	218	45	132	305
3	228	53	127	335

Same answer to rounding — roughly \$250 with an interval of $\pm\$70$ -ish — with the small differences instructive: delta and Krinsky–Robb share inputs and differ only by linearization; the Bayesian row differs slightly more because it comes from a different model fit (the HB’s population mean) on different data-processing of the same study. When the three *disagree materially*, the usual culprit is a denominator too close to zero for Equation 21.1’s linearization — at which point trust the sampling-based rows.

21.2 Willingness-to-Pay, and Its Traps

Equation 21.2 is a **marginal WTP**: the rate at which a consumer trades dollars for an attribute, all else equal. For small, single-attribute comparisons it is the right object. Two traps surround it.

Trap one: the heterogeneous ratio. With HB output it is tempting to compute each *person’s* WTP by applying Equation 21.2 to their β_n draws. Legitimate — and dangerous, because our population model makes $\beta_{\text{price},n}$ normal, so some mass sits near (and past) zero, and dividing by near-zero produces WTP draws in the tens of thousands of dollars with the wrong sign. Watch:

```
wtp_n <- 1000 * (hb$B_draws[, , 4] - hb$B_draws[, , 3]) / (-hb$B_draws[, , 6])
round(c(mean = mean(wtp_n), median = median(wtp_n),
        q01 = quantile(wtp_n, .01), q99 = quantile(wtp_n, .99)), 0)
```

mean	median	q01.1%	q99.99%
-155	210	-6552	7865

The mean is inflated (or even nonsensical) while the median and inner quantiles are sane — the textbook signature of a ratio with a denominator crossing zero. Remedies, in rising order of principle: report medians and quantiles, never means; re-estimate with a sign-constrained (lognormal) price coefficient, as flagged in Chapter 17; or reparameterize the whole model in *WTP space* so the ratio is a parameter rather than a quotient Ben-Akiva, McFadden, and Train (2019). What is not acceptable is publishing the mean of an unconstrained ratio — a mistake with a long paper trail.

Trap two: pseudo-WTP versus market value. The ratio answers “utility compensation for one attribute, one option, no competition.” It does *not* answer “what is the feature worth to a firm” — in a market, a better product steals share from rivals, rivals reprice, and the value of a feature is an equilibrium object. Allenby et al. (2014) make the accounting vivid: ratio-based “WTP” can overstate what pricing can capture by an order of magnitude or more, and the honest firm-side calculation runs through the market simulator below (and, fully, through equilibrium pricing — beyond our scope but built from exactly these pieces). The economist’s consumer-surplus version for the logit has a closed form worth knowing. The expected maximum utility of a Gumbel choice set is $\ln \sum_j e^{V_j}$ plus a constant (Euler’s $\gamma \approx 0.577$, the Gumbel mean — it cancels the moment we take *differences* between scenarios), a classic result we state rather than derive (Small and Rosen 1981; K. E. Train 2009, sec. 3.5) — Chapter 7’s log-sum, in its third and final starring role. Dividing the change in expected utility by $-\beta_{\text{price}}$, the marginal utility of a dollar, converts utils to dollars — legitimate because our linear-in-price utility makes that marginal utility constant. The *compensating variation* for any change in the choice set is therefore:

$$CV = \frac{\ln \sum_j e^{V_j^{\text{after}}} - \ln \sum_j e^{V_j^{\text{before}}}}{-\beta_{\text{price}}} \quad (21.3)$$

```
## market context: three laptops; upgrade the Acer's memory 16 -> 32
V_of <- function(prods, b) as.vector(prods %*% b)
market <- rbind(acer = c(0, 0, 1, 0, 1, 1.2), # dell,apple,ram16,ram32,screen15,price
               dell = c(1, 0, 0, 1, 1, 1.6),
               apple = c(0, 1, 0, 1, 0, 2.0))
market2 <- market; market2["acer", 3:4] <- c(0, 1)
```

```
cv <- 1000 * (log(sum(exp(V_of(market2, mle$par)))) -
             log(sum(exp(V_of(market, mle$par)))) / (-mle$par[6])
round(c(ratio_wtp = g(mle$par), market_cv = cv), 0)
```

```
ratio_wtp market_cv
      217         73
```

The market-context value is a fraction of the all-else-equal ratio — because in a market with good substitutes, consumers escape a mediocre product rather than pay for its upgrade. Neither number is wrong; they answer different questions, and attaching the right number to the right question is the analyst’s job.

21.3 Market Simulation

The workhorse deliverable. A **market simulator** takes a hypothetical market — a set of products with attributes — and returns predicted choice shares, built from nothing but the model’s probability formula and the estimated (distribution of) coefficients. With HB draws, shares come with uncertainty attached: compute the share under each posterior draw of each person’s β_n , average over people (the market), and summarize over draws (the uncertainty):

```
simulate_shares <- function(products, B_draws) {
  S <- dim(B_draws)[1]; Jm <- nrow(products)
  shares <- matrix(NA_real_, S, Jm)
  for (s in 1:S) {
    V <- B_draws[s, , ] %*% t(products)      # persons x products
    V <- V - apply(V, 1, max)                 # @sec-logsumexp, always
    P <- exp(V) / rowSums(exp(V))
    shares[s, ] <- colMeans(P)                # average person = market share
  }
  colnames(shares) <- rownames(products)
  shares
}

sh <- simulate_shares(market, hb$B_draws)
round(rbind(share = colMeans(sh),
            lo95 = apply(sh, 2, quantile, .025),
            hi95 = apply(sh, 2, quantile, .975)), 3)
```

```
acer dell apple
```

```
share 0.302 0.394 0.304
lo95  0.273 0.368 0.280
hi95  0.329 0.421 0.329
```

Read the structure of `simulate_shares()` once more, because it encodes the two averaging levels that must not be confused: `colMeans(P)` averages *people* (heterogeneity — part of the market, inside each draw), while summaries over `s` average *posterior draws* (uncertainty — reported as intervals). Collapse them in the wrong order — computing shares at the *average* β — and the simulator inherits the MNL's IIA at the population level, forfeiting exactly the substitution realism the heterogeneity bought (Chapter 13's aggregation insight, now with money on the line).

Demand curves are the simulator in a loop — sweep one product's price, watch its share:

```
grid <- seq(0.8, 2.4, by = 0.1)
dc <- sapply(grid, function(p) {
  m <- market; m["acer", 6] <- p
  simulate_shares(m, hb$B_draws)[, "acer"]
})
dfd <- data.frame(price = grid, share = colMeans(dc),
                  lo = apply(dc, 2, quantile, .025),
                  hi = apply(dc, 2, quantile, .975))
ggplot(dfd, aes(price, share)) +
  geom_ribbon(aes(ymin = lo, ymax = hi), fill = "grey80") +
  geom_line() +
  labs(x = "Acer price ($1,000s)", y = "preference share")
```

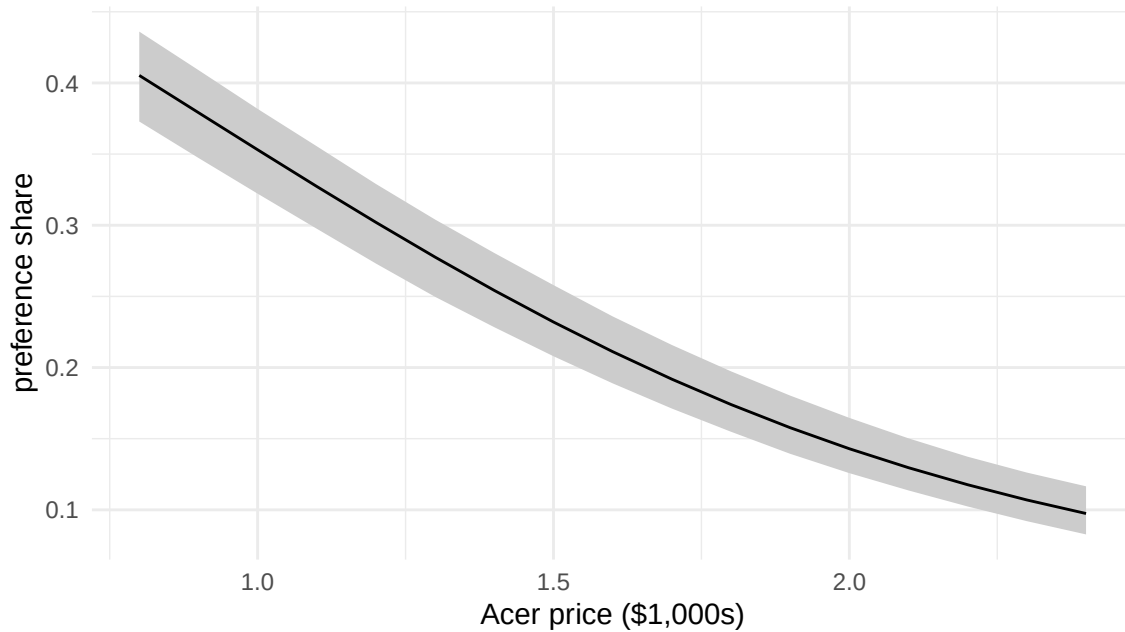


Figure 21.1: Simulated demand curve for the Acer configuration against its two rivals, with 95% posterior bands: the market simulator swept over price. The curve’s every point carries honest uncertainty inherited from the full posterior.

One vocabulary discipline, due to Chapman (2013) and hard-won industry experience: the simulator outputs **preference shares** — shares of choices among these products, by these respondents, under forced attention to the choice task. Real *market* shares also run through awareness, distribution, availability, and the option to buy nothing. Preference shares inform decisions powerfully (especially *differences* between scenarios, where the omitted factors partially cancel); presenting them as market share forecasts is the fastest way for a good model to lose a client’s trust. (The “buy nothing” piece, at least, the model can absorb — the no-choice option of Chapter 22.)

21.4 Elasticities and Substitution

Differentiating the MNL probability gives the textbook elasticity formulas — own-price $e_{jj} = \beta_p p_j (1 - P_j)$ and the IIA-flavored cross-price elasticity that is *identical for all rivals* (K. E. Train 2009, sec. 3.6). Rather than memorize formulas, note that the simulator computes any elasticity numerically — nudge a price, difference the shares — and, run on HB draws, it exhibits the realistic substitution the population mixture creates. The Chapter 10 experiment, replayed with estimated parameters:

```
bump <- market; bump["dell", 6] <- bump["dell", 6] * 0.9 # Dell cuts price 10%
delta <- colMeans(simulate_shares(bump, hb$B_draws)) - colMeans(sh)
round(rbind(base_share = colMeans(sh), change = delta,
            pct_change = delta / colMeans(sh)), 3)
```

	acer	dell	apple
base_share	0.302	0.394	0.304
change	-0.025	0.045	-0.020
pct_change	-0.084	0.114	-0.064

Dell's gain comes disproportionately from the rival whose *buyers look most like Dell's buyers* in taste space — proportional-substitution arithmetic broken by heterogeneity alone, no nests required. This table, produced for a client's actual scenario, is the applied payoff of every substitution-pattern discussion since Chapter 7.

21.5 Using the Individual-Level Estimates

Finally, the deliverable that only Part III could license: person-level quantities. Each respondent's posterior WTP (medians, per the trap above) supports targeting — and one summary picture enforces this book's most repeated interpretive warning:

```
wtp_person <- apply(wtp_n, 2, median) # per-person posterior median
ggplot(data.frame(w = wtp_person), aes(w)) +
  geom_histogram(bins = 50, fill = "grey70") +
  geom_vline(xintercept = mean(wtp_person), linetype = "dashed") +
  coord_cartesian(xlim = quantile(wtp_person, c(.005, .995))) +
  labs(x = "median WTP for 32GB upgrade ($)", y = "respondents")
```

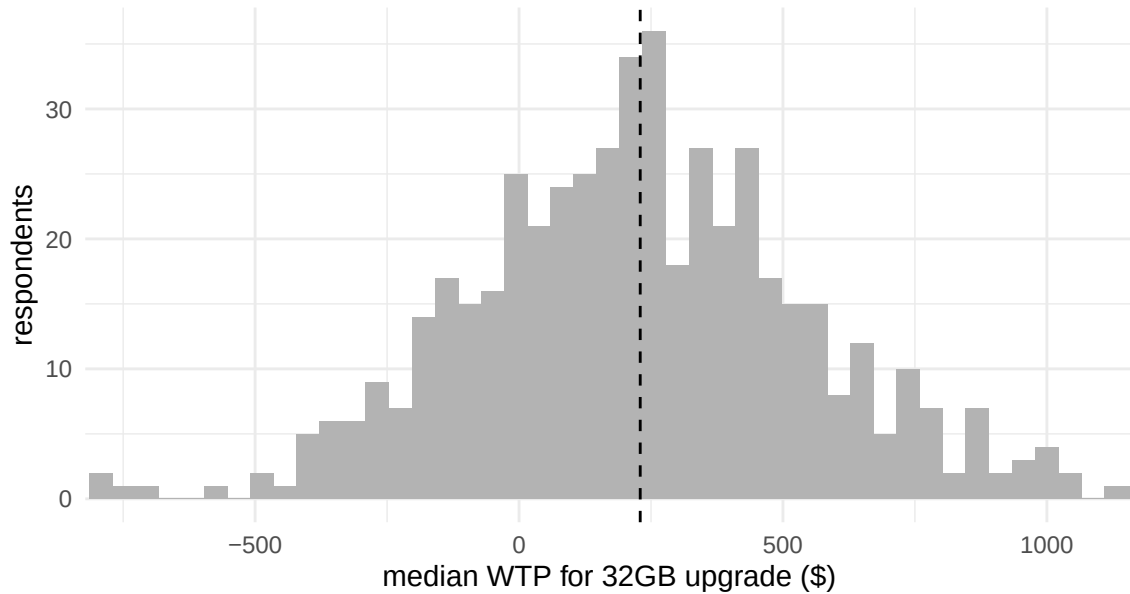


Figure 21.2: The distribution of respondent-level median WTP for the 32GB upgrade. The dashed line is the population average – a number that describes almost nobody: most respondents sit well below it, a long tail sits far above. Averages hide the distribution; the distribution is the market.

Segments fall out of such distributions directly (the right tail is the premium-upgrade target list, by respondent ID), and every claim about them inherits posterior uncertainty from the draws. This is Chapter 14’s wish list, fully cashed: person-level, honest, decision-ready.

21.6 Key Learnings

- Decision quantities are functions of parameters, and their uncertainty comes from three tools: the delta method (Equation 21.1; fast, linearized), Krinsky–Robb (simulate the asymptotic normal), and posterior draws (exact, trivial). They agree when g is tame; when they diverge, suspect the linearization.
- Marginal WTP is a scale-free ratio (Equation 21.2) — but ratios of heterogeneous coefficients explode when the denominator crosses zero (report medians; or constrain signs; or move to WTP-space), and ratio-WTP is *not* market value: the log-sum compensating variation (Equation 21.3) and the simulator answer the market question, usually with much smaller numbers (Allenby et al. 2014).
- The market simulator is the model’s probability formula pointed at hypothetical products; with HB draws it averages people *inside* each draw and summarizes uncertainty *across* draws — never share-at-the-average-beta, which resurrects IIA.

- Simulated shares are preference shares, not market shares (Chapman (2013)); differences between scenarios travel better than levels.
- Heterogeneity is the deliverable: person-level WTP distributions, targeting lists, and substitution patterns that put competition where the taste overlap is. The average consumer, meanwhile, describes almost nobody — plot the distribution.

22 Extensions: Building Your Own Model

Every chapter until now taught you models other people built. This one teaches the move that matters for your own research: *changing* a model — because your data have a feature the standard models ignore, or your theory implies behavior they cannot express. The introduction promised that the true value of choice modeling is the ability to create tailor-made models; here is where the promise is kept, three times, on three extensions drawn from live research literatures.

The method is the one this book has been rehearsing since Chapter 1, now stated as a recipe:

1. **Change the model** — a new term in utility, a new choice rule h , a new distribution somewhere in the hierarchy.
2. **Re-derive the choice probability** (or recognize that a known one still applies).
3. **Write the simulator first.** If you cannot simulate it, you do not understand it yet.
4. **Modify the likelihood or the sampler** — usually a handful of lines in code you already have.
5. **Simulate-and-recover before real data ever enters.** The answer key is the only proof your estimator works.

Watch the recipe’s economy in what follows: each extension changes remarkably little code, because Part I’s architecture — stacked data, kernel likelihood functions, modular samplers — was designed to be extended. That was the point all along.

22.1 Extension 1: The No-Choice Option

The problem. Our laptop study forces a choice: every task ends with one of three laptops “bought.” Real shoppers walk away, and a model estimated on forced choices systematically overstates demand — every simulated market in Chapter 21 implicitly assumed everyone buys *something*. The standard repair adds a **no-choice option** (“I would not buy any of these”) to each task (Brazell et al. 2006).

The model change. The none option is a fourth alternative whose utility has no attributes — nothing about it varies — so it is pure constant plus noise:

$$U_{nt,\text{none}} = \gamma + \varepsilon_{nt,\text{none}} \quad (22.1)$$

One new parameter. And here Section 4.5’s groundwork pays off: in the long data format, the none option is *just another row* — attribute columns zero, a none dummy equal to one — and the MNL machinery does not know or care that one alternative is special. The choice probabilities, likelihood, gradient, and code are all unchanged.¹

Simulate, estimate, recover — the recipe executed in one block. The simulator is Section 7.6’s with a fourth row per task; the estimator is *literally* `loglik_mnl()`:

```
set.seed(220)
sim_none_data <- function(N, T, beta, gamma_none) {
  J <- 3; n_rows <- N * T * J
  df <- data.frame(
    id = rep(seq_len(N), each = T*J),
    task = rep(rep(seq_len(T), each = J), times = N),
    alt = rep(1:J, times = N*T),
    brand = factor(sample(c("Acer", "Dell", "Apple"), n_rows, TRUE),
      levels = c("Acer", "Dell", "Apple")),
    ram = factor(sample(c("8", "16", "32"), n_rows, TRUE),
      levels = c("8", "16", "32")),
    screen = factor(sample(c("13", "15"), n_rows, TRUE),
      levels = c("13", "15")),
    price = round(runif(n_rows, 0.8, 2.4), 2))
  X <- cbind(model.matrix(~ brand + ram + screen + price, df)[, -1],
    none = 0)
  ## append one none row per task: all-zero attributes, none dummy = 1
  Xn <- X[df$alt == 1, ]; Xn[] <- 0; Xn[, "none"] <- 1
  X <- rbind(X, Xn)
  ids <- c(df$id, df$id[df$alt == 1])
  tsks <- c(df$task, df$task[df$alt == 1])
  ord <- order(ids, tsks, c(df$alt, rep(4, sum(df$alt == 1))))
  X <- X[ord, ]; ids <- ids[ord]; tsks <- tsks[ord]

  task_id <- cumsum(!duplicated(cbind(ids, tsks)))
  U <- as.vector(X %*% c(beta, gamma_none)) - log(-log(runif(nrow(X))))
  choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))
  list(X = X, y = choice, task_id = task_id, id = ids)
}
```

¹The identification bookkeeping: adding a constant-utility alternative is what *anchors* the utility scale for “buying nothing,” making statements like “share who buy nothing at these prices” meaningful — the quantity Chapter 21 flagged as missing. The deeper behavioral question — is the none utility really constant across tasks? — is live research: Hung, Kurz, et al. (2025) find respondents update the none option’s appeal as tasks teach them market prices, and model exactly that.

```

beta_true <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9,
              screen15 = 0.3, price = -1.2)
gamma_true <- -0.5 # none is mildly unattractive
d1 <- sim_none_data(500, 8, beta_true, gamma_true)
mean(rowsum(d1$y * (d1$X[, "none"] == 1), d1$task_id)) # share choosing none

```

```
[1] 0.264
```

About a fifth of tasks end in “none” — real demand information the forced design threw away. Estimation requires nothing new:

```

loglik_mnl <- function(beta, y, X, task_id) { # @sec-mnl-mle, untouched
  V <- as.vector(X %*% beta); Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}
fit1 <- optim(rep(0, 7), loglik_mnl, y = d1$y, X = d1$X, task_id = d1$task_id,
             method = "BFGS", control = list(fnscale = -1), hessian = TRUE)
round(rbind(truth = c(beta_true, none = gamma_true),
            est = fit1$par,
            se = sqrt(diag(solve(-fit1$hessian)))), 2)

```

	dell	apple	ram16	ram32	screen15	price	none
truth	0.50	1.00	0.60	0.90	0.30	-1.20	-0.50
est	0.53	1.02	0.67	0.84	0.33	-1.20	-0.52
se	0.06	0.06	0.06	0.06	0.05	0.05	0.10

Recovery, including the new parameter. Total new code: a simulator variant. Total new estimation code: zero — the none option was a *data* change, which is the least expensive kind of extension there is.

The dual-response refinement, in brief, because practice uses it heavily (Diener, Orme, and Yardley 2006; Brazell et al. 2006): asking “which would you choose?” *and then* “would you actually buy it?” keeps the forced task’s statistical efficiency while measuring purchase intent. The likelihood treats the two answers as two choices sharing parameters — the first over the J products, the second between the winner and none — and Diener, Orme, and Yardley (2006)’s practical insight is that the second response can be *stacked as an additional pseudo-task* in the long data, after which standard MNL/HB software estimates it unmodified: another extension executed entirely in the data layer. The modern frontier connects the none utility to economic primitives — Hung, Liu, et al. (2025) derive it from a budget constraint, which is precisely where our second extension goes.

22.2 Extension 2: A Budget Constraint

The problem. For laptops — or cars, or houses — some shoppers simply *cannot* pay premium prices, no matter their tastes. A linear price term says a \$2,400 laptop is merely less attractive; a **budget** says it is not an option. Models that ignore binding budgets misestimate price response exactly where pricing decisions live, overstating premium demand — the bias Pachali, Kurz, and Otter (2022) document in conjoint studies of exactly our product category.

The model change — and this one, unlike Extension 1, changes the behavioral rule h itself. Following the spirit of Pachali, Kurz, and Otter (2022) (simplified to a common budget τ for transparency): alternatives priced above τ are *screened out* — infeasible, never chosen — and money left under the budget enters utility with diminishing returns:

$$U_{ntj} = \begin{cases} \mathbf{x}'_{ntj}\boldsymbol{\beta} + \alpha \log(\tau - p_{ntj}) + \varepsilon_{ntj} & p_{ntj} < \tau \\ -\infty & p_{ntj} \geq \tau \end{cases} \quad (22.2)$$

(The $\log(\tau - p)$ term is the utility of the outside numeraire — what the unspent budget buys — and it is where the price sensitivity now lives; there is no separate linear price coefficient.²) The MNL probability survives with two amendments: the choice set shrinks to the feasible alternatives, and utilities gain curvature in price that grows explosive as $p \rightarrow \tau$. Both are *visible in behavior* — shoppers never choosing above a ceiling, and price sensitivity concentrating near it — which is what identifies τ .

Simulator first, per the recipe — and note how Equation 22.2 transcribes:

```
set.seed(221)
sim_budget_data <- function(N, T, beta, alpha, tau) {
  J <- 3; n_rows <- N * T * J
  df <- data.frame(
    id = rep(seq_len(N), each = T*J),
    task = rep(rep(seq_len(T), each = J), times = N),
    alt = rep(1:J, times = N*T),
    brand = factor(sample(c("Acer","Dell","Apple"), n_rows, TRUE),
      levels = c("Acer","Dell","Apple")),
    ram = factor(sample(c("8","16","32"), n_rows, TRUE),
      levels = c("8","16","32")),
    screen = factor(sample(c("13","15"), n_rows, TRUE),
      levels = c("13","15")),
```

²This is the direct-utility formulation of Allenby, Hardt, and Rossi (2019) and Pachali, Kurz, and Otter (2022): choose one product plus outside spending subject to the budget; substitute the constraint into utility; the log delivers diminishing marginal utility of money. Our simplification is the common τ ; the research version makes τ_n heterogeneous, which the road map below restores.

```

    price = round(runif(n_rows, 0.8, 2.4), 2))
  X <- model.matrix(~ brand + ram + screen, data = df)[, -1] # note: no price column
  V <- as.vector(X %*% beta) +
    ifelse(df$price < tau, alpha * log(tau - df$price), -Inf)
  U <- V - log(-log(runif(n_rows)))
  task_id <- cumsum(!duplicated(df[, c("id", "task")]))
  df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))
  list(df = df, X = X, task_id = task_id)
}

beta_np <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9, screen15 = 0.3)
alpha_true <- 1.4; tau_true <- 2.6 # budget $2,600: binding for top prices
d2 <- sim_budget_data(500, 8, beta_np, alpha_true, tau_true)
max(d2$df$price[d2$df$choice == 1]) # nobody chose above tau... check

```

[1] 2.4

(A task where *all three* laptops exceed τ would have no feasible option; with our price range topping out at $\$2,400 < \tau$, the issue does not arise here, but your own version must decide what shoppers do then — a none option, Extension 1, is the natural coupling. Model changes interact; simulators surface the interactions before referees do.)

The likelihood amends the Chapter 8 kernel in exactly the two places the model changed — the utility line gains the budget term, and infeasible rows carry $-\infty$ (which the log-sum-exp machinery handles gracefully: $e^{-\infty} = 0$ drops them from the denominator):

```

loglik_budget <- function(theta, y, X, price, task_id) {
  K <- ncol(X)
  beta <- theta[1:K]; alpha <- theta[K+1]; tau <- theta[K+2]
  if (tau <= max(price[y == 1])) return(-1e10) # chosen>tau impossible
  V <- as.vector(X %*% beta) + alpha * log(pmax(tau - price, 1e-12))
  V[price >= tau] <- -Inf
  Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  sum((V - Vmax - log(rowsum(eV, task_id)[task_id]))[y == 1])
}

```

One wrinkle deserves its sentence: the feasibility screen makes the likelihood *zero* for any τ below the highest chosen price — the data flatly refute such budgets — so the parameter space has a data-dependent boundary, and the surface is non-smooth in τ at every observed price

point. Textbook MLE regularity has left the building.³ Practical estimation is untroubled: profile the likelihood on a grid of τ (grid search’s redemption arc, from Section 5.7) with the smooth parameters maximized at each point:

```
taus <- seq(2.45, 3.2, by = 0.025)
prof <- sapply(taus, function(tt) {
  f <- optim(c(rep(0, 5), 1), function(th, ...)
    loglik_budget(c(th, tt), ...),
    y = d2$df$choice, X = d2$X, price = d2$df$price,
    task_id = d2$task_id,
    method = "BFGS", control = list(fnscale = -1, maxit = 300))
  c(tau = tt, ll = f$value, f$par)
})
best <- prof[, which.max(prof["ll", ])]
round(rbind(truth = c(beta_np, alpha = alpha_true, tau = tau_true),
  est = c(best[3:7], best[8], best["tau"])), 2)
```

	dell	apple	ram16	ram32	screen15	alpha	tau
truth	0.50	1.00	0.60	0.90	0.30	1.4	2.60
est	0.57	1.04	0.72	0.98	0.23	1.5	2.65

```
ggplot(data.frame(tau = prof["tau", ], ll = prof["ll", ]), aes(tau, ll)) +
  geom_line() + geom_point(size = 0.8) +
  geom_vline(xintercept = tau_true, linetype = "dashed") +
  labs(x = expression(tau~"($1,000s)"), y = "profile log-likelihood")
```

³To numerical-comparison precision for MCMC: posterior means from finite chains carry Monte Carlo error, so “agreement” means within a couple of MC standard errors, not five decimals. The Chapter 8 five-decimal standard was for deterministic optimizers.

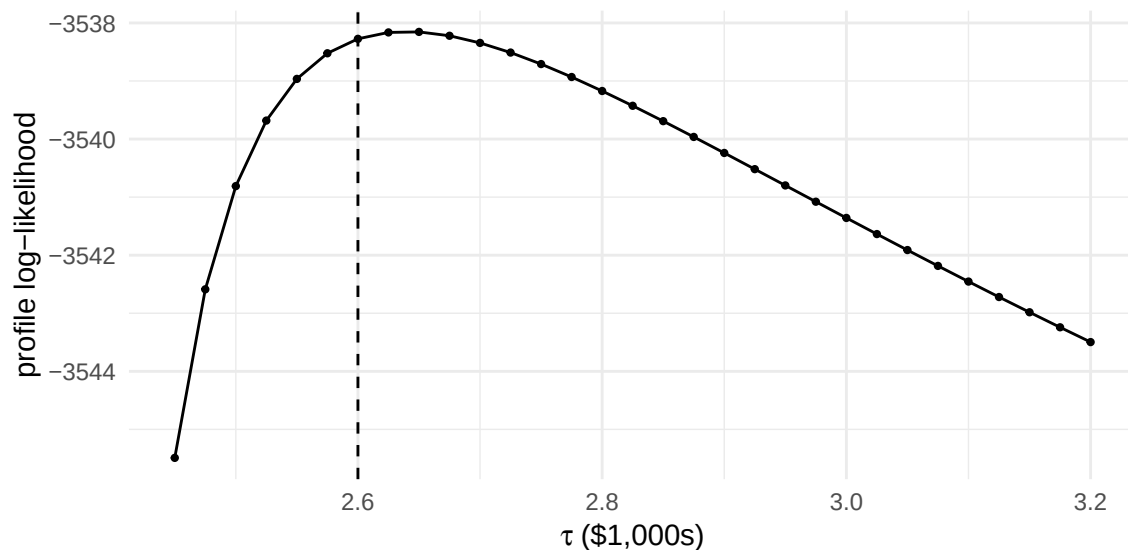


Figure 22.1: The profile log-likelihood over the budget parameter. The data reject budgets below the highest chosen price outright (left boundary), climb to a well-defined peak near the true \$2,600, and fall away as larger budgets flatten the log($\tau - p$) curvature the choices exhibit.

Recovery — including a latent budget no survey question asked about, identified purely from where price sensitivity concentrates. **The research road map** from here is Pachali, Kurz, and Otter (2022)’s: make τ_n person-specific with a hierarchical prior (the Chapter 17 machinery — the budget becomes one more transformed coefficient, log-normal to stay positive, sampled in the person-level Metropolis block), let stated budget questions enter as noisy *indicators* of τ_n rather than truth, and watch premium-price demand curves correct themselves by double-digit percentages. Every component is in your toolkit already.

22.3 Extension 3: A Flexible Mixing Distribution

The problem. Parts II–III assumed tastes are *normally* distributed — always unimodal, symmetric, thin-tailed. Suppose the true market is polarized: a budget segment and a premium segment with little in between. A normal mixing distribution will report the average of two camps and the spread between them — a distribution centered where almost nobody lives (Chapter 21’s warning, at the population level).

The model change, following K. Train (2016)’s *logit-mixed logit* in one dimension: replace the price coefficient’s normal distribution with a **nonparametric distribution on a grid**. Fix G candidate values $b_1, \dots, b_G$ spanning the plausible range; estimate the probability mass w_g on

each, via a softmax in free parameters ($w_g \propto e^{\theta_g}$ — Chapter 11’s device). One normalization before coding it: adding a constant to every θ_g leaves the softmax weights unchanged — the level unidentifiability of Section 7.5, back again — so we pin $\theta_1 = 0$ and estimate the remaining $G - 1$ free parameters. That is the `c(0, theta)` in the code below. The reader who has reached this chapter sees immediately what this *is*: a latent class model where the class locations are frozen to a grid and only the shares are estimated — flexibility through many cheap classes rather than a few expensive ones.

The estimation insight that makes it practical — and this is the trick worth stealing for other models: person n ’s sequence likelihood at grid point g ,

$$L_{ng} = \prod_{t=1}^T P_{nt}(y_{nt} \mid \beta_{\text{price}} = b_g)$$

— the ordinary MNL likelihood of person n ’s whole sequence with the price coefficient frozen at grid value b_g (and the other coefficients at their step-1 values; `persll_at()` returns its log, one column of an $N \times G$ matrix per grid point) — *does not depend on the weights*. Compute the $N \times G$ matrix of L_{ng} **once**; thereafter the log-likelihood

$$\ell(\theta) = \sum_n \log \sum_g w_g(\theta) L_{ng} \quad (22.3)$$

costs a matrix-vector product per evaluation — the optimizer runs on a *precomputed* likelihood, and estimation takes seconds regardless of how expensive the kernel was [K. Train (2016) reports 69 mixing parameters estimated in 18 seconds; ours is humbler]:

```
set.seed(222)
## simulate a polarized market: price coefficient is bimodal, others fixed
sim_laptop_design <- function(N, T, J) {
  n_rows <- N * T * J
  data.frame(id = rep(seq_len(N), each = T*J),
             task = rep(rep(seq_len(T), each = J), times = N),
             alt = rep(seq_len(J), times = N*T),
             brand = factor(sample(c("Acer","Dell","Apple"), n_rows, TRUE),
                           levels = c("Acer","Dell","Apple")),
             ram = factor(sample(c("8","16","32"), n_rows, TRUE),
                          levels = c("8","16","32")),
             screen = factor(sample(c("13","15"), n_rows, TRUE),
                              levels = c("13","15")),
             price = round(runif(n_rows, 0.8, 2.4), 2))
}
N <- 1000; T <- 8
df <- sim_laptop_design(N, T, 3)
```

```

X <- model.matrix(~ brand + ram + screen + price, data = df)[, -1]
K <- ncol(X)
task_id <- cumsum(!duplicated(df[, c("id","task")]))

b_fixed <- c(dell = 0.5, apple = 1.0, ram16 = 0.6, ram32 = 0.9, screen15 = 0.3)
bp_n <- ifelse(runif(N) < 0.55, rnorm(N, -2.0, 0.25), rnorm(N, -0.6, 0.25))
V <- as.vector(X[, 1:5] %*% b_fixed) + X[, 6] * bp_n[df$id]
U <- V - log(-log(runif(nrow(X))))
df$choice <- as.integer(ave(U, task_id, FUN = function(u) u == max(u)))

## step 1 (approximation, disclosed): fix non-price coefficients at MNL estimates
mnl <- optim(rep(0, K), loglik_mnl, y = df$choice, X = X, task_id = task_id,
             method = "BFGS", control = list(fnscale = -1))

## step 2: precompute person-by-gridpoint log-likelihoods
grid_b <- seq(-3.2, 0.2, length.out = 18)
id_of_task <- df$id[!duplicated(task_id)]
persll_at <- function(bp) {
  V <- as.vector(X[, 1:5] %*% mnl$par[1:5]) + X[, 6] * bp
  Vmax <- ave(V, task_id, FUN = max)
  eV <- exp(V - Vmax)
  logP <- (V - Vmax - log(rowsum(eV, task_id)[task_id]))[df$choice == 1]
  as.vector(rowsum(logP, id_of_task))
}
llng <- sapply(grid_b, persll_at) # N x G, computed ONCE

## step 3: maximize over the weights -- each evaluation is trivial
loglik_lml <- function(theta, llng) {
  lw <- c(0, theta) - log(sum(exp(c(0, theta)))) # log softmax weights
  M <- apply(sweep(llng, 2, lw, `+`), 1, max)
  sum(M + log(rowsums(exp(sweep(llng, 2, lw, `+`) - M))))
}
fit3 <- optim(rep(0, length(grid_b) - 1), loglik_lml, llng = llng,
             method = "BFGS", control = list(fnscale = -1, maxit = 1000))
w_hat <- exp(c(0, fit3$par)); w_hat <- w_hat / sum(w_hat)

true_dens <- function(x) 0.55*dnorm(x, -2.0, 0.25) + 0.45*dnorm(x, -0.6, 0.25)
gap <- diff(grid_b)[1]
ggplot(data.frame(b = grid_b, w = w_hat / gap), aes(b, w)) +
  geom_col(width = gap * 0.9, fill = "grey70") +
  geom_function(fun = true_dens, linewidth = 0.8) +
  labs(x = expression(beta[price]), y = "density")

```

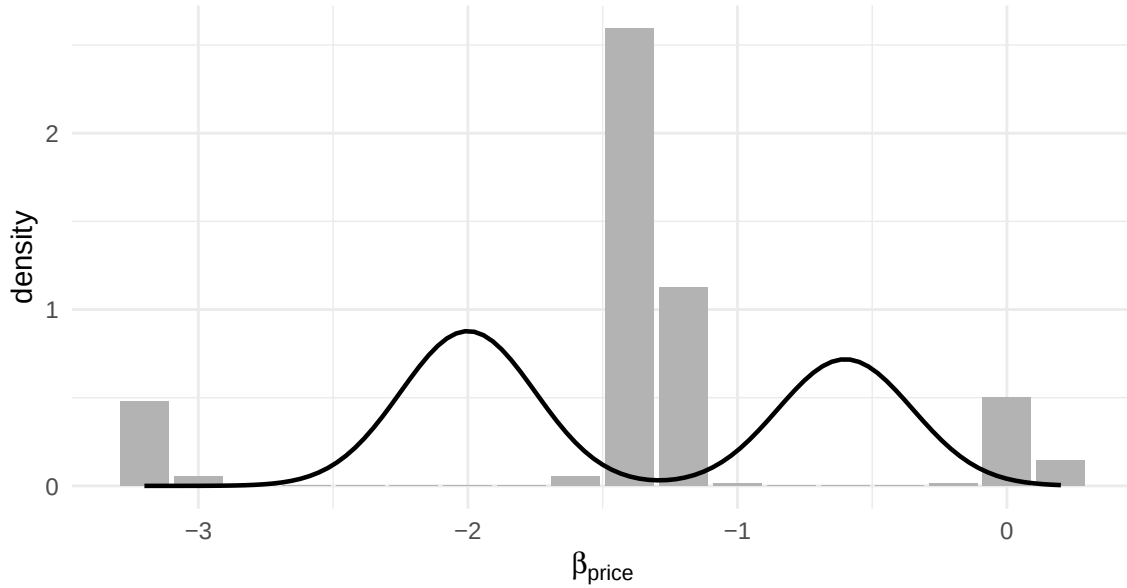


Figure 22.2: The estimated nonparametric distribution of the price coefficient (bars: mass at each grid point) against the true bimodal density (line). Nobody told the model there were two segments; eighteen grid weights found them. A normal mixing distribution, by construction, could report only the average of the two camps.

Two modes, correct locations, correct proportions — from choice data alone, with a mixing distribution the model was free to shape.⁴ The disclosed shortcut (fixing the other coefficients first) is the honest price of a demonstration; K. Train (2016)’s full treatment estimates everything jointly, adds smoothness through spline or polynomial structure on the weights, and extends to multiple dimensions. And the Bayesian reader has already noticed that Equation 22.3’s grid weights would make a lovely Dirichlet-prior Gibbs block — extensions compose across Parts II and III alike.

22.4 Where to Go From Here

Three extensions, three different entry points into the machinery — the data layer (none option), the behavioral rule (budget), the population distribution (flexible mixing) — and none required more than a page of new code. The pattern generalizes to the research frontier: volumetric and multiple-discreteness models change the choice rule to Kuhn–Tucker conditions (Allenby, Hardt, and Rossi 2019); WTP-space models reparameterize utility so preference heterogeneity

⁴Sharp eyes will connect this figure to Chapter 17’s normal-prior HB: run on these polarized data, the normal hierarchy would shrink both camps toward a middle where no one lives. Flexible mixing is not decoration; on the wrong population shape, the normal assumption distorts every downstream deliverable in Chapter 21.

lives in dollar units (K. Train 2016; Ben-Akiva, McFadden, and Train 2019); context-dependent none utilities track what respondents learn across tasks (Hung, Kurz, et al. 2025); and the unified framework of Marshall and Bradlow (2002) shows ratings, rankings, and choices as different observation layers over one latent-utility core — extensions of the *data* model rather than the utility model.

Your own extension will not be on this list; that is the point of it being yours. The recipe at the chapter’s head does not care: change the model, re-derive the probability, *simulate first*, adjust the estimator, recover before real data. You now own every tool the recipe calls for — which is what this book was for.

22.5 Key Learnings

- Custom models are the recipe: model change $\rightarrow$ probability $\rightarrow$ **simulator first** $\rightarrow$ estimator tweak $\rightarrow$ simulate-and-recover. The architecture built in Part I is what makes each step small.
- The no-choice option is a *data-layer* extension: one all-zero row with an ASC per task, zero new estimation code, and it restores the walk-away margin that forced choice suppresses (with dual-response designs as the practice-standard refinement, also executable in the data layer).
- The budget constraint changes h itself: screening plus $\log(\tau - p)$ curvature. Its likelihood has a data-dependent boundary and kinks in τ — profiled grid search handles what BFGS regularity cannot — and a latent budget is identified from behavior alone. The hierarchical version is Pachali, Kurz, and Otter (2022) and everything it needs is in Chapter 17.
- Flexible mixing replaces the normal population with estimated grid weights — latent class with frozen locations — and Train’s precomputed-likelihood trick (L_{ng} once, then Equation 22.3 in microseconds) makes it nearly free. On polarized data it finds the two camps a normal distribution would average away.
- Extensions enter through the data, the rule, or the population — and compose, with each other and with both estimation philosophies. Go build yours.

References

- Abramovich, Felix, and Ya'acov Ritov. 2023. *Bayesian Statistics*. CRC press.
- Albert, James H., and Siddhartha Chib. 1993. "Bayesian Analysis of Binary and Polychotomous Response Data." *Journal of the American Statistical Association* 88 (422): 669–79.
- Allenby, Greg M., Jeff D. Brazell, John R. Howell, and Peter E. Rossi. 2014. "Economic Valuation of Product Features." *Quantitative Marketing and Economics* 12: 421–56.
- Allenby, Greg M., Nino Hardt, and Peter E. Rossi. 2019. "Economic Foundations of Conjoint Analysis." In *Handbook of the Economics of Marketing, Volume 1*, edited by Jean-Pierre Dubé and Peter E. Rossi, 151–92. North-Holland.
- Ben-Akiva, Moshe, Daniel McFadden, and Kenneth Train. 2019. *Foundations of Stated Preference Elicitation: Consumer Behavior and Choice-Based Conjoint Analysis*. Vol. 10. Foundations and Trends in Econometrics 1–2. Now Publishers.
- Brazell, Jeff D., Christopher G. Diener, Ekaterina Karniouchina, William L. Moore, Václav Séverin, and Pierre-Francois Uldry. 2006. "The No-Choice Option and Dual Response Choice Designs." *Marketing Letters* 17 (4): 255–68.
- Chapman, Christopher N. 2013. "9 Things Clients Get Wrong about Conjoint Analysis." In *Sawtooth Software Conference Proceedings*.
- Croissant, Yves. 2020. "Estimation of Random Utility Models in R: The mlogit Package." *Journal of Statistical Software* 95 (11): 1–41.
- Diener, Christopher, Bryan Orme, and David Yardley. 2006. "Dual Response 'None' Approaches: Theory and Practice." In *Sawtooth Software Conference Proceedings*.
- Eggers, Felix, Henrik Sattler, Thorsten Teichert, and Franziska Völckner. 2022. "Choice-Based Conjoint Analysis." In *Handbook of Market Research*, edited by Christian Homburg, Martin Klarmann, and Arnd Vomberg, 781–819. Springer.
- Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. CRC press.
- Gelman, Andrew, Aki Vehtari, Daniel Simpson, Charles C Margossian, Bob Carpenter, Yuling Yao, Lauren Kennedy, Jonah Gabry, Paul-Christian Bürkner, and Martin Modrák. 2020. "Bayesian Workflow." *arXiv Preprint arXiv:2011.01808*.
- Goldberger, Arthur S. 1991. *A Course in Econometrics*. Harvard University Press.
- Greene, William H. 2008. "Discrete Choice Modeling." In *Palgrave Handbook of Econometrics, Volume 2: Applied Econometrics*, edited by Terence C. Mills and Kerry Patterson. Palgrave Macmillan.
- Hansen, Bruce. 2022a. *Econometrics*. Princeton University Press.
- . 2022b. *Probability & Statistics for Economists*. Princeton University Press.

- Hensher, David A., John M. Rose, and William H. Greene. 2015. *Applied Choice Analysis*. 2nd ed. Cambridge University Press.
- Hess, Stephane, and Andrew Daly, eds. 2024. *Handbook of Choice Modelling*. 2nd ed. Edward Elgar.
- Hung, Ya-Han, Peter Kurz, Rachael Bailey, Joel Huber, and Greg M. Allenby. 2025. "Re-Examining the No-Choice Option in Conjoint Analysis." *Journal of Choice Modelling*.
- Hung, Ya-Han, Peng Liu, Jeff D. Brazell, and Greg M. Allenby. 2025. "Dual Response in Conjoint Analysis." *Marketing Letters*.
- Kennedy, Peter. 2008. *A Guide to Econometrics*. John Wiley & Sons.
- Krinsky, Itzhak, and A. Leslie Robb. 1986. "On Approximating the Statistical Properties of Elasticities." *The Review of Economics and Statistics* 68 (4): 715–19.
- Marshall, Pablo, and Eric T. Bradlow. 2002. "A Unified Approach to Conjoint Analysis Models." *Journal of the American Statistical Association* 97 (459): 674–82.
- McCulloch, Robert, and Peter E. Rossi. 1994. "An Exact Likelihood Analysis of the Multinomial Probit Model." *Journal of Econometrics* 64 (1–2): 207–40.
- McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian Course with Examples in r and Stan*. Chapman; Hall/CRC.
- McFadden, Daniel. 2001. "Economic Choices." *American Economic Review* 91 (3): 351–78.
- McFadden, Daniel, and Kenneth Train. 2000. "Mixed MNL Models for Discrete Response." *Journal of Applied Econometrics* 15 (5): 447–70.
- Molloy, Joseph, Felix Becker, Basil Schmid, and Kay W. Axhausen. 2021. "Mixl: An Open-Source R Package for Estimating Complex Choice Models on Large Datasets." *Journal of Choice Modelling* 39.
- Pachali, Max J., Peter Kurz, and Thomas Otter. 2022. "The Perils of Ignoring the Budget Constraint in Single-Unit Demand Models." *Journal of Marketing Research*.
- Rao, Vithala R. 2014. *Applied Conjoint Analysis*. Springer.
- Rossi, Peter E., Greg M. Allenby, and Robert McCulloch. 2005. *Bayesian Statistics and Marketing*. John Wiley & Sons.
- Rossi, Peter E., Robert E. McCulloch, and Greg M. Allenby. 1996. "The Value of Purchase History Data in Target Marketing." *Marketing Science* 15 (4): 321–40.
- Sarrias, Mauricio. 2016. "Discrete Choice Models with Random Parameters in R: The Rchoice Package." *Journal of Statistical Software* 74 (10): 1–31.
- Sarrias, Mauricio, and Ricardo Daziano. 2017. "Multinomial Logit Models with Continuous and Discrete Individual Heterogeneity in R: The gmnl Package." *Journal of Statistical Software* 79 (2): 1–46.
- Small, Kenneth A., and Harvey S. Rosen. 1981. "Applied Welfare Economics with Discrete Choice Models." *Econometrica* 49 (1): 105–30.
- Train, Kenneth. 2001. "A Comparison of Hierarchical Bayes and Maximum Simulated Likelihood for Mixed Logit." *Working Paper, University of California, Berkeley*.
- . 2016. "Mixed Logit with a Flexible Mixing Distribution." *Journal of Choice Modelling* 19: 40–53.
- Train, Kenneth E. 2009. *Discrete Choice Methods with Simulation*. 2nd ed. Cambridge University Press. <https://eml.berkeley.edu/books/choice2.html>.

- Wickham, Hadley. 2019. *Advanced r*. 2nd ed. Chapman; Hall/CRC.
- Wickham, Hadley, Mine Cetinka-Rundel, and Garrett Golemund. 2023. *R for Data Science*. 2nd ed. O'Reilly Media.

A Appendix

A.1 R Packages for Discrete Choice

This book estimated everything by hand on principle; your research need not. The ecosystem below is where hand-coded understanding meets production convenience — and each package served as a validation target somewhere in these pages, so you have seen their output next to ours.

mlogit (Croissant 2020) is the canonical starting point: multinomial, nested, and mixed logit (by MSL with Halton draws), plus heteroskedastic and probit variants, built on the **dfidx** indexed-data-frame layer whose conventions mirror our long-format contract (Chapter 4). It validated our MNL (Chapter 8), nested logit (Chapter 10), and mixed logit (Chapter 13). Its formula interface — three parts, separated by `|`, for alternative-varying/generic, individual-specific, and alternative-varying/alternative-specific-coefficient variables — takes an afternoon to internalize and repays it.

gmnl (Sarrias and Daziano 2017) extends the frequentist heterogeneity menu: latent class, mixed-mixed, and G-MNL models with one interface. A caution from experience (Chapter 11): its current CRAN build has decayed against the modern **mlogit** data layer — verify against its own examples before trusting it with your data. **Rchoice** (Sarrias 2016) covers random-parameter binary and ordered models, and its documentation contains one of the most compact published tutorials on simulated-likelihood mechanics. **mixl** (Molloy et al. 2021) compiles user-specified utilities to C++ for very large mixed logit problems — the industrial version of Chapter 9’s advice.

bayesm (Rossi, Allenby, and McCulloch 2005) is the Bayesian workhorse and the direct ancestor of this book’s Part III–IV samplers: **rhierMnlRwMixture** is Chapter 17 with a mixture-of-normals population layer, **rmnpGibbs** is Chapter 19’s route 2, and hierarchical MNP, volumetric models, and more live alongside. Its data formats are list-heavy and its identification conventions (differencing base, post-sampling normalization) must be matched carefully — Chapter 19’s validation section is a worked example of exactly that care.

Beyond CRAN’s choice-specific corner: **Stan** (via **rstan**/**brms**/**cmdstanr**) estimates any model in this book by Hamiltonian Monte Carlo — less transparent than your own Gibbs sampler, far more general; the **Apollo** package is the transportation field’s full-featured platform; and Sawtooth Software’s Lighthouse Studio is the commercial conjoint standard whose HB engine is, at heart, Chapter 17’s sampler.

A.2 Notation Glossary

Table A.1: Notation used throughout the book.

Symbol	Meaning	First appears
$n = 1, \dots, N$	decision-makers (respondents)	Chapter 3
$t = 1, \dots, T$	choice situations (tasks) per person	Chapter 4
$j = 1, \dots, J$	alternatives within a task	Chapter 4
$r = 1, \dots, R$	simulation draws	Chapter 1
$s = 1, \dots, S$	MCMC iterations	Chapter 15
y, y_{nt}, y_{ntj}	choice outcome; indicator form	Chapter 1, Chapter 8
$\mathbf{x}_{ntj}, \mathbf{X}$	attribute vector; stacked design matrix	Chapter 1, Chapter 4
β, β_n	utility coefficients; person-specific version	Chapter 1, Chapter 13
$\varepsilon, \varepsilon_{ntj}$	unobserved utility components	Chapter 1
$h(\cdot), f(\cdot)$	behavioral rule; density of unobservables	Chapter 1
U_{ntj}, V_{ntj}	total and representative utility	Chapter 7
P_{ntj}	choice probability	Chapter 7
$\ell(\cdot), L(\cdot)$	log-likelihood; likelihood	Chapter 5
$s(\beta), H(\beta)$	score (gradient); Hessian	Chapter 5
$\hat{\beta}$	an estimate (MLE unless stated)	Chapter 5
π_c, β_c	class share; class coefficients	Chapter 11
$\bar{\beta}, \Sigma$	population mean and covariance of tastes	Chapter 13
λ	nested logit dissimilarity parameter	Chapter 10
I_{ntm}	inclusive value (log-sum) of nest m	Chapter 10
Ω	raw $J \times J$ MNP error covariance	Chapter 18
$\tilde{\Omega}$	differenced $(J - 1) \times (J - 1)$ covariance, normalized so $\tilde{\omega}_{11} = 1$: $\tilde{\Omega} = M\Omega M' / (M\Omega M')_{11}$	Chapter 18

Symbol	Meaning	First appears
W	unnormalized differenced covariance sampled in Gibbs; identified: W/w_{11} and $\beta/\sqrt{w_{11}}$	Chapter 19
τ	proposal step size (MCMC); budget (ext. 2)	Chapter 15, Chapter 22
γ	no-choice (none) constant	Chapter 22
$IW(\nu_0, V_0)$	inverse-Wishart prior	Chapter 17
$\hat{R}$, ESS	convergence ratio; effective sample size	Chapter 15
$\mathbf{d}_{nt}$	latent base-differenced utilities (d_2, d_3)	Chapter 19
M, A_j	differencing matrix; per-chosen-alternative transformation	Chapter 18, Chapter 19
$g = 1, \dots, G; w_g$	grid points and weights (flexible mixing) — not the covariance elements	Chapter 22
$g(\beta)$	w_{11}, w_{12}, w_{22} of Chapter 19 function of parameters (WTP ratio, shares) — distinct from the grid index g	Chapter 21
CV	compensating variation (log-sum change, in dollars)	Chapter 21
α	acceptance probability (MCMC); budget-utility weight (ext. 2)	Chapter 15, Chapter 22

A.3 Common Numerical Errors and Their Symptoms

A field guide to the failures this book staged deliberately, for the day they happen to you accidentally.

The likelihood returns -Inf or NaN. Almost always `exp()` overflow/underflow or `log(0)`. Check for a missing log-sum-exp max-shift (Section 8.3), a probability that underflowed before its log (Chapter 5), or mis-scaled data (price in dollars instead of thousands, Chapter 6).

The optimizer “converges” instantly at the starting values. Gradient numerically zero at the start — typically a flat region from saturated probabilities (again scaling) or a bug

returning a constant. Evaluate the objective at a few hand-picked points before blaming the optimizer (Chapter 5’s candidate-scoring table is the template).

Estimates match no truth, but two implementations agree. Then the estimator is fine and the *comparison* is wrong: check identification normalizations (scale, base alternative, reference levels — Section 7.5, Chapter 18) before checking code. Sign-flipped, uniformly-rescaled estimates are the signature of a differencing-convention mismatch (Chapter 19’s `bayesm` footnote).

An MSL fit wanders or won’t converge. Are the draws fixed across evaluations? Re-drawn randomness inside the objective is the classic self-inflicted wound (Chapter 12, demonstrated). Also check draw counts via the doubling diagnostic.

Wild individual-level estimates. Per-person estimation with small T — perfect separation and incidental parameters (Chapter 14). The cure is pooling: conditional means or a hierarchy, never a bigger iteration cap.

MCMC traces drift and never settle, but the answers look right. If the drifting quantities are unidentified (MNP scale) while identified *functions* of them are stable, the sampler is healthy — diagnose the normalized series (Chapter 19). If identified quantities drift: too little burn-in, a step size at an extreme acceptance rate (Chapter 15), or blocks updating on stale values (a coding bug).

Mixture or hierarchy estimates depend on the run. Local maxima (multiple starts, compare summits — Chapter 11) or label switching (order classes by a coefficient before comparing). For hierarchical variance parameters that refuse to approach zero: that is the inverse-Wishart floor, a prior property, not a bug (Chapter 17).

Grouped computations silently wrong. The sort-order contract (Chapter 4): `rowsum()`, `ave()`, and index-broadcasting all assume rows grouped as promised, and person-level weights indexed at the wrong level (task vs person, Chapter 11) fail without errors. `stopifnot()` guards are cheap; use them.